UNIVERSIDAD CEU - SAN PABLO

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA DE SISTEMAS DE TELECOMUNICACIÓN



TRABAJO FIN DE GRADO

Diseño e implementación de una aplicación generativa de explicación de contenidos de Ingeniería de dominio específico

Autor: Arturo Fierro Pardo
Directores: Ana Sanmartín Domenech, Pablo Pérez Tirador

Junio 2026

UNIVERSIDAD SAN PABLO-CEU
ESCUELA POLITÉCNICA SUPERIOR
División de Ingeniería

Datos del alumno

NOMBRE:

Datos del Trabajo

TÍTULO DEL PROYECTO:

Tribunal calificador

PRESIDENTE:	FDO.:
SECRETARIO:	FDO.:
VOCAL:	FDO.:

Reunido este tribunal el _____/_____/_____, acuerda otorgar al Trabajo Fin de Grado presentado por Don_____la calificación de _____.

AGRADECIMIENTOS

Ni yo, ni este trabajo habríamos llegado tan lejos sin las personas que me han acompañado durante estos años de carrera universitaria.

Me gustaría agradecer en primer lugar a Pablo Pérez Tirador y a Ana Sanmartín Domenech, por aceptar dirigir este proyecto y por confiar en mí para llevar a cabo este trabajo. A Ana, en particular, quiero agradecerle algo que va más allá de este TFG. Gracias por enseñarme lo que significa la investigación de verdad, la paciencia que requiere, la exigencia que implica y la satisfacción que da cuando todo encaja. Gracias por estar pendiente de cada avance y de cada tropiezo, gracias por ayudarme y estar ahí en todo momento. Gracias por, además de ser profesora y dirigir este proyecto, por ser persona, por hacerme reír y por haberme hecho disfrutar tanto de este proyecto.

A mis padres, por haberme apoyado en cada decisión de esta carrera sin cuestionarla. Sé que las cenas en las que intentaba explicar qué es un modelo de lenguaje o por qué llevaba tres días peleándome con un *script* de *Python* no eran fáciles de seguir. Pero siempre han hecho el esfuerzo de preguntar, de interesarse, de estar ahí.

A todos mis amigos, por haber hecho que esta carrera tan difícil se convierta en los mejores años de mi vida. Gracias por todas las risas, llantos y enfados durante los exámenes. Gracias por todos los planes que hemos hecho para despejarnos o solo para vernos una vez más. Y gracias por darme una amistad que me llevo para toda la vida.

Y finalmente a mi novia, porque esta carrera también es mérito suyo. Gracias por obligarme a estudiar cuando yo no quería, gracias por levantarme en los momentos en los que pensaba que no iba a poder, y gracias por haber aguantado con paciencia los días o semanas en los que no tenía tiempo para hacer nada y me atascaba en un bucle. Si he llegado hasta aquí es, en buena parte, porque no has dejado que me rinda.

ABSTRACT

Large language models produce high-quality text but depend on external APIs, have a per-query cost, and require network connectivity, making them unsuitable for offline educational tools. This work compares three mode families (LLM, SLM and Mini-SLM) on Data-to-Text tasks across three generic corpora (WebNLG, ToTTo, KELM) and a custom telecommunications dataset of 24.680 question-answer pairs built from the CEU studies plan.

Evaluation employs ROUGE-L, METEOR, BLEU, and BERTScore, with statistical analysis via Friedman tests and Nemenyi post-hoc. A GAN trained from scratch and LoRA fine-tuning for each Mini-SLM model on the custom corpus are also evaluated as alternative generation approaches.

A prior survey showed that 86.4% of participants would be willing to use a tool of this kind as a study aid. Results show that the quality gap between LLM and SLM is statistically significant but practically small across all datasets and metrics. After fine-tuning on the telecommunications corpus, Qwen 3 1.7B FT outperforms all base Mini-SLM and untuned SLM with zero inference cost. The model is deployed in the TelecoSLM application, a Flutter app that runs entirely offline.

RESUMEN

Los modelos de lenguaje de gran tamaño producen texto de alta calidad, pero dependen de API externas, tienen coste por consulta y requieren conexión a red, lo que los hace inadecuados para herramientas educativas sin conexión. Este trabajo compara tres familias de modelos (LLM, SLM, y Mini-SLM) en tareas de *Data-to-Text* sobre tres corpus genéricos (WebNLG, ToTTo, KELM) y un *dataset* propio de telecomunicaciones con 24.680 pares pregunta-respuesta construidos a partir del plan de estudios del CEU.

La evaluación emplea ROUGE-L, METEOR, BLEU y BERTScore, con análisis estadístico mediante tests de Friedman y post-hoc de Nemenyi. Se evalúa también una GAN entrenada desde cero y *fine-tuning* con LoRA para los modelos Mini-SLM sobre el corpus propio como enfoque alternativo de generación.

Una encuesta previa mostró que el 86.4% de los participante estaría dispuesto a usar una herramienta de este tipo como apoyo al estudio. Los resultados muestran que la brecha entre LLM y SLM es estadísticamente significativa pero prácticamente pequeña. Tras el *fine-tuning* sobre el corpus de telecomunicaciones, Qwen 3 1.7B FT supera a todos los modelos Mini-SLM base con coste de inferencia cero. El modelo se despliega en la aplicación TelecoSLM, una aplicación *Flutter* que funciona completamente sin conexión.

ÍNDICE

1 INTRODUCCIÓN 1

1.1 MOTIVACIÓN 2
1.2 OBJETIVOS 3
1.3 ESTRUCTURA 3

2 ESTADO DEL ARTE 4

2.1 INTRODUCCIÓN A REDES GENERATIVAS ANTAGÓNICAS (GAN) 4
2.1.1 El auge del modelo generativo 4
2.1.2 Verosimilitud y parametrización 4
2.1.3 Aprendizaje de representaciones y espacio latente 5
2.1.4 Redes Generativas Antagónicas (GAN) 5
2.1.5 Aplicaciones prácticas y limitaciones de las GAN en NLP 8
2.2 INTRODUCCIÓN A LLM Y SLM 9
2.2.1 Large Language Models 9
2.2.2 Small Language Models 11
2.3 NATURALEZA DE LA INFORMACIÓN 12
2.3.1 Selección de datasets para la experimentación 12

3 MATERIAL Y MÉTODOS 14

3.1 DISEÑO EXPERIMENTAL 15
3.2 VALIDACIÓN DE NECESIDAD 16
3.3 DATASETS GENÉRICOS Y PREPROCESAMIENTO 17
3.4 MODELOS EVALUADOS Y PIPELINE DE INFERENCIA 19
3.4.1 Pipeline de inferencia 20
3.5 IMPLEMENTACIÓN DE LA GAN TEXTUAL 21
3.6 CONSTRUCCIÓN DEL CORPUS PROPIO DE INGENIERÍA 27
3.7 FINE-TUNING DE MINI-SLM 29
3.8 MARCO DE EVALUACIÓN 31
3.9 DESARROLLO DE LA APLICACIÓN (TELECOSLM) 33

4 RESULTADOS Y DISCUSIÓN 35

4.1 RESULTADOS DE LA ENCUESTA DE VALIDACIÓN 35
4.2 COMPARATIVA LLM, SLM Y MINI-SLM 36
4.2.1 Análisis estadístico 37
4.3 IMPACTO DEL FINE-TUNING 40
4.3.1 Análisis estadístico 42
4.4 ANÁLISIS INTEGRADOR 44
4.5 APLICACIÓN MÓVIL: TELECOSLM 45

5 CONCLUSIONES Y TRABAJO FUTURO 49

6 BIBLIOGRAFÍA 51

7 USO DE LA IA 59

ANEXO A. FUNDAMENTOS TEÓRICOS DE INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO 64

A.1 INTRODUCCIÓN A LA INTELIGENCIA ARTIFICIAL 64
A.2 INTRODUCCIÓN A MACHINE LEARNING 66
A.3 INTRODUCCIÓN A NATURAL LANGUAGE PROCESSING (NLP) 72
A.4 INTRODUCCIÓN A LA ARQUITECTURA TRANSFORMER 74

ANEXO B. ENCUESTA DE FIABILIDAD DE TEXTO GENERADO POR IA 78

B.1 DESCRIPCIÓN DE LA MUESTRA 78

ANEXO C. CORPUS PROPIO DE TELECOMUNICACIONES 82

ANEXO D. RESULTADOS 86

D.1 ANÁLISIS DE PARAMETRICIDAD 87
D.2 COMPARATIVA 1: LLM VS SLM VS MINI-SLM 98
D.3 COMPARATIVA 2: MINI-SLM VS FINE-TUNING VS GAN 122

ANEXO E. INTERFAZ Y FUNCIONALIDADES DE TELECOSLM 138

E.1 PANTALLA DE INICIO 138
E.2 PANTALLA DE CHAT 139
E.3 PANTALLA DE CONFIGURACIÓN 140

ÍNDICE DE FIGURAS

FIGURA 1: CONCEPTO DE APRENDIZAJE DE REPRESENTACIONES (*REPRESENTATION LEARNING*). ADAPTADO DE FOSTER (2019) [10]. 5
FIGURA 2: ANALOGÍA CONCEPTUAL DEL FUNCIONAMIENTO DE UNA RED GAN. ADAPTADO DE FOSTER (2019) [10]. 6
FIGURA 3: PROCESOS DE ENTRENAMIENTO DEL MODELO DISCRIMINADOR Y GENERADOR. ADAPTADO DE FOSTER (2019) [10]. 7
FIGURA 4: COMPARATIVA ESPACIAL ENTRE EL DOMINIO DE DATOS CONTINUOS (IMÁGENES) Y EL DOMINIO DE DATOS ESTRICTAMENTE DISCRETOS (TEXTO). ADAPTADO DE BENGIO ET AL. (2019) [17]. 8
FIGURA 5: FLUJO DE TRABAJO DEL PROYECTO. FUENTE PROPIA. 15
FIGURA 6: EJEMPLOS DE ENTRADA Y REFERENCIA HUMANA DE CADA *DATASET* GENÉRICO EMPLEADO. FUENTE PROPIA 18
FIGURA 7: ARQUITECTURA GENERAL DEL *PIPELINE* DE GENERACIÓN Y EVALUACIÓN DE LOS MODELOS LLM, SLM Y MINI-SLM. FUENTE PROPIA. 21
FIGURA 8: EFECTO DEL PARÁMETRO DE TEMPERATURA EN LA DISTRIBUCIÓN *GUMBEL-SOFTMAX*. ADAPTADO DE JANG ET AL. (2017) [54] 22
FIGURA 9: ARQUITECTURA Y FLUJO DEL MODELO GENERADOR. FUENTE PROPIA. 22
FIGURA 10: ARQUITECTURA DEL MODELO DISCRIMINADOR. FUENTE PROPIA. 24
FIGURA 11: EJEMPLOS DE LOS CINCO *TOKENS* ESPECIALES Y SU FUNCIÓN. SE MUESTRA EN ORDEN (1) PAD, (2) BOS, (3) EOS, (4) UNK Y (5) SEP. FUENTE PROPIA. 26
FIGURA 12: PROCESO DE RECOPILACIÓN, GENERACIÓN Y VALIDACIÓN DE PARES DE PREGUNTAS-RESPUESTAS DEL *DATASET*. FUENTE PROPIA. 28
FIGURA 13: ARQUITECTURA DE *FINE-TUNING* CON *LORA*. FUENTE PROPIA. 29
FIGURA 14: ARQUITECTURA DE LA APLICACIÓN EN *WINDOWS*. FUENTE PROPIA. 33
FIGURA 15: BOXPLOTS DE TODAS LAS MÉTRICAS EN EL *DATASET* WEBNLG. FUENTE PROPIA. 36
FIGURA 16: Q-Q TESTS DE NORMALIDAD PARA METEOR Y BERTSCORE PARA WEBNLG. FUENTE PROPIA. 37
FIGURA 17: DIAGRAMA DE DIFERENCIA CRÍTICA PARA TODAS LAS MÉTRICAS EN EL *DATASET* KELM. FUENTE PROPIA 39
FIGURA 18: TIEMPO ACUMULADO DE INFERENCIA POR MODELO EN EL *DATASET* TOTTO. FUENTE PROPIA. 40
FIGURA 19: BOXPLOTS DE TODAS LAS MÉTRICAS EN EL CORPUS PROPIO. FUENTE PROPIA. 41
FIGURA 20: Q-Q TESTS DE NORMALIDAD PARA METEOR Y BERTSCORE PARA WEBNLG. FUENTE PROPIA. 42
FIGURA 21: DIAGRAMA DE DIFERENCIA CRÍTICA PARA TODAS LAS MÉTRICAS EN EL CORPUS PROPIO. FUENTE PROPIA. 43
FIGURA 22: PANTALLA PRINCIPAL DE TELECOSLM EN WINDOWS. FUENTE PROPIA 45
FIGURA 23: EJEMPLO DE RESPUESTA CON RENDERIZADO LATEX Y MARKDOWN EN TELECOSLM. FUENTE PROPIA. 45

FIGURA 24: PANTALLA DE AJUSTES: FUENTES DE INFERENCIA DISPONIBLES (MODELO LOCAL, ARCHIVO GGUF Y SERVIDOR REMOTO). FUENTE PROPIA. 46
FIGURA 25: VALIDACIÓN BETA DE TELECOSLM. *TESTERS* DURANTE EL PERIODO DE PRUEBA. FUENTE PROPIA. 48

ÍNDICE DE TABLAS

TABLA 1: COMPARATIVA TÉCNICA DE MODELOS LLM. ELABORACIÓN PROPIA. 10
TABLA 2: COMPARATIVA TÉCNICA DE MODELOS SLM. ELABORACIÓN PROPIA. 11
TABLA 3: COMPARATIVA DE DATOS ESTRUCTURADOS Y NO ESTRUCTURADOS PARA LA GENERACIÓN DE TEXTO. ELABORACIÓN PROPIA. 13
TABLA 4: PRINCIPALES PAQUETES Y DEPENDENCIAS DEL ENTORNO PYTHON 15
TABLA 5: HIPERPARÁMETROS DEL GENERADOR DE LA GAN 23
TABLA 6: HIPERPARÁMETROS DEL DISCRIMINADOR DE LA GAN 25
TABLA 7: HIPERPARÁMETROS DE ENTRENAMIENTO DE LA GAN 27
TABLA 8: HIPERPARÁMETROS UTILIZADOS PARA EL *FINE-TUNING*. 30
TABLA 9: RESULTADOS DE LA DISPOSICIÓN DE LOS PARTICIPANTES A UTILIZAR UNA HERRAMIENTA DE IA MEDIDO 35
TABLA 10: MÉTRICAS DE RENDIMIENTO DE QWEN 3 1.7B FT EN TELECOSLM. 47
TABLA 11: RESULTADOS Y COMENTARIOS POR PARTE DE 12 *TESTERS* CON PERFIL OBJETIVO. 48

ÍNDICE DE ECUACIONES

ECUACIÓN 1: FUNCIÓN DE VEROSIMILITUD (*LIKELIHOOD*) 4
ECUACIÓN 2: ESTIMACIÓN DE MÁXIMA VEROSIMILITUD (MLE). 5
ECUACIÓN 3: FUNCIÓN OBJETIVO DE LAS GAN (VALOR MIN-MAX). 6
ECUACIÓN 4: ECUACIÓN DE *GUMBEL-SOFTMAX*. 21
ECUACIÓN 5: ECUACIÓN DE LA MÉTRICA BLEU. 31
ECUACIÓN 6: ECUACIÓN DE LA MÉTRICA METEOR. 31
ECUACIÓN 7: ECUACIÓN DE LA MÉTRICA ROUGE-L. 32
ECUACIÓN 8: ECUACIÓN DE LA MÉTRICA BERTSCORE. 32
ECUACIÓN 9: ECUACIÓN DE EMISIONES DE CARBONO. 33
ECUACIÓN 10: MODELO DE REGRESIÓN LINEAL MATRICIAL. 68
ECUACIÓN 11: FUNCIÓN DE PERDIDA CON REGULARIZACIÓN. 69
ECUACIÓN 12: MODELO MATEMÁTICO DE LA NEURONA ARTIFICIAL. 72
ECUACIÓN 13: MECANISMO DE ATENCIÓN CRUZADA (*CROSS-ATTENTION*). 76

ÍNDICE DE ABREVIATURAS O ACRÓNIMOS

AdamW	Adaptive Moment Estimation with Weight Decay
AI	Artificial Intelligence
ALBERT	A Lite BERT
ANN	Artificial Neural Network
API	Application Programming Interface
APK	Android Package Kit
ARM	Advanced RISC Machine
AWS	Amazon Web Services
B	Billion
BART	Bidirectional and Auto-Regressive Transformers
BBDD	Bases de Datos
BERT	Bidirectional Encoder Representations from Transformers
BERTScore	Bidirectional Encoder Representations from Transformers Score
BLEU	Bilingual Evaluation Understudy
BOS	Beginning of Sequence
BPE	Byte Pair Encoding
CD	Critical Difference
CEU	Centro de Estudios Universitarios (Universidad CEU San Pablo)
CNN	Convolutional Neural Network
Co2	Dióxido de Carbono
CPU	Central Processing Unit
CSV	Comma-Separated Values
DART	DAta Record to Text
DDR4	Double Data Rate 4
E2E	End-to-End
EE.UU.	Estados Unidos
ELM	Extreme Learning Machine
EOS	End of Sequence
ESO	Educación Secundaria Obligatoria
FDM	Frequency Division Multiplexing
FFI	Foreign Function Interface
FLOP	Floating Point Operation
FT	Fine-Tuning

GAN	Generative Adversarial Network
GB	Gigabyte
GELU	Gaussian Error Linear Unit
GGUF	GPT-Generated Unified Format
GPT	Generative Pre-trained Transformer
GPTQ	GPT Quantization
GPU	Graphics Processing Unit
GQA	Grouped Query Attention
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
IA	Inteligencia Artificial
IC	Intervalo de Confianza
IP	Internet Protocol
JSON	JavaScript Object Notation
KELM	Knowledge Enhanced Language Model
LaTeX	Lamport TeX
LCS	Longest Common Subsequence
LLM	Large Language Model
LoRA	Low-Rank Adaptation
LSTM	Long Short-Term Memory
MB	Megabyte
MDP	Markov Decision Process
METEOR	Metric for Evaluation of Translation with Explicit Ordering
Mini-SLM	Mini Small Language Model
ML	Machine Learning
MLE	Maximum Likelihood Estimation
MLP	Multi-Layer Perceptron
MoE	Mixture of Experts
MT	Machine Translation
NER	Named Entity Recognition
NLG	Natural Language Generation
NLP	Natural Language Processing
NLU	Natural Language Understanding
NVMe	Non-Volatile Memory Express

PAD	Padding (token)
PEFT	Parameter-Efficient Fine-Tuning
Q4_K_M	Quantization 4-bit K-Means
QA	Question Answering
QAM	Quadrature Amplitude Modulation
Q-Q	Quantile-Quantile
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
RDF	Resource Description Framework
REINFORCE	REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility
ReLU	Rectified Linear Unit
RGPD	Reglamento General de Protección de Datos
RMSNorm	Root Mean Square Normalization
RNN	Recurrent Neural Network
RoBERTa	Robustly Optimized BERT Pretraining Approach
ROUGE-L	Recall-Oriented Understudy for Gisting Evaluation – Longest Common Subsequence
RTX	Ray Tracing Texel eXtreme
SD	Standard Deviation
SEP	Separator (token)
SeqGAN	Sequence Generative Adversarial Network
SLM	Small Language Model
SQL	Structured Query Language
SSD	Solid State Drive
SSE	Server-Sent Events
STEM	Science, Technology, Engineering and Mathematics
SwiGLU	Swish-Gated Linear Unit
T5	Text-to-Text Transfer Transformer
TCP	Transmission Control Protocol
TDM	Time Division Multiplexing
TelecoSLM	Telecomunicaciones Small Language Model
TFG	Trabajo Fin de Grado
ToTTo	Table-to-Text
TPU	Tensor Processing Unit

TTFT	Time to First Token
UE	Unión Europea
UNK	Unknown (token)
URL	Uniform Resource Locator
VRAM	Video Random Access Memory
WebNLG	Web Natural Language Generation
XML	eXtensible Markup Language

1 Introducción

La inteligencia artificial generativa ha transformado la forma en que los usuarios acceden al conocimiento. Sin embargo, esta transformación no ha llegado de manera uniforme a todos los contextos. Las herramientas disponibles hoy en día son potentes, pero ninguna que sea simultáneamente especializada en su dominio, ejecutable en su propio dispositivo y accesible sin conexión a internet.

Los Grandes Modelos de Lenguaje (LLM), construidos sobre la arquitectura Transformer [1] y entrenados con cientos de miles de millones de parámetros sobre corpus de escala web [2], representan el estado del arte en coherencia semántica y amplitud de conocimiento [3]. No obstante, su uso implica una dependencia estructural de infraestructura en la nube que los excluye de escenarios que exigen privacidad, funcionamiento sin red o ejecución en hardware limitado [4]. Cada consulta viaja a centros de datos equipados con miles de GPU, donde es procesada y potencialmente almacenada. Presenta un coste económico y energético asociado que recae sobre el usuario en forma de suscripciones, límites por hora o tarifas por token generado. Si el servicio se interrumpe o el usuario carece de conexión, la productividad se detiene por completo.

Frente a este paradigma, los Modelos de Lenguaje Pequeños (SLM), que compensan su menor tamaño (típicamente entre 1.000 y 10.000 millones de parámetros) con datos de entrenamiento altamente filtrados o sintéticos [5], han abierto una vía alternativa. En particular, los denominados Mini-SLM, con menos de 3.000 millones de parámetros, son los únicos candidatos viables para el despliegue en dispositivos locales mediante Edge Computing [6]. Un modelo ejecutado en local elimina la dependencia de red, garantiza que ningún tercero pueda acceder a las consultas del usuario, reduce el impacto ambiental al prescindir de centros de datos remotos y convierte el coste marginal por consulta en cero.

Así mismo, seleccionar un modelo para despliegue local no es una decisión arbitraria. En la literatura actual coexisten tres paradigmas de generación de texto: las GAN, introducidas por Goodfellow et al. en 2014 [7]; los LLM; y los SLM y los Mini-

SLM. Cada paradigma presenta compromisos distintos que justifican una evaluación comparativa sistemática antes de tomar cualquier decisión de integración.

1.1 Motivación

Este Trabajo de Fin de Grado nace de la carencia descrita. Se propone para solventarla, el diseño de **TelecoSLM**, una aplicación orientada al apoyo al estudio en el ámbito de la ingeniería de telecomunicaciones, que ejecuta un modelo de lenguaje especializado directamente en el dispositivo del usuario, sin dependencia de servidores externos, sin transmisión de datos personales y sin coste por consulta. Sin embargo, integrar un modelo en una aplicación local no es una decisión que pueda tomarse arbitrariamente ya que exige una comparativa experimental rigurosa. Siendo necesaria la evaluación de calidad lingüística, eficiencia computacional y viabilidad de despliegue de diferentes paradigmas. Esa comparativa es el núcleo de la investigación que sustenta este trabajo.

La primera motivación que orienta ese proceso es la especialización de dominio. Los modelos generalistas disponibles en el mercado no garantizan el rigor conceptual que exigen disciplinas como las telecomunicaciones [8]. Su conocimiento paramétrico, entrenado sobre corpus genéricos, puede producir respuestas superficialmente válidas, pero técnicamente imprecisas (mezclar conceptos, omitir matices relevantes o generar datos inexistentes). En un contexto académico o profesional, una respuesta incorrecta presentada con aparente seguridad puede inducir a errores con consecuencias reales.

La segunda motivación es la privacidad de datos. La transmisión de consultas técnicas a servidores de terceros introduce riesgos de confidencialidad que resultan inaceptables tanto en entornos académicos como industriales [9]. El despliegue local elimina esa dependencia de raíz ya que el procesamiento ocurre en la memoria del propio dispositivo y ningún agente externo puede acceder a las consultas.

La tercera motivación es la independencia computacional. Un modelo ejecutado en local funciona en ausencia de red y en zonas sin cobertura, sin que la disponibilidad del sistema quede condicionada a la estabilidad de un servicio externo. Elimina además el coste recurrente asociado a los modelos en la nube y reduce el impacto ambiental derivado del uso de centros de datos remotos.

1.2 Objetivos

El objetivo principal de este Trabajo de Fin de Grado desplegar el modelo de mejor rendimiento en una aplicación móvil local de asistencia técnica. Para ello, se establecen los siguientes ocho (8) objetivos específicos:

(1) Analizar el estado del arte de las arquitecturas GAN textuales, LLM y SLM, así como las técnicas de evaluación automática de generación de lenguaje natural.
(2) Definir un marco de evaluación cuantitativo basado en métricas automáticas consolidadas (BLEU, ROUGE-L, METEOR, BERTScore) y métricas de eficiencia computacional (tiempo de inferencia y coste por token).
(3) Desarrollar un pipeline automatizado de inferencia reproducible para la ingesta, preprocesamiento y evaluación sistemática de los modelos sobre los *datasets* seleccionados.
(4) Ejecutar la primera comparativa experimental entre LLM, SLM y Mini-SLM sobre tres *datasets* estandarizados de generación *Data-to-Text*.
(5) Construir un corpus propio de aproximadamente 10.000 entradas especializadas en conceptos de ingeniería, aplicando criterios de calidad, diversidad temática y rigor terminológico.
(6) Implementar una GAN textual desde cero y ejecutar la segunda comparativa experimental enfrentándola a los modelos Mini-SLM sobre el corpus propio especializado en conceptos de ingeniería.
(7) Evaluar el impacto del *fine-tuning* sobre el corpus propio, cuantificando la mejora de rendimiento de las SLM base frente a su versión especializada.
(8) Diseñar y desarrollar la aplicación móvil con la Mini-SLM *fine-tuned* con mejores resultados.

1.3 Estructura

La presente memoria se ha organizado en cinco secciones: La *Sección 1* expone el contexto general del proyecto. La *Sección 2* aborda el estado del arte y los fundamentos teóricos. La *Sección 3* detalla la metodología y el desarrollo práctico del proyecto. La *Sección 4* presenta los resultados obtenidos y su correspondiente discusión. Finalmente, la *Sección 5* recoge las conclusiones del trabajo, las limitaciones técnicas encontradas durante el desarrollo y las líneas de trabajo futuro.

2 Estado del Arte

Para comprender la evolución técnica de estos sistemas, se presenta un marco teórico exhaustivo en el Anexo A con conceptos básicos de Inteligencia Artificial, Procesamiento del Lenguaje Natural (NLP) y *Transformers*. En la presente sección se explica las Redes Generativas Antagónicas (GAN), los modelos de lenguaje grandes y pequeños (LLM y SLM) y una introducción a las bases de datos utilizadas en el campo de la Inteligencia Artificial.

2.1 Introducción a Redes Generativas Antagónicas (GAN)

En el ámbito del aprendizaje automático, un modelo generativo se define como aquel (en términos probabilísticos) capaz de sintetizar nuevas observaciones estadísticamente coherentes con la distribución original [10].

2.1.1 El auge del modelo generativo

En los últimos años, el modelado generativo ha cobrado importancia, ya que refleja de forma más precisa el funcionamiento de la cognición humana, la cual actúa en gran medida como un sofisticado modelo generativo de la realidad [10].

2.1.2 Verosimilitud y parametrización

Los modelos generativos se basan en modelos paramétricos, es decir, dependen de pesos y configuraciones internas llamadas θ. Para medir la calidad del ajuste, se emplea la función verosimilitud (*Likelihood*), que cuantifica la probabilidad de observar los datos reales bajo la configuración actual del modelo.

Dado un conjunto de datos de entrenamiento $X = \{x_1, x_2, \dots, x_n\}$ y asumiendo que las muestras son independientes, la función de verosimilitud total $L(\theta; X)$ se define como el producto de las probabilidades de cada muestra individual [10]:

$$L(\theta; X) = \prod_{i=1}^{n} P(x_i; \theta) \quad (1)$$

El objetivo del entrenamiento es encontrar los parámetros $\hat{\theta}$ que la maximicen, conocido como el principio de Estimación de Máxima Verosimilitud (MLE, *Maximum*

Likelihood Estimation). Con ello, se intenta garantizar que estadísticamente los datos generados por la red se asemejen lo máximo posible a la realidad [10]:

$$\hat{\theta} = arg\max_{\theta} \sum_{i=1}^{n} \log P(x_i; \theta) \quad (2)$$

2.1.3 Aprendizaje de representaciones y espacio latente

El modelado generativo afronta modelar dependencias de alta dimensionalidad entre características (por ejemplo, píxeles adyacentes). La solución reside en el Aprendizaje de Representaciones (*Representation Learning*) [10]. En lugar de modelar directamente el espacio original, el modelo aprende una proyección sobre un espacio latente de menor dimensionalidad (*latent space* o *manifold*) y una función de mapeo inverso f que permite sintetizar datos realistas a partir de ese espacio comprimido (véase Figura 1).



Figura 1: Concepto de Aprendizaje de Representaciones (*Representation Learning*). Adaptado de Foster (2019) [10].

2.1.4 Redes Generativas Antagónicas (GAN)

En 2014, [10] Ian Goodfellow et al. presentaron el artículo fundacional *"Generative Adversarial Nets"* [7], introduciendo una arquitectura basada en la confrontación algorítmica entre dos redes neuronales. El modelado discriminativo estima P(Y|X), que representa la probabilidad condicional de una etiqueta Y dado un dato de entrada X. Por el contrario, el modelo generativo estima P(X) (la densidad de probabilidad intrínseca de observar la muestra X) o la distribución conjunta P(X|Y) (en escenarios con datos etiquetados) [11].

Para conceptualizar esta interacción de forma intuitiva, suele emplearse la analogía del falsificador de arte y el experto autenticador. Como se ilustra en la Figura 2, el Generador actúa como el falsificador que intenta crear obras sintéticas a partir de ruido aleatorio, mientras que el Discriminador asume el rol del experto, analizando las piezas para determinar su autenticidad. A medida que el Discriminador perfecciona su capacidad de detección, impone algorítmicamente al Generador a incrementar la similitud estadística de sus creaciones para seguir evadiendo el control [10].

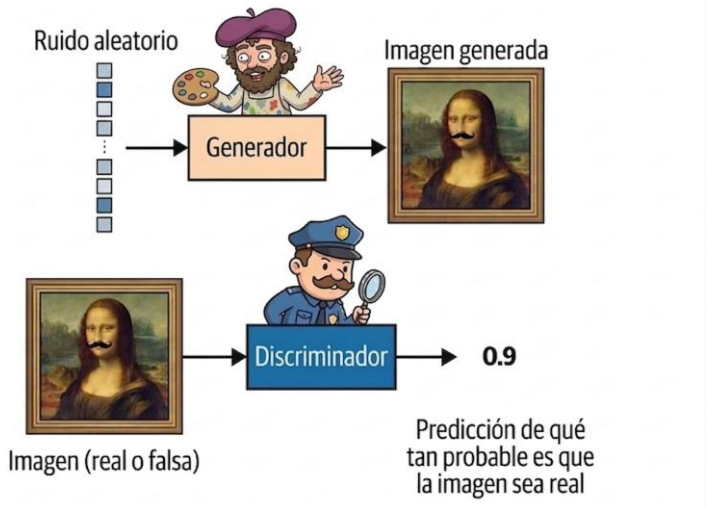


Figura 2: Analogía conceptual del funcionamiento de una red GAN. Adaptado de Foster (2019) [10].

Desde un punto de vista formal, el funcionamiento de las GAN se fundamenta en la teoría de juegos, específicamente en un juego *Mini-Max* de suma cero, en el que ambas redes compiten de forma simultánea [7]. La función de valor $\boldsymbol{V(D,G)}$ que rige este entrenamiento competitivo se expresa mediante la ecuación de la pérdida *Mini-Max* propuesta por Goodfellow et al. [7]:

$$\min_{\boldsymbol{G}} \max_{\boldsymbol{D}} \boldsymbol{V(D,G)} = \mathbb{E}_{\boldsymbol{x}\sim \boldsymbol{p_{data}}(\boldsymbol{x})}[\log \boldsymbol{D}(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim \boldsymbol{p_z}(\boldsymbol{z})}[\log(1 - \boldsymbol{D}(\boldsymbol{G}(\boldsymbol{z})))] \quad (3)$$

Dónde

- $\boldsymbol{x}$ es la muestra de datos reales proveniente de la verdadera distribución empírica de los datos, denotada como $\boldsymbol{p_{data}}(\boldsymbol{x})$.
- $\boldsymbol{z}$ es el vector de ruido aleatorio extraído de una distribución previa conocida, denotada como $\boldsymbol{p_z}(\boldsymbol{z})$ (típicamente una distribución normal o uniforme).

- $\boldsymbol{G(z)}$ es la red neuronal Generadora. Toma el vector de ruido latente $\boldsymbol{z}$ y lo mapea al espacio original de los datos, creando una muestra sintética.

- $\boldsymbol{D(x)}$ o $\boldsymbol{D(G(z))}$ es la red neuronal Discriminadora que opera como un clasificador binario y emite un valor escalar en [0, 1], representando la probabilidad de que la muestra de entrada sea real.

- $\mathbb{E}$ es el operador de esperanza matemática (el valor esperado) sobre la distribución de probabilidad correspondientes.

El Discriminador ($\boldsymbol{D}$) busca maximizar esta función asignando probabilidades cercanas a 1 a todas las muestras reales (maximizando $\mathbf{log}\ \boldsymbol{D(x)}$) y, probabilidades cercanas a 0 a las muestras sintéticas (maximizando $\mathbf{log(1 - D(G(z)))}$). El Generador ($\boldsymbol{G}$) busca minimizar la función, engañando al Discriminador para que clasifique sus muestras como reales. Su meta algorítmica es minimizar $\log(1 - \boldsymbol{D(G(z))})$.

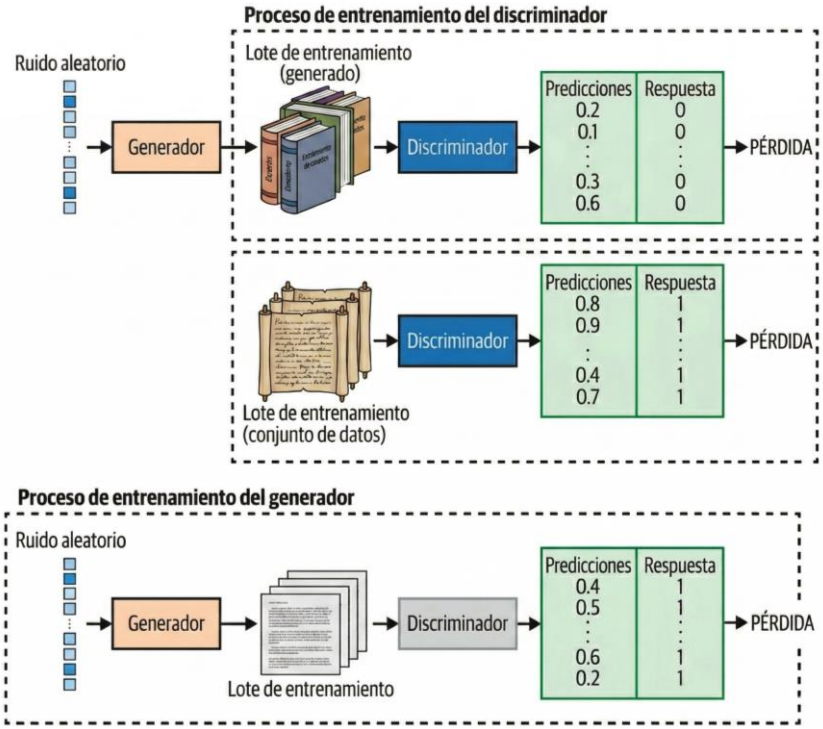


Figura 3: Procesos de entrenamiento del modelo Discriminador y Generador. Adaptado de Foster (2019) [10].

El entrenamiento converge teóricamente hacia un Equilibrio de Nash, en el que el Generador logra emular la distribución de datos reales (p_{data}) con tal precisión que alcanza una probabilidad de clasificación aleatoria $\boldsymbol{D(x)} = 0.5$ [7].

2.1.5 Aplicaciones prácticas y limitaciones de las GAN en NLP

Las aplicaciones principales de las arquitecturas GAN se han centrado mayoritariamente en visión computacional (síntesis de imágenes fotorrealistas [12], superresolución [13], transferencia de estilo [14]) [15].

A pesar de este éxito en el procesamiento de imágenes, la traslación de las arquitecturas antagónicas al Procesamiento del Lenguaje Natural (NLP) presentó un desafío matemático crítico conocido como el problema de la no diferenciabilidad, ya que las imágenes están compuestas por píxeles con valores continuos, lo que permite que el gradiente del error fluya suavemente durante la retro propagación. Por el contrario, el texto es estrictamente discreto, como se ilustra en la Figura 4. En la generación de texto, la última capa del modelo debe seleccionar una palabra (un *token* exacto) de un vocabulario fijo utilizando la operación matemática de maximización $argmax$, que es discontinua y carece de derivada matemática válida, bloqueando por completo el flujo del gradiente desde el Discriminador hacia el Generador, impidiendo la actualización de sus parámetros [16].

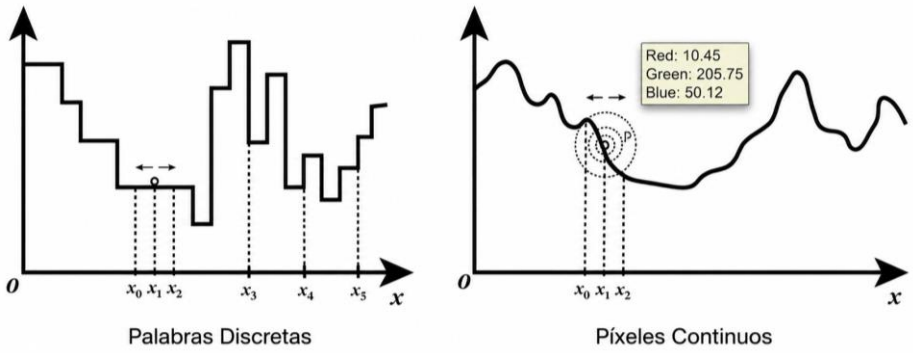


Figura 4: Comparativa espacial entre el dominio de datos continuos (imágenes) y el dominio de datos estrictamente discretos (texto). Adaptado de Bengio et al. (2019) [17].

El modelo SeqGAN (por sus siglas en inglés, *Sequence GAN)*, propuesto por Yu et al. (2017) [18], abordó este problema modelando la generación como un Proceso de Decisión de Markov (MDP) [19] y utilizando el algoritmo REINFORCE para actualizar el Generador mediante señales de recompensa del Discriminador [18].

De forma paralela, las arquitecturas de Redes Neuronales Recurrentes (RNN) y de Memoria a Corto y Largo Plazo (LSTM, *Long Short-Term Memory*) también presentaron limitaciones estructurales significativas puesto que su naturaleza

estrictamente secuencial impedía la paralelización del entrenamiento en *hardware* moderno y, al igual que las GAN, tendían a olvidar información clave en textos muy largos [10].

Ante esta problemática, se desarrolló un nuevo paradigma, los Grandes Modelos de Lenguaje (LLM) y, posteriormente, sus variantes eficientes (SLM), ambos cimentados sobre la arquitectura *Transformer* [10].

2.2 Introducción a LLM y SLM

La evolución del procesamiento del lenguaje natural experimentó un punto de inflexión al demostrarse empíricamente que el rendimiento y la capacidad de generalización de los modelos basados en la arquitectura *Transformer* mejoran al incrementar el número de parámetros y el volumen del corpus de entrenamiento [3]. Esta correlación, llamada *Scaling Laws*, impulsó el desarrollo de los Grandes Modelos de Lenguaje (LLM, *Large Language Models*) [5], aunque con elevados costes computacionales y dependencia estricta de infraestructuras *cloud*.

Como respuesta, surgieron los Modelos de Lenguaje Pequeños (SLM, *Small Language Models*). En la literatura científica, el término "pequeño" no denota una arquitectura simplificada, sino una escala paramétrica contenida (habitualmente en el rango de los 1.000 a 10.000 millones de parámetros).

2.2.1 Large Language Models

Los LLM se consideran sistemas generalistas de propósito amplio en vez de simplemente herramientas de texto, además, exhiben capacidades emergentes que les permiten simular procesos de razonamiento, generar código funcional o resolver problemas lógicos complejos sin haber sido programados explícitamente para dichas tareas [20].

Sin embargo, su principal limitación es la infraestructura requerida, puesto que necesitan *clústeres* de servidores equipados con unidades de procesamiento gráfico especializado (GPU o TPU) con requisitos extremos de memoria de video (VRAM) para alojar los pesos del modelo durante la inferencia. Esta dependencia de infraestructura implica un consumo energético elevado, una mayor latencia y una subordinación casi

total a los servicios en la nube de las grandes corporaciones tecnológicas, dificultando su despliegue en entornos locales o con restricciones de privacidad [21].

La Tabla 1 recoge los modelos más relevantes del estado del arte. De su análisis se extraen dos tendencias estructurales: la transición hacia la multimodalidad (procesamiento integrado de texto, imagen y audio) y la adopción generalizada de arquitecturas Mezcla de Expertos (Moe, *Mixture of Experts*) [22]. Los modelos MoE subdividen la red en subredes especializadas (expertos) y emplean un módulo enrutador que selecciona dinámicamente el subconjunto de expertos más adecuado para cada *token*.

Tabla 1: Comparativa técnica de modelos LLM. Elaboración propia.

Modelo	Parámetros	Input	Output	Arquitectura	Empresa Desarrolladora	Ref.
GPT-4o	No revelado	Multimodal	Multimodal	Transformer (MoE▲)	OpenAI (EE.UU., 2024)	[23]
Gemini 1.5 Pro	No revelado	Multimodal	Multimodal	Transformer (MoE▲)	Google DeepMind (EE.UU., 2024)	[24]
Claude 3.5 Sonnet	No revelado	Multimodal	Texto	Transformer (Decoder-only)	Anthropic (EE.UU., 2024)	[25]
Deepseek-V3	671 B*	Texto	Texto	Transformer (MoE▲)	DeepSeek AI (China, 2024)	[26]
Llama 3.3	70 B*	Texto	Texto	Transformer (Decoder-only)	Meta AI (EE.UU., 2024)	[27]
Mixtral 8x22B	141 B*	Texto	Texto	Transformer (MoE▲)	Mistral AI (Francia, 2024)	[28]
Qwen 2.5	72 B*	Texto	Texto	Transformer (Decoder-only)	Alibaba Cloud (China, 2024)	[29]
Grok-2	No revelado	Multimodal	Multimodal	Transformer (MoE▲)	xAI (EE.UU., 2024)	[30]
Gemma 4	26 B* y 31 B*	Multimodal	Texto	Transformer (MoE▲ y Decoder-only)	Google Deepmind (EE.UU., 2026)	[31]

Los modelos LLM escogidos son **DeepSeek-V3 Chat** [26], **Llama 3.3 70B** [27] y **Qwen 2.5 72B** [29]. La selección inicial contemplaba modelos como GPT-4 [23] (*OpenAI*) o Gemini [24] (*Google*), pero se descartaron al tratarse de modelos de código cerrado. Los tres modelos elegidos, son de código abierto y disponen de documentación técnica pública, lo que garantiza la reproducibilidad del estudio. Además, al pertenecer a tres organizaciones distintas (*DeepSeek*, *Meta* y *Alibaba*), se evita un sesgo de proveedor y se obtiene una muestra representativa del estado actual de los LLM.

▲ MoE = *Mixture of Experts*

* B = *Billions (miles de millones de parámetros)*

2.2.2 Small Language Models

La Tabla 2 muestra los SLM de código abierto más destacados del ecosistema actual. Se observa una estandarización de la arquitectura *Transformer* densa en todo el segmento con familias de modelos con distintos escalones paramétricos. Cabe destacar que las organizaciones tecnológicas (como *Google* con *Gemini* [24] o *Meta* con *Llama* [27]) paralelamente a sus modelos insignia, presentan versiones paramétricamente reducidas (SLM) que heredan la arquitectura y el conocimiento destilado de sus homólogos de mayor escala.

Tabla 2: Comparativa técnica de modelos SLM. Elaboración propia.

Modelo	Parámetros	Input	Output	Arquitectura	Empresa Desarrolladora	Ref.
SmolLM2	1.7 B*	Texto	Texto	Transformer (Decoder-only)	Hugging Face (EE.UU., 2024)	[32]
OpenELM	1.1 B* y 3 B*	Texto	Texto	Transformer (Decoder-only)	Apple (EE.UU., 2024)	[33]
Llama 3.2	1 B* y 3 B*	Texto	Texto	Transformer (Decoder-only)	Meta AI (EE.UU., 2024)	[27]
Phi-3 Mini	3.8 B*	Texto	Texto	Transformer (Decoder-only)	Microsoft (EE.UU., 2024)	[34]
Phi-4-Mini	3.8 B*	Texto	Texto	Transformer (Decoder-only)	Microsoft (EE.UU., 2025)	[35]
OLMo	1 B* y 7 B*	Texto	Texto	Transformer (Decoder-only)	Allen Institute for AI (EE.UU., 2024)	[36]
Qwen 2.5	0.5 B*, 1.5 B*, 3 B* y 7 B*	Texto	Texto	Transformer (Decoder-only)	Alibaba Cloud (China, 2024)	[29]
Qwen 3	0.6 B* y 1.7 B*	Multimodal	Texto	Transformer (Decoder-only)	Alibaba Cloud (China, 2025)	[37]
Aya 23	8 B*	Texto	Texto	Transformer (Decoder-only)	Cohere (Canadá, 2024)	[38]
Ministral	3 B* y 8 B*	Texto	Texto	Transformer (Decoder-only)	Mistral AI (Francia, 2024)	[39]
Gemma 2	2 B*, 9 B* y 27 B*	Multimodal	Texto	Transformer (Decoder-only)	Google DeepMind (EE.UU., 2026)	[31]
Gemma 3	1 B* y 4 B*	Multimodal	Texto	Transformer (Decoder-only)	Google DeepMind (EE.UU., 2025)	[40]

Mientras que un SLM estándar opera cómodamente en la franja de los 3B a los 10B de parámetros (acercándose a la frontera de los LLM pequeños), los Mini-SLM se restringen estrictamente al rango inferior, habitualmente entre 0.5 B y 3 B de parámetros. Esta reducción sacrifica deliberadamente el volumen de conocimiento general para maximizar la velocidad de inferencia y minimizar el consumo de memoria.

Los SLM seleccionados son **Gemma 2 9B**, **Llama 3.2 3B** y **Qwen 2.5 7B**. Al igual que con la selección de LLM, esta elección se debe a que, son de código abierto y

disponen de documentación técnica pública, lo que garantiza la reproducibilidad del estudio. Además, al pertenecer a tres organizaciones distintas (*Google*, *Meta* y *Alibaba*), se evita un sesgo de proveedor y se obtiene una muestra representativa del estado actual de los SLM.

Lo mismo ocurre con los Mini-SLM, que son los mismos modelos que los seleccionados para la categoría SLM, pero con un número de parámetros bastante reducido, garantizando así, la reproducibilidad del estudio sin cambiar de modelos, solo de tamaño. Los modelos elegidos son **Gemma 3 1B**, **Llama 3.2 1B** y **Qwen 3 1.7B**.

2.3 Naturaleza de la información

Datos estructurados y no estructurados

Aquellos que se organizan siguiendo un esquema lógico, tabular o jerárquico estricto. Residen típicamente en sistemas de gestión de bases de datos relacionales (SQL) donde cada dato tiene asignado un tipo, una fila, una columna definidos [41]. Por su parte, los datos semiestructurados no se ajustan a modelos relacionales rígidos, pero contienen etiquetas y marcadores que separan los elementos semánticos. Los formatos más representativos en el desarrollo de software son los archivos JSON *(JavaScript Object Notation*) y los archivos tabulares CSV o Excel.

Datos no estructurados

Aquellos que carecen de un formato predefinido o un modelo de datos rígido. Representan la inmensa mayoría de la información generada en entornos digitales, incluyendo documentos de texto libre, correos electrónicos o artículos periodísticos [42]. Para procesar este tipo de datos, el modelo de lenguaje debe inferir el contexto basándose puramente en la semántica. Si bien los LLM destacan en este ámbito gracias a su amplia memoria paramétrica general, el procesamiento de texto libre es incrementa la propensión a las alucinaciones [43].

2.3.1 Selección de datasets para la experimentación

Para llevar a cabo una evaluación rigurosa y reproducible, se han seleccionado conjuntos de datos ampliamente utilizados en la literatura (véase Tabla 3).

Tabla 3: Comparativa de datos estructurados y no estructurados para la generación de texto. Elaboración propia.

Base de datos	Repositorio	Formato	Evaluación	Breve Descripción	Ref.
E2E NLG Challenge	Hugging Face (EE.UU., 2017)	Estructurado (JSON)	Generación de descripciones	Descripciones en lenguaje natural de restaurantes a partir de pares atributo-valor	[44]
WebNLG	Hugging Face (Francia / Reino Unido, 2017)	Estructurado (Tripletas RDF)	Traducción de datos a texto	Generación de oraciones a partir de tripletas lógicas (sujeto-predicado-objeto).	[45]
ToTTo	Hugging Face (EE.UU., 2020)	Estructurado (Tablas Web / JSON)	Generación de texto desde tablas	Generación de descripciones precisas a partir de celdas resaltadas de Wikipedia.	[46]
DART	Hugging Face (EE.UU., 2020)	Estructurado (Tripletas / JSON)	Data-to-Text	Generación de texto a partir de representaciones en forma de grafos.	[47]
WikiBio	Hugging Face / TensorFlow (Suiza / EE.UU., 2016)	Estructurado (JSON / Tabular)	Generación desde *infoboxes*	Redacción de biografías cortas a partir de las fichas técnicas (*infoboxes*) de Wikipedia.	[48]
KELM	Google Research (EE.UU., 2021)	Estructurado (Grafos)	Traducción de datos a texto	Conversión del grafo de conocimiento de Wikidata a oraciones sintéticas en lenguaje natural	[49]
SQuAD 2.0	Hugging Face / Stanford (EE.UU., 2018)	No estructurado (JSON)	Comprensión lectora (QA)	Preguntas y respuestas sobre texto libre con preguntas sin respuesta posible	[50]
CNN / DailyMail	Hugging Face (Reino Unido, 2015)	No estructurado (Texto plano)	Resumen de documentos	Generación de resúmenes abstractivos de artículos de noticias.	[51]
PubMed Summarization	Hugging Face (EE.UU., 2018)	No estructurado (Texto plano)	Resumen de documentos médicos	Generación de resúmenes abstractivos a partir de artículos científicos clínicos y biomédicos	[52]

Se identifican tres grupos funcionales diferenciados: (i) los *datasets* puramente estructurados orientados a la tarea *Data-to-Text*, como WebNLG [45], ToTTo [46] o DART [47],(ii) los *datasets* de dominio cerrado, como E2E NLG (restauración) [44] o WikiBio (biografías) [48]; y (iii) los repositorios de texto no estructurado, como CNN/DailyMail [51] o PubMed *Summarization* [52].

Para el desarrollo experimental de este Trabajo de Fin de Grado se han seleccionado tres *datasets* (resaltados en verde en la Tabla 3): **WebNLG** [45], **ToTTo** [46], y **KELM** [49]. Dado que el objetivo de este trabajo es evaluar la capacidad de los modelos para generar texto en lenguaje natural, se han descartado deliberadamente los *benchmarks* orientados a tareas de clasificación o verificación.

3 MATERIAL Y MÉTODOS

Esta sección presenta la propuesta técnica, describiendo las especificaciones del entorno de programación, del hardware utilizado y cada una de las fases que componen el proceso de desarrollo. Cada fase se presenta de forma secuencial, reflejando el flujo natural del desarrollo del proyecto y las decisiones técnicas adoptadas en cada momento.

Especificaciones del hardware utlizado (CPU/GPU)

Todo el desarrollo local (código, ejecución y la gestión de los *datasets*, la evaluación y la prueba de la aplicación en *Windows*) se ha realizado en un portátil Lenovo (Intel Core i5-1135G7, 8 GB RAM DDR4, SSD NVMe 512 GB, Intel Iris Xe Graphics en Windows 11 de 64 bits) sin GPU dedicada. Dado que los LLM y SLM requieren hardware con una gran cantidad de VRAM, la inferencia de los modelos se ha delegado a *Openrouter*[1]. Se trata de un servicio de enrutamiento que agrega proveedores como *DeepSeek*, *Meta*, *Google* y *Albibaba* bajo un único *endpoint* con infraestructura basada en GPU NVIDIA A100, H100 y H200.

Para las fases de *fine-tuning* y entrenamiento de la GAN, se ha utilizado un servidor con GPU NVIDIA GeForce RTX 4070 (12 GB VRAM) cedido por la Universidad CEU San Pablo.

Las pruebas de la aplicación en *Android* se han realizado en un Samsung Galaxy A70 (Qualcomm Snapdragon 675, 6 GB RAM, 128 GB de almacenamiento, GPU Adreno 612, Android 11), un terminal de gama media sin aceleración dedicada.

Entorno de programación

El desarrollo se ha realizado en *Visual Studio Code* con *Python* 3.11.9 dentro de un entorno virtual (venv). La Tabla 4 recoge los paquetes principales por área principal (inferencias, métricas, datos, visualización, NLP y procesamiento de texto).

[1] https://openrouter.ai

Tabla 4: Principales paquetes y dependencias del entorno Python.

Paquete	Versión	Uso
Inferencia y modelos		
openai	2.26.0	Cliente API para OpenRouter
huggingface_hub	1.7.1	Descarga y gestión de modelos desde HuggingFace Hub
transformers	5.7.0	Tokenizadores y utilidades de modelos HuggingFace
torch	2.11.0	Backend de cómputo para BERTScore y transformers
Métricas y evaluación		
nltk	3.9.1	Cálculo de METEOR y utilidades de NLP
Datos y visualización		
datasets	2.21.0	Carga de WebNLG, ToTTo y KELM desde HuggingFace Hub
pandas	3.0.1	Manipulación de DataFrames y CSVs de resultados
matplotlib	3.10.1	Generación de gráficas y boxplots
seaborn	0.13.2	Visualización estadística sobre matplotlib
numpy	2.4.3	Operaciones numéricas
scikit-learn	1.8.0	Métricas adicionales y utilidades estadísticas
scipy	1.17.1	Computación científica (backend de scikit-learn/seaborn/tqdm)
NLP y procesamiento de texto		
spacy	3.8.5	Procesamiento lingüístico y modelos de idioma

Todo el código desarrollado se encuentra alojado en un repositorio GitHub privado, disponible bajo petición: [ArturoFierroPardo/TFG]

3.1 Diseño experimental

El proyecto se ha organizado en 6 fases (véase Figura 5). A diferencia de un *pipeline* lineal, varias fases se ejecutan en paralelo y convergen en puntos concretos del flujo de trabajo.

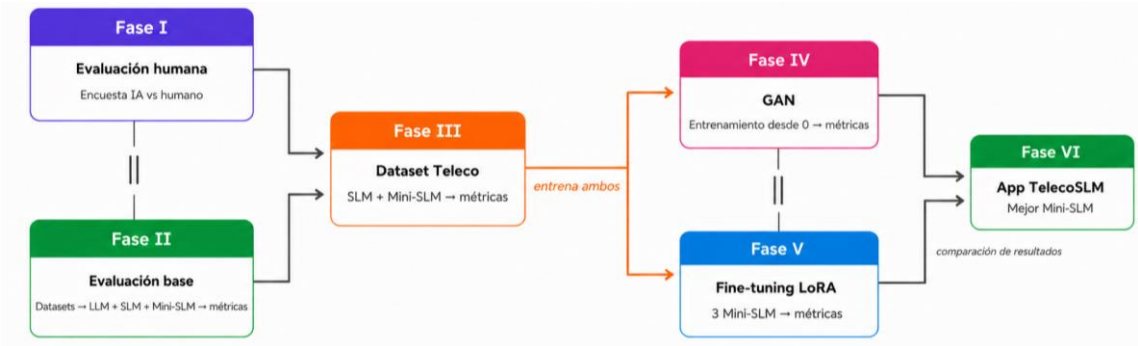


Figura 5: Flujo de trabajo del proyecto. Fuente propia.

La Fase I consiste en una evaluación humana mediante encuesta en dónde los participantes debían distinguir 20 textos (10 generados por IA y 10 por humanos). La finalidad de la encuesta es doble: (i) valorar el interés en usar una herramienta de IA educativa y (ii) medir si la calidad del texto generado es indistinguible de una explicación escrita por un humano.

La Fase II evalúa tres categorías de modelos (LLM, SLM y Mini-SLM) sobre los tres *datasets* genéricos (*WebNLG* [45], *ToTTo* [46] y *KELM* [49]), midiendo la degradación del rendimiento conforme se reduce el tamaño del modelo. La Fase III introduce un *dataset* propio y especifico de telecomunicaciones y repite la evaluación con SLM y Mini-SLM. Este *dataset* alimenta en paralelo la Fase IV (entrenamiento de una GAN textual desde cero) y la Fase V (*fine-tuning* con LoRA de los tres Mini-SLM), permitiendo comparar directamente ambos enfoques con el modelo base sin ajustar. Finalmente, la Fase VI se cuantiza el mejor modelo y se despliega en la aplicación multiplataforma.

3.2 Validación de necesidad

Con el objetivo de validar la necesidad y el interés del sistema desarrollado, se diseñó un cuestionario para analizar dos dimensiones complementarias: evaluación de la naturalidad de los textos generados por sistemas de IA (con el fin de determinar si los usuarios son capaces de distinguirlos de textos de autoría humana) y evaluación del interés del público objetivo (principalmente estudiantes universitarios y de posgrado) en utilizar herramientas de IA especializadas como apoyo al aprendizaje.

El cuestionario se compone de 20 oraciones de diversas temáticas. Para cada oración, el participante indicó su nivel de confianza en la autoría percibida mediante una escala Likert de 5 puntos (1 = ninguna confianza, 5 = total confianza). Esta elección permite que el participante identifique la posible fuente (IA vs. Humano) y la certeza subjetiva. Además, se recogió variables sociodemográficas (nivel de estudios y rango de edad) y el uso y frecuencia de herramientas de IA generativa, variables que permiten analizar si el perfil del participante influye en su capacidad de detección y en su disposición hacia este tipo de herramientas. La pregunta de opinión final sobre la disposición a utilizar una herramienta de IA especializada como apoyo al aprendizaje

constituye el indicador central de demanda potencial del sistema y fundamenta directamente la decisión del desarrollo.

Procedimiento de recogida de datos

El cuestionario se implementó mediante *Google Forms* y se distribuyó durante una semana por difusión digital (correo electrónico y redes sociales). La participación fue voluntaria, sin incentivo alguno. Se obtuvieron un total de 220 respuestas válidas, lo que proporciona una base suficiente para extraer conclusiones sobre el interés del público objetivo.

Consideraciones éticas y de privacidad

La investigación se llevó a cabo en España en el contexto de actividades docentes universitarias ordinarias. La participación en el cuestionario fue voluntaria y anónima. No se recogieron datos personales identificables ni datos sensibles. De acuerdo con el marco legal español, la aprobación obligatoria por parte de un comité de ética se requiere principalmente para investigaciones biomédicas que impliquen intervenciones médicas, muestras biológicas o experimentación clínica (Ley 14/2007 de Investigación Biomédica). El cuestionario constituye una investigación educativa y no se encuentra dentro del ámbito de aplicación de dicha legislación. Todo el tratamiento de datos cumplió con el Reglamento General de Protección de Datos (UE) 2016/679 y con la Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales. Por estas razones, de acuerdo con el marco legal español aplicable, no fue necesaria la aprobación formal de un comité de ética para este tipo de investigación educativa.

3.3 Datasets genéricos y preprocesamiento

Los tres *datasets* empleados en la Fase II (Evaluación base) se eligieron porque cubren tres formatos de entrada distintos (tripletas RDF, tablas con celdas resaltadas y tripletas de *Wikidata*), lo que permite evaluar la capacidad de abstracción y generalización de los modelos ante estructuras de datos diferentes.



Figura 6: Ejemplos de entrada y referencia humana de cada *dataset* genérico empleado. Fuente propia.

Dataset WebNLG

WebNLG [45] es un *dataset* de referencia en la tarea *Data-to-Text* que forma parte de las campañas de evaluación *WebNLG Challenge*. Los datos de entrada son conjuntos de tripletas RDF (sujeto, predicado, objeto) extraídas de *Dbpedia*, cuenta con 12.876 filas y la referencia es un texto en inglés que verbaliza esas tripletas de forma fluida (véase Figura 6). Se ha utilizado la versión *18ootstr_v2.1* del *dataset* [45], que incluye 16 categorías temáticas. No se ha aplicado ningún filtrado adicional, pero, se ha procesado el *dataset* completo tal como lo proporciona la librería de *datasets* de *HuggingFace* [53].

Dataset ToTTo

ToTTo (*Table-to-Text with Open-domain tables*) [46] es un dataset de *Google Research* donde la entrada es una tabla de Wikipedia con una o varias celdas resaltadas (*highlighted cells*) con 120.761 filas de datos, y la referencia es una frase en inglés que describe la información contenida en esas celdas. La entrada es una estructura tabular con cabeceras de fila y columna (véase Figura 6), lo que obliga al modelo a interpretar relaciones posicionales dentro de la tabla. Se ha utilizado el *dataset* completo sin filtrado. El preprocesamiento consiste en serializar la tabla y las celdas resaltadas en un formato textual que el modelo pueda recibir como *prompt*.

Dataset KELM

KELM (*Knowledge-Enhanced Language Model*) [49] es un *dataset* de *Google Research* que contiene pares de tripletas de *Wikidata* y verbalizaciones en inglés generadas automáticamente (véase Figura 6). El *dataset* original es muy grande y temáticamente heterogéneo, por lo que se ha aplicado un filtrado en dos fases para obtener el subconjunto del dominio STEM (ciencia, tecnología, ingeniería o matemáticas). El resultado es un subconjunto de aproximadamente 59.751 filas, frente a los 15 millones de filas del *dataset* original.

3.4 Modelos evaluados y Pipeline de inferencia

A continuación, se detallan los modelos evaluados en la Fase II (Evaluación base) pertenecientes a las cuatro familias (Llama, Qwen, Gemma y Deepseek) y se explica el proceso de la inferencia.

Modelos LLM

DeepSeek Chat V3 [26] es un modelo de 685.000 millones de parámetros con arquitectura *Mixture of Experts* (MoE), de los cuales solo 37.000 millones se activan por token. Fue entrenado por DeepSeek (China) sobre 2.048 GPUs NVIDIA H800 usando precisión mixta. Se ha incluido como representante del extremo superior de escala, para establecer un techo de rendimiento frente al que medir la capacidad de los modelos más pequeños.

Llama 3.3 70B Instruct [27] es el modelo de Meta con 70.000 millones de parámetros, entrenado con más de 15 billones de tokens. Utiliza una arquitectura *decoder-only* con GQA y una ventana de contexto de 128 mil tokens. Se seleccionó por ser el modelo abierto de referencia en la comunidad NLP y por su rendimiento competitivo frente a modelos cerrados de tamaño similar. Se utiliza como representante de la familia GPT, ya que su código fuente no es público.

Qwen 2.5 72B Instruct [29] es el modelo grande de la familia Qwen, con 72.000 millones de parámetros y preentrenado con 18 billones de *tokens*. Comparte muchas decisiones de diseño con DeepSeek (SwiGLU, RMSNorm, GQA) y se incluye para tener un segundo punto de comparación en la escala 70B.

Modelos SLM

Gemma 2 9B IT [31] es el modelo mediano de *Google*, con 9.000 millones de parámetros, entrenado sobre 8 billones de tokens. Es una versión destilada de la familia de Gemini y se seleccionó como el SLM de mayor tamaño del estudio

Llama 3.2 3B Instruct [27] es la variante compacta de la familia Llama, con 3.000 millones de parámetros. Mantiene la misma arquitectura que su hermano de 70B, pero con menos capas y dimensión reducida. Se incluyó para medir cuanto rendimiento se pierde al bajar de un SLM de 9B a 3B.

Qwen 2.5 7B Instruct [29] es la versión optimizada de su hermano de 72B, cuenta con 7.000 millones de parámetros. Se ha seleccionado como punto intermedio entre los 9B de Gemma y los 3B de Llama.

Modelos Mini-SLM

Gemma 3 1B [40] es la versión más compacta de la familia Gemma, con aproximadamente 1.000 millones de parámetros.

Llama 1B [27] es la variante mínima de la familia Llama, también con alrededor de los 1.000 millones de parámetros.

Qwen 2 1.7B [29] es el modelo más pequeño de la última generación de Qwen, con 1.700 millones de parámetros. Se incluye como el Mini-SLM de mayor tamaño para ver si esos 700 millones de parámetros extra frente a los modelos de 1B suponen una diferencia notable.

3.4.1 Pipeline de inferencia

Se observa en la Figura 7 cómo cada *pipeline* se comunica con la API de *OpenRouter* a través de la librería de *openai* de *Python*, configurando la *base_url* hacia el *endpoint* correspondiente. Cada modelo recibe el mismo *prompt*, siendo este un mensaje de sistema con la descripción de la tarea y un mensaje de usuario con los datos estructurados. Las peticiones van en paralelo mediante 20 hilos. Al terminar se generan 27 pares de archivos CSV (9 modelos x 3 *datasets*). El primer CSV son los textos generados y el segundo son las métricas calculadas fila a fila.

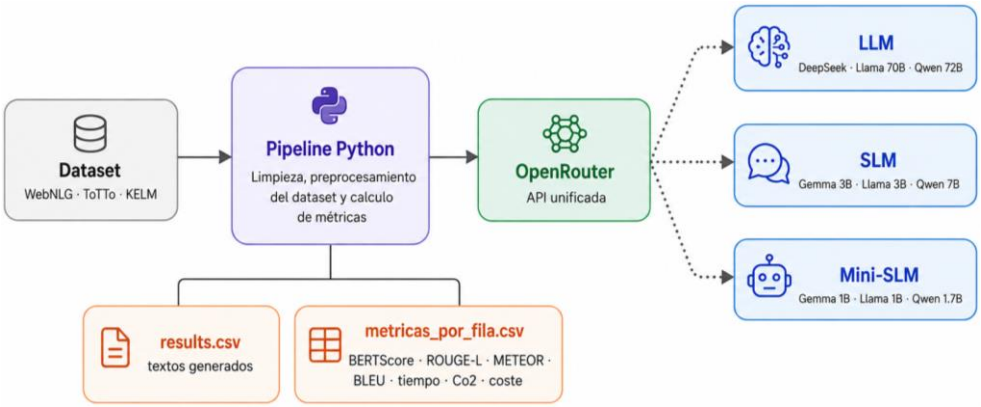


Figura 7: Arquitectura general del *pipeline* de generación y evaluación de los modelos LLM, SLM y Mini-SLM. Fuente propia.

3.5 Implementación de la GAN textual

La GAN desarrollada en la Fase IV del proyecto sigue una arquitectura *Conditional SeqGAN* [18] con *Gumbel-Softmax* [54] (τ = 0.8) para resolver la no diferenciabilidad de las salidas discretas (texto).

El problema principal de aplicar una GAN a generación de texto es la salida discreta (*tokens*), lo que impide propagar gradientes desde el Discriminador al Generador durante el entrenamiento. Para resolver este problema, se emplea el truco de *Gumbel-Softmax* [54], que sustituye el muestreo discreto de *tokens* por una aproximación continua diferenciable. Cuando el Generador intenta predecir el siguiente *token*, primero calcula unas puntuaciones para cada palabra del vocabulario, conocidas como *logits* (z). Para transformar estas puntuaciones en probabilidades suaves y manejables (y_i), se aplica [54]

$$y_i = \frac{\exp\left((z_i + g_i)/\tau\right)}{\sum_{j=1}^{V} \exp\left((z_j + g_j)/\tau\right)}, \quad (4)$$

Dónde g_i representa el ruido aleatorio. Este ruido es vital para que el modelo no sea totalmente determinista y le permita probar diferentes rutas al generar el texto. Por otro lado, τ representa la temperatura del sistema, y actúa como regulador directo de la confianza del modelo (véase Figura 8). Si bajamos la temperatura acercándola a cero, el sistema se vuelve muy "tajante", estando la probabilidad de la próxima palabra generada concentrada en un solo *token*. Por el contrario, si se sube la temperatura, la distribución se aplana, repartiendo las opciones de forma mucho más uniforme entre varias palabras.



Figura 8: Efecto del parámetro de temperatura en la distribución *Gumbel-Softmax*. Adaptado de Jang et al. (2017) [54].

Se ha fijado en una temperatura de 0.8, permitiendo que el modelo pueda aprender y propagar los errores, pero es suficientemente estricto como para garantizar que la selección de palabras es precisa y coherente.

Generador

El Generador ejecuta el proceso de mapeo inverso, proyecta hacia el espacio de alta dimensionalidad de las respuestas en lenguaje natural para sintetizar la salida [55]. Esta proyección se ha implementado mediante una arquitectura *Transformer encoder-decoder* (430M parámetros, 14 + 14 capas, dimensión 1024) con *teacher forcing* y *weight tying,* lo que permite que el modelo utilice la misma representación semántica para entender las palabras de entrada y para escribirlas a la salida. (véase Figura 9).

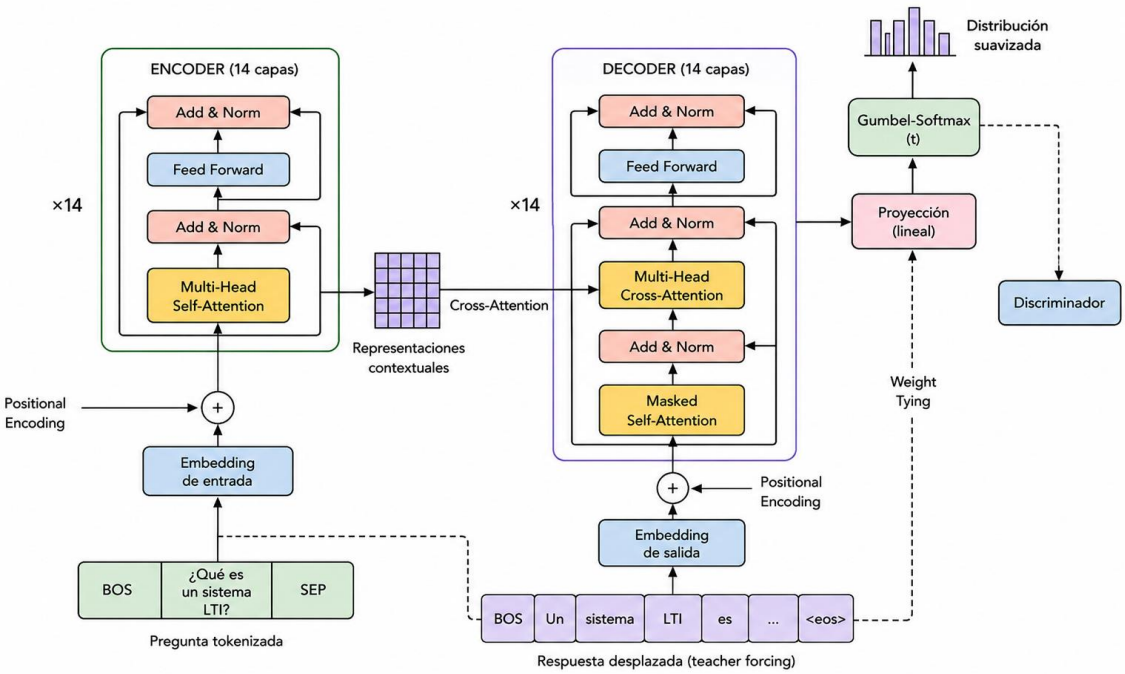


Figura 9: Arquitectura y flujo del modelo Generador. Fuente propia.

La entrada al *encoder* es la pregunta, tokenizada y precedida por un *token* BOS (inicio de secuencia) y seguida de un *token* SEP (separador). Este, recibe la pregunta y

genera un mapa de representaciones contextuales que el *decoder* utilizará mediante mecanismos de *cross-attention*. Durante el entrenamiento, el *decoder* emplea *teacher forcing* (alimentándose de la respuesta desplazada un paso) para acelerar el proceso y la convergencia. A diferencia de las arquitectura sgenerativas clñasicas que aplican *Batch Normalization* [56] para estabilizar el entrenamiento, el *Transformer* integra *Layer Normalization* [57] en cada subcapa, lo que elimina la dependencia del tamaño del *batch* y estabiliza las distribuciones de activación. Finalmente, para permitir que el Discriminador evalúe la salida, la capa de proyección pasa por *Gumbel-Softmax* para producir la distribución suavizada. La Tabla 5 recoge los hiperparámetros seleccionados para el Generador.

Tabla 5: Hiperparámetros del Generador de la GAN

Parámetro	Valor
Parámetros totales	430M
Dimensión del modelo	1024
Número de capas (encoder + decoder)	14 + 14
Cabezas de atención	16
Dimensión feedforward	4096
Dropout	0.1
Longitud máxima de secuencia	256 tokens
Inicialización de embeddings	$\mathcal{N}(0,1/\sqrt{d_model})$
Inicialización del resto de pesos	Xavier uniform

La elección de la dimensión del modelo con *feedforward* sigue la convención habitual de que la capa *feedforward* sea 4 veces la dimensión del modelo [1]. Las 14 capas tanto en *encoder* como en *decoder* y la dimensión del modelo de 1024, sitúan al Generador en un rango de capacidad comparable al de modelos como GPT-2 Medium [58], aunque con una arquitectura *encoder-decoder* en vez de *decoder-only*.

Discriminador

El Discriminador se ha diseñado con una arquitectura *Transformer encoder-only* (véase Figura 10) que recibe la concatenación de la pregunta y la respuesta (real o generada) y produce una puntuación escalar que indica la probabilidad de que la respuesta sea real. A diferencia de las arquitecturas convolucionales profundas, el mecanismo de atención del *Transformer* mitiga el problema del desvanecimiento del gradiente [59] al mantener gradientes estables a lo largo de toda la secuencia. Para llevar a cabo esta clasificación, se debe transformar la secuencia de entrada en un vector paramétrico. Este proceso se realiza mediante una operación llamada *masked mean pooling* a la salida del *encoder*. La representación vectorial resultante pasa directamente por un clasificador compuesto por dos capas lineales, el cual incorpora una función de activación GELU y regularización mediante *Dropout* [60] (desactiva aleatoriamente una fracción de neuronas durante el entrenamiento) para garantizar la estabilidad y prevenir el sobreajuste.

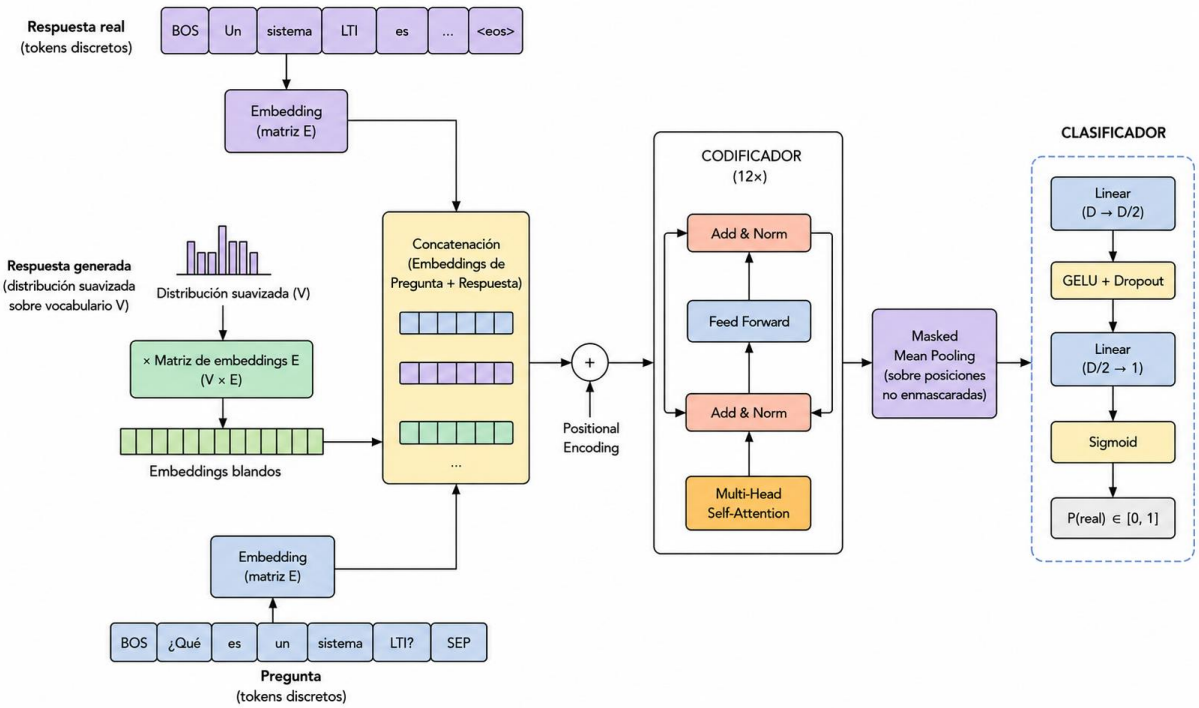


Figura 10: Arquitectura del modelo Discriminador. Fuente propia.

Cuando la respuesta es real, el Discriminador trabaja de forma estándar utilizando la matriz de *embeddings* de los *tokens* discretos. Sin embargo, cuando la respuesta es generada, el modelo no recibe *tokens* fijos, sino que recibe la distribución de probabilidad generada por la capa *Gumbel-Softmax* del Generador. Al multiplicar esta distribución por su matriz de *embeddings*, el Discriminador logra operar con representaciones continuas

(*soft embeddings*). Esto permite que el gradiente de error fluya de vuelta para optimizar los pesos del Generador. Los hiperparámetros empleados para el Discriminador se pueden observar en la Tabla 6.

La asimetría entre Generador y Discriminador (430M frente a 100M parámetros) es deliberada. Un Discriminador demasiado potente aprende a distinguir textos reales de generados con facilidad, lo que colapsa el entrenamiento. Un Discriminador más pequeño fuerza al Generador a mejorar de forma más gradual [61].

Tabla 6: Hiperparámetros del discriminador de la GAN

Parámetro	Valor
Parámetros totales	100 millones
Dimensión del modelo	768
Número de capas	12
Cabezas de atención	12
Dimensión feedforward	3072
Dropout	0.1
Pooling	Mean Pooling
Clasificador	Proyección lineal con GELU y Dropout

Tokenizador

Se entrena un *tokenizador* con algoritmo BPE (*Byte Pair Encoding*) [62]. BPE es un algoritmo de segmentación de texto que parte de caracteres individuales y va fusionando los pares más frecuentes hasta alcanzar el tamaño del vocabulario deseado [62]. El resultado es un vocabulario de 16.000 *tokens* que cubre tanto palabras frecuentes como palabras para términos menos comunes, lo que permite representar cualquier texto. El tokenizador se entrena exclusivamente sobre el *split* de entrenamiento del *dataset* de telecomunicaciones, nunca sobre los *splits* de validación ni de test. El vocabulario incluye cinco *tokens* especiales con funciones concretas en la arquitectura de la GAN (véase Figura 11): (1) PAD (*Paddimg*) rellena las secuencias más cortas hasta la longitud máxima del *batch*; (2) BOS (*Beginning Of Sequence*) marca el inicio de una secuencia; (3) EOS (*End Of Sentence*) marca el final; (4) UNK (*Unknown*) representa caracteres que no aparecieron durante el entrenamiento del tokenizador; y (5) SEP (*Separator*) separa la pregunta de las respuestas cuando se concatenan.

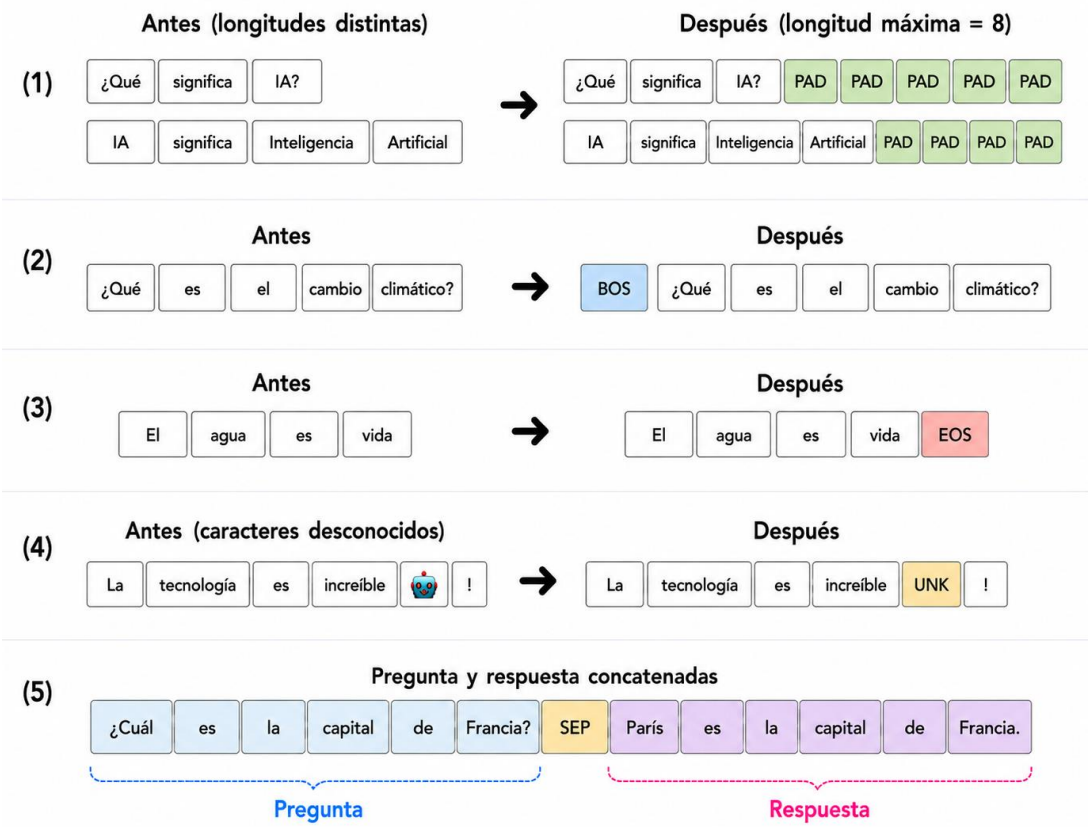


Figura 11: Ejemplos de los cinco *tokens* especiales y su función. Se muestra en orden (1) PAD, (2) BOS, (3) EOS, (4) UNK y (5) SEP. Fuente propia.

Entrenamiento

El entrenamiento sigue el esquema adversarial estándar, donde el Generador crea respuestas falsas para intentar engañar al Discriminador en cada *batch* [1]. La función de pérdida del discriminador es *binary cross-entropy* con *logits* (*BCEWithLogitsLoss*). La pérdida del generador combina dos términos: la señal adversarial del Discriminador (que empuja al generador a producir texto que el Discriminador clasifique como real) y una pérdida de *cross-entropy* supervisada (que ancla al Generador a las respuestas de referencia e impide que el entrenamiento diverja).

Se divide el *dataset* de entrada en tres *splits* (70% entrenamiento, 15% validación, 15% test) mediante un *split* por subtema para evitar *data leakage* (aparición de las mismas preguntas simultáneamente en entrenamiento y test). Una validación cruzada *k-fold* no es viable en este contexto dado el coste computacional del entrenamiento adversarial (100 épocas sobre GPU), por lo que el *split* único dividido por subtema se ha considerado la alternativa adecuada [64].

Ambas redes (Generador y Discriminador) se han optimizado con *AdamW* (con parámetros de 0.9 y 0.98, *weight decay* de 0.01) y un *scheduler* con *lineal warmup* durante las dos primeras épocas seguido de *lineal decay* hasta el final del entrenamiento. El *learning rate* es de $5x10^{-5}$ para ambas redes. El *batch size* es 16 y el entrenamiento se ejecuta durante 100 épocas. El *batch size* se ajusta de forma dinámica, el *pipeline* intenta arrancar con *batch* 16, y si la GPU no tiene suficiente memoria reduce automáticamente a 8 y después a 4. Los hiperparámetros de entrenamiento se resumen en la Tabla 7.

Tabla 7: Hiperparámetros de entrenamiento de la GAN.

Parámetro	Valor
Optimizador	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay = 0.01)
Learning rate	$5 \cdot 10^{-5}$
Scheduler	Warmup lineal (2 épocas) + decay lineal
Épocas	100
Batch size	16 (con fallback automático a 8 o 4)
Temperatura Gumbel-Softmax	0.8

3.6 Construcción del corpus propio de ingeniería

Se ha construido un corpus propio en la Fase III del trabajo, centrado en la titulación de Ingeniería de Sistemas de Telecomunicación de la Universidad CEU San Pablo. Se empleará para, inferir los mismos modelos evaluados en la Fase II y para evaluar el efecto de *transfer learning*.

Se ha organizado el contenido de la base de datos creada siguiendo el plan de estudios del grado en Ingeniería de Sistemas de Telecomunicación[2]. Es decir, se han tomado las guías docentes de las 33 asignaturas referentes de los 4 años de estudio del grado (más las asignaturas optativas).

Como se observa en la Figura 12, de cada guía docente se extrajo el temario completo, así como los puntos específicos vistos en cada asignatura y bibliografía principal y complementaria. Se formularon 10 preguntas teóricas (con su respuesta) y 10

[2] Grado en Ingeniería de Sistemas de Telecomunicación | CEU San Pablo

preguntas prácticas (con su respuesta) por punto, es decir que en total la base de datos alberga **24.680 pares de preguntas-respuestas distribuidos en 33 asignaturas** organizadas en un CSV plano (dataset_teleco.csv) con las columnas id, curso, asignatura, tema, subtema, tipo, pregunta y respuesta, pensado para el pipeline de inferencia y evaluación con métricas.

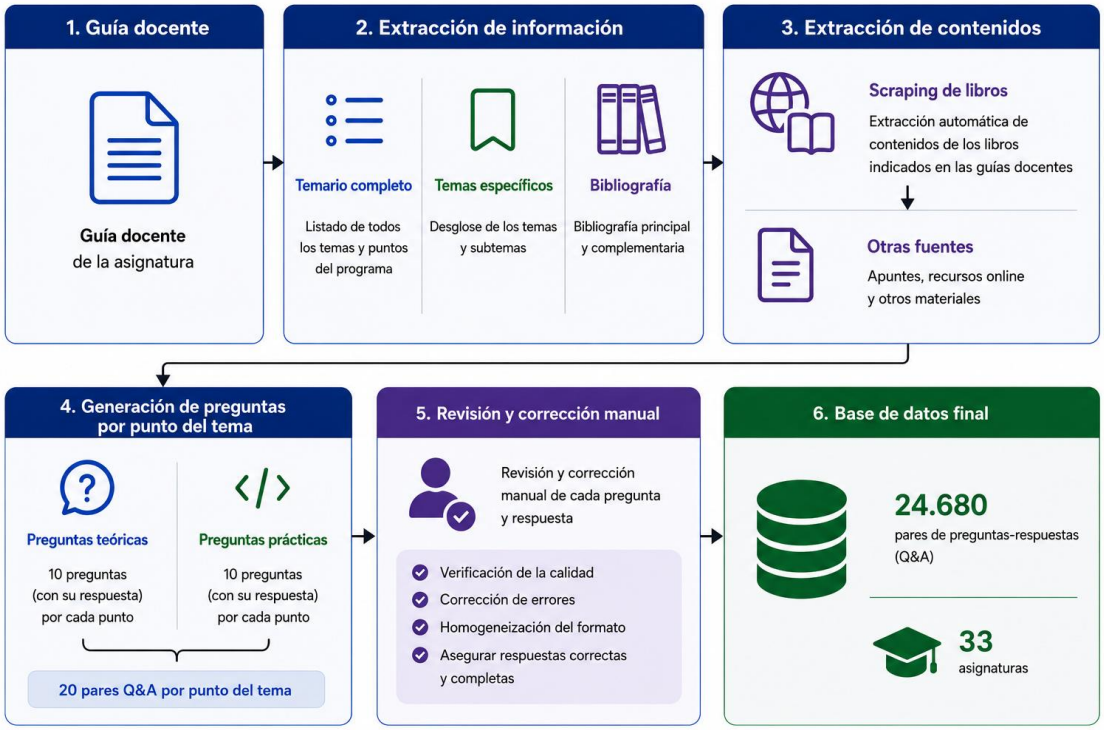


Figura 12: Proceso de recopilación, generación y validación de pares de preguntas-respuestas del *dataset*. Fuente propia.

Las preguntas se realizaron de forma manual y algunas de las preguntas se crearon mediante *scraping* de los libros indicados en las guías docentes. Se ha evitado emplear IA generativa para la creación de preguntas, puesto que se considera que, aunque es inevitable que las futuras IA se entrenen con contenido creado por IA anteriores, esto podría llevar a un círculo vicioso que tiene el peligro de acabar en la masificación de conocimientos falsos o errados. Las preguntas y respuestas se han redactado con sintaxis LaTeX para representar correctamente la notación matemática propia de la ingeniería.

La estructura completa del corpus propio se recoge en el Anexo C.

3.7 Fine-tuning de mini-SLM

Para realizar *transfer learning* con el corpus creado de los tres modelos Mini-SLM se ha utilizado la técnica *LoRA* (*Low-Rank Adaptation*) [65], que, en lugar de reentrenar todos los parámetros del modelo, congela los pesos originales e introduce unas matrices de bajo rango entrenables en las capas de atención. De este modo se consigue un *fine-tuning* eficiente tanto en tiempo como en consumo de VRAM, con resultados similares a un *fine-tuning* completo.

La elección de *LoRA* frente a un *fine-tuning* completo responde a la optimización en eficiencia y almacenamiento. A nivel computacional, aunque los modelos base ocupan unos 3 GB de VRAM en precisión bf16, un *fine-tuning* completo exige almacenar gradientes y estados del optimizador para toda la red, consumiendo entre 12 y 15 GB de VRAM. *LoRA* evita este cuello de botella calculando los gradientes exclusivamente sobre matrices adaptadoras (menos del 2% de los parámetros totales), lo que permite entrenar en la GPU disponible.

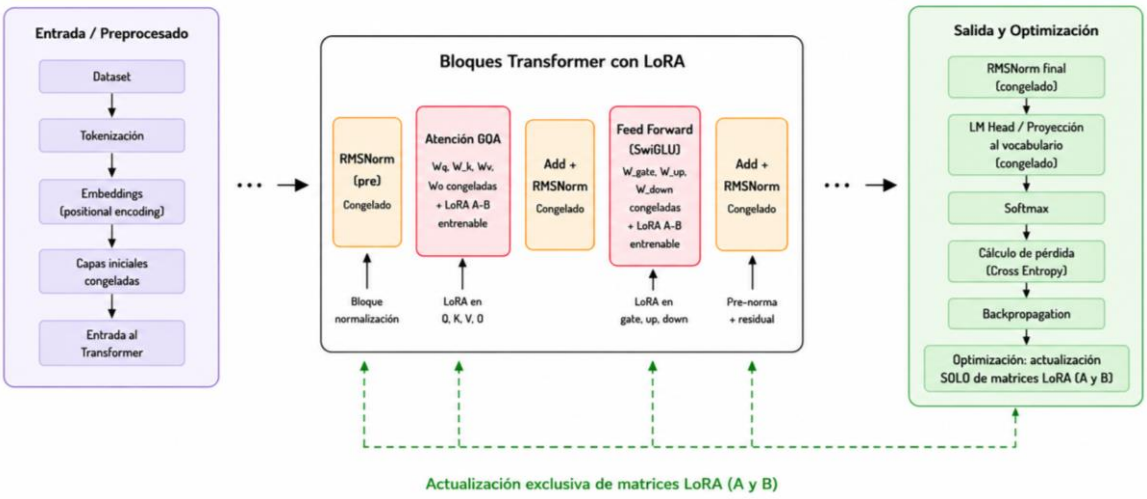


Figura 13: Arquitectura de *fine-tuning* con *LoRA*. Fuente propia.

Los hiperparámetros utilizados y la arquitectura, se pueden observar en la Tabla 8 y en la Figura 13. Se han fijado siguiendo las recomendaciones del *paper* original de *LoRA* [65] y la documentación de librería PEFT [66] para modelos en el rango de 1 – 2B parámetros.

Tabla 8: Hiperparámetros utilizados para el *fine-tuning.*

Hiperparámetro	Valor	Función
Rango (r)	8	Determina la dimensión de las matrices adaptadoras. Controla la capacidad de aprendizaje y el riesgo de sobreajuste
Alpha (lora_alpha)	16	Actúa como factor de escala sobre la contribución de los adaptadores
Dropout	0.05	Técnica de regularización aplicada específicamente sobre las capas *LoRA* para prevenir el sobreajuste
Learning rate	0.0002	Tasa de aprendizaje máxima para actualizar los pesos del modelo
Optimizador	AdamW	Algoritmo de optimización encargado de ajustar los pesos de los adaptadores
Scheduler	Cosine con warmup del 3%	Estabiliza el entrenamiento subiendo gradualmente la tasa de aprendizaje durante el 3% inicial de los pasos, luego decae hasta 0
Épocas	3	Número total de veces que el modelo itera sobre el conjunto de datos completo
Batch size efectivo	16 (batch 4 x gradient accumulation 4)	Simula el procesamiento de 16 muestras acumulando los gradientes de 4 iteraciones consecutivas de tamaño 4
Longitud máxima de secuencia	512 tokens	Limita el tamaño de la entrada para acotar el consumo de memoria
Capas objetivo	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	Aplica adaptadores tanto en el mecanismo de atención como en el MLP
Precisión	bf16	Formato numérico en punto flotante, permite guardar los modelos en memoria

El *dataset* de telecomunicaciones (24.680 pares) se divide por subtema y posteriormente en tres *splits* dentro de cada subtema (70% entrenamiento, 15% validación, 15% test). Se ha realizado un *fine-tuning* independiente para cada uno de los tres modelos. Cada 500 pasos se evalúa la pérdida sobre el *split* de validación y se guarda el *checkpoint* si la función de perdida (*Masked Cross-Entropy Loss*) ha mejorado respecto al anterior. Al finalizar el entrenamiento se recupera automáticamente el mejor *checkpoint* (el de menor eval_loss) y se descartan los intermedios.

Una vez completado el entrenamiento, se carga el modelo base junto con el adaptador *LoRA* y ejecuta la inferencia sobre los *splits* de validación y test por separado.

3.8 Marco de evaluación

Bilingual Evaluation Understudy (BLEU)

BLEU [67] mide la coincidencia de n-gramas entre el texto generado y la referencia, evaluando no solo el vocabulario sino la fluidez gramatical. Para evitar "engaños" por una respuesta extremadamente corta se introduce una penalización por brevedad (*Brevity Penalty*) que evita que el modelo obtenga puntuaciones artificialmente altas generando respuestas muy cortas. Su fórmula matemática es [67]:

$$BLEU = BP \cdot \exp\left(\sum_n w_n \cdot \log(p_n)\right) \quad (5)$$

Donde p_n es la precisión modificada del n-grama de orden n, w_n es el peso asociado (1/N) y BP vale 1 si la frase generada es al menos tan larga como la referencia, o exp $(1 - r/c)$ en caso contrario, siendo r la longitud de la referencia y c la de la de la frase generada. BLEU toma valores entre 0 y 1, donde 1 indica coincidencia total con la referencia.

Metric for Evaluation of Translation with Explicit Ordering (METEOR)

METEOR [68] evalúa la frase generada y la referencia buscando coincidencias exactas, raíces de palabras (*stemming*) y sinónimos mediante bases de datos (*WordNet*).

$$METEOR = \left(\frac{10 \cdot P \cdot R}{R + 9P}\right) \cdot \left(1 - 0.5 \cdot \left(\frac{c}{u_m}\right)^3\right) \quad (6)$$

Donde P es la precisión, R la exhaustividad, u_m el total de palabras emparejadas y c el número de "fragmentos" en orden [68]. Penaliza fuertemente la omisión de información dándole más valor a la precisión (palabras iguales que en la referencia) del modelo que a añadir palabras de más. METEOR toma valores entre 0 y 1.

Recall-Oriented Understudy for Gisting Evaluation – Longest Common Subsequence (ROUGE-L)

ROUGE-L [69] evalúa la coherencia estructural del texto mediante la subsecuencia común más larga (LCS, *Larger Common Subsequence*) sin exigir que las palabras coincidentes sean consecutivas. Su cálculo global equilibra la precisión y exhaustividad de dicha subsecuencia [69]:

$$ROUGE - L = \frac{(1 + \beta^2) \cdot R_{LCS} \cdot P_{LCS}}{R_{LCS} + \beta^2 \cdot P_{LCS}} \quad (7)$$

R_{LCS} representa la exhaustividad (la longitud de la subsecuencia común dividida entre el número total de palabras de la referencia) y P_{LCS} representa la precisión (la longitud de la subsecuencia común dividida entre el total de palabras generadas). β permite ajustar qué factor tiene más peso en la evaluación final. ROUGE-L penaliza los cambios drásticos en el orden de la oración, puntuando del 0 al 1.

Bidirectional Encoder Representations from Transformers Score (BERTScore)

BERTScore [70] abandona la coincidencia léxica y compara los textos en el espacio semántico, convirtiendo cada *token* en un vector o *embedding* contextualizado (mediante BERT [71] o RoBERTa [72]) y calculando la similitud coseno:

$$BERTScore = \frac{2 \cdot P_{BERT} \cdot R_{BERT}}{P_{BERT} + R_{BERT}} \quad (8)$$

P_{BERT} representa la precisión y R_{BERT} la exhaustividad [70]. La puntuación global de BERTScore se obtiene mediante la media armónica entre la precisión y la exhaustividad [70]. Esto le permite reconocer términos totalmente distintos (paráfrasis) pero que conservan el mismo significado que BLEU, METEOR o ROUGE-L penalizarían por no compartir vocabulario, siendo la métrica más robusta ante variaciones léxicas.

Métricas operativas

Más allá de la calidad textual, se evalúa la viabilidad del en producción mediante tres métricas:

- *Tiempo de inferencia y latencia:* Cuantifica el retardo computacional del modelo. Se evalúa mediante la latencia (ms/token) y el caudal o *throughput* (token/s). Se mide desde que el modelo recibe la secuencia de entrada hasta que genera el texto de salida completo.
- *Consumo de recursos y coste:* Se monitoriza el consumo máximo de memoria virtual (VRAM) durante la inferencia y el número de operaciones de coma flotante por segundo (FLOP) para calcular el coste económico de mantener el modelo alojado en infraestructuras en la nube (como AWS, *Google Cloud* o *Azure*).

- *Huella de carbono (Co2):* Se cuantifica el impacto ambiental del sistema en kg de Co2 [73]:

$$Emisiones_{Co2} = \left(\sum P_i \cdot t\right) \cdot I_C \quad (9)$$

Donde P_i es la potencia de cada componente, t el tiempo de inferencia e I_C la intensidad de carbono de la red eléctrica local.

3.9 Desarrollo de la aplicación (TelecoSLM)

TelecoSLM ha sido desarrollada con *Flutter* [74], un *framework* de *Google* que permite compilar una única base de código en el lenguaje de programación Dart tanto para *Android* como para *Windows*, evitando mantener dos versiones separadas.

La arquitectura de la aplicación está organizada en capas. El estado global (modelo cargado, conversaciones activas, si está generando una respuesta) se gestiona con *Provider* [75], una librería de *Flutter* que permite compartir datos entre distintas capas de la aplicación sin tener que pasarlos manualmente de una a otra. Las conversaciones se almacenan en *SQLite* [76], una base de datos local que no requiere servidor externo, y las preferencias del usuario (ruta del modelo, URL del servidor) se guardan con *SharedPreferences* [77], un sistema de almacenamiento *key-value* que persiste entre sesiones.

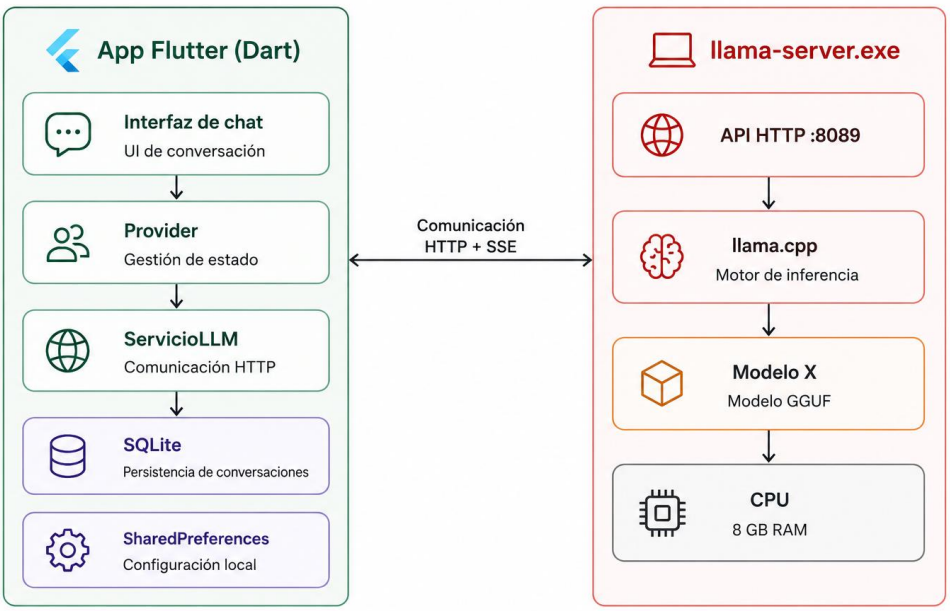


Figura 14: Arquitectura de la aplicación en *Windows*. Fuente Propia.

La inferencia del modelo difiere según la plataforma. En *Windows* (véase Figura 14), la aplicación llama a `llama-server.exe` [78][79] , que implementa la versión C/C++ del modelo de lenguaje (`llama.cpp`), como proceso independiente que expone una API HTTP en *localhost* (es decir, solo accesible desde el propio equipo). La comunicación entre la aplicación y el servidor se hace mediante peticiones HTTP con respuestas en *34ootstrap* a través de *Server-Sent Events* (SSE) [80], lo que permite que el usuario vea cómo se genera la respuesta palabra a palabra en tiempo real.

En *Android*, la inferencia local se realiza mediante `llama_flutter_android` [81], un paquete de *Flutter* que compila `llama.cpp` de forma nativa para procesadores ARM64. Esto permite ejecutar el modelo directamente en el procesador del teléfono sin necesidad de conexión a internet. Ambas plataformas soportan conexión a un servidor remoto en la red local, lo que permite aprovechar *hardware* más potente si está disponible en otro equipo.

El modelo desplegado se distribuye en formato GGUF con cuantización Q4_K_M [82][83], que comprime los pesos a 4 bits reduciendo su tamaño a unos 900 MB. El formato GGUF es un estándar para almacenar modelos de lenguaje optimizados para inferencia en CPU y la cuantización se trata de una técnica ampliamente estudiada en la literatura para reducir los requisitos de memoria y computación de lenguaje [84][85].

Para integrar el modelo *fine-tuned* en la aplicación se requiere convertirlo a un formato compatible con `llama.cpp`. El adaptador LoRA se fusiona con el modelo base usando PEFT (*Parameter-Efficient Fine-Tuning*), lo que produce un modelo completo en formato *safetensors*. Después se convierte a GGUF con el *script* `convert_hf_to_gguf.py` de `llama.cpp` y se cuantiza a Q4_K_M con `llama-quantize`, pasando de 3,4 GB en f16 a 1.1 GB.

El instalador de *Windows* y el archivo APK para *Android*, se encuentran en disponibles en el siguiente enlace de *OneDrive*: TelecoSLM. Para acceder, es posible que se requiera solicitar permiso previamente.

4 RESULTADOS Y DISCUSIÓN

La presente sección recoge y analiza los resultados obtenidos a lo largo de las distintas fases del desarrollo descritas en el capítulo anterior.

4.1 Resultados de la encuesta de validación

El diseño completo de la encuesta, las instrucciones, los ítems y el análisis detallado en el Anexo B. Se evaluó la disposición de los 220 participantes a utilizar una herramienta de IA especializada como apoyo al aprendizaje. La Tabla 9 compara los resultados entre la muestra global y el subgrupo de estudiantes de grado y máster, perfil de usuario objetivo del sistema *Likert* de 5 puntos. Los resultados de la encuesta se recogen en el Anexo D (Figura Anexo 11; Tabla Anexo 1).

Tabla 9: Resultados de la disposición de los participantes a utilizar una herramienta de IA medido.

Opción	Global (n = 220)		Grado/Máster (n = 178)	
	n	%	N	%
Totalmente de acuerdo	**107**	**48,6 %**	**85**	**47,8 %**
De acuerdo	**83**	**37,7 %**	**70**	**39,3 %**
Ni de acuerdo ni en desacuerdo	16	7,3 %	13	7,3 %
En desacuerdo	7	3,2 %	4	2,2 %
Totalmente en desacuerdo	7	3,2 %	6	3,4 %
De acuerdo o totalmente de acuerdo	**190**	**86,4 %**	155	**87,1 %**

El 86,4 % de la muestra global se mostró de acuerdo o totalmente de acuerdo, porcentaje prácticamente idéntico en el subgrupo de grado y máster (87,1 %), lo que indica que la alta disposición es transversal a todos los perfiles. Se justifica pues, con estos resultados, que existe una demanda clara y generalizada de herramientas de IA especializadas como apoyo al aprendizaje entre el público universitario.

No obstante, esta disposición no implica una aceptación incondicional. En las respuestas abiertas, un porcentaje notable de participantes condicionó su valoración positiva con expresiones como “apoyo sí, como base no”, “siempre y cuando sea de apoyo, no de sustitución”, distinguiendo con claridad entre una herramienta que agiliza el estudio y un reemplazo al material académico. Esta distinción es coherente con la orientación del sistema desarrollado, diseñado como complemento al proceso de aprendizaje.

4.2 Comparativa LLM, SLM y Mini-SLM

La comparativa se estructura en tres niveles de capacidad descendente, LLM, SLM y Mini-SLM. La Figura 15 muestra los *boxplots* de las cuatro métricas de calidad en WebNLG para los nueve modelos.

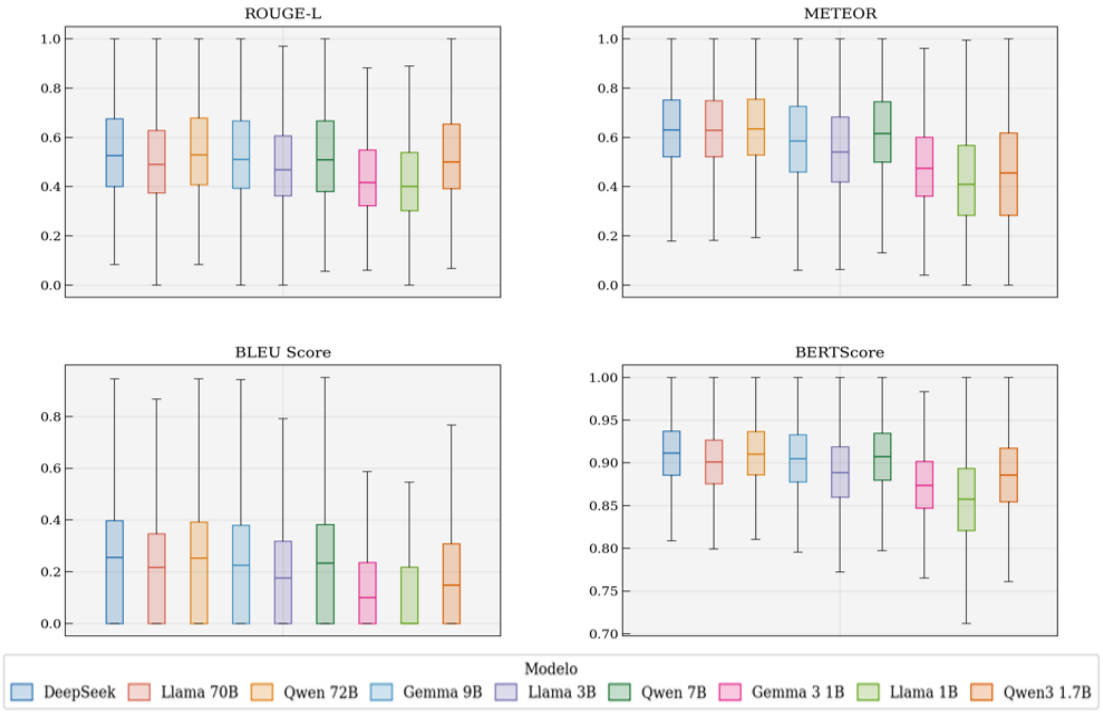


Figura 15: Boxplots de todas las métricas en el *dataset* WebNLG. Fuente propia.

En WebNLG los LLM lideran con media de 0,540 en ROUGE-L y 0,640 en METEOR, seguido de los SLM (0,525 y 0,591). Dentro de los Mini-SLM, Qwen 3 1.7B alcanza un ROUGE-L de 0,536, por encima de varios SLM, mientras que Gemma 3 1B y Llama 3.2 1B en valores razonables (0,452 y 0,436). En ToTTo la brecha entre los LLM y SLM se reduce, donde Gemma 2 9B alcanza 0,499 puntos frente a una media de 0,505 de los dos mejores LLM. En KELM los valores descienden de forma generalizada para todos los grupos (ROUGE-L de 0,331 para LLM, 0,321 para SLM, y 0,307 para Mini-SLM) lo que refleja que el vocabulario técnico especializado del *dataset* penaliza el solapamiento léxico con independencia del tamaño del modelo.

BERTScore invierte esa dispersión con cajas más largas (desviación típica de 0,04 a 0,07), valores concentrados entre 0,84 y 0,91 para todos los modelos en WebNLG y ToTTo, sin que ningún modelo quede fuera de ese rango. BERTScore es la métrica que mejor ilustra que la similitud semántica es alta y homogénea con independencia del tamaño del modelo. BLEU presenta el comportamiento opuesto con alta varianza y

bigotes que alcanzan los 0,8 a 1,0 puntos, ya que, el solapamiento de n-gramas depende mucho de la longitud de la referencia en cada ejemplo concreto. Todas las gráficas y tablas se recogen en el Anexo D (Figura Anexo 26 – Figura Anexo 29; Tabla Anexo 6).

4.2.1 Análisis estadístico

Se ha hecho un análisis estadístico con el objetivo de determinar si las métricas de calidad varían significativamente entre LLM, SLM y Mini-SLM para identificar qué familia de modelos se propone para el despliegue. Las hipótesis planteadas son:

- H_0: El valor medio de las métricas es igual para todos los modelos evaluados.
- H_1: El valor medio de al menos una métrica difiere significativamente entre modelos.

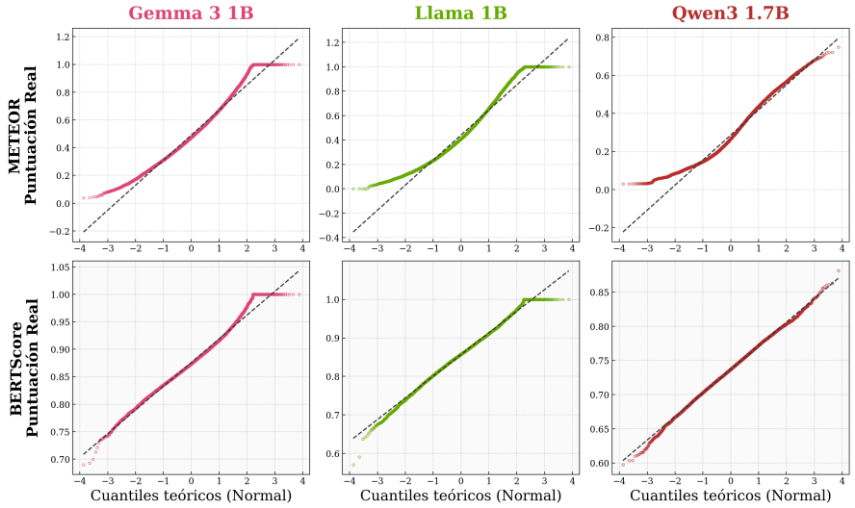


Figura 16: Q-Q tests de normalidad para METEOR y BERTScore para WebNLG. Fuente propia.

La parametricidad se evaluó mediante Q-Q tests en lugar de Shapiro-Wilk [86]. Con tamaños muestrales mayores a 50 elementos, Shapiro-Wilk rechaza la normalidad de forma casi sistemática, incluso ante desviaciones sin efecto práctico sobre la elección del test [86]. La Figura 16 muestra los Q-Q *plots* de METEOR y BERTScore en WebNLG para los tres Mini-SLM (Gemma 3 1B (—), Llama 3.2 1B (—), Qwen 3 1.7B (—)), donde la línea discontinua representa la distribución normal teórica. Todas las figuras se recogen en el Anexo D (Figura Anexo 15 – Figura Anexo 24).

Se aprecia que para BERTScore, los puntos siguen la diagonal con relativa fidelidad, pero ese comportamiento no es consistente en otros modelos y *datasets* donde

se presentan colas pesadas y desviaciones visibles. Por lo que se aplicó el test de Friedman a todas las métricas [87].

Los estadísticos del test de Friedman fueron significativos en todos los casos ($p < 0,05$), por lo que se rechaza la hipótesis nula ($H0$) y se acepta $H1$ [87]. Las tablas completas se recogen en el Anexo D (Tabla Anexo 7).

Con muestras tan grandes, el test de Friedman tiene potencia estadística casi infinita, haciendo que cualquier diferencia, por mínima que sea, alcanza significación. Por esta razón, el criterio de interpretación principal en el análisis *post-hoc* no ha sido el p-valor sino el tamaño del efecto r de Wilcoxon ($r = Z/\sqrt{N}$), siendo Z el estadístico del test y N el número de pares comparados [88]. Siguiendo los umbrales de efecto convencionales de Cohen [89] (nulo = $r < 0,10$; pequeño = $0,10 \leq r < 0,30$; mediano = $0,30 \leq r < 0,50$); y grande = $r \geq 0,50$). Por ejemplo, pares con $p < 0,001$ pero $r < 0,10$ se consideran estadísticamente detectables, pero prácticamente irrelevantes.

Post-hoc

Se ha aplicado Nemenyi con corrección de Wilcoxon [90]. En WebNLG, los tres LLM (DeepSeek, Llama 3.3 70B y Qwen 2.5 72B) superaron consistentemente a los Mini-SLM (Gemma 3 1B, Llama 3.2 1B y Qwen 3 1.7B) en METEOR con efectos grandes (r = 0,57 – 0,69) y en ROUGE-L con efectos de pequeños a medianos (r = 0,08 – 0,48). En BERTScore los efectosLLM vs Mini-SLM también son relevantes (r = 0,27 – 0,71), mientras que las diferencias LLM vs SLM son en su mayoría nulas o pequeñas (r = 0,05 – 0,41), lo que cofirma que los SLM son semánticamente competitivocon los LLM en corpus genérico.

En ToTTo, sin embargo, la mayoría de las diferencias LLM vs SLM (Gemma 2 9B, Llama 3.2 3B, Qwen 2.5 7B) son significativas, pero de magnitud nula o pequeña (r = 0,01 – 0,21) en todas las métricas.

En KELM los efectos son nulos o pequeños en todas las métricas y grupos, lo que refleja la mayor homogeneidad entre modelos sobre corpus técnico especializado. BLEU es la métrica con menos pares significativos (solo 27 de 36), con efectos exclusivamente nulos o pequeños (r = 0,01 – 0,15), coherente con la mayor dificultad léxica del corpus.

Dentro de la categoría SLM, Gemma 2 9B obtuvo el mejor rendimiento global, aunque las diferencias frente a Qwen 2.5 7B no alcanzaron significación práctica ($r < 0,15$ en todas las métricas). Las tablas completas del *post-hoc* se recogen en el Anexo D (Tabla Anexo 8 – Tabla Anexo 19).

Se crearon diagramas de diferencia crítica (*CD diagrams*) para ilustrar visualmente los resultados del test de Nemenyi. Todos los diagramas de diferencia crítica de cada métrica por cada *dataset* se recogen en el Anexo D (Figura Anexo 39 – Figura Anexo 50).

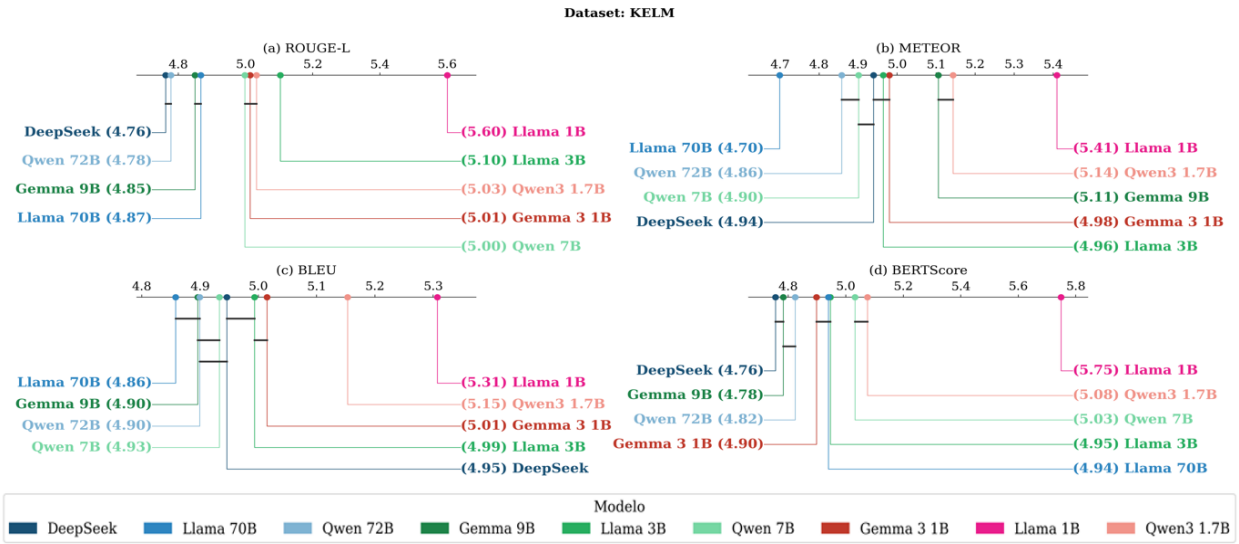


Figura 17: Diagrama de diferencia crítica para todas las métricas en el *dataset* KELM. Fuente propia.

La Figura 17 muestra los *CD diagrams* para las cuatro métricas en KELM, donde los modelos unidos por una barra horizontal no presentan diferencias estadísticamente significativas entre sí ($\alpha = 0,05$) y a menor rango, mejor rendimiento. En BERTScore (Figura 17d), DeepSeek (rango 4,76), Gemma 9B (4,78) y Qwen 2.5 72B (4,82) forman un bloque sin diferencias significativas al que se una Gemma 3 1B (4,90) (los LLM y el mejor Mini-SLM son indistinguibles en semántica sobre corpus técnico). En ROUGE-L (Figura 17ª) los rangos se comprimen entre 4,76 y 5,60, con todos los modelos agrupados salvo Llama 3.2 1B (5,60), que queda aislado. El patrón se repite en METEOR y BLEU, rangos muy próximos entre sí, sin una separación clara entre LLM, SLM y Mini-SLM.

La diferencia media entre LLM y SLM en BERTScore es de 0,002 puntos en KELM (prácticamente nula). El *post-hoc* confirma que la mayoría de los pares, LLM frente a SLM en ToTTo y KELM, tiene efectos nulos o pequeños ($r = 0,01 – 0,21$),

diferencias estadísticamente detectables, pero sin relevancia práctica. Tres familias de modelos con tamaños muy distintos (de 72B a 1B parámetros) producen texto semánticamente casi indistinguibles sobre corpus técnico especializado.

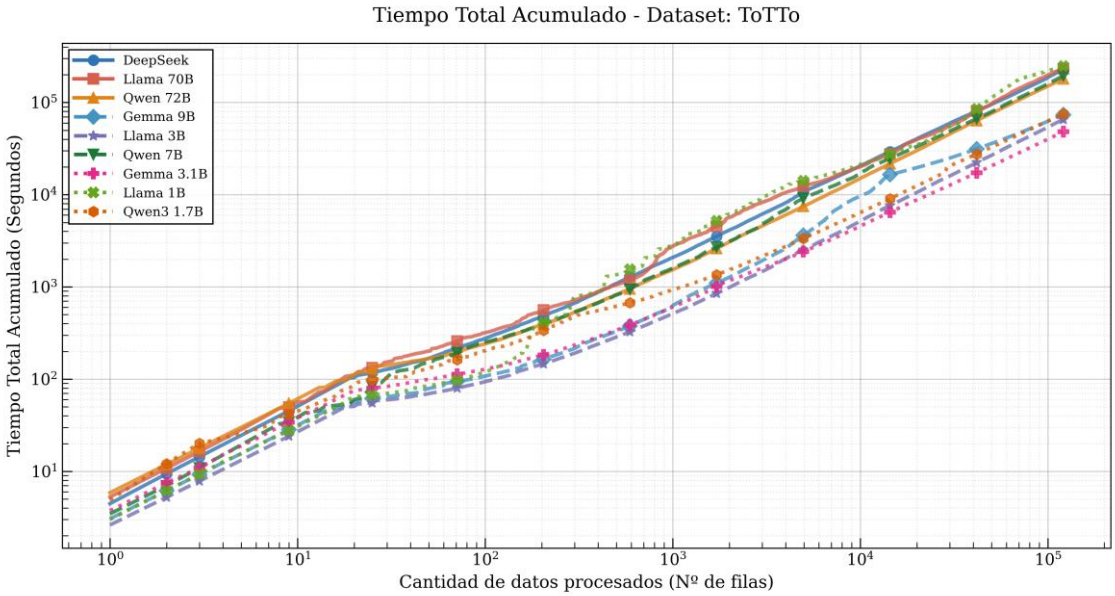


Figura 18: Tiempo acumulado de inferencia por modelo en el *dataset* ToTTo. Fuente propia.

Junto a las métricas de calidad también se han recopilado el coste de inferencia, las emisiones de Co2 y el tiempo por fila de cada modelo. Los resultados completos, se recogen en el Anexo D (Figura Anexo 30 – Figura Anexo 38; Tabla Anexo 4 – Tabla Anexo 5). Con este contexto, la elección de un Mini-SLM para la aplicación es una decisión técnica razonada. Los LLM requieren API externa, tienen costes de 1,15$ a 10,27$ por corpus evaluado, latencias de 1,4 – 2,4 segundos por fila y emisiones de entre 333 y 5.127 gramos de Co2 por *dataset*. Un Mini-SLM puede ejecutarse en el dispositivo sin red, sin coste marginal por consulta y con latencia compatible con uso interactivo. Entre Gemma 3 1B, Qwen 3 1.7B y Llama 3.2 1B, **Gemma 3 1B muestra mayor consistencia** en BERTScore y ROUGE-L a lo largo de los tres *datasets*.

4.3 Impacto del fine-tuning

El rendimiento sobre un corpus de dominio específico puede diferir, por ello, se evalúan los tres Mini-SLM base junto a sus versiones *fine-tuned* sobre el corpus propio de telecomunicaciones, y la GAN entrenado desde cero sobre lo mismos datos. La Figura 19 muestra los *boxplots* de las métricas de calidad para la BBDD de telecomunicaciones

(corpus propio). El resto de las gráficas y las tablas se recogen en el Anexo D (Figura Anexo 51 – Figura Anexo 65; Tabla Anexo 20 – Tabla Anexo 22).

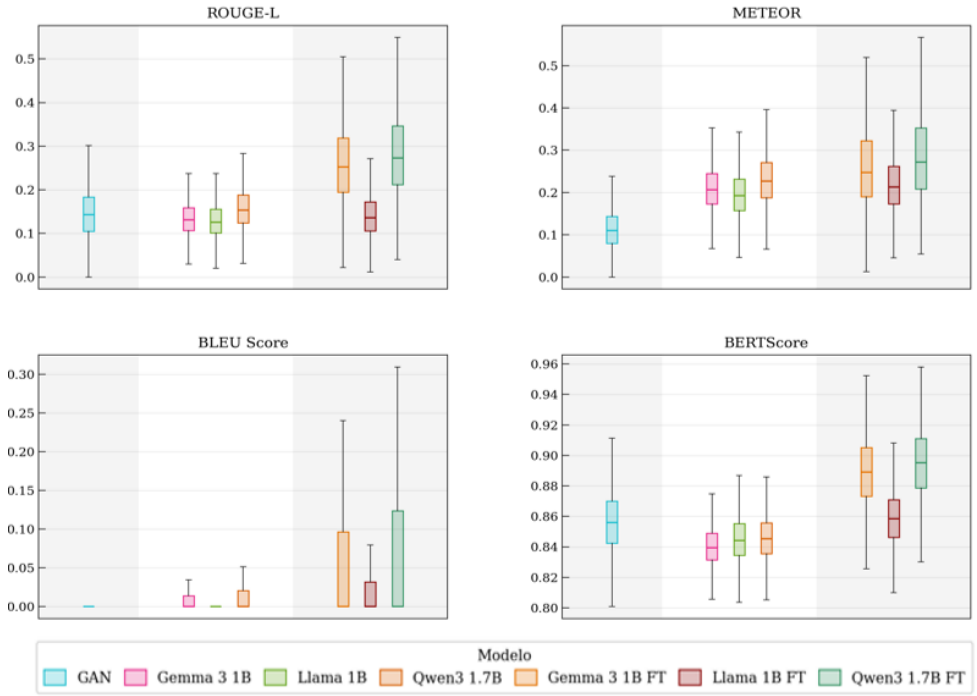


Figura 19: Boxplots de todas las métricas en el corpus propio. Fuente propia.

En ROUGE-L los *fine-tuned* lideran con claridad, Qwen 3 1.7B FT con 0,286 (+80,5% sobre su modelo base) puntos y Gemma 3 1B FT con 0,264 puntos (+95,5%), seguidos de la GAN (0,147) y Llama 3.2 1B FT (0,146) prácticamente empatados. Los Mini-SLM base se agrupan por debajo con Qwen 3 1.7B (0,159), Gemma 3 1B (0,135) y Llama 3.2 1B (0,132). En METEOR la GAN (0,116) cae al último puesto, por debajo de los Mini-SLM base, lo que refleja que su mecanismo de entrenamiento sacrifica precisión semántica frente a coherencia global. Gemma 3 1B FT mejora un 25% en METEOR respecto a su base y Llama 1B FT un 11,9%.

BLEU es donde la separación es más visible, la GAN colapsa en 0,001 y los Mini-SLM base apenas llegan entre 0,008 y 0,011, mientras que los modelos *fine-tuned* dan el salto más grande de toda la figura con Gemma 3 1B FT de 0,008 a 0,053 (+562,5%) y Llama 1B FT de 0,009 a 0,020 (+122,2%). El *fine-tuning* mejora moderadamente los modelos, siendo coherente con que los modelos base ya capturan bien la semántica general antes del ajuste.

4.3.1 Análisis estadístico

Las hipótesis son las mismas que en el apartado anterior, ahora aplicadas al corpus propio de telecomunicaciones. La parametricidad se evaluó con Q-Q tests bajo el mismo criterio, con 7.699 observaciones por modelo, los tests formales rechazan la normalidad de forma casi garantizada [86]. Debido a ello, se aplica el test Friedman a todas las métricas [87]. Todas las gráficas de los Q-Q test se recogen en el Anexo D (Figura Anexo 25).

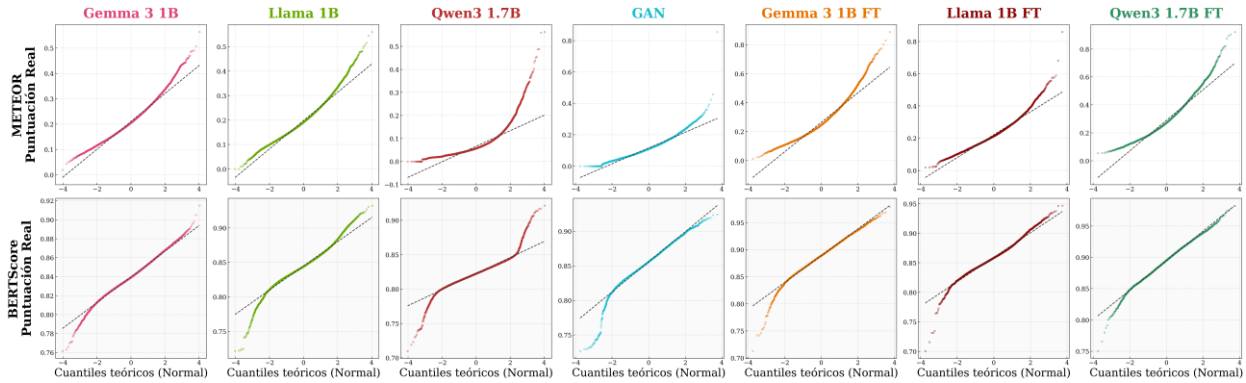


Figura 20: Q-Q tests de normalidad para METEOR y BERTScore para WebNLG. Fuente propia.

Los estadísticos del test de Friedman resultaron significativos en todos los casos ($p < 0,05$), por lo que se rechaza la hipótesis nula ($H0$) y se acepta $H1$ [87]. Las tablas completas se recogen en el Anexo D (Tabla Anexo 23).

Post-hoc

El análisis *post-hoc* par a par con el test de Nemenyi mostró 20 de 21 pares significativos en ROUGE-L y 21 de 21 en METEOR y BERTScore [90]. El único par no significativo en ROUGE-L es el de la GAN con Llama 3.2 1B FT ($p = 0,2775$, $r = 0,02$, efecto nulo), lo que indica que Llama 3.2 1B FT no mejora sobre el GAN en solapamiento léxico. Los efectos entre los modelos *fine-tuned* y la GAN son grandes ($r = 0,79$ - $0,83$), mientras que los efectos de los modelos *fine-tuned* con los modelos Mini-SLM base son más heterogéneos (de pequeños a grandes según el par concreto). En BLEU, los tres modelos *fine-tuned* superan a la GAN con efectos grandes ($r = 0,52$ - $0,58$). Las tablas completas del *post-hoc* se recogen en el Anexo D (Tabla Anexo 24 – Tabla Anexo 27).

Como apoyo al estudio se ha evaluado si el *fine-tuning* degrada el rendimiento fuera de dominio, comparando los Mini-SLM base con sus versiones *fine-tuned* en

KELM. Los tamaños de efecto resultaron mayoritariamente nulos o pequeños ($r < 0,3$), por lo que se puede suponer que el *fine-tuning* al corpus propio de telecomunicaciones no degrada la generalización del modelo. Los resultados completos están disponibles bajo petición.

Se crearon diagramas de diferencia crítica (*CD diagrams*) para ilustrar visualmente los resultados del test de Nemenyi. La mayoría de los pares entre categorías presentan diferencias significativas, con la excepción de GAN frente a Llama 1B FT en ROUGE-L. Todos los diagramas de diferencia crítica de cada métrica se recogen en el Anexo D (Figura Anexo 66 – Figura Anexo 69).

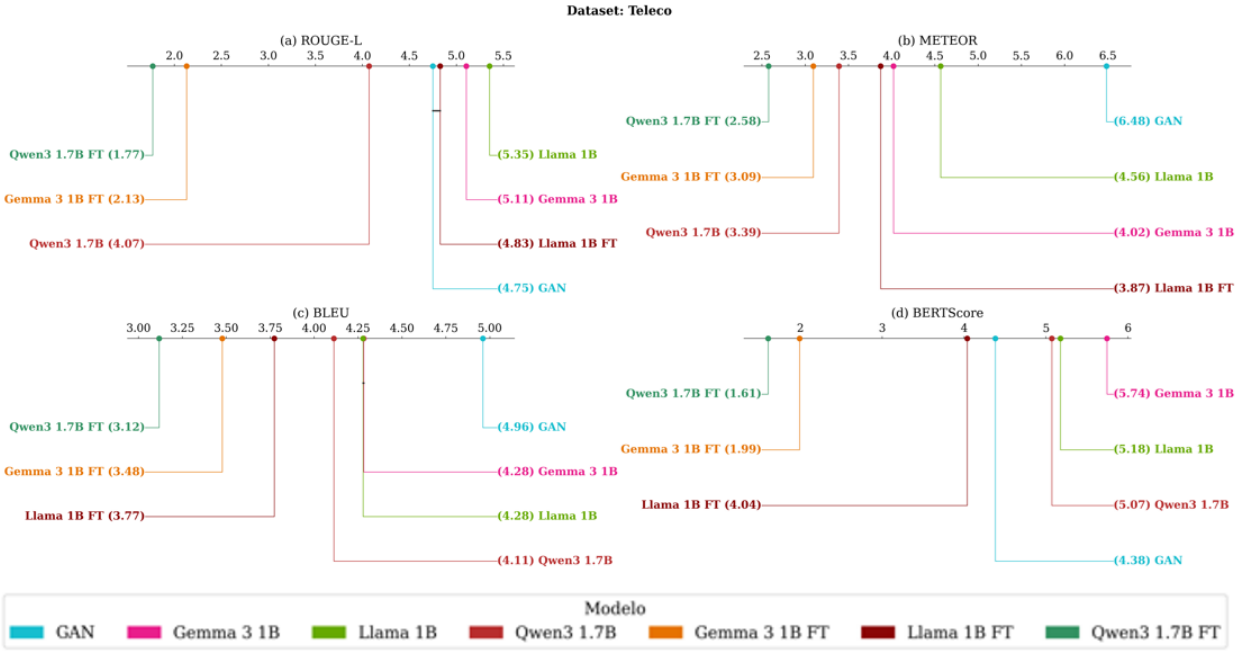


Figura 21: Diagrama de diferencia crítica para todas las métricas en el corpus propio. Fuente propia.

La Figura 21 muestra los *CD diagrams* para las cuatro métricas en el corpus de telecomunicaciones. En ROUGE-L y BERTScore (Figura 21ª y Figura 21d), Qwen 3 1.7B FT (rangos 1,77 y 1,61) y Gemma 3 1B FT (2,13 y 1,99) lideran con amplitud, separados del resto. En METEOR la GAN queda último con diferencia (6,48), confirmando su debilidad en precisión semántica. En BLEU los tres modelos *fine-tuned* encabezan el ranking y la GAN lo cierra (4,96).

Los modelos *fine-tuned* lideran en todas las métricas, Qwen 3 1.7b FT con media en ROUGE-L de 0,286 y en BERTScore con 0,895 en primer lugar y Gemma 3 1B FT en segundo lugar con medias de 0,264 y 0,889. La brecha media en BERTScore entre los

modelos *fine-tuned* y los modelos base es de 0,037 puntos, mayor que la observada entre LLM y SLM en corpus genéricos. La GAN supera a Gemma y Llama base en ROUGE-L (0,147 frente a 0,135 y 0,132) pero su BLEU es prácticamente nulo (0,001) lo que lo descarta para generación estructurada. **Qwen 3 1.7B FT ofrece el mejor equilibrio entre rendimiento y coste computacional, además de liderar en métricas**.

4.4 Análisis integrador

Cada paso hacia abajo en la jerarquía de tamaño sacrifica menos rendimiento del esperado y gana más de lo que se pierde en coste y autonomía de despliegue.

En corpus genéricos, la ventaja de los LLM sobre los SLM es pequeña. En ROUGE-L, el mejor LLM supera al mejor SLM en 0,012 putnos en WebNLG, 0,005 en ToTTo y 0,005 en KELM. En BERTScore es prácticamente nula en KELM (0,0004). A cambio, los LLM cuestan cinco veces más (1,15$ frente a 0,22$ en WebNLG; 10,27 $ frente a 1,97 $ en ToTTo) y requieren API con red. El *fine-tuning* invierte la situación en dominio específico, Qwen 3 1.7B FT alcanza ROUGE-L de 0,286 y BERTScore de 0,895, superando a todos los SLM sin ajustar en ambas métricas, con coste de inferencia cero.

La GAN queda fuera de esta jerarquía por una limitación estructural puesto que presenta BLEU prácticamente nulo (0,001). Esto refleja la inestabilidad inherente al uso de *Gumbel-Softmax* y el ratio parámetros/datos desfavorable (430 M parámetros sobre 17.000 pares). Es competitiva en semántica (BERTScore de 0,856) y la más eficiente en emisiones (2,6 g de CO_2 frente a los 15-34 g de los modelos *fine-tuned*), pero su falta de precisión léxica la descarta para generación estructurada orientada a consulta.

Entre los modelos *fine-tuned,* **Qwen 3 1.7B FT es el seleccionado para la aplicación**. Qwen 3 1.7B FT lidera en métricas brutas (ROUGE-L = 0,286, BERTScore = 0,895) y combina una mejora sólida frente a su modelo base con ROUGE-L de 0,159 a 0,286 (+80,5 %), BLEU de 0,011 a 0,069 (+531%), BERTScore de 0,846 a 0,895 (+5,8%), y 1.7B parámetros compatible con dispositivos de gama media sin red ni coste marginal. Gemma 3 1B FT queda en segundo lugar con resultados sólido (ROUGE-L de 0,264 puntos y BERTScore de 0,889 puntos) y con un comportamiento más predecible, pero sus métricas no alcanzan las de Qwen. Llama 1B FT mejora solo un 10,5 % en ROUGE-L sobre su base y no compite en este rango con sus métricas.

4.5 Aplicación móvil: TelecoSLM

La aplicación **TelecoSLM** ha sido desarrollada en *Flutter*, lo que permite mantener una única base de código para *Android* y *Windows*. Ambas versiones comparten arquitectura y funcionalidades, adaptando el *layout* a las dimensiones de cada plataforma.

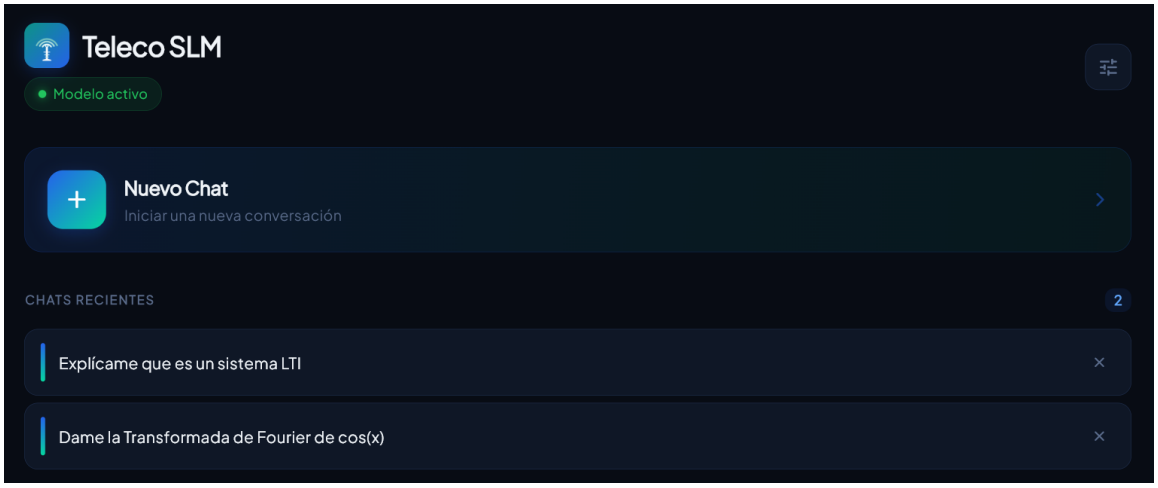


Figura 22: Pantalla principal de TelecoSLM en Windows. Fuente propia

La interfaz usa un tema oscuro con un sistema de *chat* similar al de aplicaciones como *ChatGPT* [23], debido a que reduce la fatiga visual en sesiones largas de estudio (véase Figura 22). El estado del modelo se indica en todo momento mediante un *badge* de color gris cuando no hay modelo cargado (Sin modelo), naranja cuando se está cargando (Cargando) y verde cuando la inferencia está disponible (Modelo activo). Las respuestas se renderizan con soporte completo de Markdown y las fórmulas matemáticas en LaTeX (véase Figura 23) mediante la librería `gpt_markdown` [91].

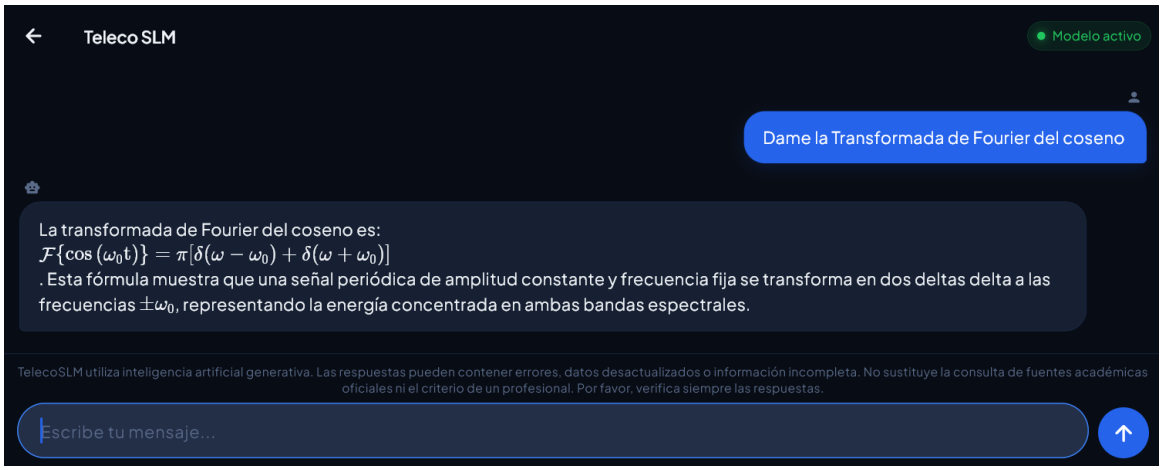


Figura 23: Ejemplo de respuesta con renderizado LaTeX y Markdown en TelecoSLM. Fuente propia.

La aplicación incluye un aviso explícito en cada conversación "*TelecoSLM es utiliza inteligencia artificial generativa. Las respuestas pueden contener errores, datos desactualizados o información incompleta. No sustituye la consulta de fuentes académicas oficiales ni el criterio de un profesional. Por favor, verifica siempre las respuestas.*". Este mensaje sirve de recordatorio constante para recordar al usuario que la inteligencia artificial no es infalible ya que existen varios riesgos como las alucinaciones (invención de datos), no es un reemplazo de expertos (no emplearla como única fuente) y presenta límites de conocimiento (respuestas obsoletas, parciales o sesgadas).

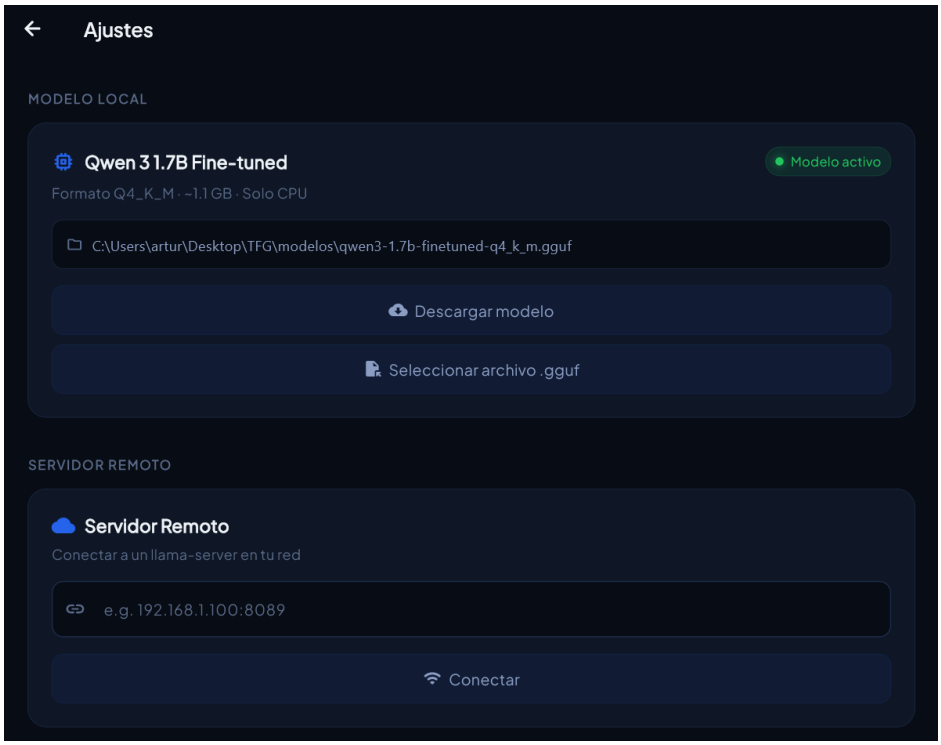


Figura 24: Pantalla de ajustes: fuentes de inferencia disponibles (modelo local, archivo GGUF y servidor remoto). Fuente propia.

La pantalla de ajustes centraliza las tres fuentes de inferencia disponible (véase Figura 24): (1) Modelo Local (el usuario puede descargar el modelo en la aplicación desde el repositorio de *HuggingFace*[3]), (2) seleccionar un archivo GGUF que ya tenga en el dispositivo o (3) Servidor Remoto (conexión de la aplicación a una instancia llama-server en red local mediante IP y puerto). Las opciones (1) y (2) no requieren conexión a Internet, en cambio la (3) sí.

[3] arturofierrop/qwen-3-1.7B-teleco-slm-GGUF · Hugging Face

El historial de conversaciones recientes es accesible desde la pantalla de inicio (véase Figura 22). Se almacena exclusivamente en el dispositivo mediante la base de datos local de *Flutter* (`sqflite`) y no se transmite a ningún servidor externo. En modo local o GGUF, ningún dato abandona el dispositivo. En modo servidor remoto, las consultas se envían únicamente a la instancia llama-server configurada por el propio usuario en su red local, sin intermediarios externos. Con ello garantizamos privacidad total de todos los datos de la aplicación.

La interfaz y funcionalidades de **TelecoSLM** se recogen en detalle en el Anexo E, además de en un manual de usuario bajo petición.

Métricas internas

La aplicación también incorpora un sistema de métricas de rendimiento oculto que registra, por cada petición al modelo, el número de *tokens* generados, el tiempo hasta el primer *token* (TTFT), el tiempo total de respuesta y la velocidad de *tokens* por segundo. Esta funcionalidad está oculta por defecto, ya que existe solo para los desarrolladores. Los valores medios recogidos en la Tabla 103 corresponden a mediciones sobre el modelo Qwen 3 1.7B FT en las versiones de *Windows* y Android.

Tabla 10: Métricas de rendimiento de Qwen 3 1.7B FT en TelecoSLM.

Métrica	Windows	Android
Dispositivo	Lenovo Intel Core i5	Samsung Galaxy A70
Tokens generados	89	119
Tiempo hasta primer token	2.384 ms	28.896 ms
Tiempo total de respuesta	10,20 s	79,5 s
Velocidad de generación	9,6 tok/s	1,48 tok/s
Uso de memoria RAM	13,0%	7,5%

La diferencia más llamativa es el tiempo hasta el primer *token*, en *Android* tarda 12 veces más en generar el primer *token* (28,9s frente a 2,4s en *Windows*), consecuencia directa de ejecutar el modelo en CPU ARM sin aceleración de *hardware*. La velocidad de generación también cae drásticamente (1.48 tok/s frente a 9,6 tok/s) aunque el resultado sigue siendo usable para un asistente de estudio donde el usuario no espera respuestas instantáneas.

Evaluación y validación de la aplicación

Con el fin de obtener una validación cualitativa en condiciones de uso real, la versión beta de la aplicación se ha distribuido a 12 *testers* con perfil objetivo (estudiantes de Ingeniería de Telecomunicaciones) durante un período de 14 días. La evaluación con usuarios se realizó conforme al Reglamento General de Protección de Datos (RGPD, UE 2016/679). Este test complementa la validación de producción y despliegue de *Google Play Store*. Los participantes han utilizado la aplicación como apoyo al estudio y al finalizar el periodo reportaron una evaluación resumida en la Tabla 11.

Tabla 11: Resultados y comentarios por parte de 12 *testers* con perfil objetivo.

Criterio	Puntuación media (1-5)	Comentarios
Facilidad de uso	4,4	La interfaz es intuitiva y sencilla. Es muy parecida a las interfaces de otros modelos.
Calidad de las respuestas	4,2	Las respuestas suelen ser correctas y bien explicadas, aunque a veces comete algún fallo o faltan detalles
Utilidad como apoyo al estudio	4,7	Muy útil para resolver dudas rápidas, sacar fórmulas u obtener explicaciones más breves.
Naturalidad del lenguaje	4,1	Las respuestas son claras y fáciles de entender. A veces se nota mucho que es una respuesta generada.
Valoración global	4,4	En general me parece una muy buena aplicación para apoyar el estudio y una buena herramienta para resolver dudas rápido. En Windows el modelo responde bastante rápido, pero en Android responde muy lento.
Recomendación de la app (% Sí)	88%	Se la recomendaría a todos mis amigos de la carrera o cualquier estudiante de ingeniería.



Figura 25: Validación beta de TelecoSLM. *Testers* durante el periodo de prueba. Fuente propia.

5 CONCLUSIONES Y TRABAJO FUTURO

El objetivo principal de este trabajo era comparar empíricamente los tres paradigmas de generación de lenguaje natural evaluar el impacto del *fine tuning* sobre un dominio específico y desplegar el mejor modelo en una aplicación local. Los ocho objetivos específicos han sido cubiertos en su totalidad.

En corpus genéricos, la jerarquía entre familias existe, pero es más estrecha de lo esperado. Los LLM lideran en todas las métricas, pero la ventaja sobre los SLM en BERTScore no supera los 0,009 puntos en WebNLG y se vuelve prácticamente nula en KELM: Los Mini-SLM sin ajustar quedan por debajo, aunque Qwen 3 1.7B alcanza a los LLM en ROUGE-L sobre WebNLG, por encima de varios SLM de mayor tamaño. El *fine-tuning* con LoRA sobre el corpus de telecomunicaciones invierte parcialmente esa jerarquía. Qwen 3 1.7B FT pasa de una media en ROUGE-L de 0,176 a 0,286 (+62,5%) y en BLEU de 0,017 a 0,069 (+317,5%), liderando en todas las métricas. El *dataset* de telecomunicaciones (24.680 pares de pregunta-respuesta construidos a partir del plan de estudios del CEU) fue determinante para esa ganancia, sin un corpus de dominio suficientemente denso, el *fine-tuning* no habría producido mejoras de esta magnitud.

La GAN tiene una lectura más matizada. En ROUGE-L (media de 0,147) supera a los Mini-SLM base de Gemma 3 1B y Llama 3.2 1B, y su coste de inferencia es de 0,00$ con emisiones de solo 2,6 gramos de Co2 frente a los 15 a 24 gramos de los modelos *fine-tuned*. Si el criterio fuera eficiencia de despliegue puro, la GAN sería el mejor modelo. Pero su BLEU es prácticamente nulo (0,001) reflejando una limitación estructural, la no diferenciabilidad del *agrmax* en generación discreta obliga a usar *Gumbel-Softmax*, que introduce ruido en el gradiente y hace el entrenamiento inherentemente menos estable. A eso se suma el ratio desfavorable de parámetros/datos (430 millones de parámetros sobre 17.000 pares de entrenamiento), frente a los Mini-SLM que llegaron preentrenados con billones de *tokens*. El resultado es un modelo competitivo en semántica, pero sin precisión léxica, lo que lo descarta para generación estructurada orientada a consulta.

Qwen 3 1.7B FT es el modelo seleccionado para la aplicación **TelecoSLM**. La aplicación se ha desarrollado en *Flutter* para *Android* y *Windows*, funciona completamente en local (sin red y sin coste) y genera respuestas con soporte *Markdown*

y LaTeX. El 86,4% de los participantes en la encuesta previa se mostró favorable a usar una herramienta de este tipo, con la condición de que actuase como complemento y no como sustituto del material académico, condición que el diseño de la aplicación incorpora explícitamente.

La limitación más visible son los recursos de cómputo disponibles para el entrenamiento de la GAN, los modelos *fine-tuning* y la inferencia de las bases de datos para estudiar a los modelos. Para el entrenamiento de la GAN el *hadrware* utilizado limitó el tamaño de los parámetros del modelo, debido a ello la GAN no es capaz de alcanzar a los modelos *fine-tuned*. Lo mismo ocurre para entrenar a los modelos *fine-tuned*. Para la inferencia de las bases de datos se tuvo que alquilar servidores donde los modelos estudiados estaban alojados, ya que en los recursos de computó disponibles para el estudio no se podía alojar a ningún modelo LLM, SLM o Mini-SLM. Otra limitación muy visible es la latencia de inferencia de la aplicación en *Android*. En un *Samsung Galaxy A70* sin acelerador dedicado, el tiempo hasta el primer *token* ronda los 28,9 segundos y el tiempo total de respuesta los 79,5 segundos, valores que quedan fuera del rango de lo que un usuario consideraría interactivo. Por último, el corpus de telecomunicaciones, aunque suficiente para producir mejoras medibles, tiene cobertura desigual entre asignaturas y está construido a partir de un único plan de estudios. Eso limita la generalización a otros programas de grado o a contenidos no cubiertos en el temario del plan de estudios utilizado. Las métricas automáticas empleadas capturan solapamiento léxico y semántico con las referencias, pero no evalúan corrección factual ni razonamiento técnico, dimensiones que solo la evaluación humana puede cubrir de forma fiable

A medio plazo, como líneas de trabajo futuro se propone ampliar el corpus con más preguntas por temática, explorar RAG sobre el corpus propio para reducir alucinaciones, evaluar técnicas de destilación del conocimiento desde LLM hacia Mini-SLM ajustados, e incorporar evaluación humana sistemática de las respuestas generadas como complemento a las métricas automáticas empleadas en este estudio. Debido a que la latencia en *Android* es el problema más inmediato, se propone utilizar cuantizaciones de menos precisión (4-bit o 2-bit) y optimizaciones para arquitecturas ARM que podrían reducir el tiempo de respuesta a un rango más interactivo sin perdida significativa de calidad.

6 BIBLIOGRAFÍA

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*..

[2] Sevilla, J., Heim, L., Ho, A., Besiroglu, T., Hobbhahn, M., & Villalobos, P. (2022). Compute trends across three eras of machine learning. En *2022 International Joint Conference on Neural Networks (IJCNN)* (pp. 1-8). IEEE.

[3] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., … & Amodei, D. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, *33*, 1877-1901.

[4] Strubell, E., Ganesh, A., & McCallum, A. (2019, July). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 3645-3650).

[5] Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., … & Fiedel, N. (2023). Palm: Scaling language modeling with pathways. *Journal of machine learning research*, *24*(240), 1-113.

[6] Lu, Z., Li, X., Cai, D., Yi, R., Liu, F., Liu, W., … & Xu, M. (2025, July). Demystifying small language models for edge deployment. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 14747-14764).

[7] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., … & Bengio, Y. (2014). Generative adversarial nets. Advances in neural information processing systems, 27.

[8] Gatt, A., & Krahmer, E. (2018). Survey of the state of the art in natural language generation: Core tasks, applications and evaluation. *Journal of Artificial Intelligence Research*, *61*, 65-170.

[9] Voigt, P., & Von dem Bussche, A. (2017). The eu general data protection regulation (gdpr). *A practical guide, 1st ed., Cham: Springer International Publishing*, *10*(3152676), 10-5555.

[10] Foster, D. (2022). *Generative deep learning*. O'Reilly Media.

[11] Ng, A. Y., & Jordan, M. I. (2001). On discriminative vs. generative classifiers: A comparison of logistic regression and naive bayes. *Advances in neural information processing systems*, 14.

[12] Karras, T., Laine, S., & Aila, T. (2019). A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 4401-4410).

[13] Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., … & Shi, W. (2017). Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4681-4690).

[14] Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1125-1134).

[15] Wang, Z., She, Q., & Ward, T. E. (2021). Generative adversarial networks in computer vision: A survey and taxonomy. *ACM Computing Surveys*, 54(2), 1-38.

[16] Goodfellow, I. (2016). *NIPS 2016 tutorial: Generative adversarial networks* (Conference Tutorial). Neural Information Processing Systems.

[17] Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798-1828.

[18] Yu, L., Zhang, W., Wang, J., & Yu, Y. (2017). SeqGAN: Sequence generative adversarial nets with policy gradient. En *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 31, No. 1).

[19] Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning: An introduction* (Vol. 1, No. 1, pp. 9-11). Cambridge: MIT press.

[20] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., … & Fedus, W. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*.

[21] Li, R., Fu, D., Shi, C., Huang, Z., & Lu, G. (2024). Efficient LLMs training and inference: An introduction. IEEE Access, 13, 32944-32970.

[22] Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. En *5th International Conference on Learning Representations (ICLR)*.

[23] OpenAI. (2024). *Hello GPT-4o*. OpenAI Blog. Recuperado de https://openai.com/index/hello-gpt-4o/.

[24] Google DeepMind, "Gemini: A next-generation AI model," *Google DeepMind*, Dic. 2023. Recuperado de: https://deepmind.google/technologies/gemini/

[25] Anthropic. (2024). *Claude 3.5 Sonnet*. Anthropic News. Recuperado de https://www.anthropic.com/news/claude-3-5-sonnet.

[26] DeepSeek-AI. (2024). *DeepSeek-V3 Technical Report: An open-source MoE model with 671B parameters*. DeepSeek Research. Recuperado de https://www.deepseek.com/.

[27] Meta AI. (2024). Llama 3 [Software]. Meta. Recuperado de: https://llama.meta.com/

[28] Mistral AI. (2023). *Mixtral of experts*. Mistral AI News. Recuperado de https://mistral.ai/news/mixtral-of-experts/.

[29] Qwen Team. (2024). Qwen2.5: A Party of Foundation Models! Qwen LM Blog. Recuperado de: https://qwenlm.github.io/blog/qwen2.5/

[30] xAI. (2024). *Grok-2 and Grok-2 mini*. xAI Technical Blog. Recuperado de https://x.ai/blog/grok-2.

[31] Gemma Team, Google. (2026). Gemma 4: Frontier multimodal intelligence on device. Technical Report. Google DeepMind. Recuperado de: https://deepmind.google/models/gemma/gemma-4/

[32] Hugging Face. (2025). *SmolLM2: State-of-the-art small language models*. Hugging Face Blog. Recuperado de https://huggingface.co/blog/smollm2.

[33] Mehta, S., Sekhavat, M. H., Cao, Q., Horton, M., Jin, Y., Sun, C., … & Rastegari, M. (2024). OpenELM: An Efficient Language Model Family with Open Training and Inference Framework. Technical Report. Apple Machine Learning Research. Recuperado de: https://machinelearning.apple.com/research/openelm

[34] Microsoft. (2024). *Introducing Phi-3: Redefining what's possible with SLMs*. Microsoft Official Blog. Recuperado de https://blogs.microsoft.com/blog/2024/04/23/introducing-phi-3/.

[35] Microsoft. (2025). Phi-4: Highly Capable Small Language Models. Technical Report. Microsoft Research. Recuperado de: https://azure.microsoft.com/en-us/blog/introducing-phi-4/

[36] Groeneveld, D., Beltagy, I., Walsh, E., Bhagia, A., Kinney, R., Tafjord, O., … & Hajishirzi, H. (2024). OLMo: Accelerating the science of language models. En *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* (pp. 15789-15809).

[37] Qwen Team. (2025). Qwen3 Technical Report & Open Source Repository. *Alibaba Cloud / Hugging Face*. Recuperado de: https://github.com/QwenLM

[38] Aryabumi, V., Dang, J., Talupuru, D., Dash, S., Cairuz, D., Lin, H., … & Hooker, S. (2024). Aya 23: Open Weight Releases to Further Multilingual Progress. Technical Report. Cohere For AI. Recuperado de https://cohere.com/research/aya.

[39] Mistral AI. (2026). *Ministral 3 for edge computing*. Mistral AI News. Recuperado de https://mistral.ai/news/.

[40] Gemma Team, Google. (2025). Gemma 3: Open Models Based on Gemini Research and Technology. Technical Report. Google DeepMind. Recuperado de: https://ai.google.dev/gemma/

[41] Rao, S., & Daumé III, H. (2018, July). Learning to ask good questions: Ranking clarification questions using neural expected value of perfect information. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 2737-2746).

[42] Cai, L., & Zhu, Y. (2015). The challenges of data quality and data quality assessment in the big data era. *Data science journal*, *14*, 2-2.

[43] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., … & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM computing surveys*, *55*(12), 1-38.

[44] Novikova, J., Dušek, O., & Rieser, V. (2017, August). The E2E dataset: New challenges for end-to-end generation. In *Proceedings of the 18th annual SIGdial meeting on discourse and dialogue* (pp. 201-206).

[45] Gardent, C., Shimorina, A., Narayan, S., & Perez-Beltrachini, L. (2017, July). Creating training corpora for NLG micro-planners. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 179-188).

[46] Parikh, A., Wang, X., Gehrmann, S., Faruqui, M., Dhingra, B., Yang, D., & Das, D. (2020, November). ToTTo: A controlled table-to-text generation dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1173-1186).

[47] Nan, L., Radev, D., Zhang, R., Rau, A., Sivaprasad, A., Hsieh, C., … & Rajani, N. F. (2021, June). Dart: Open-domain structured data record to text generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 432-447).

[48] Lebret, R., Grangier, D., & Auli, M. (2016, November). Neural text generation from structured data with application to the biography domain. In *Proceedings of the 2016 conference on empirical methods in natural language processing* (pp. 1203-1213).

[49] Agarwal, O., Ge, H., Shakeri, S., & Al-Rfou, R. (2021, June). Knowledge graph based synthetic corpus generation for knowledge-enhanced language model pre-training. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (pp. 3554-3565).

[50] Rajpurkar, P., Jia, R., & Liang, P. (2018, July). Know what you don't know: Unanswerable questions for SquAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 784-789).

[51] Hermann, K. M., Kocisky, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., & Blunsom, P. (2015). Teaching machines to read and comprehend. *Advances in neural information processing systems*, *28*.

[52] Cohan, A., Dernoncourt, F., Kim, D. S., Bui, T., Kim, S., Chang, W., & Goharian, N. (2018, June). A discourse-aware attention model for abstractive summarization of long documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)* (pp. 615-621).

[53] Lhoest, Q., Del Moral, A. V., Jernite, Y., Thakur, A., Von Platen, P., Patil, S., … & Wolf, T. (2021, November). Datasets: A community library for natural language processing. In *Proceedings of the 2021 conference on empirical methods in natural language processing: system demonstrations* (pp. 175-184).

[54] Jang, E., Gu, S., y Poole, B. (2017). Categorical Reparameterization with Gumbel-Softmax. Proceedings of the 5th International Conference on Learning Representations (ICLR).

[55] Radford, A., Metz, L., & Chintala, S. (2016). Unsupervised representation learning with deep convolutional generative adversarial networks. En *4th International Conference on Learning Representations (ICLR)*.

[56] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. En *International Conference on Machine Learning* (pp. 448-456). PMLR.

[57] Xu, J., Sun, X., Zhang, Z., Zhao, G., & Lin, J. (2019). Understanding and improving layer normalization. *Advances in neural 55ootstrap55g processing systems*, *32*.

[58] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, *1*(8), 9.

[59] Hochreiter, S. (1998). The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2), 107-116.

[60] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), 1929-1958.

[61] Arjovsky, M., & Bottou, L. (2017). Towards Principled Methods for Training Generative Adversarial Networks. En *International Conference on Learning Representations (ICLR)*.

[62] Sennrich, R., Haddow, B., & Birch, A. (2016, August). Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 1715-1725).

[63] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in neural 55ootstrap55g processing systems*, *27*.

[64] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *The Journal of machine Learning research*, *12*, 2825-2830.

[65] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., y Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. En *Proceedings of the 10th International Conference on Learning Representations (ICLR 2022)*.

[66] HuggingFace (2024). PEFT: Parameter-Efficient Fine-Tuning. https://github.com/huggingface/peft

[67] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002, July). Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics* (pp. 311-318).

[68] Banerjee, S., & Lavie, A. (2005, June). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization* (pp. 65-72).

[69] Lin, C. Y. (2004, July). Rouge: A package for automatic evaluation of summaries. In Text summarization branches out (pp. 74-81).

[70] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. In Proceedings of the 8th International Conference on Learning Representations (ICLR).

[71] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019, June). Bert: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers) (pp. 4171-4186).

[72] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., … & Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach. Technical Report. Facebook AI Research.

[73] Lacoste, A., Luccioni, A., Schmidt, V., & Dandres, T. (2019). Quantifying the Carbon Emissions of Machine Learning. *Tackling Climate Change with Machine Learning Workshop at NeurIPS 2019*.

[74] Google. (2024). *Flutter: Build apps for any screen*. Recuperado de: https://flutter.dev

[75] Rousselet, R. (2024). *Provider package (v6.x)*. Pub.dev. Recuperado de: https://pub.dev/packages/provider

[76] SQLite Consortium. (2024). *SQLite*. Recuperado de: https://sqlite.org

[77] Flutter Team. (2024). *Shared_preferences package*. Pub.dev. Recuperado de: https://pub.dev/packages/shared_preferences

[78] ggml-org. (2024). *Llama.cpp: LLM inference in C/C++*. GitHub. Recuperado de: https://github.com/ggml-org/llama.cpp

[79] ggml-org. (2024). *Llama.cpp releases (binarios precompilados)*. GitHub. Recuperado de: https://github.com/ggml-org/llama.cpp/releases

[80] Hickson, I. (2015). *Server-Sent Events*. W3C Recommendation. Recuperado de: https://html.spec.whatwg.org/multipage/server-sent-events.html

[81] fllama package (v0.0.1). Pub.dev. Recuperado de: https://pub.dev/packages/fllama

[82] ggml-org. (2024). *GGUF format specification*. GitHub. Recuperado de: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md

[83] ggml-org. (2024). *Quantization methods in llama.cpp*. GitHub. Recuperado de: https://github.com/ggml-org/llama.cpp/blob/master/examples/quantize/README.md

[84] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). Gpt3. Int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural 56ootstrap56g processing systems*, *35*, 30318-30332.

[85] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *Proceedings of the International Conference on Learning Representations (ICLR 2023)*.

[86] Shapiro, S. S., Wilk, M. B., & Chen, H. J. (1968). A comparative study of various tests for normality. *Journal of the American 57ootstrap57g association*, *63*(324), 1343-1372.

[87] Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the american 57ootstrap57g association*, *32*(200), 675-701.

[88] Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics bulletin*, *1*(6), 80-83.

[89] Cohen, J. (1988). Statistical power analysis for the behavioral sciences, Lawrence Erlbaum Associates. *Hillsdale, NJ*, 20-26

[90] Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research*, *7*(Jan), 1-30.

[91] gpt_markdown package. Pub.dev. Recuperado de: https://pub.dev/packages/gpt_markdown

[92] Russell, S., & Norvig, P. (2010). *Artificial intelligence: A modern approach* (3.ª ed.). Prentice Hall

[93] Turing, A. M. (2009). Computing machinery and intelligence. En *Parsing the Turing Test: Philosophical and methodological issues in the quest for the thinking computer* (pp. 23-65). Springer.

[94] Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer

[95] Mitchell, T. M. (1997). Does machine learning really work? *AI Magazine*, 18(3), 11.

[96] Shyam, R., & Singh, R. (2021). A taxonomy of machine learning techniques. *J. Adv. Robot*, *8*(3), 18-25.

[97] Murphy, K. P. (2012). *Machine learning: a probabilistic perspective*. MIT press.

[98] Emmert-Streib, F., & Dehmer, M. (2022). Taxonomy of machine learning paradigms: A data-centric perspective. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, *12*(5), e1470.

[99] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436-444.

[100] Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). Deep learning (Vol. 1, No. 2, pp. 1-800). Cambridge: MIT press.

[101] Jurafsky, D., & Martin, J. H. (2009). Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition. Pearson/Prentice Hall.

[102] Manning, C. D., & Schütze, H. (1999). Foundations of statistical natural language processing. MIT Press.

[103] Brown, P. F., Della Pietra, V. J., Desouza, P. V., Lai, J. C., & Mercer, R. L. (1992). Class-based n-gram models of natural language. Computational linguistics, 18(4), 467-480.

[104] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. En 1st International Conference on Learning Representations (ICLR).

[105] Allen, C., & Hospedales, T. (2019). Analogies explained: Towards understanding word embeddings. En International Conference on Machine Learning (pp. 223-231). PMLR.

[106] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.

[107] Hugging Face, "How Transformers Work," *Hugging Face NLP Course*, 2021. Recuperado de: https://huggingface.co/learn/nlp-course/chapter1/4.

[108] Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2020). ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. Eighth International Conference on Learning Representations (ICLR).

[109] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., … & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of machine learning research, 21(140), 1-67.

[110] Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., … & Zettlemoyer, L. (2020, July). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In Proceedings of the 58th annual meeting of the association for computational linguistics (pp. 7871-7880).

7 Uso de la IA

Por la presente, yo, Arturo Fierro Pardo, estudiante del Grado en Ingeniería de Sistemas de Telecomunicación de la Universidad CEU San Pablo, al presentar mi Trabajo Fin de Grado titulado "Diseño e implementación de una aplicación generativa de explicación de contenidos de telecomunicaciones de dominio especifico", declaro que he utilizado herramientas de inteligencia artificial generativa durante la elaboración de este trabajo. A continuación, detallo de forma transparente las herramientas empleadas, la finalidad de su uso, las partes afectadas y los controles de calidad aplicados.

Herramientas empleadas:

- **Claude** (Anthropic). Modelos Claude Sonnet 4.6 y Claude Opus 4.5. Uso continuado entre enero de 2025 y junio de 2026.
- **Gemini** (Google). Modelo Gemini Pro. Uso continuado entre enero de 2025 y junio de 2026.

Finalidad y alcance del uso:

- **Depuración y refactorización de código:** Asistencia en la localización y explicación de errores en código Python y Dart/Flutter. Todo el código fue escrito por el estudiante, y las herramientas se emplearon como apoyo para depurar, comprender errores y optimizar el código ya existente.
- **Revisión lingüística y de estilo:** Corrección ortográfica y gramatical del texto de la memoria. La redacción, la estructura argumental, las decisiones sobre qué contenido incluir y el hilo narrativo de cada sección son íntegramente de autoría propia. Las herramientas se utilizaron como herramienta de revisión, no de redacción.
- **Referencias:** Identificar referencias preliminares que luego he contrastado y validado.
- **Imágenes:** Edición de imágenes para el propio documento.

Se estima que entre un 10 % y un 15 % del contenido del presente documento ha sido afectado por herramientas de inteligencia artificial.

ÍNDICE DE FIGURAS DEL ANEXO

FIGURA ANEXO 1: LAS SEIS DIMENSIONES FUNDAMENTALES DE LA INTELIGENCIA ARTIFICIAL. FUENTE PROPIA. 65
FIGURA ANEXO 2: REPRESENTACIÓN VISUAL DEL PARADIGMA DE APRENDIZAJE AUTOMÁTICO PROPUESTO POR TOM MITCHELL. FUENTE PROPIA. 66
FIGURA ANEXO 3: JERARQUÍA DE LOS PRINCIPALES ALGORITMOS DE *MACHINE LEARNING*. ADAPTADO DE SHYAM ET AL. (2021) [96]. 67
FIGURA ANEXO 4: COMPARATIVA DE TAREAS FUNDAMENTALES EN EL APRENDIZAJE SUPERVISADO. FUENTE PROPIA. 68
FIGURA ANEXO 5: REPRESENTACIÓN DE UN PROCESO DE AGRUPAMIENTO (*CLUSTERING*) EN APRENDIZAJE NO SUPERVISADO. FUENTE PROPIA 69
FIGURA ANEXO 6: REPRESENTACIÓN BIDIMENSIONAL DEL APRENDIZAJE SEMI-SUPERVISADO MEDIANTE EL CONJUNTO DE DATOS SINTÉTICO *HALF-MOONS*. FUENTE PROPIA 70
FIGURA ANEXO 7: CICLO DE INTERACCIÓN AGENTE-ENTORNO EN EL PARADIGMA DE APRENDIZAJE POR REFUERZO. FUENTE PROPIA. 70
FIGURA ANEXO 8: TOPOLOGÍA ESTÁNDAR DE UNA RED NEURONAL PROFUNDA (PERCEPTRÓN MULTICAPA). ADAPTADO DE LECUN ET AL. (2015) [99]. 71
FIGURA ANEXO 9: ILUSTRACIÓN DE LA ANALOGÍA LÉXICA "REY – HOMBRE + MUJER = REINA". ADAPTADO DE ALLEN ET AL. (2019) [105]. 73
FIGURA ANEXO 10: ARQUITECTURA DE RED NEURONAL *TRANSFORMER* COMPLETA. ADAPTADO DE VASWANI ET AL. (2017) [1]. 75
FIGURA ANEXO 11: RESULTADOS DEL USO DE HERRAMIENTAS DE IA GENERATIVA DE LOS PARTICIPANTES. FUENTE PROPIA 79
FIGURA ANEXO 12: MAPA VISUAL DEL PLAN DE ESTUDIOS. FUENTE PROPIA. 82
FIGURA ANEXO 13: ÍNDICE JERÁRQUICO DE CONTENIDOS. FUENTE PROPIA. 83
FIGURA ANEXO 14: *DATASET* COMPLETO CON PARES PREGUNTAS-RESPUESTAS. FUENTE PROPIA 83
FIGURA ANEXO 15: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS LLM EN WEBNLG. FUENTE PROPIA .. 87
FIGURA ANEXO 16: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS LLM EN TOTTO. FUENTE PROPIA. 88
FIGURA ANEXO 17: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS LLM EN KELM. FUENTE PROPIA. 89
FIGURA ANEXO 18: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS SLM EN WEBNLG. FUENTE PROPIA... 90
FIGURA ANEXO 19: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS SLM EN TOTTO. FUENTE PROPIA..... 91
FIGURA ANEXO 20: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS SLM EN KELM. FUENTE PROPIA. 92
FIGURA ANEXO 21: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS MINI-SLM EN WEBNLG. FUENTE PROPIA. 93
FIGURA ANEXO 22: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS MINI-SLM EN TOTTO. FUENTE PROPIA. 94
FIGURA ANEXO 23: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS MINI-SLM EN KELM. FUENTE PROPIA. 95

FIGURA ANEXO 24: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS MINI-SLM EN EL CORPUS PROPIO. FUENTE PROPIA.....96
FIGURA ANEXO 25: Q-Q TESTS DE NORMALIDAD DE LOS MODELOS *FINE-TUNED* Y LA GAN EN EL CORPUS PROPIO. FUENTE PROPIA. 97
FIGURA ANEXO 26: BOXPLOTS DE ROUGE-L POR MODELO Y *DATASET*. *FUENTE* PROPIA. 100
FIGURA ANEXO 27: BOXPLOTS DE METEOR POR MODELO Y *DATASET*. *FUENTE* PROPIA. 100
FIGURA ANEXO 28: BOXPLOTS DE BLEU POR MODELO Y DATASET. FUENTE PROPIA. 100
FIGURA ANEXO 29: BOXPLOTS DE BERTSCORE POR MODELO Y DATASET. FUENTE PROPIA.....101
FIGURA ANEXO 30: BOXPLOTS DE COSTE POR MODELO Y DATASET. FUENTE PROPIA. 101
FIGURA ANEXO 31: BOXPLOTS DE CO2 POR MODELO Y DATASET. FUENTE PROPIA.....101
FIGURA ANEXO 32: BOXPLOTS DE TIEMPO DE INFERENCIA POR MODELO Y DATASET. FUENTE PROPIA. 102
FIGURA ANEXO 33: BARPLOST DE COSTE TOTAL POR MODELO Y DATASET. FUENTE PROPIA. 102
FIGURA ANEXO 34: BARPLOTS DE CO2 TOTAL POR MODELO Y DATASET. FUENTE PROPIA.....102
FIGURA ANEXO 35: BARPLOTS DE TIEMPO TOTAL POR MODELO Y DATASET. FUENTE PROPIA.....103
FIGURA ANEXO 36: TIEMPO TOTAL ACUMULADO DE INFERENCIA POR MODELO EN WEBNLG. FUENTE PROPIA. 103
FIGURA ANEXO 37: TIEMPO TOTAL ACUMULADO DE INFERENCIA POR MODELO EN TOTTO. FUENTE PROPIA. 104
FIGURA ANEXO 38: TIEMPO TOTAL ACUMULADO DE INFERENCIA POR MODELO EN KELM. FUENTE PROPIA. 104
FIGURA ANEXO 39: DIAGRAMA DE DIFERENCIA CRÍTICA PARA ROUGE-L EN WEBNLG. FUENTE PROPIA. 118
FIGURA ANEXO 40: DIAGRAMA DE DIFERENCIA CRÍTICA PARA METEOR EN WEBNLG. FUENTE PROPIA. 118
FIGURA ANEXO 41: DIAGRAMA DE DIFERENCIA CRÍTICA PARA BLEU EN WEBNLG. FUENTE PROPIA.....118
FIGURA ANEXO 42: DIAGRAMA DE DIFERENCIA CRÍTICA PARA BERTSCORE EN WEBNLG. FUENTE PROPIA. 119
FIGURA ANEXO 43: DIAGRAMA DE DIFERENCIA CRÍTICA PARA ROUGE-L EN TOTTO. FUENTE PROPIA...119
FIGURA ANEXO 44: DIAGRAMA DE DIFERENCIA CRÍTICA PARA METEOR EN TOTTO. FUENTE PROPIA....119
FIGURA ANEXO 45: DIAGRAMA DE DIFERENCIA CRÍTICA PARA BLEU EN TOTTO. FUENTE PROPIA.....120
FIGURA ANEXO 46: DIAGRAMA DE DIFERENCIA CRÍTICA PARA BERTSCORE EN TOTTO. FUENTE PROPIA. 120
FIGURA ANEXO 47: DIAGRAMA DE DIFERENCIA CRÍTICA PARA ROUGE-L EN KELM. FUENTE PROPIA. ..120
FIGURA ANEXO 48: DIAGRAMA DE DIFERENCIA CRÍTICA PARA METEOR EN KELM. FUENTE PROPIA. ...121
FIGURA ANEXO 49: DIAGRAMA DE DIFERENCIA CRÍTICA PARA BLEU EN KELM. FUENTE PROPIA. 121
FIGURA ANEXO 50: DIAGRAMA DE DIFERENCIA CRÍTICA PARA BERTSCORE EN KELM. FUENTE PROPIA. 121
FIGURA ANEXO 51: BOXPLOTS DE ROUGE-L POR MODELO EN EL CORPUS PROPIO. FUENTE PROPIA. 124
FIGURA ANEXO 52: BOXPLOTS DE METEOR POR MODELO EN EL CORPUS PROPIO. FUENTE PROPIA.....125
FIGURA ANEXO 53: BOXPLOTS DE BLEU POR MODELO EN EL CORPUS PROPIO. FUENTE PROPIA.....125

FIGURA ANEXO 54: BOXPLOTS DE BERTSCORE POR MODELO EN EL CORPUS PROPIO. FUENTE PROPIA. ... 126
FIGURA ANEXO 55: BOXPLOTS DE COSTE POR MODELO EN EL CORPUS PROPIO. FUENTE PROPIA. ... 126
FIGURA ANEXO 56: BOXPLOTS DE CO2 POR MODELO EN EL CORPUS PROPIO. FUENTE PROPIA. ... 127
FIGURA ANEXO 57: BOXPLOTS DE TIEMPO DE INFERENCIA POR MODELO EN EL CORPUS PROPIO. FUENTE PROPIA. ... 127
FIGURA ANEXO 58: BARPLOTS DE COSTE TOTAL DE TODOS LOS MODELOS EN EL CORPUS PROPIO. FUENTE PROPIA. ... 128
FIGURA ANEXO 59: BARPLOTS DE CO2 TOTAL DE TODOS LOS MODELOS EN EL CORPUS PROPIO. FUENTE PROPIA. ... 128
FIGURA ANEXO 60: BARPLOTS DE TIEMPO TOTAL DE TODOS LOS MODELOS EN EL CORPUS PROPIO. FUENTE PROPIA. ... 129
FIGURA ANEXO 61: TIEMPO TOTAL ACUMULADO POR MODELO EN EL CORPUS PROPIO. FUENTE PROPIA... 129
FIGURA ANEXO 62: CURVAS DE APRENDIZAJE DE LA GAN. FUENTE PROPIA. ... 130
FIGURA ANEXO 63: CURVA DE APRENDIZAJE DE GEMMA 3 1B FINE-TUNED. FUENTE PROPIA. ... 130
FIGURA ANEXO 64: CURVA DE APRENDIZAJE DE LLAMA 3.2 1B FINE-TUNED. FUENTE PROPIA. ... 131
FIGURA ANEXO 65: CURVA DE APRENDIZAJE DE QWEN 3 1.7B FINE-TUNED. FUENTE PROPIA. ... 131
FIGURA ANEXO 66: DIAGRAMA DE DIFERENCIA CRÍTICA PARA ROUGE-L EN EL CORPUS PROPIO. FUENTE PROPIA. ... 136
FIGURA ANEXO 67: DIAGRAMA DE DIFERENCIA CRÍTICA PARA METEOR EN EL CORPUS PROPIO. FUENTE PROPIA. ... 136
FIGURA ANEXO 68: DIAGRAMA DE DIFERENCIA CRÍTICA PARA BLEU EN EL CORPUS PROPIO. FUENTE PROPIA. ... 137
FIGURA ANEXO 69: DIAGRAMA DE DIFERENCIA CRÍTICA PARA BERTSCORE EN EL CORPUS PROPIO. FUENTE PROPIA. ... 137
FIGURA ANEXO 70: PANTALLA DE INICIO DE TELECOSLM. FUENTE PROPIA. ... 138
FIGURA ANEXO 71: PANTALLA DE CHAT DE TELECOSLM. FUENTE PROPIA. ... 139
FIGURA ANEXO 72: PANTALLA DE CONFIGURACIÓN DE TELECOSLM. FUENTE PROPIA. ... 140

ÍNDICE DE TABLAS DEL ANEXO

TABLA ANEXO 1: RESULTADOS DEL GRADO DE CONFIANZA DE LOS USUARIOS SOBRE LAS FRASES GENERADAS POR UNA IA GENERATIVA O UN HUMANO. 80
TABLA ANEXO 2: DISTRIBUCIÓN DEL CORPUS POR CURSO ACADÉMICO. 84
TABLA ANEXO 3: EJEMPLOS REPRESENTATIVOS DE PARES PREGUNTA-RESPUESTA DEL *DATASET*. 85
TABLA ANEXO 4: COSTES Y CO2 POR MODELO Y *DATASET*. 98
TABLA ANEXO 5: TIEMPO DE INFERENCIA POR MODELO Y *DATASET*. 98
TABLA ANEXO 6: RESULTADOS DE LAS MÉTRICAS DE CALIDAD POR MODELO Y *DATASET*. 99
TABLA ANEXO 7: RESULTADOS DEL TEST DE FRIEDMAN POR MÉTRICA Y *DATASET*. 105
TABLA ANEXO 8: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA ROUGE-L EN WEBNLG. 106
TABLA ANEXO 9: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA METEOR EN WEBNLG. 107
TABLA ANEXO 10: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA BLEU EN WEBNLG. 108
TABLA ANEXO 11: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA BERTSCORE EN WEBNLG. 109
TABLA ANEXO 12: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA ROUGE-L EN TOTTO. 110
TABLA ANEXO 13: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA METEOR EN TOTTO. 111
TABLA ANEXO 14: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA BLEU EN TOTTO. 112
TABLA ANEXO 15: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA BERTSCORE EN TOTTO. 113
TABLA ANEXO 16: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA ROUGE-L EN KELM. 114
TABLA ANEXO 17: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA METEOR EN KELM. 115
TABLA ANEXO 18: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA BLEU EN KELM. 116
TABLA ANEXO 19: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA BERTSCORE EN KELM. 117
TABLA ANEXO 20: COSTES Y CO2 POR MODELO EN EL CORPUS PROPIO. 122
TABLA ANEXO 21: TIEMPO DE INFERENCIA POR MODELO EN EL CORPUS PROPIO. 122
TABLA ANEXO 22: MÉTRICAS DE CALIDAD POR MODELO EN EL CORPUS PROPIO. 123
TABLA ANEXO 23: RESULTADOS DEL TEST DE FRIEDMAN POR MODELO EN EL CORPUS PROPIO. 124
TABLA ANEXO 24: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA ROUGE-L EN EL CORPUS PROPIO. 132
TABLA ANEXO 25: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA METEOR EN EL CORPUS PROPIO. 133
TABLA ANEXO 26: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA BLEU EN EL CORPUS PROPIO. 134
TABLA ANEXO 27: ANÁLISIS POST-HOC (NEMENYI + WILCOXON R) PARA BERTSCORE EN EL CORPUS PROPIO. 135

Anexo A. Fundamentos teóricos de Inteligencia Artificial y Aprendizaje Profundo

En la presente sección se establece el marco teórico fundamental, estructurando una revisión básica del estado del arte. Se introducen los conceptos de Inteligencia Artificial (IA) y *Machine Learning* (ML), el Procesamiento del Lenguaje Natural (NLP), así como la arquitectura *Transformer*.

A.1 Introducción a la Inteligencia Artificial

En su concepción más fundamental, la Inteligencia Artificial (IA) se define como *la rama de la informática dedicada al diseño y desarrollo de sistemas capaces de ejecutar tareas que, tradicionalmente, requieren de la cognición humana, tales como el aprendizaje, el razonamiento y la percepción* [92]. Sin embargo, debido a la amplitud y constante evolución del campo, acotar una definición universal resulta complejo.

Por ello, para establecer un marco teórico sólido, este trabajo adopta la taxonomía presentada por Stuart Russel y Peter Norvig en su obra referencial *Artificial Intelligence: A Modern Approach* (2010) [92]. Estos autores proponen seis áreas o capacidades operativas (véase Figura Anexo 1). Esta subdivisión se fundamenta en las competencias técnicas que un sistema requeriría de forma simultánea para superar *el Test de Turing* [93]; es decir, para que su comportamiento y capacidad de comunicarse resultase indistinguible de la de un ser humano.



Figura Anexo 1: Las seis dimensiones fundamentales de la inteligencia artificial. Fuente propia.

Bajo este enfoque, la Inteligencia Artificial se identifica como el conjunto de las siguientes seis dimensiones fundamentales:

- El *Procesamiento del Lenguaje Natura*l (NLP) representa la capacidad fundamental de comunicarse, no solo procesando símbolos, sino comprendiendo y generando lenguaje de manera contextual y coherente.
- La *representación del conocimiento* consiste en la habilidad para almacenar información del mundo real en estructuras lógicas que permitan su posterior recuperación y uso.
- El *razonamiento automático* aporta la capacidad de manipular dicho conocimiento para resolver problemas, inferir nuevas conclusiones o ideas.
- El *Aprendizaje Automático (Machine Learning)* define la facultad critica de adquirir nuevas destrezas a partir de la experiencia, adaptándose y mejorando sin programación explicita.
- La *visión por computador* proporciona a la máquina la capacidad de percibir e interpretar el entorno visual.
- La *robótica* supone la integración de la percepción y el razonamiento para interactuar físicamente con el mundo real.

El presente trabajo se enmarca principalmente en la intersección entre el aprendizaje automático y el procesamiento del lenguaje natural.

A.2 Introducción a Machine Learning

El aprendizaje automático o *Machine Learning* (ML) se define como el campo de estudio que dota a los sistemas computacionales de la capacidad de identificar patrones complejos y aprender a partir de la ingesta de datos, sin necesidad de ser programados mediante reglas condicionales explícitas [94].

Para trasladar este concepto abstracto a un modelo evaluable, se adoptó de forma estándar la formalización propuesta por Mitchell et al. (1997) [95]. En lugar de ofrecer una simple descripción, Mitchell modela el aprendizaje como un problema matemático de optimización basado en tres variables fundamentales. Según este postulado, se dice que un programa de ordenador aprende de una experiencia E con respecto a una clase de tareas T y una medida de rendimiento P, si su desempeño en dichas tareas T, evaluado mediante la métrica P, mejora de forma progresiva a medida que adquiere mayor experiencia E (véase Figura Anexo 2).

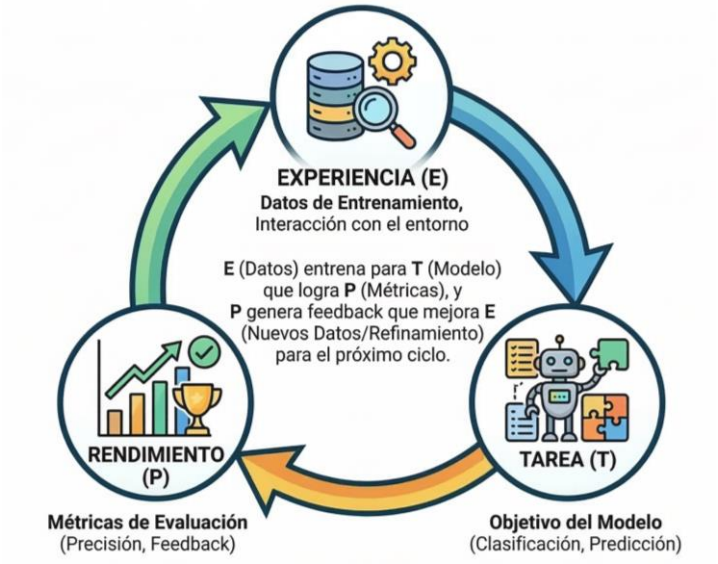


Figura Anexo 2: Representación visual del paradigma de aprendizaje automático propuesto por Tom Mitchell. Fuente propia.

A.2.1 Paradigmas tradicionales de Machine Learning

Para sistematizar su estudio, los distintos paradigmas se clasifican en función de la naturaleza de la señal de aprendizaje o *feedback* disponible durante el entrenamiento. Siguiendo la taxonomía tradicional (véase Figura Anexo 3), el aprendizaje automático

tradicional se articula fundamentalmente en cuatro grandes áreas: aprendizaje supervisado, aprendizaje no supervisado, aprendizaje semi-supervisado y aprendizaje por refuerzo.

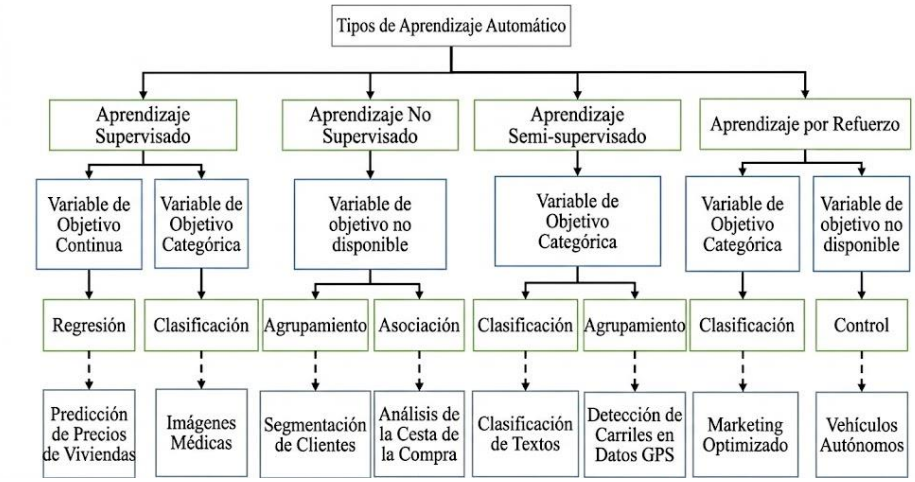


Figura Anexo 3: Jerarquía de los principales algoritmos de *Machine Learning*. Adaptado de Shyam et al. (2021) [96].

Aprendizaje Supervisado

El aprendizaje supervisado se fundamenta en la inducción de una función a partir de un conjunto de datos de entrenamiento previamente etiquetado, compuesto por pares de entrada y salida esperada. Las tareas fundamentales dentro de este paradigma son la regresión (predicción de valores continuos) y la clasificación (asignación de categorías discretas).

Como se ilustra en la Figura Anexo 4, ambas tareas abordan el problema de manera distinta empleando modelos diferentes. En la regresión (Figura Anexo 4ª y Figura Anexo 4b), se utilizan algoritmos como la regresión lineal o polinómica para generar una función matemática que se ajusta y sigue la tendencia continua de los datos. En las tareas de clasificación (Figura Anexo 4c y Figura Anexo 4d), se emplean modelos como la regresión logística para calcular una frontera de decisión cuyo objetivo es separar el espacio muestral y diferenciar las distintas clases.

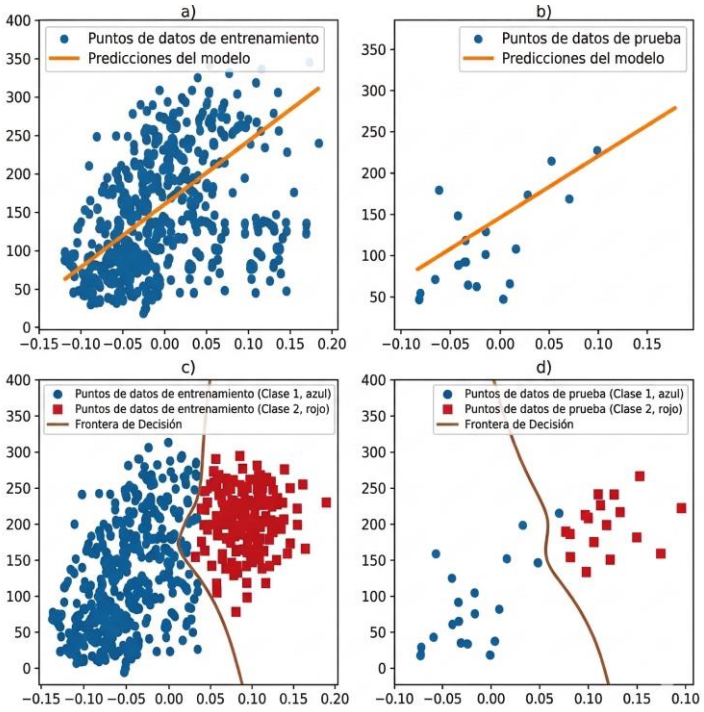


Figura Anexo 4: Comparativa de tareas fundamentales en el aprendizaje supervisado. Fuente propia.

Desde un punto de vista formal y matricial, el aprendizaje supervisado se modela operando con un cuerpo de datos de entrada representado por la matriz $\boldsymbol{X}$ y un vector de categorías o valores reales de salida $\boldsymbol{Y}$. El proceso de entrenamiento consiste en iterar hasta encontrar una matriz de pesos $\boldsymbol{W}$ óptima, tal que la combinación lineal de los datos logre la siguiente equivalencia [97]:

$$\boldsymbol{X} \cdot \boldsymbol{W} = \widehat{\boldsymbol{Y}} \tag{10}$$

El objetivo del algoritmo es que la predicción generada $\widehat{\boldsymbol{Y}}$ sea lo más equivalente posible a la salida real esperada $\boldsymbol{Y}$, minimizando para ello el error empírico mediante una función de coste o pérdida [97].

Aprendizaje No Supervisado

El aprendizaje no supervisado opera sobre conjuntos de datos carentes de etiquetas explícitas. Su objetivo no es predecir una salida conocida, sino descubrir la estructura subyacente, la distribución o los patrones ocultos inherentes en los datos [98].

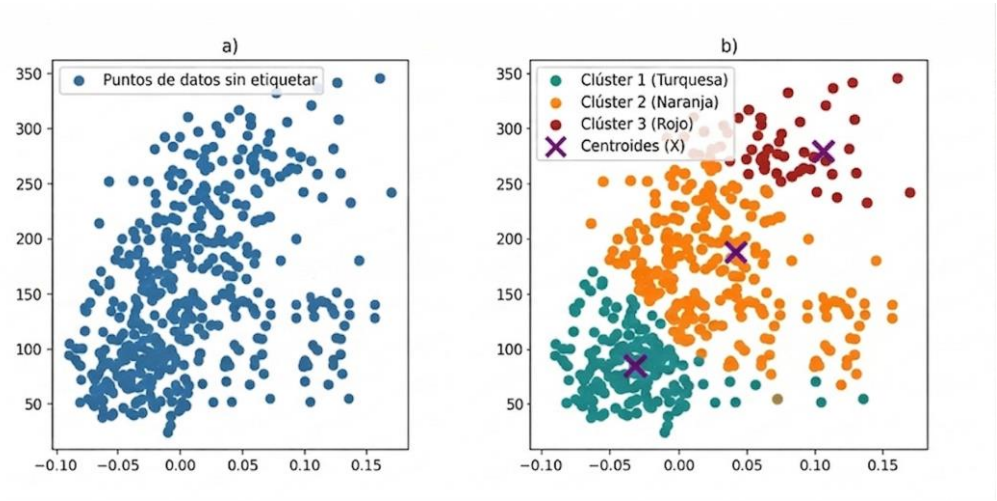


Figura Anexo 5: Representación de un proceso de agrupamiento (*clustering*) en aprendizaje no supervisado. Fuente propia.

La tarea más representativa de este enfoque es el agrupamiento o *clustering*. Como se ilustra en la Figura Anexo 5ª, el modelo recibe una distribución de puntos sin categorizar y, de forma autónoma, evalúa la similitud entre ellos para dividirlos en subconjuntos homogéneos (véase Figura Anexo 5b). Las marcas en forma de equis ('x') representan los centroides, es decir, el centro de masa matemático calculador por el algoritmo para definir el núcleo de cada grupo.

Aprendizaje Semi-supervisado

Este paradigma no solo aborda el problema práctico y económico de la escasez de datos etiquetados en la industria, sino que se emplea de forma estratégica para identificar patrones estructurales inherentes en grandes volúmenes de datos no anotados. Tomando como ejemplo el conjunto de datos sintético *Half-Moons* (véase Figura Anexo 6), el algoritmo extrae el conocimiento directivo de una pequeña muestra etiquetada (véase Figura Anexo 6ª) y lo propaga aprovechando la forma topológica del conjunto de datos sin etiquetar (véase Figura Anexo 6b) [98]. Matemáticamente, esto se formula optimizando una función de pérdida total ($\mathcal{L}$) que combina la pérdida supervisada ($\mathcal{L}_s$) con un término de regularización basado en la estructura de la parte no supervisada ($\mathcal{L}_u$), habitualmente ponderado por (λ) [98]:

$$\mathcal{L} = \mathcal{L}_s + \lambda \mathcal{L}_u \qquad (11)$$

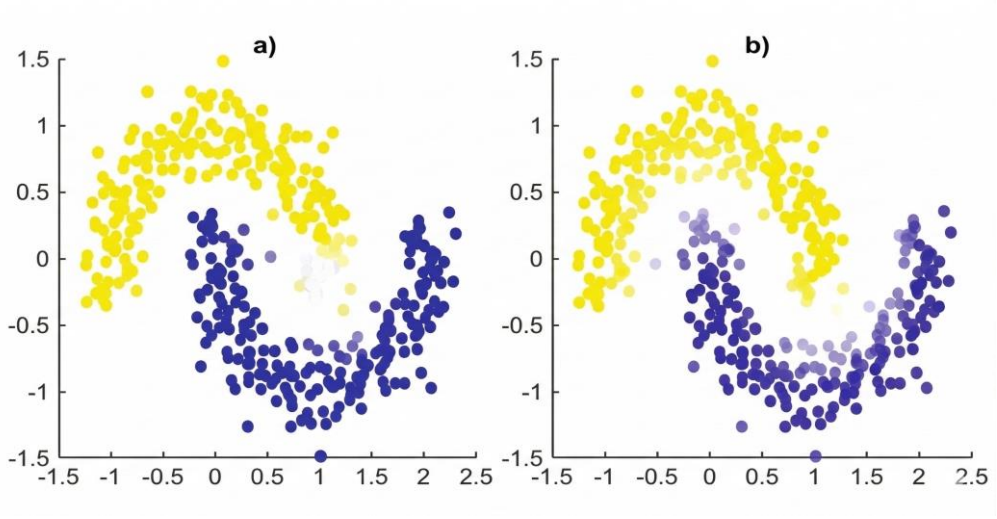


Figura Anexo 6: Representación bidimensional del aprendizaje semi-supervisado mediante el conjunto de datos sintético *Half-Moons*. Fuente propia

Aprendizaje por refuerzo

El aprendizaje por refuerzo abandona la dependencia de un cuerpo de datos estático predefinido. En este paradigma, un agente autónomo aprende a tomar secuencias de decisiones interactuando directamente con un entorno dinámico. La señal de aprendizaje es una métrica de retroalimentación evaluativa (recompensa o penalización) que el agente recibe tras ejecutar una acción en un estado determinado (véase Figura Anexo 7). Este problema se formaliza habitualmente mediante los Procesos de Decisión de Markov (MDP, por sus siglas en inglés *Markov Decision Process*) [19], cuyo objetivo matemático es descubrir una política de actuación que maximice la recompensa acumulada a lo largo del tiempo.

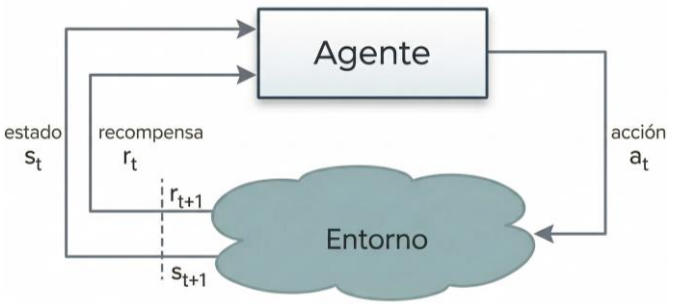


Figura Anexo 7: Ciclo de interacción Agente-Entorno en el paradigma de aprendizaje por refuerzo. Fuente propia.

A.2.2 Redes Neuronales Artificiales y Deep Learning

A pesar del indudable éxito empírico de los paradigmas de aprendizaje automático tradicional descritos anteriormente, estas arquitecturas presentan una limitación inherente cuando se enfrentan a datos no estructurados de alta dimensionalidad, como el procesamiento de imágenes o texto en lenguaje natural. Su rendimiento depende críticamente de un proceso previo de ingeniería de características (*feature 71ootstrap71g*), donde un experto humano debe extraer, limpiar y definir matemáticamente los atributos relevantes del conjunto de datos antes de poder entrenar el modelo [99].

Para superar este cuello de botella representacional, se consolida el aprendizaje profundo (*Deep Learning*). Técnicamente, el *Deep Learning* no constituye un paradigma aislado, sino que es la evolución y aplicación directa de las redes neuronales artificiales (ANN, por sus siglas en inglés, *Artificial Neural Networks*) caracterizadas por disponer de múltiples capas ocultas. Es precisamente esta profundidad arquitectónica (formada por unidades de procesamiento interconectadas o neuronas artificiales) la que permite al modelo procesar los datos en bruto y aprender jerarquías de características de forma completamente automática, eliminando así la necesidad de intervención humana para la extracción de atributos.

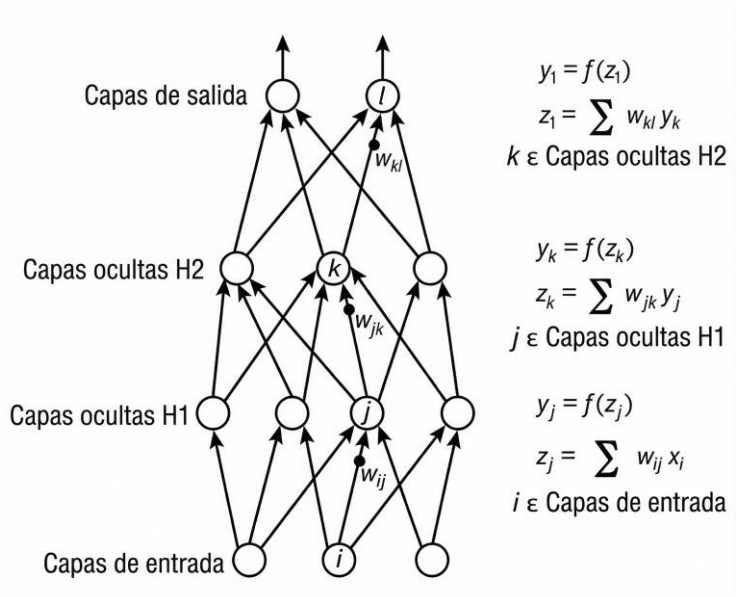


Figura Anexo 8: Topología estándar de una Red Neuronal Profunda (Perceptrón Multicapa). Adaptado de LeCun et al. (2015) [99].

Matemáticamente, el modelo computacional de una neurona artificial estándar recibe un vector de señales de entrada x, aplica un vector de pesos sinápticos w, suma un término de sesgo (bias) b, y proyecta el resultado a través de una función de activación no lineal f (como ReLU, sigmoide o tanh), produciendo una señal de salida y [99]:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{12}$$

La verdadera capacidad del *Deep Learning* reside en la estructuración de estas neuronas en múltiples capas superpuestas. Una arquitectura fundamental, conocida como Perceptrón Multicapa (MLP, por sus siglas en inglés, *Multilayer Perceptron*), consta de una capa de entrada, una o varias capas ocultas (*hidden layers*) y una capa de salida (véase Figura Anexo 8).

La profundidad de la red (definida por el número de capas ocultas) es lo que permite al modelo realizar un aprendizaje jerárquico de características (*Hierarchical Feature Learning*). En lugar de depender de descriptores diseñados manualmente, la red neuronal descubre de forma autónoma las representaciones óptimas de los datos en distintos niveles de abstracción durante el proceso de entrenamiento. Para lograrlo, ajusta de forma iterativa un conjunto parámetros de alta dimensionalidad con el objetivo de encontrar los mínimos de una función de coste o error (*loss function*). Este proceso de búsqueda de mínimos se lleva a cabo mediante algoritmos de optimización (siendo el descenso del gradiente, *gradient descent*, es el más extendido en el contexto de redes neuronales) combinados con el algoritmo de retropropagación del error (*backpropagation*) [100].

A.3 Introducción a Natural Language Processing (NLP)

El Procesamiento del Lenguaje Natural (NLP) es la rama de la inteligencia artificial dedicada al diseño de algoritmos que permitan a las máquinas comprender, interpretar y generar el lenguaje humano [101]. El propósito de esta área radica en lograr que los sistemas informáticos atribuyan el valor semántico adecuado a las secuencias lingüísticas, modelando fielmente el contexto, la intención y la ambigüedad inherente al discurso humano [102].

Históricamente, predominaron los enfoques basados en reglas gramaticales y diccionarios ontológicos (que definen las relaciones semánticas entre conceptos, como la relación entre perro y mamífero) programados manualmente, un paradigma conocido como NLP simbólico. Estos sistemas demostraron una baja capacidad de generalización ante la fluidez y complejidad del lenguaje real [101]. Posteriormente, surgió el NLP estadístico, basado en modelos de probabilidad sobre grandes corpus de texto (como N-gramas [103] o modelos de Markov [19]), que mejoraron la robustez, pero carecían de una representación semántica profunda.

La reconfiguración estructural definitiva del NLP surge con la implementación de redes neuronales, que permitió transitar de la manipulación de símbolos discretos a la representación continua mediante vectores densos de baja dimensionalidad [104]. Este cambio conceptual marcó un punto de inflexión en el campo, impulsado finalmente por los modelos de representación vectorial del lenguaje o *73oot-embeddings*, siendo *Word2Vec* [104] el trabajo fundamental que catalizó esta revolución tecnológica.

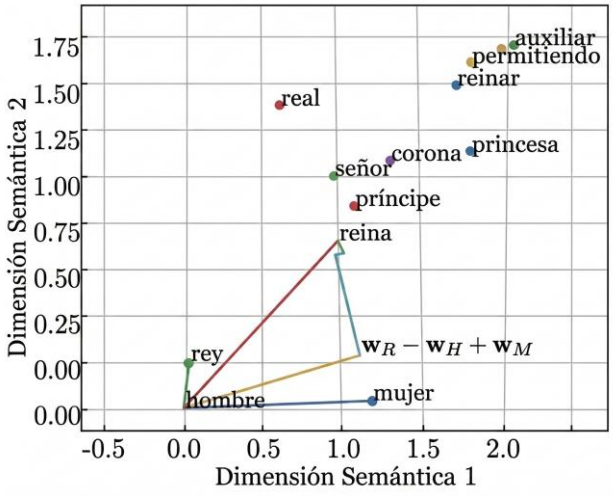


Figura Anexo 9: Ilustración de la analogía léxica "rey – hombre + mujer = reina". Adaptado de Allen et al. (2019) [105].

Bajo este nuevo enfoque, ilustrado en la Figura Anexo 9, las palabras se transforman en coordenadas matemáticas dentro de un espacio multidimensional, de modo que la similitud semántica del lenguaje humano queda reflejada en distancias euclídeas. Por ejemplo, al restar el vector "hombre" al vector "rey" y sumarle el vector "mujer", el vector resultante se proyecta sobre la coordenada correspondiente a la palabra "reina" [105]**.** Esta capacidad de cartografiar sustenta a los modelos generativos actuales.

A.4 Introducción a la arquitectura Transformer

Partiendo de las bases del NLP explicadas en el Anexo A.3, es importante entender por qué la inteligencia artificial necesitó dar un salto tecnológico tan brusco hacia nuevas arquitecturas. Los modelos clásicos (como las Redes Neuronales Recurrentes) tenían un gran obstáculo, procesaban el texto de forma secuencial, es decir, palabra por palabra y en estricto orden.

Esta forma de procesar la información suponía un problema por dos motivos principales. En primer lugar, al leer textos muy largos, el modelo acababa olvidando el contexto del principio de la frase cuando llegaba al final. En segundo lugar, obligaba a los ordenadores a trabajar muy despacio, impidiendo que las tarjetas gráficas (GPU) pudieran hacer varios cálculos a la vez.

Hacía falta crear un sistema nuevo que no leyera palabra a palabra, sino que pudiera procesar párrafos enteros de golpe para no perder el significado global. Esta necesidad de rapidez y buena memoria fue lo que impulsó a crear la arquitectura *Transformer*.

A.4.1 Transformers

La arquitectura *Transformer* es una red neuronal artificial presentada en 2017 en el artículo "*Attention Is All You Need*" [1]. Se fundamenta en mecanismos de atención, en los cuales el texto se convierte en representaciones numéricas llamadas *tokens*, y cada *token* se mapea a un vector mediante una tabla de *embeddings*.

Anteriormente, el NLP dependía casi exclusivamente en arquitecturas basadas en redes neuronales recurrentes (RNN, por sus siglas en inglés, *Recurrent Neural Networks*) y sus variantes con memoria a corto-largo plazo (LSTM, por sus siglas en inglés, *Long Short-Term Memory*) [106]. Desde un punto de vista computacional, estas arquitecturas presentaban un cuello de botella estructural derivado del procesamiento estrictamente secuencial de la información. La arquitectura *Transformer* [1] prescinde por completo de la recurrencia y las convoluciones. En su lugar, el modelo se fundamenta en un mecanismo de atención escalada por producto escalar (*Scaled Dot-Product Self-Attention*). Este mecanismo calcula la afinidad entre palabras utilizando tres abstracciones

vectoriales para cada token: una consulta (*Query*), una clave (*Key*) y un valor (*Value*). El sistema calcula el producto escalar de la *Query* de cada token contra las *Keys* de todos los demás tokens de la secuencia. El resultado se escala y se normaliza mediante una función *softmax* para obtener los pesos de atención, que se multiplican finalmente por el *Value* de cada token.

Este mecanismo permite que cada token calcule su grado de relevancia respecto a todos los demás, extrayendo el contexto global de la frase en un solo paso computacional.

Como se muestra en la Figura Anexo 10, la arquitectura canónica se divide en dos grandes bloques de procesamiento profundo interconectados: un codificador (*Encoder*) y un decodificador (*Decoder*). A nivel interno, ambos bloques están formados por la concatenación de múltiples capas idénticas, compuestas por submódulos de atención multicabezal (*Multi-Head Attention*) y redes de avance directo (*Feed Forward*), estabilizados mediante conexiones residuales y normalización de capa (*Add & Norm*) [1].

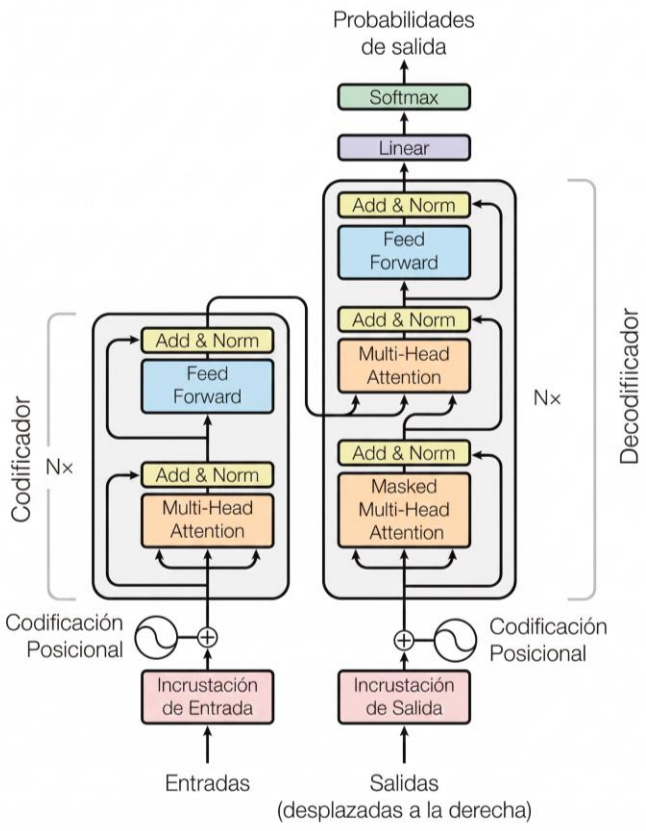


Figura Anexo 10: Arquitectura de red neuronal *Transformer* completa. Adaptado de Vaswani et al. (2017) [1].

El Codificador (*Encoder*) actúa como un extractor de características bidireccional. Su objetivo es analizar la secuencia de entrada en su totalidad (procesando todos los tokens de forma simultánea) para extraer su significado semántico y las relaciones sintácticas subyacentes. Formalmente, su objetivo matemático es mapear la secuencia discreta de entrada $\boldsymbol{x} = (x_1, \dots, x_n)$ a una secuencia continua de representaciones densas en un espacio latente, denotada como $\boldsymbol{z} = (z_1, \dots, z_n)$.

Por su parte, el Decodificador (*Decoder*) constituye el módulo de generación. Su función es producir la secuencia de salida $\boldsymbol{y} = (y_1, \dots, y_m)$ condicionada a los símbolos generados previamente (el contexto extraído por el Codificador). Para preservar esta propiedad causal durante el entrenamiento en paralelo, el primer sub-bloque de autoatención del Decodificador incorpora una máscara causal (*Masked Multi-Head Attention*): se aplica una matriz triangular superior de valores $-\infty$ antes de la función *softmax*, garantizando que la predicción de la posición i dependa exclusivamente de las posiciones previas (de 1 a i-1), evitand que el modelo acceda a información futura ("vea el futuro") de la secuencia objetivo $\boldsymbol{y}$.

Para integrar la información de la secuencia de entrada en el proceso generativo, el Decodificador incorpora un tercer sub-bloque crucial intermedio: la Atención Cruzada Codificador-Decodificador (*Encoder-Decoder Cross-Attention*). En esta capa, las consultas ($\mathbf{Q}$) provienen del bloque de atención anterior del Decodificador, mientras que las claves ($\boldsymbol{K}$) y los valores ($\mathbf{V}$) se obtienen directamente de la salida del Codificador. Este flujo de información se define extendiendo la ecuación base de atención [1]:

$$CrossAttention(\mathbf{Q}_{dec}, \mathbf{K}_{enc}, \mathbf{V}_{enc}) = softmax\left(\frac{\mathbf{Q}_{dec}\mathbf{K}_{enc}^{T}}{\sqrt[2]{d_k}}\right)\mathbf{V}_{enc} \qquad (13)$$

A.4.2 Taxonomía de los modelos basados en Transformers

Aunque la arquitectura original de los *Transformers* acoplaba ambos bloques (*Encoder-Decoder*), la evolución del campo ha demostrado que estos módulos pueden separarse funcionalmente para resolver distintos problemas, dando lugar a tres grandes familias arquitectónicas en la industria actual [107]:

Modelos Encoder-only

Conservan únicamente el bloque bidireccional. Al tener acceso a todo el contexto simultáneamente, destacan para tareas de comprensión del lenguaje natural (NLU, por sus siglas en inglés, *Natural Language Understanding*) como la clasificación de textos, análisis de sentimiento o reconocimiento de entidades nombradas (NER) [107]. La familia *BERT* de *Google* [71] y *ALBERT* [108] son modelos *Encoder-only*.

Modelos Decoder-only

Prescinden del codificador y utilizan únicamente bloques de atención enmascarados de forma autorregresiva. Constituyen el estándar dominante en la industria para tareas de generación de texto y son la base de los actuales Grandes Modelos de Lenguaje (LLM). Sus principales aplicaciones abarcan la generación de código, redacción de ensayos o creación de asistentes conversacionales (*Chatbots*) [107]. La familia *GPT* de *OpenAI* [23], *DeepSeek* [26], *Llama* de *Meta* [27] y *Qwen* [29] fundamentan sus arquitecturas de razonamiento en este paradigma.

Modelos Encoder-Decoder

Mantienen la arquitectura dual original de tipo secuencia a secuencia (*Seq2Seq*). Son especialmente adecuados para tareas que requieren un mapeo complejo entre una secuencia de entrada completa y una de salida tales como la traducción automática entre idiomas o generación de resúmenes a partir de textos extensos (*Data-to-Text*) [107]. Los modelos T5 (*Text-to-Text Transfer Transformer*) [109] y *BART* [110] son modelos destacados de esta categoría.

Anexo B. Encuesta de fiabilidad de texto generado por IA

Con el fin de evaluar de forma empírica la calidad y naturalidad de los textos generados automáticamente por el sistema desarrollado, se diseñó una encuesta de percepción humana. El objetivo principal es determinar en qué medida los participantes son capaces de distinguir entre oraciones generadas por un modelo de IA y oraciones redactadas por humanos, así como la confianza con la que realizan dicha distinción.

La encuesta se estructura en cinco secciones: (1) recogida de datos sociodemográficos, (2) frecuencia de uso de herramientas de IA generativa, (3) instrucciones, (4) bloque de valoración de oraciones y (5) opinión final. El diseño sigue un esquema de escala Likert de 5 puntos (1 = totalmente en desacuerdo, 5 = totalmente de acuerdo) para capturar tanto la decisión binaria del participante (IA vs. Humano) como su nivel de certeza subjetiva.

Consideraciones éticas y de privacidad

La encuesta fue diseñada con carácter completamente anónimo. Ningún dato recogido permite identificar a los participantes de forma directa o indirecta. El tratamiento de los datos se realizó en cumplimiento del Reglamento General de Protección de Datos (UE) 2016/679 y de la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales.

B.1 Descripción de la muestra

La encuesta fue cumplimentada por un total de 220 participantes. La distribución por nivel de estudios muestra una muestra heterogénea: el grupo más numeroso correspondió a estudios de posgrado de tipo máster o equivalente (30,0 %, n = 66), seguido de grado universitario en otras ramas (19,1 %, n = 42), ingeniería (17,3 %, n = 38), ciencias de la salud (7,7 %, n = 17) y artes (6,8 %, n = 15). El 6,4 % contaba con estudios de doctorado (n = 14), el 6,4 % con bachillerato (n = 14), el 3,2 % con formación profesional (n = 7) y el 3,2 % con ESO (n = 7).

En cuanto al rango de edad, el grupo más representado fue el de 61–80 años (31,4 %, n = 69), seguido del de 18–25 años (28,6 %, n = 63), 41–60 años (24,1 %, n = 53) y 26–40 años (10,0 %, n = 22). Los menores de 18 años supusieron el 4,5 % (n = 10) y los mayores de 80 el 1,4 % (n = 3).

Respecto al uso de herramientas de IA generativa, el 86,4 % de los participantes afirmó utilizarlas (n = 190), frente al 13,6 % que indicó no hacerlo (n = 30). Entre los usuarios, el 45,3 % las empleaba diariamente, el 25,3 % entre 3 y 5 veces por semana, el 20,0 % entre 1 y 2 veces por semana, y el 9,5 % menos de una vez por semana (véase Figura Anexo 11).

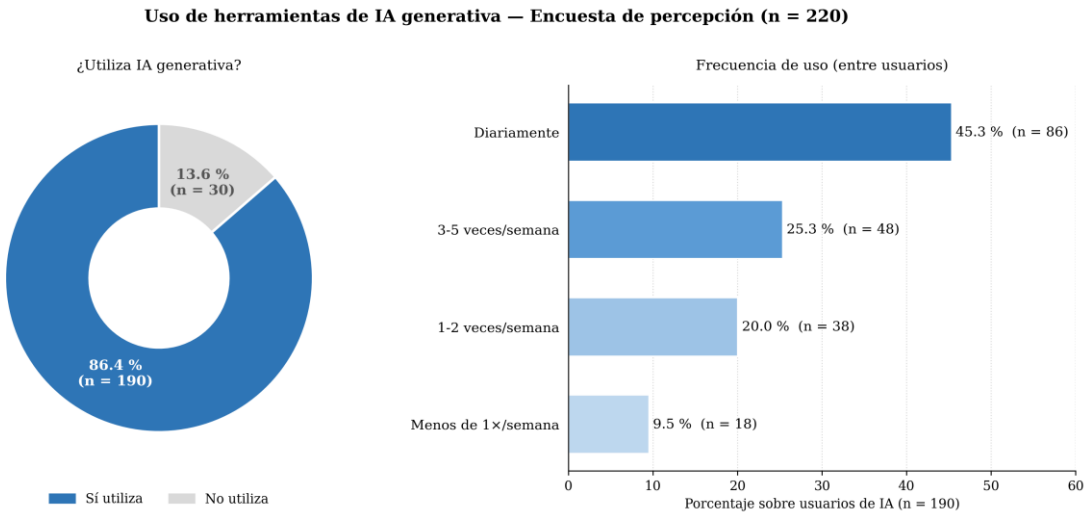


Figura Anexo 11: Resultados del uso de herramientas de IA generativa de los participantes. Fuente propia

La Tabla Anexo 1 recoge, para cada uno de los 20 ítems de la encuesta, la distribución de frecuencias de respuesta, la media y desviación típica de la puntuación de confianza declarada, el porcentaje de respuestas de alta confianza (puntuaciones 4 o 5).

Tabla Anexo 1: Resultados del grado de confianza de los usuarios sobre las frases generadas por una IA generativa o un humano.

Ítem	Oración (resumen)	1	2	3	4	5	Media	SD	% Conf. Alta (≥ 4)	Autoría real
Grado de confianza de si se encuentra la frase escrita por una IA generativa										
1	Monumento 14.º Regimiento NJ	52	41	62	39	26	2,75	1,32	29,5 %	IA
2	Amarillo, Texas, nativos americanos	48	43	50	51	28	2,85	1,34	35,9 %	IA
3	Baked Alaska, postre, Nueva York	54	50	58	34	24	2,65	1,30	26,4 %	IA
4	Jorge H. Rodríguez, manager selección	55	43	43	56	23	2,77	1,35	35,9 %	IA
5	AIDAluna, eslora 252.000 mm	42	37	53	36	52	3,09	1,43	40,0 %	Humano
Grado de confianza de si se encuentra la frase escrita por un humano										
6	ALCORS-3, motor V12	54	31	34	65	36	2,99	1,44	45,9 %	Humano
7	Universidad de Aarhus, 1928	43	40	45	60	32	2,99	1,35	41,8 %	Humano
8	Cornell, Admin. Science Quarterly	50	36	65	46	23	2,80	1,29	31,4 %	Humano
9	Abel Caballero, Juan Carlos I	57	38	37	46	42	2,90	1,48	40,0 %	Humano
10	Aeropuerto Atlantic City, pista 13/31	56	39	58	42	25	2,73	1,33	30,5 %	Humano
Grado de confianza de si se encuentra la frase escrita por una IA generativa										
11	Alan Shepard, Apolo 14	48	41	51	52	28	2,87	1,34	36,4 %	IA
12	Ariane 5, plataforma ELA-3	38	34	54	53	41	3,11	1,35	42,7 %	Humano
13	Baymax, Big Hero 6	37	44	51	57	31	3,00	1,31	40,0 %	Humano
14	Abdulsalami Abubakar, Armada nigeriana	36	39	48	58	39	3,11	1,34	44,1 %	IA
15	Ayam penyet, gastronomía banyumasan	47	43	47	56	27	2,88	1,34	37,7 %	IA
Grado de confianza de si se encuentra la frase escrita por un humano										
16	Batalla de Antietam, Lee y Daggett	47	35	51	45	42	3,00	1,41	39,5 %	IA
17	Acura TLX, motor 3,5 l	53	34	51	47	35	2,90	1,40	37,3 %	Humana
18	Buzz Aldrin, Apolo 11	37	36	50	56	41	3,13	1,35	44,1 %	Humana
19	Michele Marcolini, Lumezzane	49	30	56	61	24	2,91	1,32	38,6 %	IA
20	Putrajaya, capital de Malasia	39	29	54	62	36	3,12	1,33	44,5 %	IA

La puntuación media global de confianza fue de 2,93 (SD = 1,36), situada prácticamente en el punto central de la escala. El porcentaje de respuestas de alta confianza (puntuaciones 4 o 5) fue del 38,1 % en promedio, lo que indica que la mayoría de los participantes no se encontraba seguro de su decisión en ningún ítem.

Este resultado es coherente con la hipótesis de que los textos generados automáticamente presentan un alto grado de naturalidad perceptiva, siendo difícilmente distinguibles de los textos de autoría humana sin conocimiento previo de la fuente.

Los ítems que generaron mayor confianza fueron el ítem 20 (*Putrajaya, capital de Malasia*, 44,5 %), el ítem 18 (*Buzz Aldrin, Apolo 11*, 44,1 %) y el ítem 14 (*Abdulsalami Abubakar*, 44,1 %). En el extremo opuesto, los de menor confianza declarada fueron el ítem 3 (*Baked Alaska*, 26,4 %), el ítem 1 (*Monumento 14.º Regimiento*, 29,5 %) y el ítem 10 (*Aeropuerto Atlantic City*, 30,5 %).

Anexo C. Corpus propio de telecomunicaciones

El archivo Excel consta de tres hojas, cada una con una función distinta dentro del proceso de construcción del *dataset*.

La Hoja 1 (mapa visual del plan de estudios) presenta las 33 asignaturas del grado organizadas por curso en una disposición matricial. Cada columna corresponde a una asignatura y las filas listan sus temas (véase Figura Anexo 12). Esta hoja sirvió como referencia durante la fase de diseño para verificar la cobertura completa del temario y detectar posibles solapamientos entre asignaturas.

Curso 1	
Informática para la ingeniería	**Programación I**
Arquitectura de computadores	Introducción a la algoritmia
Sistemas Operativos y virtualización	Introducción a la programación I
Redes de ordenadores	Estructuras de control de flujo
Herramientas de programación	Prototipos y definiciones de funciones
	Introducción a la programación II
Curso 2	
Campos y ondas	**Señales aleatorias**
Análisis vectorial	Probabilidad
Campos variables con el tiempo y ecuaciones de Maxwell	Variables aleatorias
Ondas electromagnéticas planas	Múltiples variables aleatorias
Lineas de transmisión	Procesos estocásticos
Guías de onda y cavidades resonantes	

Figura Anexo 12: Mapa visual del plan de estudios. Fuente propia.

La Hoja 2 (índice jerárquico de contenidos) estructura los contenidos en cuatro niveles, curso, asignatura, tema y contenido específico (subtema). Cada asignatura incluye un contador del total de contenidos que agrupa (véase Figura Anexo 13). El archivo contiene 1.234 subtemas repartidos entre 243 temas. Esta hoja permite navegar la taxonomía completa del *dataset* sin necesidad de recorrer los miles de filas de preguntas.

Curso	Asignatura	Total temas
Curso 1	Informática para la Ingeniería	Total: 18 contenidos
Curso 1	Programación I	Total: 35 contenidos
Curso 1	Programación II	Total: 35 contenidos
Curso 1	Introducción a la Ingeniería de Telecomunicación	Total: 40 contenidos
Curso 1	Circuitos Eléctricos y Electrónicos	Total: 79 contenidos
Curso 1	Física	Total: 22 contenidos
Curso 1	Fundamentos Matemáticos de la Ingeniería de Telecomunicación I (Álgebra)	Total: 31 contenidos
Curso 1	Fundamentos Matemáticos de la Ingeniería de Telecomunicación II (Cálculo)	Total: 51 contenidos
Curso 2	Campos y ondas	Total: 34 contenidos
Curso 2	Redes de comunicaciones	Total: 40 contenidos
Curso 2	Señales Aleatorias	Total: 24 contenidos
Curso 2	Sistemas Lineales	Total: 35 contenidos
Curso 2	Sistemas Operativos	Total: 41 contenidos
Curso 2	Comunicaciones Analógicas	Total: 23 contenidos
Curso 2	Electrónica I	Total: 36 contenidos
Curso 2	Redes de Empresa	Total: 22 contenidos
Curso 2	Sistemas de Telecomunicación	Total: 35 contenidos

Figura Anexo 13: Índice jerárquico de contenidos. Fuente propia.

La Hoja 3 (*dataset* completo con pares pregunta-respuesta) contiene las 12.340 filas de contenido, organizadas en nueve columnas, curso, asignatura, contenido tema, contenido específico, total, pregunta teórica, respuesta teórica, pregunta práctica y respuesta práctica (véase Figura Anexo 14). Cada fila agrupa una pregunta teórica y una práctica sobre el mismo subtema, lo que facilita la revisión comparada de ambos enfoques. Cada subtema tiene 20 filas de pares pregunta-respuesta, lo que da lugar a 10 preguntas teóricas y 10 prácticas por subtema.

Pregunta Teórica	Respuesta Teórica
¿Qué es la CPU y cuál es su función principal en un ordenador?	La CPU (Unidad Central de Procesamiento) es el componente que ejecuta instrucciones de programas mediante operaciones aritméticas, lógicas y de control. Actúa como el cerebro del ordenador, coordinando todas las operaciones del sistema. Interpreta y procesa datos procedentes de la memoria y dispositivos de entrada/salida.
Pregunta Práctica	**Respuesta Práctica**
Un ingeniero diseña un sistema embebido de telecomunicaciones con restricciones de consumo energético. ¿Qué tipo de memoria debería priorizar y por qué?	Debería priorizar memoria Flash o EEPROM sobre RAM para almacenamiento permanente, ya que son no volátiles y no requieren energía para retener datos. Para memoria de trabajo, SRAM de bajo consumo es preferible a DRAM porque no necesita refrescos constantes. Implementar modos de bajo consumo y gestión dinámica de frecuencia también optimiza el uso energético.

Figura Anexo 14: *Dataset* completo con pares preguntas-respuestas. Fuente propia

El *dataset* cubre las 33 asignaturas del grado, organizadas en 243 temas y 1.234 subtemas. Como se observa en la Tabla Anexo 2, la distribución por curso no es uniforme, ya que los primeros cursos concentran más asignaturas de formación básica.

Tabla Anexo 2: Distribución del corpus por curso académico.

Curso	Asignaturas	Temas	Subtemas	Pares pregunta-respuesta
Curso 1	8	60	308	6.220
Curso 2	9	55	290	5.800
Curso 3	7	65	276	5.520
Curso 4	5	34	208	4.160
Optativas	4	28	149	2.980
Total	**33**	**243**	**1.234**	**24.680**

Para alimentar los *pipelines* de entrenamiento e inferencia, los datos de la Hoja 3 se exportaron a un archivo CSV con una estructura aplanada. Cada fila del Excel, que contenía un par teórico y un par práctico, se desdobla en dos filas independientes en el CSV. De las 12.340 filas del Excel resultan las 24.680 filas del archivo CSV. La elección de CSV frente a otros formatos (JSON) responde a la compatibilidad directa con las librerías `datasets` y `pandas` y a la facilidad de inspección manual con cualquier editor de texto.

La Tabla Anexo 3 muestra una selección de pares preguntas-respuesta del *dataset* de distintos cursos, asignaturas y tipos.

Tabla Anexo 3: Ejemplos representativos de pares pregunta-respuesta del *dataset*.

Curso	Asignatura	Tipo	Pregunta	Respuesta (extracto)
Curso 1	Informática para la Ingeniería	Práctica	Un ingeniero diseña un sistema embebido con restricciones de consumo energético. ¿Qué tipo de memoria debería priorizar?	Debería priorizar memoria Flash o EEPROM para almacenamiento permanente, ya que son no volátiles. Para memoria de trabajo, SRAM de bajo consumo es preferible a DRAM porque no necesita refrescos constantes.
Curso 2	Campos y Ondas	Teórica	¿Cuál es la definición de un vector en el contexto del análisis vectorial y cómo se diferencia de un escalar?	Un vector es una magnitud que posee módulo, dirección y sentido. A diferencia del escalar, que solo tiene magnitud, el vector requiere información direccional para su representación completa.
Curso 3	Electrónica II	Práctica	Un ingeniero detecta una caída de tensión inesperada en un circuito de telecomunicaciones, ¿qué posibles causas podría investigar?	Investigaría conexiones sueltas, componentes defectuosos, cortocircuitos, degradación de cables, problemas en fuentes de alimentación o variaciones en la resistencia.
Curso 4	Gestión de Redes y Servicios	Teórica	¿Qué es la gestión de redes y cuáles son sus objetivos principales?	La gestión de redes es un conjunto de procesos y herramientas para supervisar, controlar y mantener la infraestructura de comunicaciones. Sus objetivos son garantizar disponibilidad, rendimiento y seguridad.
Optativas	Robótica	Práctica	¿Qué se entiende por robot en el contexto de la ingeniería de sistemas?	Un robot es un dispositivo mecatrónico programable capaz de interactuar con su entorno y realizar tareas de forma autónoma o semiautónoma, integrando sistemas mecánicos, electrónicos y de control.

Anexo D. Resultados

Este anexo contiene los resultados estadísticos completos, separados en tres bloques.

(1) El primero recoge el análisis de parametricidad mediante Q-Q tests, que justifica el uso del test de Friedman para todas las métricas en ambas comparativas, dado que ninguna sigue la distribución normal con los tamaños muestrales empleados.
(2) El Grupo 1 enfrenta nueve modelos (LLM vs SLM vs Mini-SLM) sobre los *datasets* genéricos.
(3) El Grupo 2 compara siete modelos (Mini-SLM vs Fine-tuning vs GAN) sobre el corpus propio de telecomunicaciones.

Para cada grupo se presentan las métricas de calidad con intervalos de confianza *bootstrap* al 95%, el test global de Friedman, el análisis post-hoc par a par con tamaño del efecto r de Wilcoxon, y los diagramas de diferencia crítica.

D.1 Análisis de parametricidad

En esta sección se recogen todos los análisis de parametricidad (Q-Q tests) de los distintos modelos evaluados sobre sus respectivos *datasets* de inferencia. Modelos LLM, SLM y Mini-SLM en WebNLG, ToTTo y KELM. Modelos SLM, Mini-SLM *fine-tuned* y la GAN en el corpus propio de telecomunicaciones.

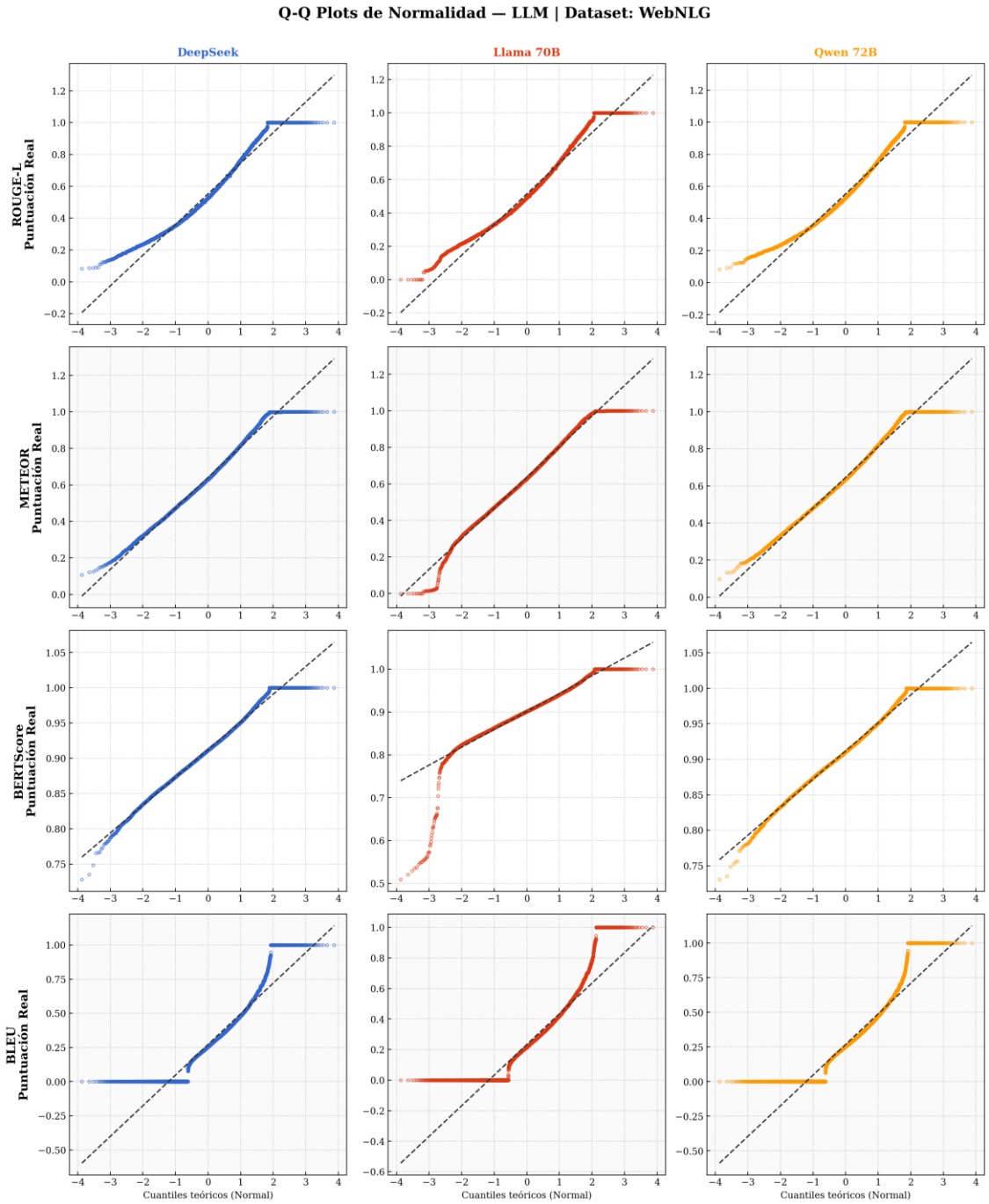


Figura Anexo 15: Q-Q tests de normalidad de los modelos LLM en WebNLG. Fuente propia

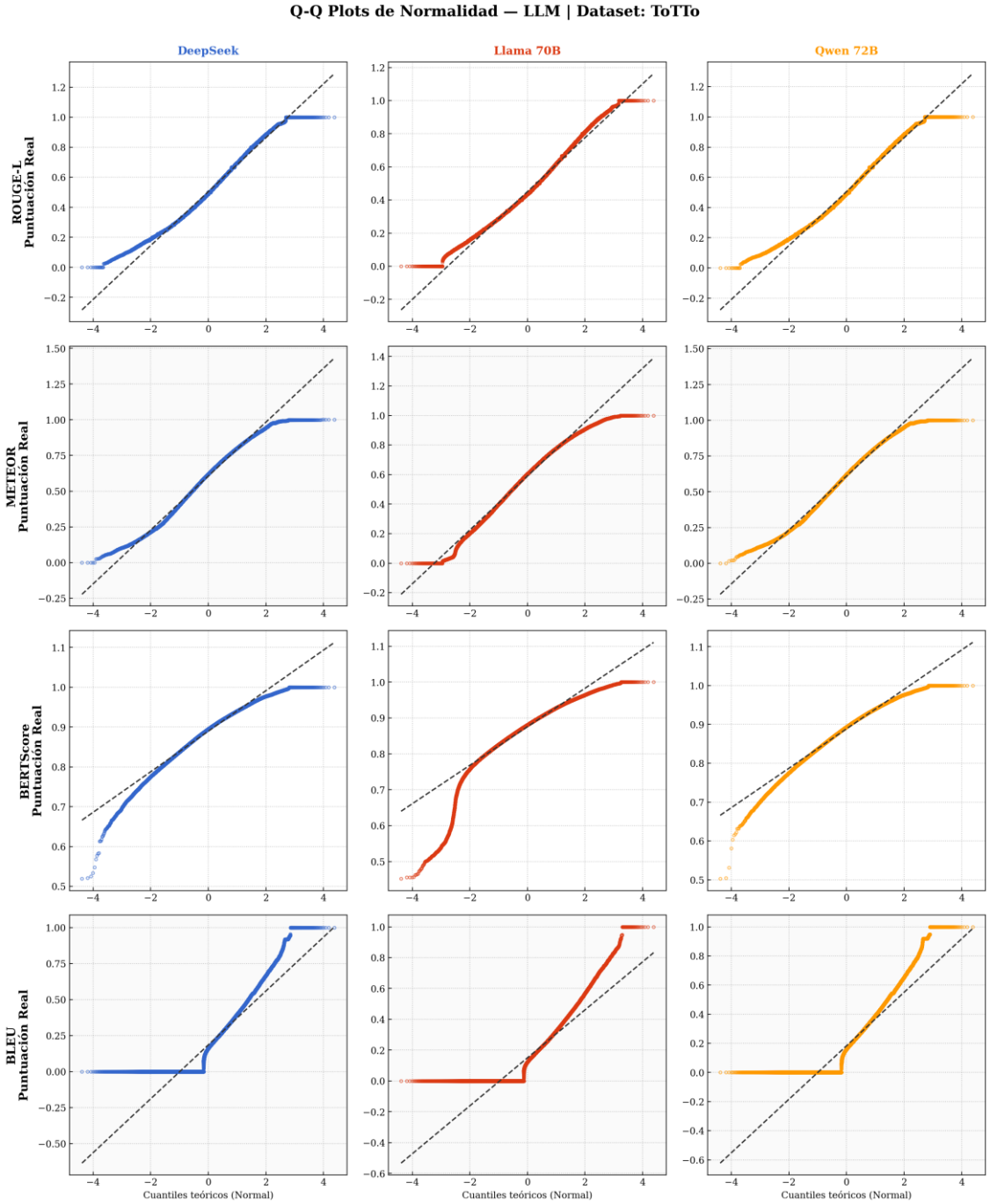


Figura Anexo 16: Q-Q tests de normalidad de los modelos LLM en ToTTo. Fuente propia.

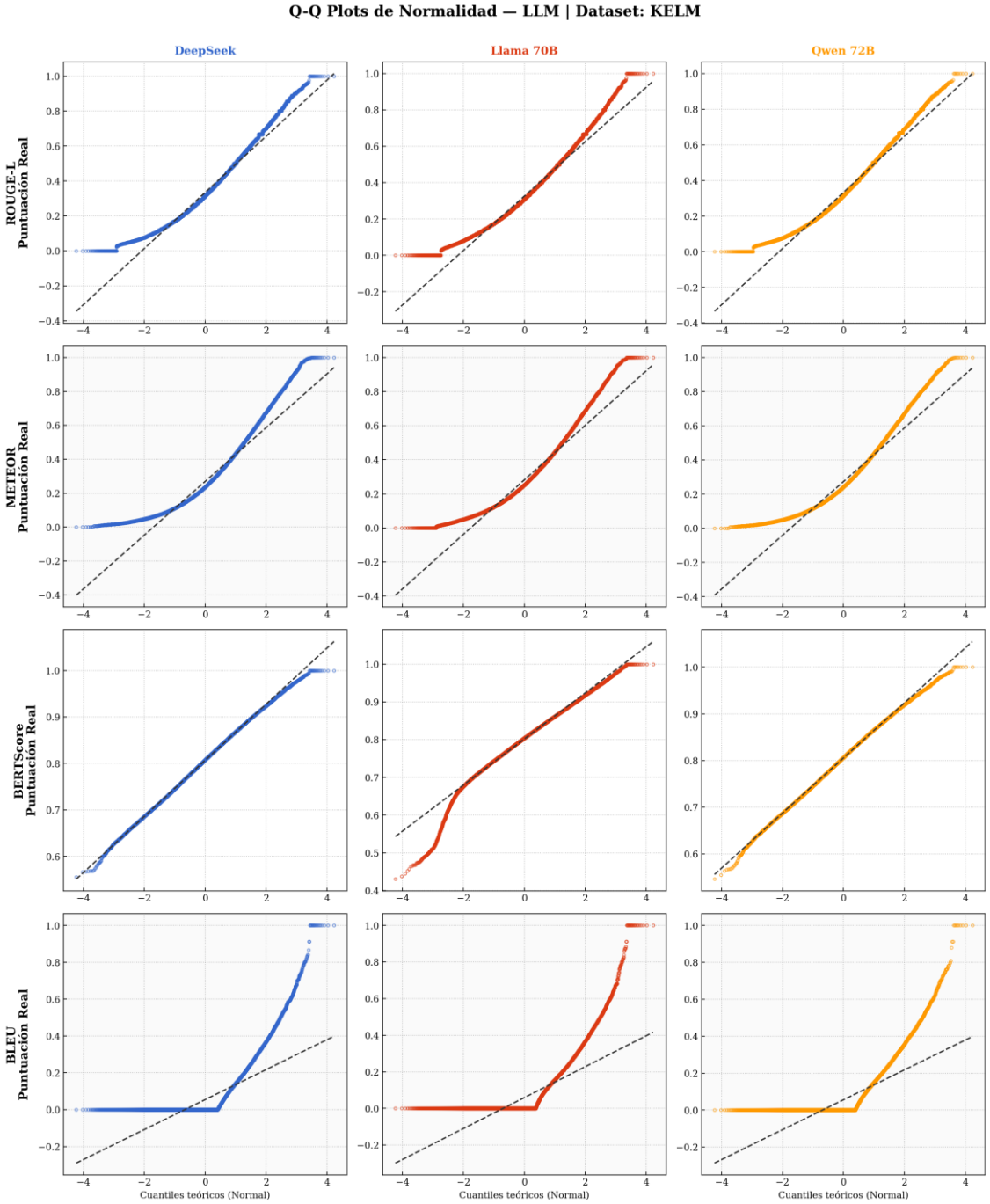


Figura Anexo 17: Q-Q tests de normalidad de los modelos LLM en KELM. Fuente propia.

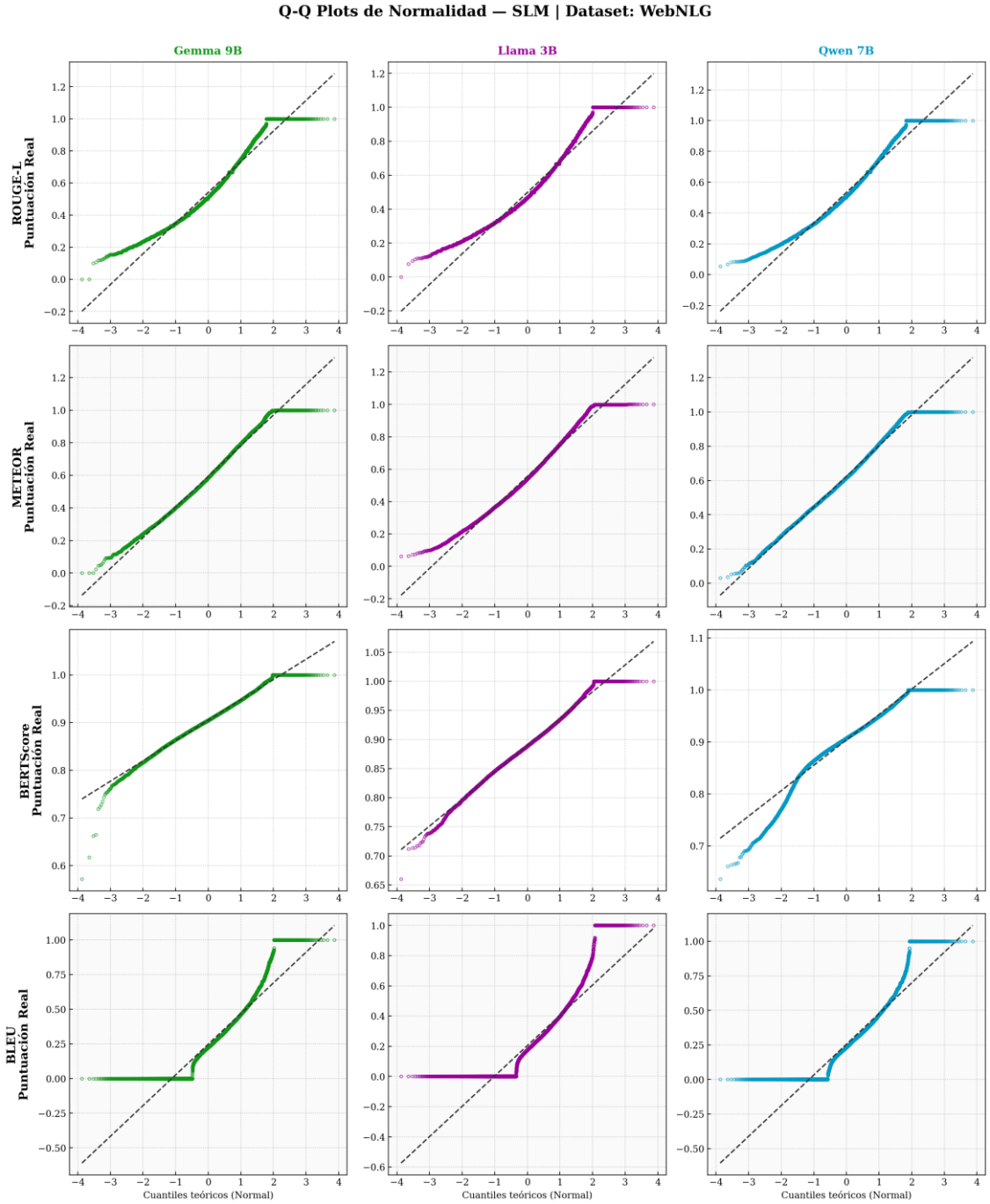


Figura Anexo 18: Q-Q tests de normalidad de los modelos SLM en WebNLG. Fuente propia.

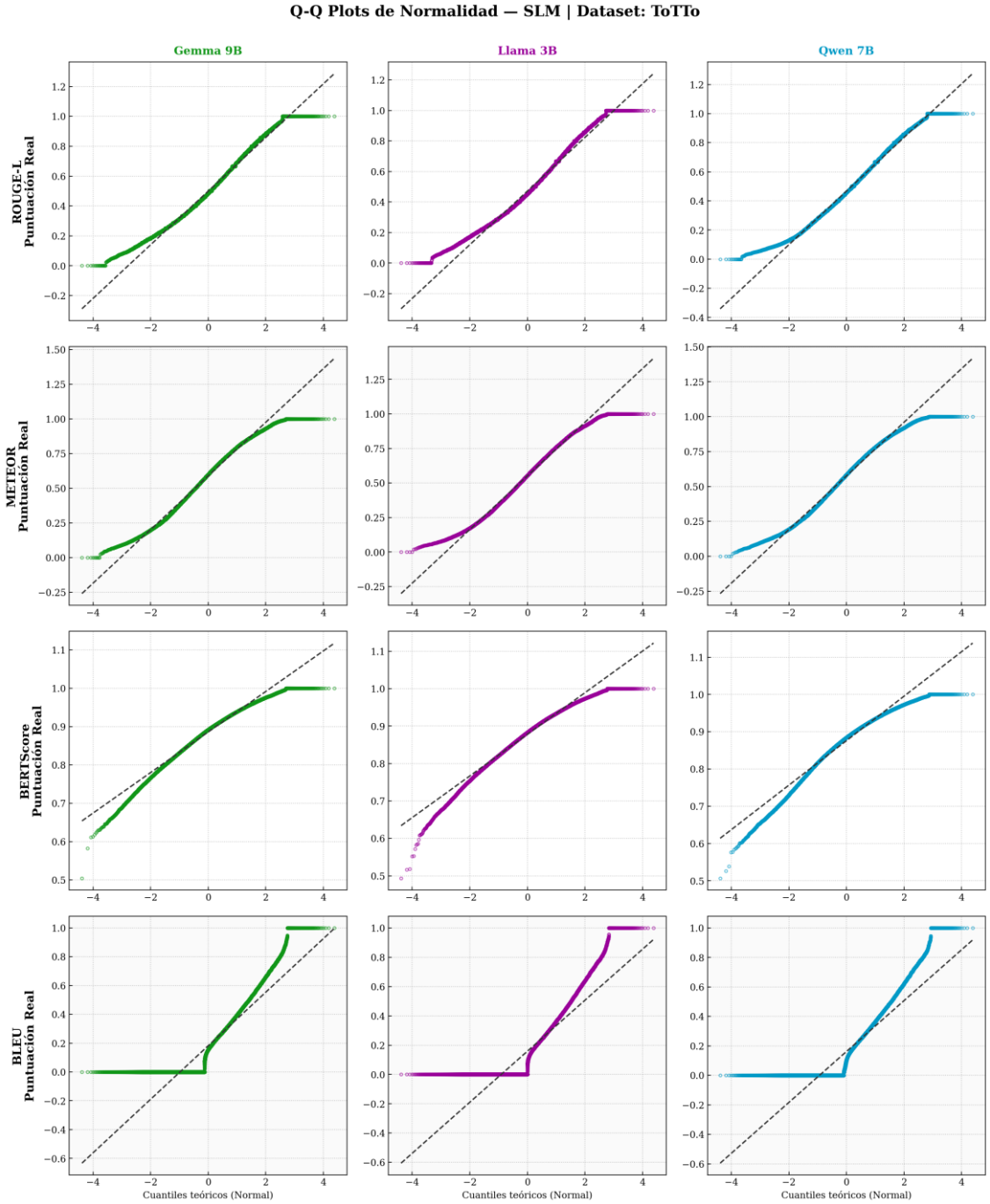


Figura Anexo 19: Q-Q tests de normalidad de los modelos SLM en ToTTo. Fuente propia.

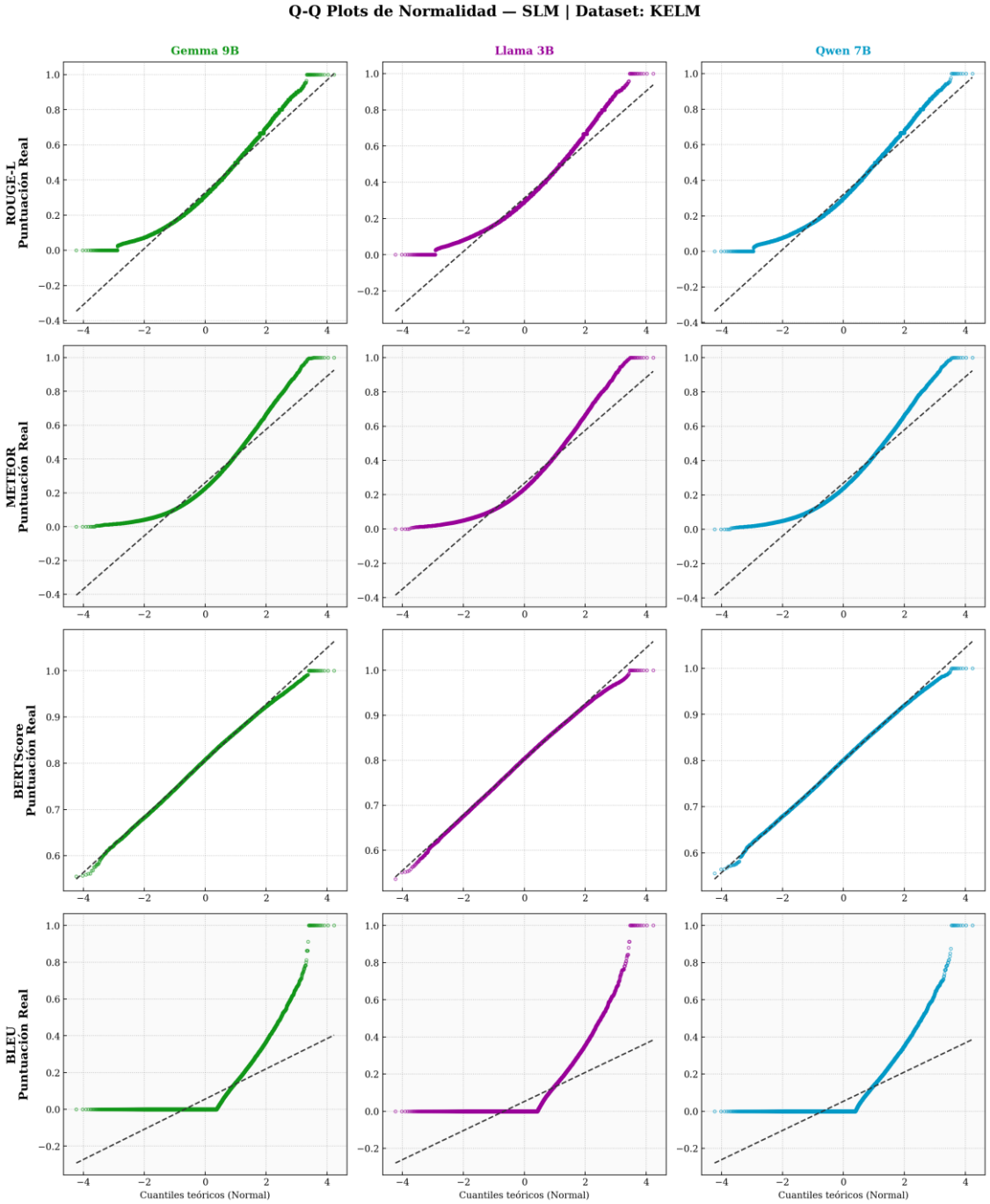


Figura Anexo 20: Q-Q tests de normalidad de los modelos SLM en KELM. Fuente propia.

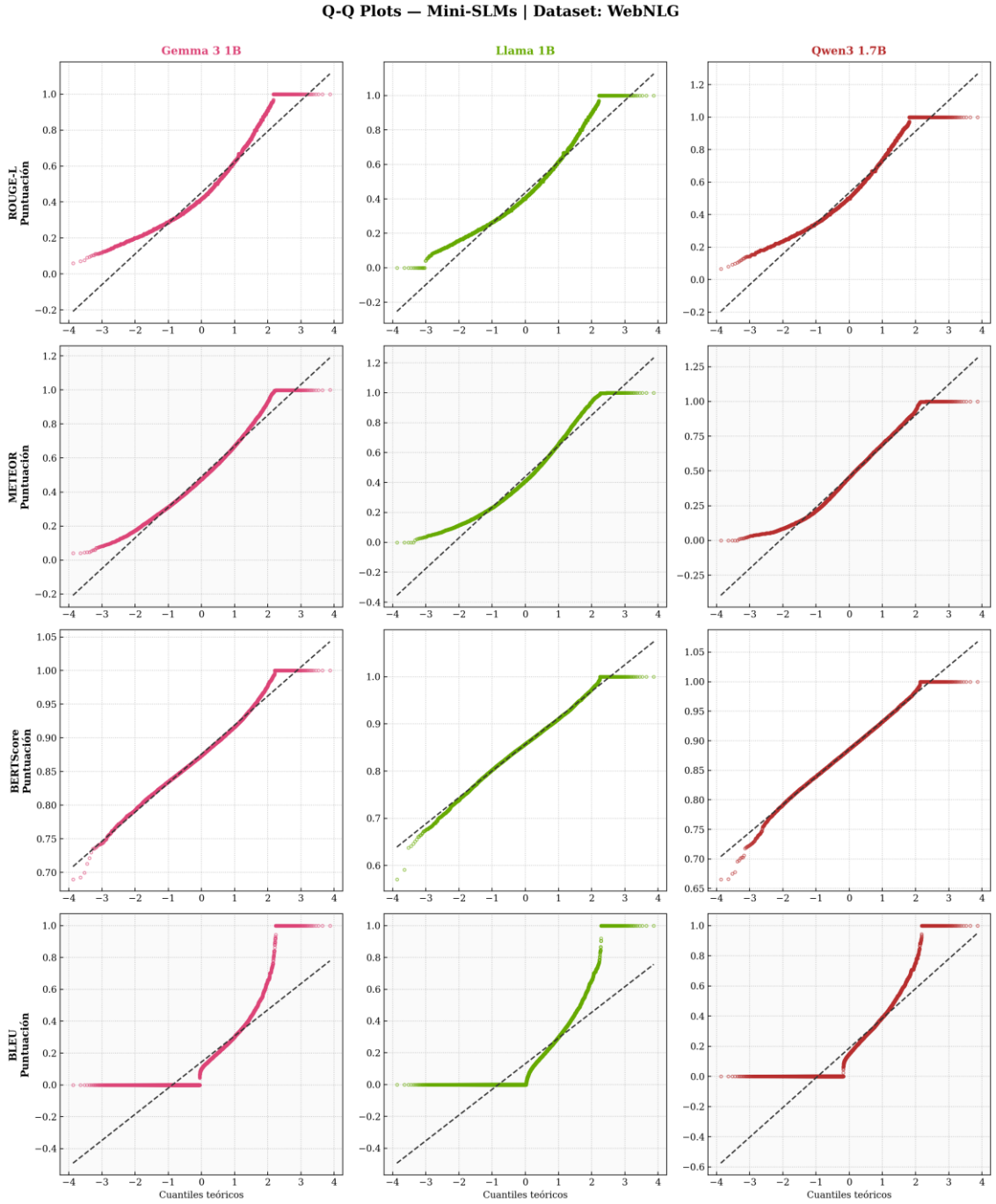


Figura Anexo 21: Q-Q tests de normalidad de los modelos Mini-SLM en WebNLG. Fuente propia.

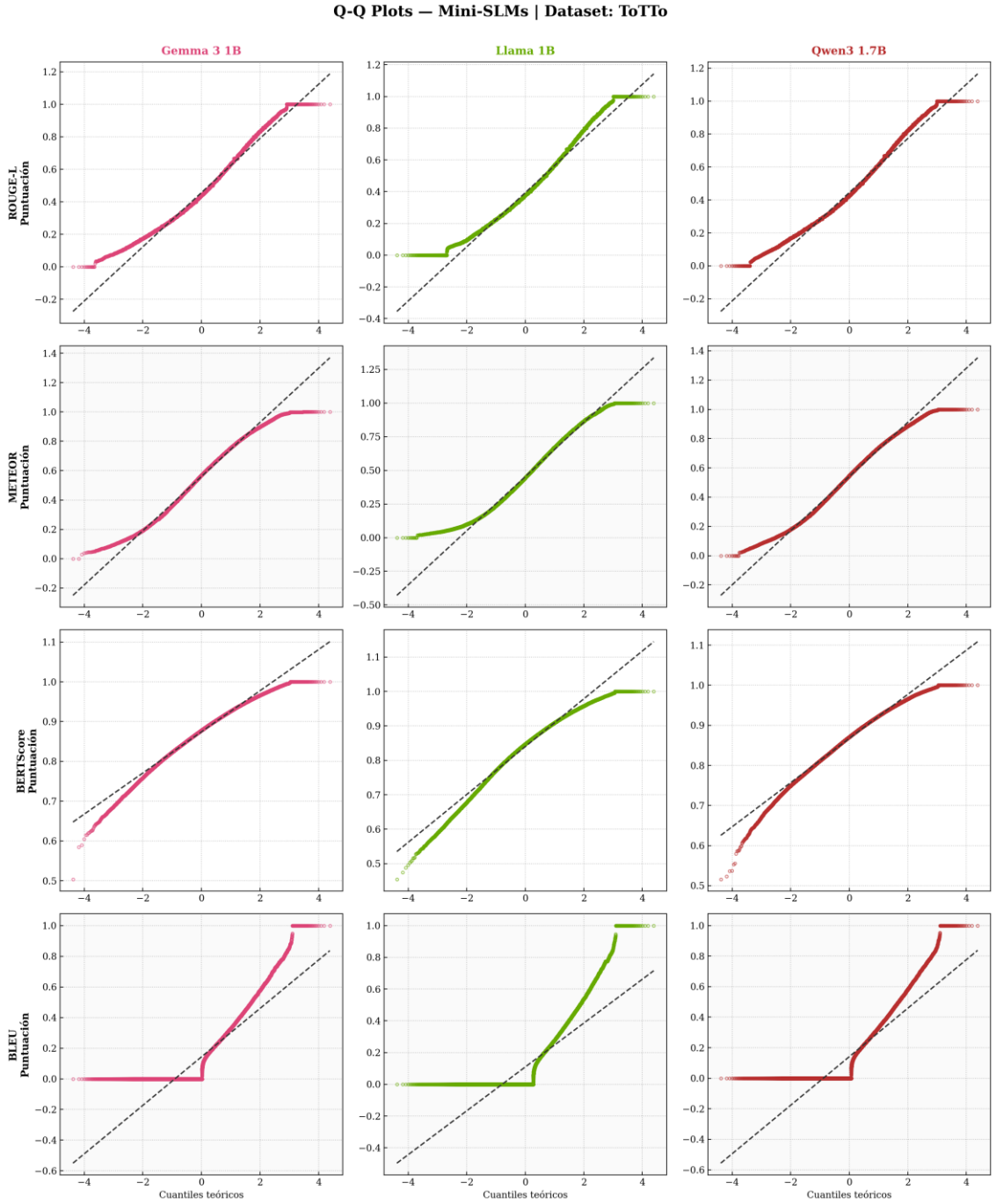


Figura Anexo 22: Q-Q tests de normalidad de los modelos Mini-SLM en ToTTo. Fuente propia.

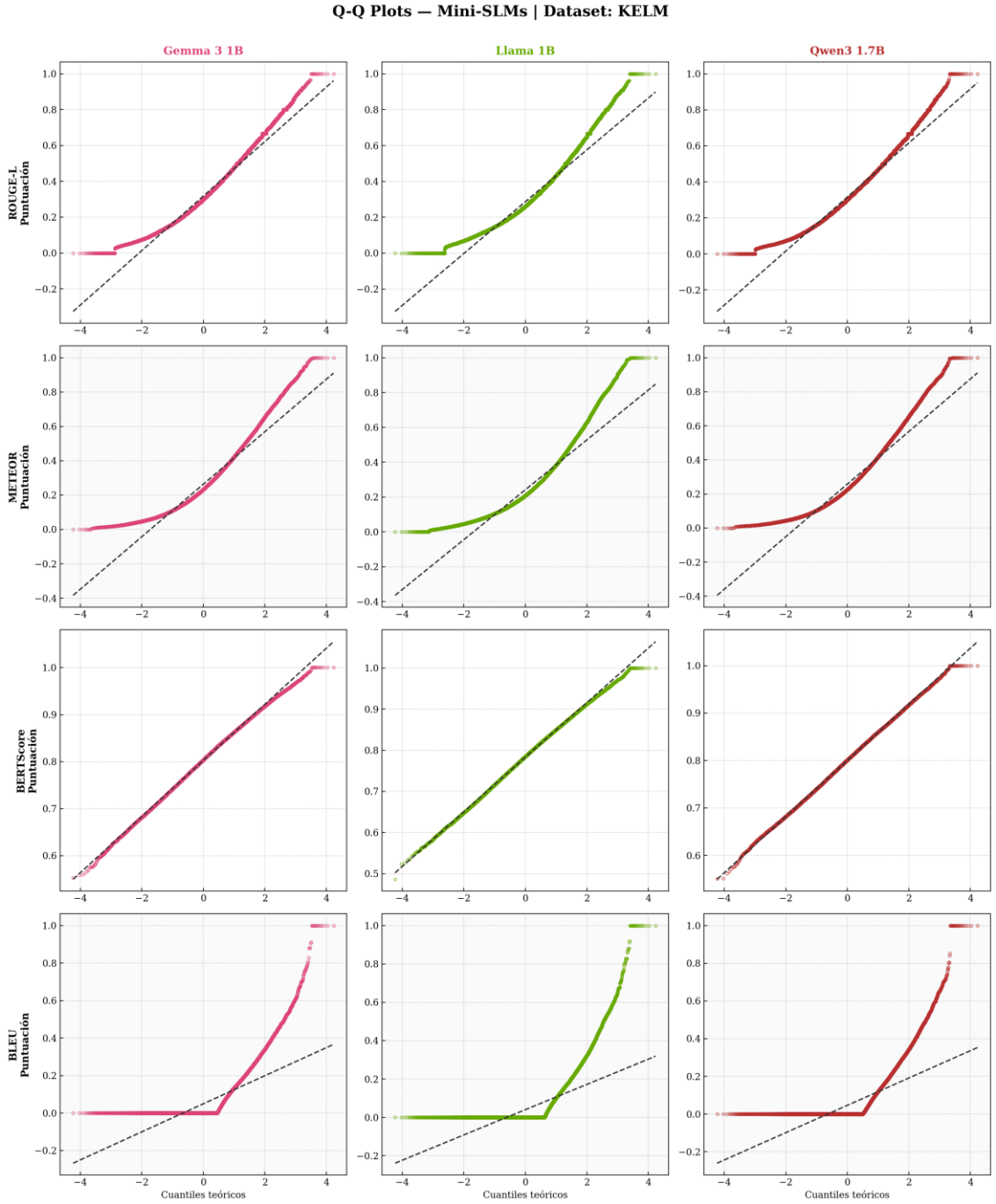


Figura Anexo 23: Q-Q tests de normalidad de los modelos Mini-SLM en KELM. Fuente propia.

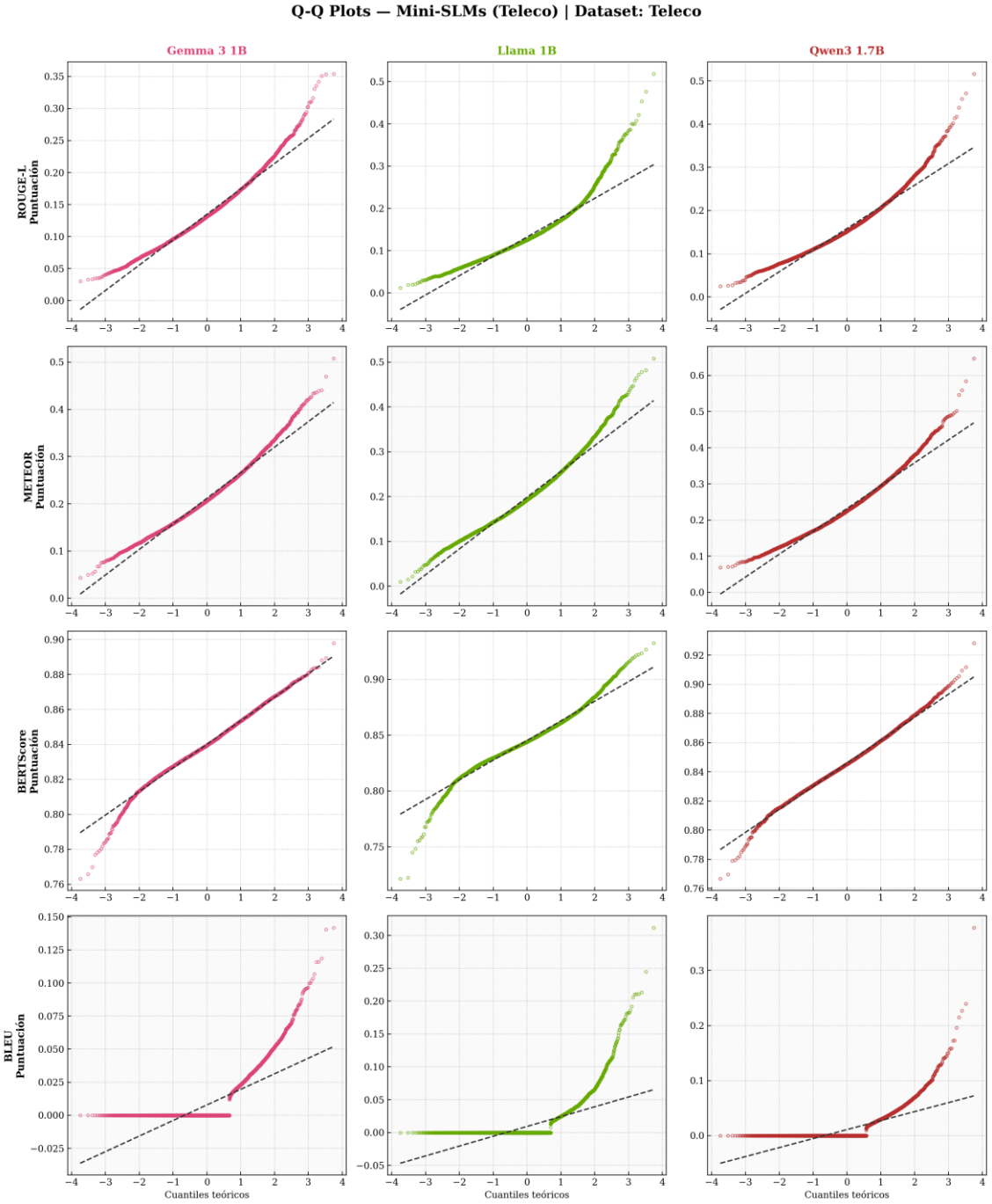


Figura Anexo 24: Q-Q tests de normalidad de los modelos Mini-SLM en el corpus propio. Fuente propia.

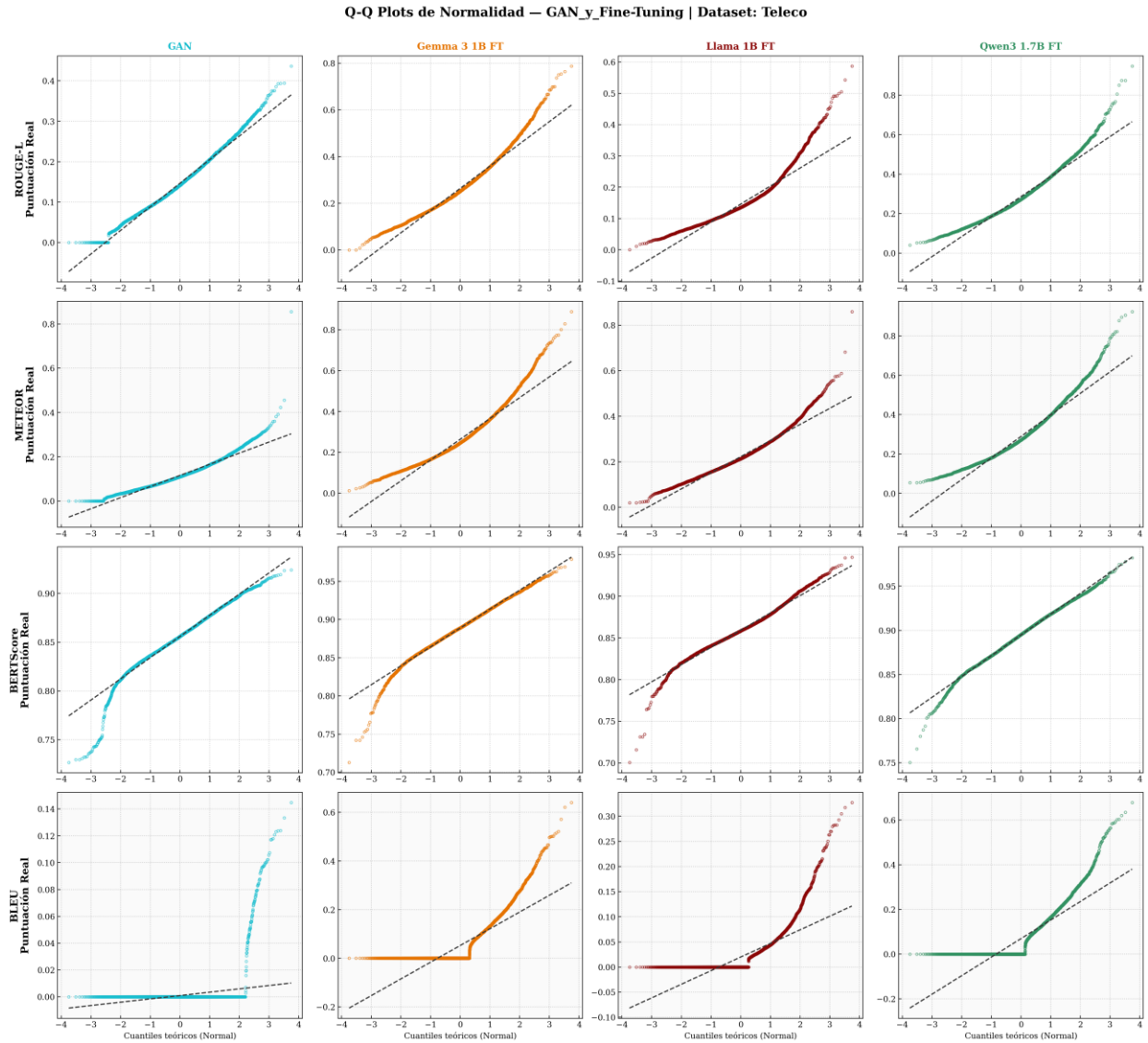


Figura Anexo 25: Q-Q tests de normalidad de los modelos *fine-tuned* y la GAN en el corpus propio. Fuente propia.

Dado que gran parte de las métricas de evaluación no siguen una distribución normal estricta, se ha establecido el uso de métodos estadísticos no paramétricos de manera uniforme. Aunque determinados subconjuntos de datos satisfacen los criterios de parametricidad, esta aproximación garantiza la homogeneidad del análisis.

D.2 Comparativa 1: LLM vs SLM vs Mini-SLM

Esta sección presenta los resultados obtenidos en la comparativa entre los modelos LLM, SLM y Mini-SLM sobre los tres datasets genéricos Se incluyen las métricas de calidad textual (ROUGE-L, METEOR, BLEU y BERTScore), los tiempos de inferencia, los costes asociados y las emisiones de Co2. Además, se aportan los resultados del test de Friedman, el análisis post-hoc de Nemenyi con tamaño del efecto de Wilcoxon y los diagramas de diferencia crítica correspondiente.

D.2.1 Métricas de calidad

Tabla Anexo 4: Costes y Co2 por modelo y *dataset.*

Tipo	Modelo	WebNLG		ToTTo		KELM	
		Coste ($)	*Co2 (g)*	*Coste ($)*	*Co2 (g)*	*Coste ($)*	*Co2 (g)*
LLM	DeepSeek	1,5651	333,5	13,9037	2.985,7	4,8807	1.104,7
LLM	Llama 3.3 70B	0,9635	563,1	8,7865	5.127,4	3,1701	1.879,4
LLM	Qwen 2.5 72B	0,9097	537,2	8,1308	4.812,1	2,9987	1.796,9
SLM	Gemma 2 9B	0,2737	130,0	2,4811	1.178,5	0,9177	435,9
SLM	Llama 3.2 3B	0,0823	52,1	0,7430	470,5	0,2848	180,4
SLM	Qwen 2.5 7B	0,2926	139,0	2,6826	1.274,2	0,9948	472,5
Mini-SLM	Gemma 3 1B	0,0456	18,2	2,7852	1.114,1	0,9369	374,8
Mini-SLM	Llama 3.2 1B	0,0169	17,0	0,1609	158,2	0,0586	57,0
Mini-SLM	Qwen 3 1.7B	0,1865	24,3	2,2739	219,9	0,7785	74,1

Tabla Anexo 5: Tiempo de inferencia por modelo y *dataset.*

Tipo	Modelo	WebNLG	ToTTo	KELM
		Tiempo (s/fila)	*Tiempo (s/fila)*	*Tiempo (s/fila)*
LLM	DeepSeek	1,8236	1,9479	1,7341
LLM	Llama 3.3 70B	2,0682	1,9143	1,9288
LLM	Qwen 2.5 72B	1,4262	1,4927	2,4130
SLM	Gemma 2 9B	0,7114	0,6114	0,6353
SLM	Llama 3.2 3B	0,5457	0,5412	0,6603
SLM	Qwen 2.5 7B	1,5769	1,6120	1,5984
Mini-SLM	Gemma 3 1B	0,6958	0,3978	0,3018
Mini-SLM	Llama 3.2 1B	1,8937	2,0477	2,4214
Mini-SLM	Qwen 3 1.7B	0,5997	0,6236	0,5959

Tabla Anexo 6: Resultados de las métricas de calidad por modelo y *dataset*.

Tipo	Modelo	ROUGE-L		METEOR		BLEU		BERTScore	
		Media + s	*IC 95%*	*Media + s*	*IC 95%*	*Media + s*	*IC 95%*	*Media + s*	*IC 95%*
WebNLG (12.876 filas)									
LLM	DeepSeek	0,5519 ± 0,1950	[0,5486; 0,5553]	0,6406 ± 0,1685	[0,6377; 0,6435]	0,2720 ± 0,2358	[0,2680; 0,2760]	0,9122 ± 0,0394	[0,9115; 0,9129]
LLM	Llama 3.3 70B	0,5139 ± 0,1861	[0,5107; 0,5172]	0,6353 ± 0,1682	[0,6323; 0,6382]	0,2335 ± 0,2125	[0,2298; 0,2373]	0,9010 ± 0,0430	[0,9003; 0,9018]
LLM	Qwen 2.5 72B	0,5530 ± 0,1935	[0,5497; 0,5563]	0,6454 ± 0,1659	[0,6425; 0,6482]	0,2683 ± 0,2330	[0,2643; 0,2723]	0,9116 ± 0,0396	[0,9109; 0,9123]
SLM	Gemma 2 9B	0,5412 ± 0,1945	[0,5378; 0,5447]	0,5937 ± 0,1890	[0,5904; 0,5970]	0,2473 ± 0,2353	[0,2432; 0,2514]	0,9050 ± 0,0428	[0,9042; 0,9057]
SLM	Llama 3.2 3B	0,4985 ± 0,1844	[0,4954; 0,5017]	0,5557 ± 0,1905	[0,5524; 0,5590]	0,2040 ± 0,2183	[0,2003; 0,2079]	0,8896 ± 0,0463	[0,8888; 0,8904]
SLM	Qwen 2.5 7B	0,5338 ± 0,2015	[0,5303; 0,5373]	0,6239 ± 0,1796	[0,6208; 0,6270]	0,2551 ± 0,2353	[0,2510; 0,2592]	0,9040 ± 0,0500	[0,9032; 0,9049]
Mini-SLM	Gemma 3 1B	0,4522 ± 0,1756	[0,4491; 0,4552]	0,4907 ± 0,1820	[0,4875; 0,4939]	0,1435 ± 0,1877	[0,1403; 0,1467]	0,8758 ± 0,0434	[0,8751; 0,8765]
Mini-SLM	Llama 3.2 1B	0,4362 ± 0,1819	[0,4331; 0,4394]	0,4400 ± 0,2085	[0,4365; 0,4436]	0,1313 ± 0,1887	[0,1280; 0,1346]	0,8567 ± 0,0563	[0,8557; 0,8576]
Mini-SLM	Qwen 3 1.7B	0,5356 ± 0,1925	[0,5323; 0,5389]	0,4596 ± 0,2226	[0,4559; 0,4634]	0,1878 ± 0,2170	[0,1841; 0,1915]	0,8859 ± 0,0470	[0,8851; 0,8867]
ToTTo (120.761 filas)									
LLM	DeepSeek	0,5043 ± 0,1804	[0,5032; 0,5053]	0,6050 ± 0,1898	[0,6040; 0,6061]	0,1858 ± 0,2031	[0,1846; 0,1869]	0,8895 ± 0,0513	[0,8892; 0,8898]
LLM	Llama 3.3 70B	0,4499 ± 0,1634	[0,4490; 0,4508]	0,5893 ± 0,1834	[0,5883; 0,5904]	0,1496 ± 0,1703	[0,1486; 0,1505]	0,8753 ± 0,0554	[0,8750; 0,8756]
LLM	Qwen 2.5 72B	0,5048 ± 0,1792	[0,5038; 0,5058]	0,6091 ± 0,1893	[0,6081; 0,6102]	0,1829 ± 0,1992	[0,1818; 0,1840]	0,8886 ± 0,0511	[0,8884; 0,8889]
SLM	Gemma 2 9B	0,4994 ± 0,1805	[0,4983; 0,5004]	0,5883 ± 0,1945	[0,5871; 0,5894]	0,1822 ± 0,2027	[0,1810; 0,1833]	0,8858 ± 0,0534	[0,8855; 0,8861]
SLM	Llama 3.2 3B	0,4711 ± 0,1770	[0,4701; 0,4721]	0,5511 ± 0,1948	[0,5500; 0,5522]	0,1580 ± 0,1941	[0,1569; 0,1591]	0,8778 ± 0,0560	[0,8775; 0,8781]
SLM	Qwen 2.5 7B	0,4672 ± 0,1848	[0,4662; 0,4683]	0,5754 ± 0,1930	[0,5743; 0,5765]	0,1616 ± 0,1903	[0,1606; 0,1627]	0,8759 ± 0,0607	[0,8755; 0,8762]
Mini-SLM	Gemma 3 1B	0,4562 ± 0,1681	[0,4553; 0,4572]	0,5597 ± 0,1853	[0,5587; 0,5607]	0,1431 ± 0,1766	[0,1421; 0,1441]	0,8742 ± 0,0519	[0,8739; 0,8745]
Mini-SLM	Llama 3.2 1B	0,3938 ± 0,1720	[0,3928; 0,3947]	0,4537 ± 0,2023	[0,4526; 0,4548]	0,1094 ± 0,1630	[0,1085; 0,1103]	0,8398 ± 0,0702	[0,8394; 0,8402]
Mini-SLM	Qwen 3 1.7B	0,4442 ± 0,1658	[0,4432; 0,4451]	0,5406 ± 0,1857	[0,5396; 0,5416]	0,1410 ± 0,1789	[0,1399; 0,1419]	0,8671 ± 0,0553	[0,8668; 0,8674]
KELM (59.751 filas)									
LLM	DeepSeek	0,3345 ± 0,1629	[0,3331; 0,3358]	0,2695 ± 0,1640	[0,2682; 0,2708]	0,0545 ± 0,1055	[0,0536; 0,0553]	0,8068 ± 0,0604	[0,8063; 0,8073]
LLM	Llama 3.3 70B	0,3259 ± 0,1514	[0,3247; 0,3271]	0,2818 ± 0,1646	[0,2805; 0,2831]	0,0589 ± 0,1067	[0,0581; 0,0598]	0,8018 ± 0,0615	[0,8013; 0,8023]
LLM	Qwen 2.5 72B	0,3331 ± 0,1596	[0,3318; 0,3343]	0,2731 ± 0,1621	[0,2718; 0,2745]	0,0558 ± 0,1027	[0,0549; 0,0566]	0,8055 ± 0,0589	[0,8050; 0,8059]
SLM	Gemma 2 9B	0,3294 ± 0,1620	[0,3281; 0,3307]	0,2606 ± 0,1626	[0,2593; 0,2619]	0,0558 ± 0,1055	[0,0549; 0,0566]	0,8064 ± 0,0608	[0,8059; 0,8069]
SLM	Llama 3.2 3B	0,3129 ± 0,1503	[0,3118; 0,3141]	0,2667 ± 0,1596	[0,2654; 0,2679]	0,0522 ± 0,1018	[0,0514; 0,0530]	0,8023 ± 0,0619	[0,8018; 0,8028]
SLM	Qwen 2.5 7B	0,3207 ± 0,1580	[0,3194; 0,3219]	0,2697 ± 0,1592	[0,2684; 0,2710]	0,0534 ± 0,1008	[0,0526; 0,0542]	0,8005 ± 0,0608	[0,8001; 0,8010]
Mini-SLM	Gemma 3 1B	0,3179 ± 0,1543	[0,3167; 0,3191]	0,2634 ± 0,1578	[0,2622; 0,2647]	0,0498 ± 0,0976	[0,0490; 0,0506]	0,8025 ± 0,0597	[0,8020; 0,8030]
Mini-SLM	Llama 3.2 1B	0,2870 ± 0,1483	[0,2858; 0,2882]	0,2415 ± 0,1498	[0,2403; 0,2427]	0,0403 ± 0,0922	[0,0396; 0,0410]	0,7831 ± 0,0664	[0,7825; 0,7836]
Mini-SLM	Qwen 3 1.7B	0,3157 ± 0,1521	[0,3144; 0,3169]	0,2581 ± 0,1597	[0,2568; 0,2594]	0,0462 ± 0,0976	[0,0455; 0,0471]	0,8001 ± 0,0592	[0,7997; 0,8006]

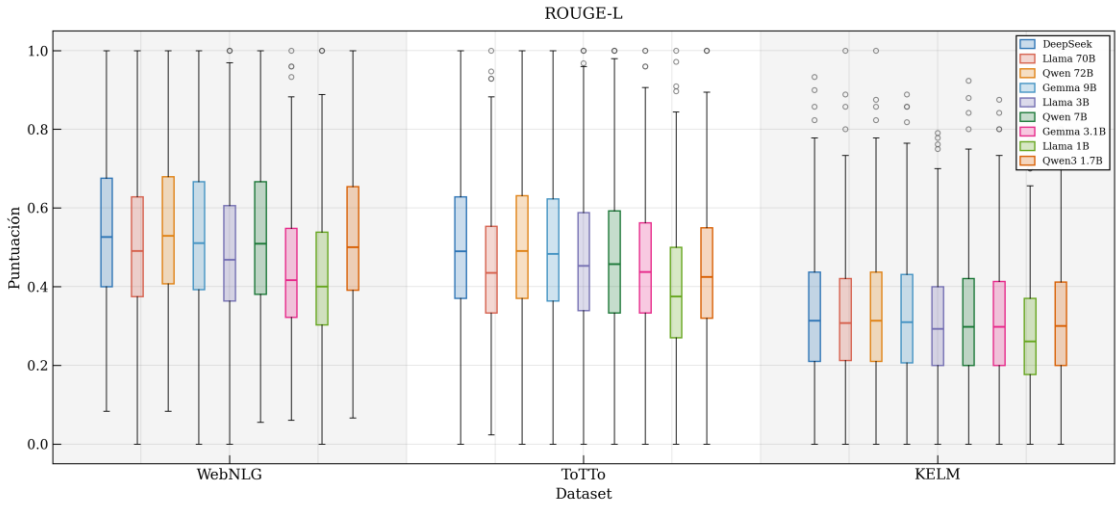


Figura Anexo 26: Boxplots de ROUGE-L por modelo y *dataset*. *Fuente* propia.

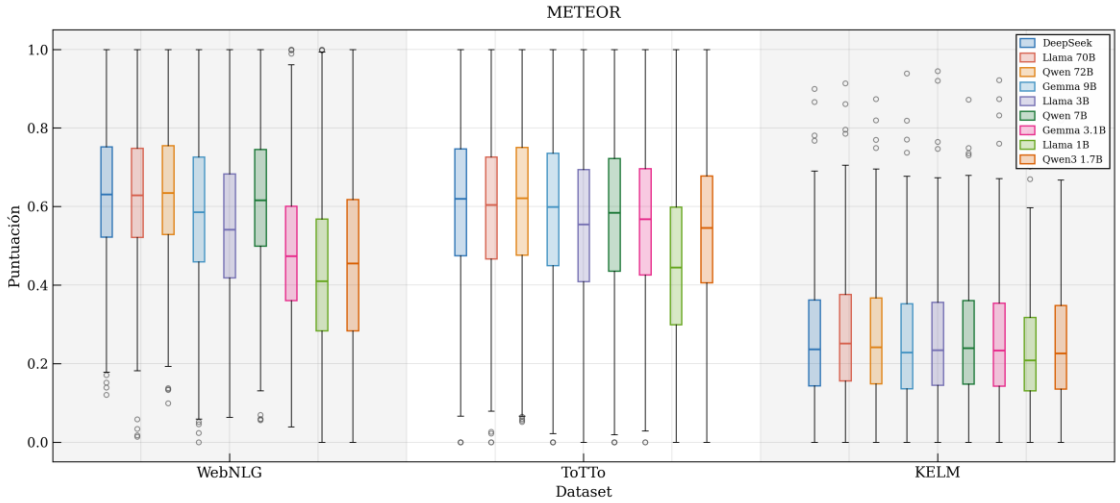


Figura Anexo 27: Boxplots de METEOR por modelo y *dataset*. *Fuente* propia.

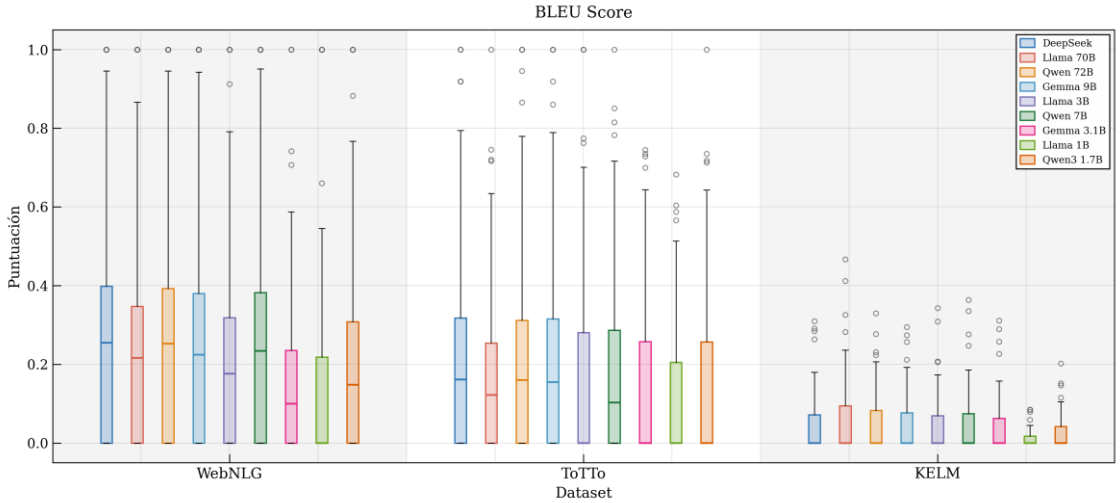


Figura Anexo 28: Boxplots de BLEU por modelo y dataset. Fuente propia.

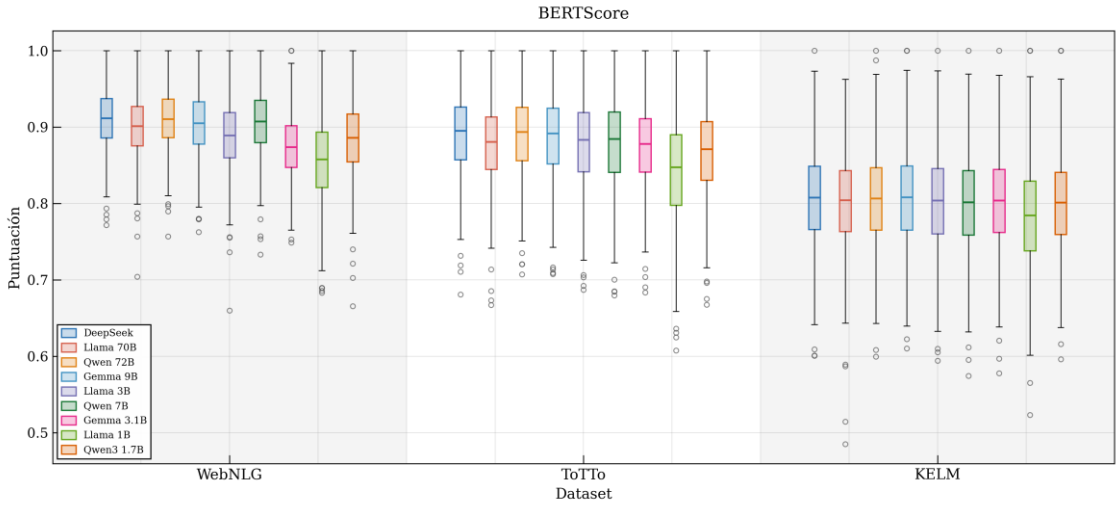


Figura Anexo 29: Boxplots de BERTScore por modelo y dataset. Fuente propia.

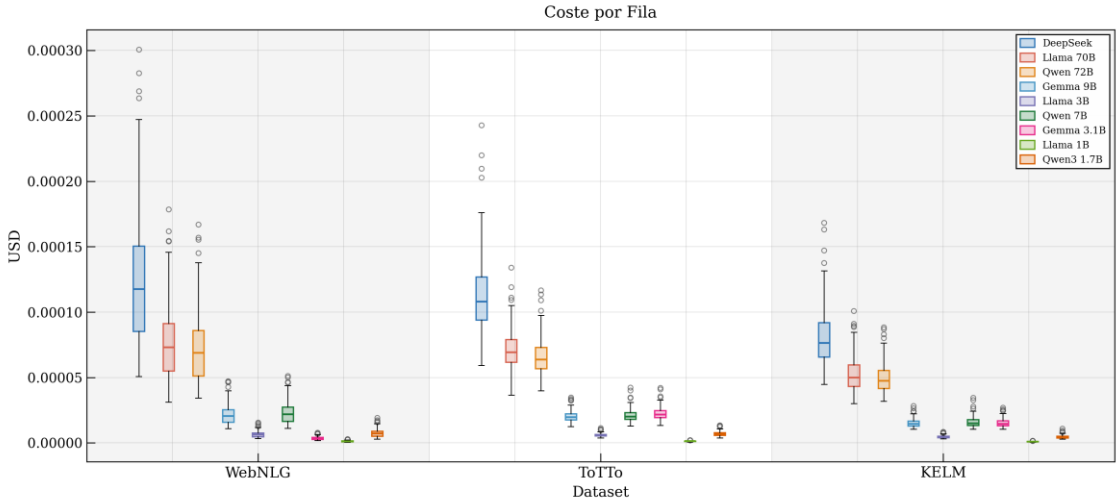


Figura Anexo 30: Boxplots de coste por modelo y dataset. Fuente propia.

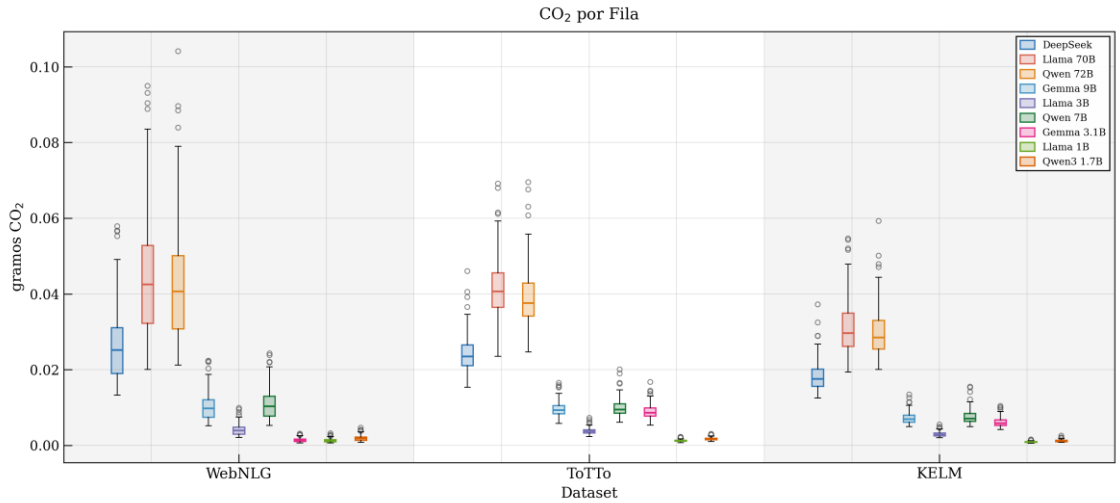


Figura Anexo 31: Boxplots de Co2 por modelo y dataset. Fuente propia.

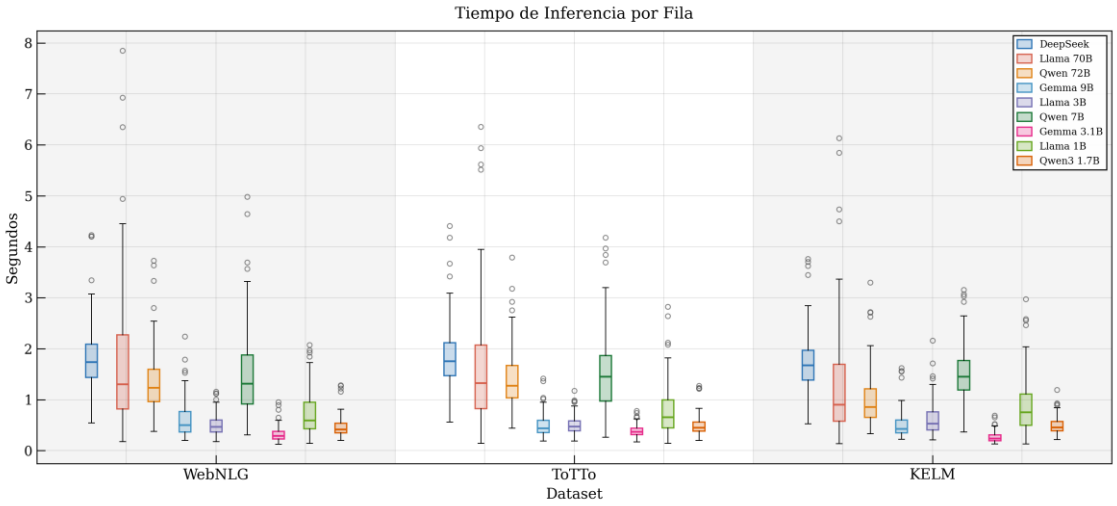


Figura Anexo 32: Boxplots de tiempo de inferencia por modelo y dataset. Fuente propia.

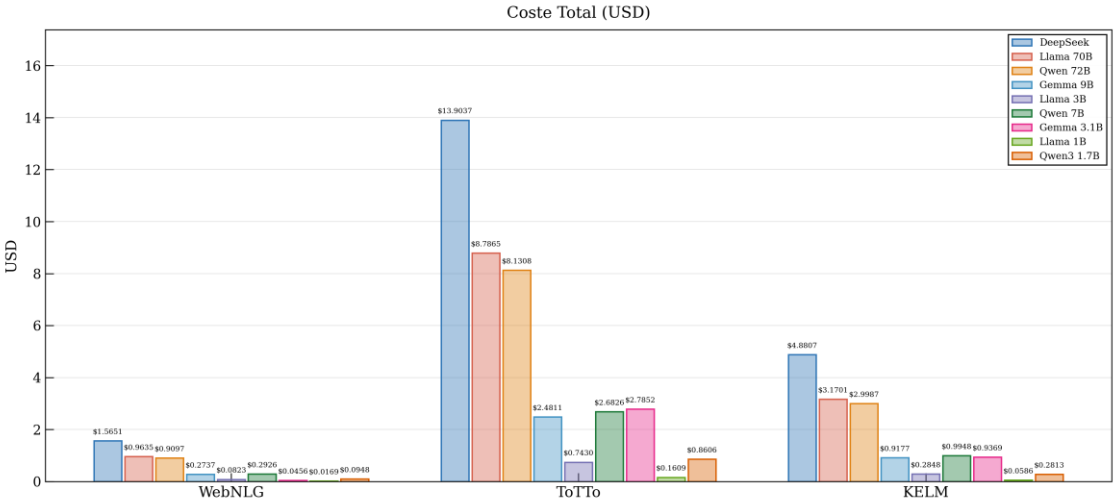


Figura Anexo 33: Barplost de coste total por modelo y dataset. Fuente propia.

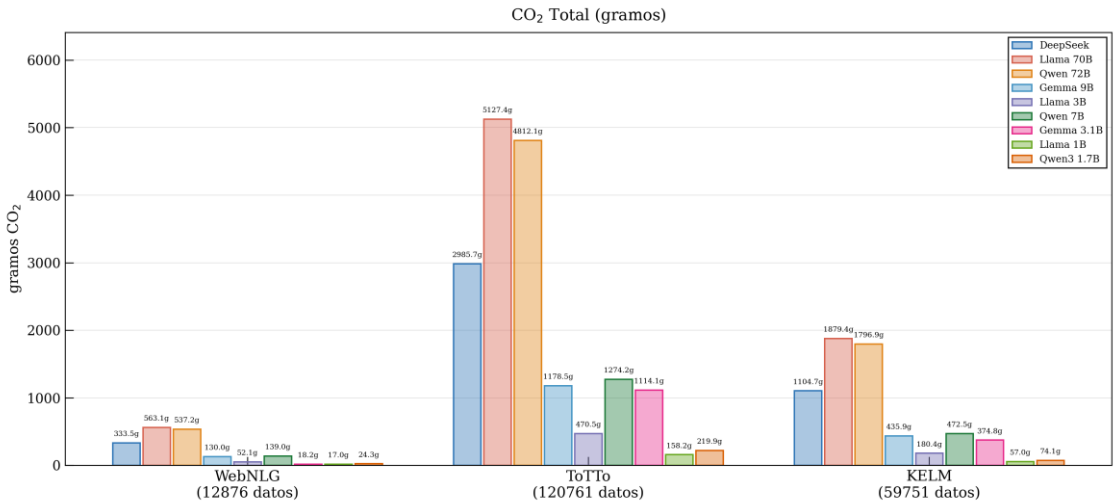


Figura Anexo 34: Barplots de Co2 total por modelo y dataset. Fuente propia.

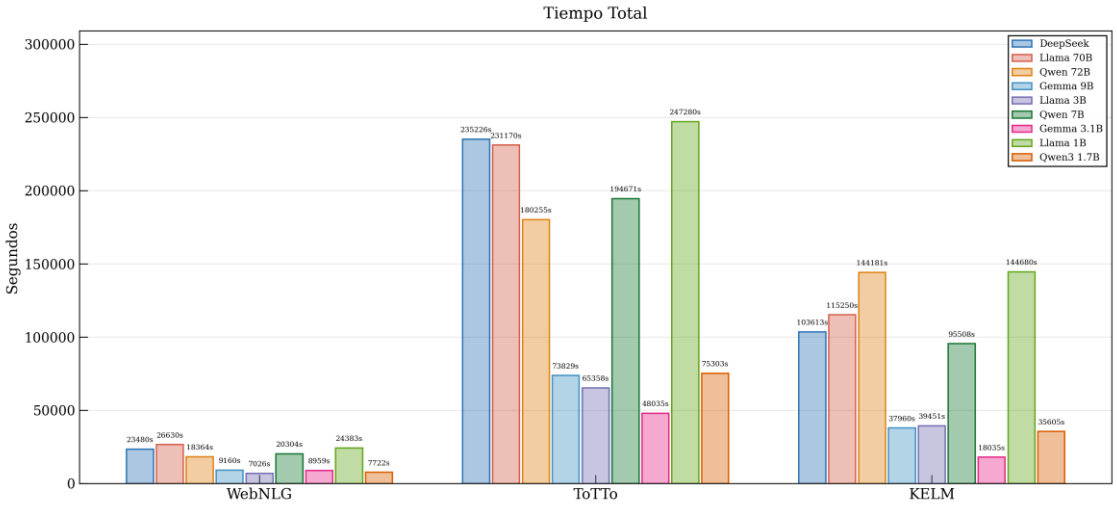


Figura Anexo 35: Barplots de tiempo total por modelo y dataset. Fuente propia.

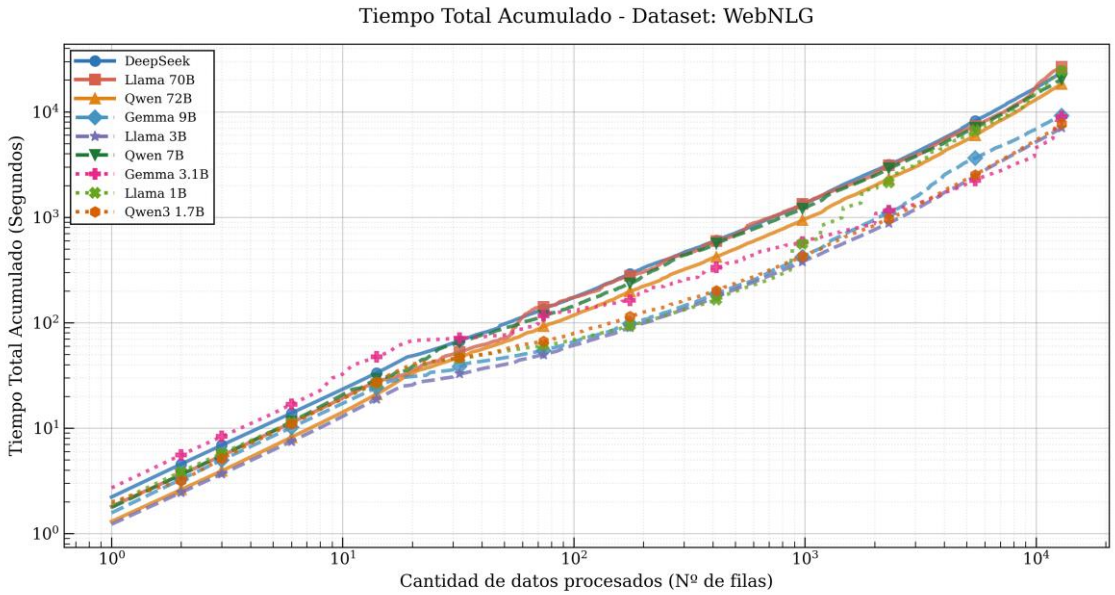


Figura Anexo 36: Tiempo total acumulado de inferencia por modelo en WebNLG. Fuente propia.

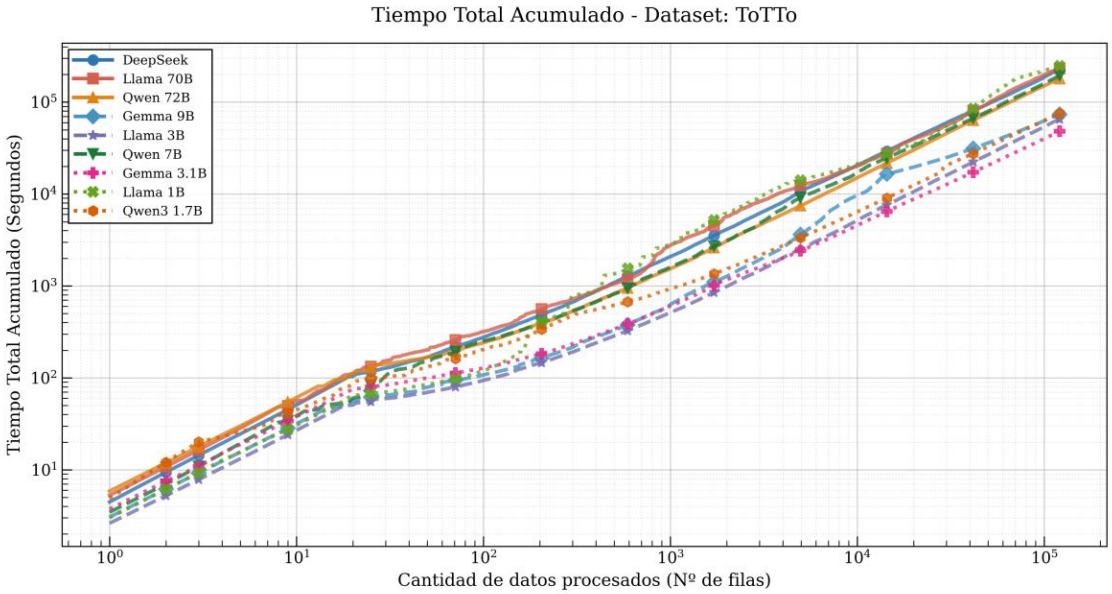


Figura Anexo 37: Tiempo total acumulado de inferencia por modelo en ToTTo. Fuente propia.

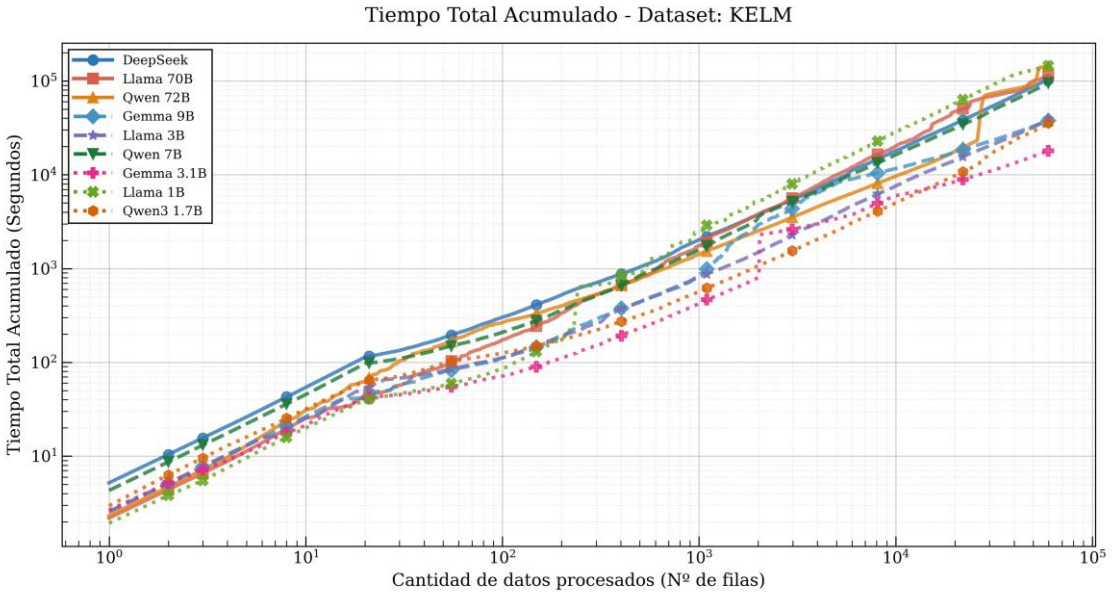


Figura Anexo 38: Tiempo total acumulado de inferencia por modelo en KELM. Fuente propia.

D.2.2 Test estadístico

Tabla Anexo 7: Resultados del test de Friedman por métrica y *dataset*.

Métrica	Test	Estadístico	p-valor	Sig.	N
WebNLG					
ROUGE-L	Friedman (χ^2)	355,16	$7{,}16 \cdot 10^{-72}$	**Sí**	12.876
METEOR	Friedman (χ^2)	787,96	$8{,}09 \cdot 10^{-165}$	**Sí**	12.876
BERTScore	Friedman (χ^2)	727,03	$1{,}08 \cdot 10^{-151}$	**Sí**	12.876
BLEU	Friedman (χ^2)	401,79	$7{,}76 \cdot 10^{-82}$	**Sí**	12.876
ToTTo					
ROUGE-L	Friedman (χ^2)	227,81	$8{,}58 \cdot 10^{-45}$	**Sí**	120.761
METEOR	Friedman (χ^2)	246,37	$1{,}01 \cdot 10^{-48}$	**Sí**	120.761
BERTScore	Friedman (χ^2)	266,60	$5{,}19 \cdot 10^{-53}$	**Sí**	120.761
BLEU	Friedman (χ^2)	68,69	$8{,}94 \cdot 10^{-12}$	**Sí**	120.761
KELM					
ROUGE-L	Friedman (χ^2)	35,70	$1{,}99 \cdot 10^{-5}$	**Sí**	59.751
METEOR	Friedman (χ^2)	18,33	$1{,}32 \cdot 10^{-2}$	**Sí**	59.751
BERTScore	Friedman (χ^2)	45,32	$3{,}20 \cdot 10^{-7}$	**Sí**	59.751
BLEU	Friedman (χ^2)	21,27	$6{,}45 \cdot 10^{-3}$	**Sí**	59.751

D.2.3 Post hoc

Tabla Anexo 8: Análisis post-hoc (Nemenyi + Wilcoxon r) para ROUGE-L en WebNLG.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0380	< 0,001	**Sí**	0,1664	Pequeño
DeepSeek	Qwen 2.5 72B	-0,0011	0,9998	**No**	0,0002	Nulo
DeepSeek	Gemma 2 9B	+0,0107	< 0,001	**Sí**	0,0512	Nulo
DeepSeek	Llama 3.2 3B	+0,0534	< 0,001	**Sí**	0,2350	Pequeño
DeepSeek	Qwen 2.5 7B	+0,0181	< 0,001	**Sí**	0,0773	Nulo
DeepSeek	Gemma 3 1B	+0,0998	< 0,001	**Sí**	0,4421	Mediano
DeepSeek	Llama 3.2 1B	+0,1157	< 0,001	**Sí**	0,4794	Mediano
DeepSeek	Qwen 3 1.7B	+0,0163	< 0,001	**Sí**	0,0768	Nulo
Llama 3.3 70B	Qwen 2.5 72B	-0,0391	< 0,001	**Sí**	0,1664	Pequeño
Llama 3.370B	Gemma 2 9B	-0,0273	< 0,001	**Sí**	0,1133	Pequeño
Llama 3.3 70B	Llama 3.2 3B	+0,0154	< 0,001	**Sí**	0,0738	Nulo
Llama 3.3 70B	Qwen 2.5 7B	-0,0199	< 0,001	**Sí**	0,0845	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0618	< 0,001	**Sí**	0,2862	Pequeño
Llama 3.3 70B	Llama 3.2 1B	+0,0777	< 0,001	**Sí**	0,3386	Mediano
Llama 3.3 70B	Qwen 3 1.7B	-0,0216	< 0,001	**Sí**	0,0888	Nulo
Qwen 2.5 72B	Gemma 2 9B	+0,0118	< 0,001	**Sí**	0,0551	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0545	< 0,001	**Sí**	0,2362	Pequeño
Qwen 2.5 72B	Qwen 2.5 7B	+0,0192	< 0,001	**Sí**	0,0763	Nulo
Qwen 2.5 72B	Gemma 3 1B	+0,1009	< 0,001	**Sí**	0,4350	Mediano
Qwen 2.5 72B	Llama 3.2 1B	+0,1168	< 0,001	**Sí**	0,4767	Mediano
Qwen 2.5 72B	Qwen 3 1.7B	+0,0175	< 0,001	**Sí**	0,0772	Nulo
Gemma 2 9B	Llama 3.2 3B	+0,0427	< 0,001	**Sí**	0,1852	Pequeño
Gemma 2 9B	Qwen 2.5 7B	+0,0074	0,0849	**No**	0,0250	Nulo
Gemma 2 9B	Gemma 3 1B	+0,0891	< 0,001	**Sí**	0,3865	Mediano
Gemma 2 9B	Llama 3.2 1B	+0,1050	< 0,001	**Sí**	0,4307	Mediano
Gemma 2 9B	Qwen 3 1.7B	+0,0056	0,9887	**No**	0,0222	Nulo
Llama 3.2 3B	Qwen 2.5 7B	-0,0353	< 0,001	**Sí**	0,1561	Pequeño
Llama 3.2 3B	Gemma 3 1B	+0,0464	< 0,001	**Sí**	0,2226	Pequeño
Llama 3.2 3B	Llama 3.2 1B	+0,0623	< 0,001	**Sí**	0,2850	Pequeño
Llama 3.2 3B	Qwen 3 1.7B	-0,0371	< 0,001	**Sí**	0,1648	Pequeño
Qwen 2.5 7B	Gemma 3 1B	+0,0816	< 0,001	**Sí**	0,3450	Mediano
Qwen 2.5 7B	Llama 3.2 1B	+0,0976	< 0,001	**Sí**	0,3949	Mediano
Qwen 2.5 7B	Qwen 3 1.7B	-0,0018	0,0033	**Sí**	0,0012	Nulo
Gemma 3 1B	Llama 3.2 1B	+0,0159	< 0,001	**Sí**	0,0858	Nulo
Gemma 3 1B	Qwen 3 1.7B	-0,0834	< 0,001	**Sí**	0,5196	Grande
Llama 3.2 1B	Qwen 3 1.7B	-0,0993	< 0,001	**Sí**	0,5262	Grande

Tabla Anexo 9: Análisis post-hoc (Nemenyi + Wilcoxon r) para METEOR en WebNLG.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0053	0,1068	**No**	0,0182	Nulo
DeepSeek	Qwen 2.5 72B	-0,0048	0,9068	**No**	0,0192	Nulo
DeepSeek	Gemma 2 9B	+0,0469	< 0,001	**Sí**	0,2168	Pequeño
DeepSeek	Llama 3.2 3B	+0,0849	< 0,001	**Sí**	0,3812	Mediano
DeepSeek	Qwen 2.5 7B	+0,0167	< 0,001	**Sí**	0,0761	Nulo
DeepSeek	Gemma 3 1B	+0,1499	< 0,001	**Sí**	0,6099	Grande
DeepSeek	Llama 3.2 1B	+0,2006	< 0,001	**Sí**	0,6853	Grande
DeepSeek	Qwen 3 1.7B	+0,1810	< 0,001	**Sí**	0,5975	Grande
Llama 3.3 70B	Qwen 2.5 72B	-0,0102	< 0,001	**Sí**	0,0400	Nulo
Llama 3.370B	Gemma 2 9B	+0,0415	< 0,001	**Sí**	0,2007	Pequeño
Llama 3.3 70B	Llama 3.2 3B	+0,0796	< 0,001	**Sí**	0,3567	Mediano
Llama 3.3 70B	Qwen 2.5 7B	+0,0114	< 0,001	**Sí**	0,0528	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,1446	< 0,001	**Sí**	0,5811	Grande
Llama 3.3 70B	Llama 3.2 1B	+0,1952	< 0,001	**Sí**	0,6632	Grande
Llama 3.3 70B	Qwen 3 1.7B	+0,1757	< 0,001	**Sí**	0,5719	Grande
Qwen 2.5 72B	Gemma 2 9B	+0,0517	< 0,001	**Sí**	0,2368	Pequeño
Qwen 2.5 72B	Llama 3.2 3B	+0,0897	< 0,001	**Sí**	0,4008	Mediano
Qwen 2.5 72B	Qwen 2.5 7B	+0,0216	< 0,001	**Sí**	0,0960	Nulo
Qwen 2.5 72B	Gemma 3 1B	+0,1547	< 0,001	**Sí**	0,6197	Grande
Qwen 2.5 72B	Llama 3.2 1B	+0,2054	< 0,001	**Sí**	0,6902	Grande
Qwen 2.5 72B	Qwen 3 1.7B	+0,1858	< 0,001	**Sí**	0,6026	Grande
Gemma 2 9B	Llama 3.2 3B	+0,0380	< 0,001	**Sí**	0,1730	Pequeño
Gemma 2 9B	Qwen 2.5 7B	-0,0302	< 0,001	**Sí**	0,1423	Pequeño
Gemma 2 9B	Gemma 3 1B	+0,1030	< 0,001	**Sí**	0,4285	Mediano
Gemma 2 9B	Llama 3.2 1B	+0,1537	< 0,001	**Sí**	0,5597	Grande
Gemma 2 9B	Qwen 3 1.7B	+0,1341	< 0,001	**Sí**	0,4400	Mediano
Llama 3.2 3B	Qwen 2.5 7B	-0,0682	< 0,001	**Sí**	0,3063	Mediano
Llama 3.2 3B	Gemma 3 1B	+0,0650	< 0,001	**Sí**	0,2808	Pequeño
Llama 3.2 3B	Llama 3.2 1B	+0,1157	< 0,001	**Sí**	0,4609	Mediano
Llama 3.2 3B	Qwen 3 1.7B	+0,0961	< 0,001	**Sí**	0,3220	Mediano
Qwen 2.5 7B	Gemma 3 1B	+0,1332	< 0,001	**Sí**	0,5331	Grande
Qwen 2.5 7B	Llama 3.2 1B	+0,1838	< 0,001	**Sí**	0,6282	Grande
Qwen 2.5 7B	Qwen 3 1.7B	+0,1643	< 0,001	**Sí**	0,5359	Grande
Gemma 3 1B	Llama 3.2 1B	+0,0507	< 0,001	**Sí**	0,2785	Pequeño
Gemma 3 1B	Qwen 3 1.7B	+0,0311	0,9590	**No**	0,1106	Pequeño
Llama 3.2 1B	Qwen 3 1.7B	-0,0195	< 0,001	**Sí**	0,1092	Pequeño

Tabla Anexo 10: Análisis post-hoc (Nemenyi + Wilcoxon r) para BLEU en WebNLG.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0385	< 0,001	Sí	0,1342	Pequeño
DeepSeek	Qwen 2.5 72B	+0,0037	0,3628	No	0,0163	Nulo
DeepSeek	Gemma 2 9B	+0,0247	< 0,001	Sí	0,0972	Nulo
DeepSeek	Llama 3.2 3B	+0,0680	< 0,001	Sí	0,2555	Pequeño
DeepSeek	Qwen 2.5 7B	+0,0169	< 0,001	Sí	0,0642	Nulo
DeepSeek	Gemma 3 1B	+0,1285	< 0,001	Sí	0,4709	Mediano
DeepSeek	Llama 3.2 1B	+0,1407	< 0,001	Sí	0,4908	Mediano
DeepSeek	Qwen 3 1.7B	+0,0842	< 0,001	Sí	0,3052	Mediano
Llama 3.3 70B	Qwen 2.5 72B	-0,0348	< 0,001	Sí	0,1196	Pequeño
Llama 3.370B	Gemma 2 9B	-0,0138	0,6945	No	0,0363	Nulo
Llama 3.3 70B	Llama 3.2 3B	+0,0295	< 0,001	Sí	0,1275	Pequeño
Llama 3.3 70B	Qwen 2.5 7B	-0,0216	< 0,001	Sí	0,0673	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0900	< 0,001	Sí	0,3613	Mediano
Llama 3.3 70B	Llama 3.2 1B	+0,1022	< 0,001	Sí	0,3980	Mediano
Llama 3.3 70B	Qwen 3 1.7B	+0,0458	< 0,001	Sí	0,1822	Pequeño
Qwen 2.5 72B	Gemma 2 9B	+0,0210	< 0,001	Sí	0,0798	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0643	< 0,001	Sí	0,2369	Pequeño
Qwen 2.5 72B	Qwen 2.5 7B	+0,0132	< 0,001	Sí	0,0496	Nulo
Qwen 2.5 72B	Gemma 3 1B	+0,1248	< 0,001	Sí	0,4528	Mediano
Qwen 2.5 72B	Llama 3.2 1B	+0,1370	< 0,001	Sí	0,4785	Mediano
Qwen 2.5 72B	Qwen 3 1.7B	+0,0806	< 0,001	Sí	0,2859	Pequeño
Gemma 2 9B	Llama 3.2 3B	+0,0433	< 0,001	Sí	0,1531	Pequeño
Gemma 2 9B	Qwen 2.5 7B	-0,0078	< 0,001	Sí	0,0313	Nulo
Gemma 2 9B	Gemma 3 1B	+0,1038	< 0,001	Sí	0,3737	Mediano
Gemma 2 9B	Llama 3.2 1B	+0,1160	< 0,001	Sí	0,4087	Mediano
Gemma 2 9B	Qwen 3 1.7B	+0,0595	< 0,001	Sí	0,2039	Pequeño
Llama 3.2 3B	Qwen 2.5 7B	-0,0511	< 0,001	Sí	0,1875	Pequeño
Llama 3.2 3B	Gemma 3 1B	+0,0605	< 0,001	Sí	0,2441	Pequeño
Llama 3.2 3B	Llama 3.2 1B	+0,0727	< 0,001	Sí	0,2955	Pequeño
Llama 3.2 3B	Qwen 3 1.7B	+0,0162	0,0017	Sí	0,0562	Nulo
Qwen 2.5 7B	Gemma 3 1B	+0,1116	< 0,001	Sí	0,3984	Mediano
Qwen 2.5 7B	Llama 3.2 1B	+0,1238	< 0,001	Sí	0,4309	Mediano
Qwen 2.5 7B	Qwen 3 1.7B	+0,0673	< 0,001	Sí	0,2328	Pequeño
Gemma 3 1B	Llama 3.2 1B	+0,0122	< 0,001	Sí	0,0861	Nulo
Gemma 3 1B	Qwen3 1.7B	-0,0442	< 0,001	Sí	0,2375	Pequeño
Llama 1B	Qwen3 1.7B	-0,0565	< 0,001	Sí	0,2888	Pequeño

Tabla Anexo 11: Análisis post-hoc (Nemenyi + Wilcoxon r) para BERTScore en WebNLG.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0112	< 0,001	**Sí**	0,2185	Pequeño
DeepSeek	Qwen 2.5 72B	+0,0006	0,4668	**No**	0,0174	Nulo
DeepSeek	Gemma 2 9B	+0,0072	< 0,001	**Sí**	0,1468	Pequeño
DeepSeek	Llama 3.2 3B	+0,0226	< 0,001	**Sí**	0,4140	Mediano
DeepSeek	Qwen 2.5 7B	+0,0082	< 0,001	**Sí**	0,1137	Pequeño
DeepSeek	Gemma 3 1B	+0,0364	< 0,001	**Sí**	0,6256	Grande
DeepSeek	Llama 3.2 1B	+0,0555	< 0,001	**Sí**	0,7084	Grande
DeepSeek	Qwen 3 1.7B	+0,0263	< 0,001	**Sí**	0,4581	Mediano
Llama 3.3 70B	Qwen 2.5 72B	-0,0106	< 0,001	**Sí**	0,2043	Pequeño
Llama 3.370B	Gemma 2 9B	-0,0039	< 0,001	**Sí**	0,0639	Nulo
Llama 3.3 70B	Llama 3.2 3B	+0,0114	< 0,001	**Sí**	0,2211	Pequeño
Llama 3.3 70B	Qwen 2.5 7B	-0,0030	< 0,001	**Sí**	0,0857	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0252	< 0,001	**Sí**	0,4601	Mediano
Llama 3.3 70B	Llama 3.2 1B	+0,0443	< 0,001	**Sí**	0,6021	Grande
Llama 3.3 70B	Qwen 3 1.7B	+0,0152	< 0,001	**Sí**	0,2699	Pequeño
Qwen 2.5 72B	Gemma 2 9B	+0,0067	< 0,001	**Sí**	0,1319	Pequeño
Qwen 2.5 72B	Llama 3.2 3B	+0,0220	< 0,001	**Sí**	0,4030	Mediano
Qwen 2.5 72B	Qwen 2.5 7B	+0,0076	< 0,001	**Sí**	0,1039	Pequeño
Qwen 2.5 72B	Gemma 3 1B	+0,0358	< 0,001	**Sí**	0,6090	Grande
Qwen 2.5 72B	Llama 3.2 1B	+0,0549	< 0,001	**Sí**	0,6998	Grande
Qwen 2.5 72B	Qwen 3 1.7B	+0,0258	< 0,001	**Sí**	0,4336	Mediano
Gemma 2 9B	Llama 3.2 3B	+0,0154	< 0,001	**Sí**	0,2789	Pequeño
Gemma 2 9B	Qwen 2.5 7B	+0,0009	0,0025	**Sí**	0,0203	Nulo
Gemma 2 9B	Gemma 3 1B	+0,0292	< 0,001	**Sí**	0,5089	Grande
Gemma 2 9B	Llama 3.2 1B	+0,0483	< 0,001	**Sí**	0,6361	Grande
Gemma 2 9B	Qwen 3 1.7B	+0,0191	< 0,001	**Sí**	0,3202	Mediano
Llama 3.2 3B	Qwen 2.5 7B	-0,0144	< 0,001	**Sí**	0,2735	Pequeño
Llama 3.2 3B	Gemma 3 1B	+0,0138	< 0,001	**Sí**	0,2558	Pequeño
Llama 3.2 3B	Llama 3.2 1B	+0,0329	< 0,001	**Sí**	0,4697	Mediano
Llama 3.2 3B	Qwen 3 1.7B	+0,0037	0,0100	**Sí**	0,0596	Nulo
Qwen 2.5 7B	Gemma 3 1B	+0,0282	< 0,001	**Sí**	0,4760	Mediano
Qwen 2.5 7B	Llama 3.2 1B	+0,0474	< 0,001	**Sí**	0,5988	Grande
Qwen 2.5 7B	Qwen 3 1.7B	+0,0182	< 0,001	**Sí**	0,3153	Mediano
Gemma 3 1B	Llama 3.2 1B	+0,0191	< 0,001	**Sí**	0,3459	Mediano
Gemma 3 1B	Qwen 3 1.7B	-0,0100	< 0,001	**Sí**	0,2410	Pequeño
Llama 3.2 1B	Qwen 3 1.7B	-0,0292	< 0,001	**Sí**	0,4719	Mediano

Tabla Anexo 12: Análisis post-hoc (Nemenyi + Wilcoxon r) para ROUGE-L en ToTTo.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0544	< 0,001	**Sí**	0,2170	Pequeño
DeepSeek	Qwen 2.5 72B	-0,0005	1,0000	**No**	0,0025	Nulo
DeepSeek	Gemma 2 9B	+0,0049	< 0,001	**Sí**	0,0198	Nulo
DeepSeek	Llama 3.2 3B	+0,0331	< 0,001	**Sí**	0,1340	Pequeño
DeepSeek	Qwen 2.5 7B	+0,0370	< 0,001	**Sí**	0,1394	Pequeño
DeepSeek	Gemma 3 1B	+0,0480	< 0,001	**Sí**	0,1949	Pequeño
DeepSeek	Llama 3.2 1B	+0,1105	< 0,001	**Sí**	0,4158	Mediano
DeepSeek	Qwen 3 1.7B	+0,0601	< 0,001	**Sí**	0,2421	Pequeño
Llama 3.3 70B	Qwen 2.5 72B	-0,0549	< 0,001	**Sí**	0,2183	Pequeño
Llama 3.370B	Gemma 2 9B	-0,0495	< 0,001	**Sí**	0,1950	Pequeño
Llama 3.3 70B	Llama 3.2 3B	-0,0212	< 0,001	**Sí**	0,0823	Nulo
Llama 3.3 70B	Qwen 2.5 7B	-0,0173	< 0,001	**Sí**	0,0692	Nulo
Llama 3.3 70B	Gemma 3 1B	-0,0063	< 0,001	**Sí**	0,0231	Nulo
Llama 3.3 70B	Llama 3.2 1B	+0,0561	< 0,001	**Sí**	0,2347	Pequeño
Llama 3.3 70B	Qwen 3 1.7B	+0,0057	< 0,001	**Sí**	0,0290	Nulo
Qwen 2.5 72B	Gemma 2 9B	+0,0054	< 0,001	**Sí**	0,0221	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0336	< 0,001	**Sí**	0,1354	Pequeño
Qwen 2.5 72B	Qwen 2.5 7B	+0,0376	< 0,001	**Sí**	0,1419	Pequeño
Qwen 2.5 72B	Gemma 3 1B	+0,0485	< 0,001	**Sí**	0,1957	Pequeño
Qwen 2.5 72B	Llama 3.2 1B	+0,1110	< 0,001	**Sí**	0,4152	Mediano
Qwen 2.5 72B	Qwen 3 1.7B	+0,0606	< 0,001	**Sí**	0,2418	Pequeño
Gemma 2 9B	Llama 3.2 3B	+0,0282	< 0,001	**Sí**	0,1125	Pequeño
Gemma 2 9B	Qwen 2.5 7B	+0,0322	< 0,001	**Sí**	0,1195	Pequeño
Gemma 2 9B	Gemma 3 1B	+0,0431	< 0,001	**Sí**	0,1719	Pequeño
Gemma 2 9B	Llama 3.2 1B	+0,1056	< 0,001	**Sí**	0,3950	Mediano
Gemma 2 9B	Qwen 3 1.7B	+0,0552	< 0,001	**Sí**	0,2185	Pequeño
Llama 3.2 3B	Qwen 2.5 7B	+0,0039	0,7580	**No**	0,0112	Nulo
Llama 3.2 3B	Gemma 3 1B	+0,0149	< 0,001	**Sí**	0,0598	Nulo
Llama 3.2 3B	Llama 3.2 1B	+0,0774	< 0,001	**Sí**	0,3034	Mediano
Llama 3.2 3B	Qwen 3 1.7B	+0,0270	< 0,001	**Sí**	0,1089	Pequeño
Qwen 2.5 7B	Gemma 3 1B	+0,0110	< 0,001	**Sí**	0,0475	Nulo
Qwen 2.5 7B	Llama 3.2 1B	+0,0734	< 0,001	**Sí**	0,2819	Pequeño
Qwen 2.5 7B	Qwen 3 1.7B	+0,0230	< 0,001	**Sí**	0,0957	Nulo
Gemma 3 1B	Llama 3.2 1B	+0,0625	< 0,001	**Sí**	0,3421	Mediano
Gemma 3 1B	Qwen 3 1.7B	+0,0121	< 0,001	**Sí**	0,0699	Nulo
Llama 3.2 1B	Qwen 3 1.7B	-0,0504	< 0,001	**Sí**	0,2880	Pequeño

Tabla Anexo 13: Análisis post-hoc (Nemenyi + Wilcoxon r) para METEOR en ToTTo.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0157	< 0,001	**Sí**	0,0576	Nulo
DeepSeek	Qwen 2.5 72B	-0,0041	0,0100	**Sí**	0,0146	Nulo
DeepSeek	Gemma 2 9B	+0,0168	< 0,001	**Sí**	0,0612	Nulo
DeepSeek	Llama 3.2 3B	+0,0539	< 0,001	**Sí**	0,1993	Pequeño
DeepSeek	Qwen 2.5 7B	+0,0297	< 0,001	**Sí**	0,1095	Pequeño
DeepSeek	Gemma 3 1B	+0,0454	< 0,001	**Sí**	0,1726	Pequeño
DeepSeek	Llama 3.2 1B	+0,1513	< 0,001	**Sí**	0,4922	Mediano
DeepSeek	Qwen 3 1.7B	+0,0645	< 0,001	**Sí**	0,2414	Pequeño
Llama 3.3 70B	Qwen 2.5 72B	-0,0198	< 0,001	**Sí**	0,0704	Nulo
Llama 3.370B	Gemma 2 9B	+0,0011	0,0108	**Sí**	0,0067	Nulo
Llama 3.3 70B	Llama 3.2 3B	+0,0382	< 0,001	**Sí**	0,1460	Pequeño
Llama 3.3 70B	Qwen 2.5 7B	+0,0139	< 0,001	**Sí**	0,0549	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0296	< 0,001	**Sí**	0,1165	Pequeño
Llama 3.3 70B	Llama 3.2 1B	+0,1356	< 0,001	**Sí**	0,4524	Mediano
Llama 3.3 70B	Qwen 3 1.7B	+0,0487	< 0,001	**Sí**	0,1870	Pequeño
Qwen 2.5 72B	Gemma 2 9B	+0,0209	< 0,001	**Sí**	0,0757	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0580	< 0,001	**Sí**	0,2112	Pequeño
Qwen 2.5 72B	Qwen 2.5 7B	+0,0338	< 0,001	**Sí**	0,1228	Pequeño
Qwen 2.5 72B	Gemma 3 1B	+0,0495	< 0,001	**Sí**	0,1840	Pequeño
Qwen 2.5 72B	Llama 3.2 1B	+0,1555	< 0,001	**Sí**	0,4984	Mediano
Qwen 2.5 72B	Qwen 3 1.7B	+0,0686	< 0,001	**Sí**	0,2513	Pequeño
Gemma 2 9B	Llama 3.2 3B	+0,0371	< 0,001	**Sí**	0,1358	Pequeño
Gemma 2 9B	Qwen 2.5 7B	+0,0129	< 0,001	**Sí**	0,0468	Nulo
Gemma 2 9B	Gemma 3 1B	+0,0286	< 0,001	**Sí**	0,1062	Pequeño
Gemma 2 9B	Llama 3.2 1B	+0,1346	< 0,001	**Sí**	0,4363	Mediano
Gemma 2 9B	Qwen 3 1.7B	+0,0477	< 0,001	**Sí**	0,1735	Pequeño
Llama 3.2 3B	Qwen 2.5 7B	-0,0243	< 0,001	**Sí**	0,0897	Nulo
Llama 3.2 3B	Gemma 3 1B	-0,0086	< 0,001	**Sí**	0,0342	Nulo
Llama 3.2 3B	Llama 3.2 1B	+0,0974	< 0,001	**Sí**	0,3325	Mediano
Llama 3.2 3B	Qwen 3 1.7B	+0,0105	< 0,001	**Sí**	0,0370	Nulo
Qwen 2.5 7B	Gemma 3 1B	+0,0157	< 0,001	**Sí**	0,0595	Nulo
Qwen 2.5 7B	Llama 3.2 1B	+0,1217	< 0,001	**Sí**	0,4038	Mediano
Qwen 2.5 7B	Qwen 3 1.7B	+0,0348	< 0,001	**Sí**	0,1297	Pequeño
Gemma 3 1B	Llama 3.2 1B	+0,1060	< 0,001	**Sí**	0,5241	Grande
Gemma 3 1B	Qwen 3 1.7B	+0,0191	< 0,001	**Sí**	0,1319	Pequeño
Llama 3.2 1B	Qwen 3 1.7B	-0,0869	< 0,001	**Sí**	0,4406	Mediano

Tabla Anexo 14: Análisis post-hoc (Nemenyi + Wilcoxon r) para BLEU en ToTTo.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0362	< 0,001	**Sí**	0,1319	Pequeño
DeepSeek	Qwen 2.5 72B	+0,0029	0,2922	**No**	0,0092	Nulo
DeepSeek	Gemma 2 9B	+0,0036	< 0,001	**Sí**	0,0128	Nulo
DeepSeek	Llama 3.2 3B	+0,0277	< 0,001	**Sí**	0,1028	Pequeño
DeepSeek	Qwen 2.5 7B	+0,0241	< 0,001	**Sí**	0,0868	Nulo
DeepSeek	Gemma 3 1B	+0,0427	< 0,001	**Sí**	0,1580	Pequeño
DeepSeek	Llama 3.2 1B	+0,0763	< 0,001	**Sí**	0,2902	Pequeño
DeepSeek	Qwen 3 1.7B	+0,0448	< 0,001	**Sí**	0,1650	Pequeño
Llama 3.3 70B	Qwen 2.5 72B	-0,0333	< 0,001	**Sí**	0,1230	Pequeño
Llama 3.370B	Gemma 2 9B	-0,0326	< 0,001	**Sí**	0,1185	Pequeño
Llama 3.3 70B	Llama 3.2 3B	-0,0084	0,9999	**No**	0,0250	Nulo
Llama 3.3 70B	Qwen 2.5 7B	-0,0121	< 0,001	**Sí**	0,0417	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0065	< 0,001	**Sí**	0,0286	Nulo
Llama 3.3 70B	Llama 3.2 1B	+0,0402	< 0,001	**Sí**	0,1754	Pequeño
Llama 3.3 70B	Qwen 3 1.7B	+0,0086	< 0,001	**Sí**	0,0380	Nulo
Qwen 2.5 72B	Gemma 2 9B	+0,0007	0,0717	**No**	0,0038	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0249	< 0,001	**Sí**	0,0937	Nulo
Qwen 2.5 72B	Qwen 2.5 7B	+0,0213	< 0,001	**Sí**	0,0782	Nulo
Qwen 2.5 72B	Gemma 3 1B	+0,0398	< 0,001	**Sí**	0,1486	Pequeño
Qwen 2.5 72B	Llama 3.2 1B	+0,0735	< 0,001	**Sí**	0,2808	Pequeño
Qwen 2.5 72B	Qwen 3 1.7B	+0,0419	< 0,001	**Sí**	0,1558	Pequeño
Gemma 2 9B	Llama 3.2 3B	+0,0242	< 0,001	**Sí**	0,0890	Nulo
Gemma 2 9B	Qwen 2.5 7B	+0,0205	< 0,001	**Sí**	0,0737	Nulo
Gemma 2 9B	Gemma 3 1B	+0,0391	< 0,001	**Sí**	0,1422	Pequeño
Gemma 2 9B	Llama 3.2 1B	+0,0727	< 0,001	**Sí**	0,2738	Pequeño
Gemma 2 9B	Qwen 3 1.7B	+0,0412	< 0,001	**Sí**	0,1497	Pequeño
Llama 3.2 3B	Qwen 2.5 7B	-0,0036	< 0,001	**Sí**	0,0162	Nulo
Llama 3.2 3B	Gemma 3 1B	+0,0149	< 0,001	**Sí**	0,0524	Nulo
Llama 3.2 3B	Llama 3.2 1B	+0,0486	< 0,001	**Sí**	0,1916	Pequeño
Llama 3.2 3B	Qwen 3 1.7B	+0,0171	< 0,001	**Sí**	0,0617	Nulo
Qwen 2.5 7B	Gemma 3 1B	+0,0185	< 0,001	**Sí**	0,0687	Nulo
Qwen 2.5 7B	Llama 3.2 1B	+0,0522	< 0,001	**Sí**	0,2081	Pequeño
Qwen 2.5 7B	Qwen 3 1.7B	+0,0207	< 0,001	**Sí**	0,0778	Nulo
Gemma 3 1B	Llama 3.2 1B	+0,0337	< 0,001	**Sí**	0,2081	Pequeño
Gemma 3 1B	Qwen 3 1.7B	+0,0021	0,1735	**No**	0,0109	Nulo
Llama 3.2 1B	Qwen 3 1.7B	-0,0315	< 0,001	**Sí**	0,1975	Pequeño

Tabla Anexo 15: Análisis post-hoc (Nemenyi + Wilcoxon r) para BERTScore en ToTTo.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0142	< 0,001	**Sí**	0,1848	Pequeño
DeepSeek	Qwen 2.5 72B	+0,0009	< 0,001	**Sí**	0,0145	Nulo
DeepSeek	Gemma 2 9B	+0,0038	< 0,001	**Sí**	0,0496	Nulo
DeepSeek	Llama 3.2 3B	+0,0117	< 0,001	**Sí**	0,1542	Pequeño
DeepSeek	Qwen 2.5 7B	+0,0136	< 0,001	**Sí**	0,1559	Pequeño
DeepSeek	Gemma 3 1B	+0,0153	< 0,001	**Sí**	0,2150	Pequeño
DeepSeek	Llama 3.2 1B	+0,0497	< 0,001	**Sí**	0,5152	Grande
DeepSeek	Qwen 3 1.7B	+0,0224	< 0,001	**Sí**	0,2945	Pequeño
Llama 3.3 70B	Qwen 2.5 72B	-0,0134	< 0,001	**Sí**	0,1709	Pequeño
Llama 3.370B	Gemma 2 9B	-0,0105	< 0,001	**Sí**	0,1322	Pequeño
Llama 3.3 70B	Llama 3.2 3B	-0,0025	< 0,001	**Sí**	0,0252	Nulo
Llama 3.3 70B	Qwen 2.5 7B	-0,0006	< 0,001	**Sí**	0,0150	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0011	1,0000	**No**	0,0285	Nulo
Llama 3.3 70B	Llama 3.2 1B	+0,0355	< 0,001	**Sí**	0,3785	Mediano
Llama 3.3 70B	Qwen 3 1.7B	+0,0082	< 0,001	**Sí**	0,1183	Pequeño
Qwen 2.5 72B	Gemma 2 9B	+0,0029	< 0,001	**Sí**	0,0365	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0108	< 0,001	**Sí**	0,1411	Pequeño
Qwen 2.5 72B	Qwen 2.5 7B	+0,0128	< 0,001	**Sí**	0,1444	Pequeño
Qwen 2.5 72B	Gemma 3 1B	+0,0145	< 0,001	**Sí**	0,1997	Pequeño
Qwen 2.5 72B	Llama 3.2 1B	+0,0488	< 0,001	**Sí**	0,5030	Grande
Qwen 2.5 72B	Qwen 3 1.7B	+0,0215	< 0,001	**Sí**	0,2793	Pequeño
Gemma 2 9B	Llama 3.2 3B	+0,0080	< 0,001	**Sí**	0,1033	Pequeño
Gemma 2 9B	Qwen 2.5 7B	+0,0099	< 0,001	**Sí**	0,1090	Pequeño
Gemma 2 9B	Gemma 3 1B	+0,0116	< 0,001	**Sí**	0,1583	Pequeño
Gemma 2 9B	Llama 3.2 1B	+0,0459	< 0,001	**Sí**	0,4689	Mediano
Gemma 2 9B	Qwen 3 1.7B	+0,0186	< 0,001	**Sí**	0,2380	Pequeño
Llama 3.2 3B	Qwen 2.5 7B	+0,0019	0,9999	**No**	0,0113	Nulo
Llama 3.2 3B	Gemma 3 1B	+0,0036	< 0,001	**Sí**	0,0542	Nulo
Llama 3.2 3B	Llama 3.2 1B	+0,0380	< 0,001	**Sí**	0,3943	Mediano
Llama 3.2 3B	Qwen 3 1.7B	+0,0107	< 0,001	**Sí**	0,1396	Pequeño
Qwen 2.5 7B	Gemma 3 1B	+0,0017	< 0,001	**Sí**	0,0409	Nulo
Qwen 2.5 7B	Llama 3.2 1B	+0,0361	< 0,001	**Sí**	0,3686	Mediano
Qwen 2.5 7B	Qwen 3 1.7B	+0,0087	< 0,001	**Sí**	0,1232	Pequeño
Gemma 3 1B	Llama 3.2 1B	+0,0344	< 0,001	**Sí**	0,5323	Grande
Gemma 3 1B	Qwen 3 1.7B	+0,0070	< 0,001	**Sí**	0,1852	Pequeño
Llama 3.2 1B	Qwen 3 1.7B	-0,0273	< 0,001	**Sí**	0,4210	Mediano

Tabla Anexo 16: Análisis post-hoc (Nemenyi + Wilcoxon r) para ROUGE-L en KELM.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0086	< 0,001	**Sí**	0,0341	Nulo
DeepSeek	Qwen 2.5 72B	+0,0014	0,9881	**No**	0,0056	Nulo
DeepSeek	Gemma 2 9B	+0,0050	< 0,001	**Sí**	0,0224	Nulo
DeepSeek	Llama 3.2 3B	+0,0215	< 0,001	**Sí**	0,0949	Nulo
DeepSeek	Qwen 2.5 7B	+0,0138	< 0,001	**Sí**	0,0614	Nulo
DeepSeek	Gemma 3 1B	+0,0165	< 0,001	**Sí**	0,0736	Nulo
DeepSeek	Llama 3.2 1B	+0,0475	< 0,001	**Sí**	0,2142	Pequeño
DeepSeek	Qwen 3 1.7B	+0,0188	< 0,001	**Sí**	0,0818	Nulo
Llama 3.3 70B	Qwen 2.5 72B	-0,0072	< 0,001	**Sí**	0,0304	Nulo
Llama 3.370B	Gemma 2 9B	-0,0036	0,9757	**No**	0,0113	Nulo
Llama 3.3 70B	Llama 3.2 3B	+0,0129	< 0,001	**Sí**	0,0650	Nulo
Llama 3.3 70B	Qwen 2.5 7B	+0,0052	< 0,001	**Sí**	0,0284	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0080	< 0,001	**Sí**	0,0391	Nulo
Llama 3.3 70B	Llama 3.2 1B	+0,0389	< 0,001	**Sí**	0,1876	Pequeño
Llama 3.3 70B	Qwen 3 1.7B	+0,0102	< 0,001	**Sí**	0,0476	Nulo
Qwen 2.5 72B	Gemma 2 9B	+0,0036	< 0,001	**Sí**	0,0186	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0201	< 0,001	**Sí**	0,0936	Nulo
Qwen 2.5 72B	Qwen 2.5 7B	+0,0124	< 0,001	**Sí**	0,0576	Nulo
Qwen 2.5 72B	Gemma 3 1B	+0,0151	< 0,001	**Sí**	0,0697	Nulo
Qwen 2.5 72B	Llama 3.2 1B	+0,0461	< 0,001	**Sí**	0,2126	Pequeño
Qwen 2.5 72B	Qwen 3 1.7B	+0,0174	< 0,001	**Sí**	0,0784	Nulo
Gemma 2 9B	Llama 3.2 3B	+0,0165	< 0,001	**Sí**	0,0738	Nulo
Gemma 2 9B	Qwen 2.5 7B	+0,0088	< 0,001	**Sí**	0,0382	Nulo
Gemma 2 9B	Gemma 3 1B	+0,0115	< 0,001	**Sí**	0,0476	Nulo
Gemma 2 9B	Llama 3.2 1B	+0,0425	< 0,001	**Sí**	0,1925	Pequeño
Gemma 2 9B	Qwen 3 1.7B	+0,0138	< 0,001	**Sí**	0,0569	Nulo
Llama 3.2 3B	Qwen 2.5 7B	-0,0077	< 0,001	**Sí**	0,0325	Nulo
Llama 3.2 3B	Gemma 3 1B	-0,0050	< 0,001	**Sí**	0,0243	Nulo
Llama 3.2 3B	Llama 3.2 1B	+0,0260	< 0,001	**Sí**	0,1282	Pequeño
Llama 3.2 3B	Qwen 3 1.7B	-0,0027	< 0,001	**Sí**	0,0152	Nulo
Qwen 2.5 7B	Gemma 3 1B	+0,0027	0,9831	**No**	0,0115	Nulo
Qwen 2.5 7B	Llama 3.2 1B	+0,0337	< 0,001	**Sí**	0,1575	Pequeño
Qwen 2.5 7B	Qwen 3 1.7B	+0,0050	0,4026	**No**	0,0199	Nulo
Gemma 3 1B	Llama 3.2 1B	+0,0309	< 0,001	**Sí**	0,1734	Pequeño
Gemma 3 1B	Qwen 3 1.7B	+0,0023	0,9619	**No**	0,0407	Nulo
Llama 3.2 1B	Qwen 3 1.7B	-0,0287	< 0,001	**Sí**	0,1620	Pequeño

Tabla Anexo 17: Análisis post-hoc (Nemenyi + Wilcoxon r) para METEOR en KELM.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	-0,0123	< 0,001	**Sí**	0,0584	Nulo
DeepSeek	Qwen 2.5 72B	-0,0037	< 0,001	**Sí**	0,0185	Nulo
DeepSeek	Gemma 2 9B	+0,0089	< 0,001	**Sí**	0,0417	Nulo
DeepSeek	Llama 3.2 3B	+0,0028	0,8184	**No**	0,0094	Nulo
DeepSeek	Qwen 2.5 7B	-0,0002	0,2812	**No**	0,0056	Nulo
DeepSeek	Gemma 3 1B	+0,0061	0,1986	**No**	0,0238	Nulo
DeepSeek	Llama 3.2 1B	+0,0280	< 0,001	**Sí**	0,1253	Pequeño
DeepSeek	Qwen 3 1.7B	+0,0114	< 0,001	**Sí**	0,0511	Nulo
Llama 3.3 70B	Qwen 2.5 72B	+0,0087	< 0,001	**Sí**	0,0379	Nulo
Llama 3.370B	Gemma 2 9B	+0,0213	< 0,001	**Sí**	0,0967	Nulo
Llama 3.3 70B	Llama 3.2 3B	+0,0152	< 0,001	**Sí**	0,0685	Nulo
Llama 3.3 70B	Qwen 2.5 7B	+0,0121	< 0,001	**Sí**	0,0534	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0184	< 0,001	**Sí**	0,0797	Nulo
Llama 3.3 70B	Llama 3.2 1B	+0,0403	< 0,001	**Sí**	0,1811	Pequeño
Llama 3.3 70B	Qwen 3 1.7B	+0,0238	< 0,001	**Sí**	0,1058	Pequeño
Qwen 2.5 72B	Gemma 2 9B	+0,0126	< 0,001	**Sí**	0,0598	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0065	< 0,001	**Sí**	0,0292	Nulo
Qwen 2.5 72B	Qwen 2.5 7B	+0,0035	0,1259	**No**	0,0147	Nulo
Qwen 2.5 72B	Gemma 3 1B	+0,0097	< 0,001	**Sí**	0,0435	Nulo
Qwen 2.5 72B	Llama 3.2 1B	+0,0316	< 0,001	**Sí**	0,1454	Pequeño
Qwen 2.5 72B	Qwen 3 1.7B	+0,0151	< 0,001	**Sí**	0,0709	Nulo
Gemma 2 9B	Llama 3.2 3B	-0,0061	< 0,001	**Sí**	0,0302	Nulo
Gemma 2 9B	Qwen 2.5 7B	-0,0091	< 0,001	**Sí**	0,0466	Nulo
Gemma 2 9B	Gemma 3 1B	-0,0029	< 0,001	**Sí**	0,0180	Nulo
Gemma 2 9B	Llama 3.2 1B	+0,0190	< 0,001	**Sí**	0,0847	Nulo
Gemma 2 9B	Qwen 3 1.7B	+0,0025	0,3107	**No**	0,0088	Nulo
Llama 3.2 3B	Qwen 2.5 7B	-0,0030	0,0022	**Sí**	0,0156	Nulo
Llama 3.2 3B	Gemma 3 1B	+0,0032	0,9864	**No**	0,0113	Nulo
Llama 3.2 3B	Llama 3.2 1B	+0,0251	< 0,001	**Sí**	0,1180	Pequeño
Llama 3.2 3B	Qwen 3 1.7B	+0,0086	< 0,001	**Sí**	0,0395	Nulo
Qwen 2.5 7B	Gemma 3 1B	+0,0063	< 0,001	**Sí**	0,0302	Nulo
Qwen 2.5 7B	Llama 3.2 1B	+0,0282	< 0,001	**Sí**	0,1334	Pequeño
Qwen 2.5 7B	Qwen 3 1.7B	+0,0116	< 0,001	**Sí**	0,0573	Nulo
Gemma 3 1B	Llama 3.2 1B	+0,0219	< 0,001	**Sí**	0,1245	Pequeño
Gemma 3 1B	Qwen 3 1.7B	+0,0054	< 0,001	**Sí**	0,0983	Nulo
Llama 3.2 1B	Qwen 3 1.7B	-0,0165	< 0,001	**Sí**	0,0889	Nulo

Tabla Anexo 18: Análisis post-hoc (Nemenyi + Wilcoxon r) para BLEU en KELM.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	-0,0044	< 0,001	**Sí**	0,0378	Nulo
DeepSeek	Qwen 2.5 72B	-0,0012	0,0845	**No**	0,0159	Nulo
DeepSeek	Gemma 2 9B	-0,0013	0,0450	**Sí**	0,0105	Nulo
DeepSeek	Llama 3.2 3B	+0,0024	0,0642	**No**	0,0139	Nulo
DeepSeek	Qwen 2.5 7B	+0,0011	0,9972	**No**	0,0017	Nulo
DeepSeek	Gemma 3 1B	+0,0047	< 0,001	**Sí**	0,0293	Nulo
DeepSeek	Llama 3.2 1B	+0,0142	< 0,001	**Sí**	0,1094	Pequeño
DeepSeek	Qwen 3 1.7B	+0,0083	< 0,001	**Sí**	0,0634	Nulo
Llama 3.3 70B	Qwen 2.5 72B	+0,0032	0,1746	**No**	0,0214	Nulo
Llama 3.370B	Gemma 2 9B	+0,0031	0,2786	**No**	0,0265	Nulo
Llama 3.3 70B	Llama 3.2 3B	+0,0068	< 0,001	**Sí**	0,0519	Nulo
Llama 3.3 70B	Qwen 2.5 7B	+0,0055	< 0,001	**Sí**	0,0395	Nulo
Llama 3.3 70B	Gemma 3 1B	+0,0091	< 0,001	**Sí**	0,0654	Nulo
Llama 3.3 70B	Llama 3.2 1B	+0,0186	< 0,001	**Sí**	0,1457	Pequeño
Llama 3.3 70B	Qwen 3 1.7B	+0,0127	< 0,001	**Sí**	0,0984	Nulo
Qwen 2.5 72B	Gemma 2 9B	-0,0000	1,0000	**No**	0,0053	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0036	< 0,001	**Sí**	0,0313	Nulo
Qwen 2.5 72B	Qwen 2.5 7B	+0,0024	0,4522	**No**	0,0190	Nulo
Qwen 2.5 72B	Gemma 3 1B	+0,0060	< 0,001	**Sí**	0,0454	Nulo
Qwen 2.5 72B	Llama 3.2 1B	+0,0155	< 0,001	**Sí**	0,1276	Pequeño
Qwen 2.5 72B	Qwen 3 1.7B	+0,0095	< 0,001	**Sí**	0,0792	Nulo
Gemma 2 9B	Llama 3.2 3B	+0,0036	< 0,001	**Sí**	0,0252	Nulo
Gemma 2 9B	Qwen 2.5 7B	+0,0024	0,3122	**No**	0,0121	Nulo
Gemma 2 9B	Gemma 3 1B	+0,0060	< 0,001	**Sí**	0,0395	Nulo
Gemma 2 9B	Llama 3.2 1B	+0,0155	< 0,001	**Sí**	0,1217	Pequeño
Gemma 2 9B	Qwen 3 1.7B	+0,0095	< 0,001	**Sí**	0,0725	Nulo
Llama 3.2 3B	Qwen 2.5 7B	-0,0012	0,0045	**Sí**	0,0125	Nulo
Llama 3.2 3B	Gemma 3 1B	+0,0024	0,9209	**No**	0,0136	Nulo
Llama 3.2 3B	Llama 3.2 1B	+0,0119	< 0,001	**Sí**	0,0967	Nulo
Llama 3.2 3B	Qwen 3 1.7B	+0,0059	< 0,001	**Sí**	0,0480	Nulo
Qwen 2.5 7B	Gemma 3 1B	+0,0036	< 0,001	**Sí**	0,0275	Nulo
Qwen 2.5 7B	Llama 3.2 1B	+0,0131	< 0,001	**Sí**	0,1095	Pequeño
Qwen 2.5 7B	Qwen 3 1.7B	+0,0071	< 0,001	**Sí**	0,0625	Nulo
Gemma 3 1B	Llama 3.2 1B	+0,0095	< 0,001	**Sí**	0,0929	Nulo
Gemma 3 1B	Qwen 3 1.7B	+0,0035	< 0,001	**Sí**	0,0734	Nulo
Llama 3.2 1B	Qwen 3 1.7B	-0,0060	< 0,001	**Sí**	0,0530	Nulo

Tabla Anexo 19: Análisis post-hoc (Nemenyi + Wilcoxon r) para BERTScore en KELM.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
DeepSeek	Llama 3.3 70B	+0,0050	< 0,001	**Sí**	0,0504	Nulo
DeepSeek	Qwen 2.5 72B	+0,0013	< 0,001	**Sí**	0,0159	Nulo
DeepSeek	Gemma 2 9B	+0,0004	0,7772	**No**	0,0028	Nulo
DeepSeek	Llama 3.2 3B	+0,0044	< 0,001	**Sí**	0,0500	Nulo
DeepSeek	Qwen 2.5 7B	+0,0062	< 0,001	**Sí**	0,0738	Nulo
DeepSeek	Gemma 3 1B	+0,0043	< 0,001	**Sí**	0,0508	Nulo
DeepSeek	Llama 3.2 1B	+0,0237	< 0,001	**Sí**	0,2538	Pequeño
DeepSeek	Qwen 3 1.7B	+0,0066	< 0,001	**Sí**	0,0818	Nulo
Llama 3.3 70B	Qwen 2.5 72B	-0,0036	< 0,001	**Sí**	0,0367	Nulo
Llama 3.370B	Gemma 2 9B	-0,0045	< 0,001	**Sí**	0,0471	Nulo
Llama 3.3 70B	Llama 3.2 3B	-0,0005	1,0000	**No**	0,0001	Nulo
Llama 3.3 70B	Qwen 2.5 7B	+0,0013	< 0,001	**Sí**	0,0224	Nulo
Llama 3.3 70B	Gemma 3 1B	-0,0007	0,1715	**No**	0,0014	Nulo
Llama 3.3 70B	Llama 3.2 1B	+0,0187	< 0,001	**Sí**	0,2062	Pequeño
Llama 3.3 70B	Qwen 3 1.7B	+0,0017	< 0,001	**Sí**	0,0290	Nulo
Qwen 2.5 72B	Gemma 2 9B	-0,0009	0,1744	**No**	0,0115	Nulo
Qwen 2.5 72B	Llama 3.2 3B	+0,0031	< 0,001	**Sí**	0,0347	Nulo
Qwen 2.5 72B	Qwen 2.5 7B	+0,0049	< 0,001	**Sí**	0,0584	Nulo
Qwen 2.5 72B	Gemma 3 1B	+0,0029	< 0,001	**Sí**	0,0346	Nulo
Qwen 2.5 72B	Llama 3.2 1B	+0,0224	< 0,001	**Sí**	0,2415	Pequeño
Qwen 2.5 72B	Qwen 3 1.7B	+0,0053	< 0,001	**Sí**	0,0660	Nulo
Gemma 2 9B	Llama 3.2 3B	+0,0040	< 0,001	**Sí**	0,0475	Nulo
Gemma 2 9B	Qwen 2.5 7B	+0,0058	< 0,001	**Sí**	0,0687	Nulo
Gemma 2 9B	Gemma 3 1B	+0,0038	< 0,001	**Sí**	0,0450	Nulo
Gemma 2 9B	Llama 3.2 1B	+0,0233	< 0,001	**Sí**	0,2459	Pequeño
Gemma 2 9B	Qwen 3 1.7B	+0,0062	< 0,001	**Sí**	0,0753	Nulo
Llama 3.2 3B	Qwen 2.5 7B	+0,0018	< 0,001	**Sí**	0,0227	Nulo
Llama 3.2 3B	Gemma 3 1B	-0,0002	0,0603	**No**	0,0020	Nulo
Llama 3.2 3B	Llama 3.2 1B	+0,0192	< 0,001	**Sí**	0,2039	Pequeño
Llama 3.2 3B	Qwen 3 1.7B	+0,0022	< 0,001	**Sí**	0,0285	Nulo
Qwen 2.5 7B	Gemma 3 1B	-0,0020	< 0,001	**Sí**	0,0242	Nulo
Qwen 2.5 7B	Llama 3.2 1B	+0,0175	< 0,001	**Sí**	0,1866	Pequeño
Qwen 2.5 7B	Qwen 3 1.7B	+0,0004	0,1348	**No**	0,0076	Nulo
Gemma 3 1B	Llama 3.2 1B	+0,0194	< 0,001	**Sí**	0,2510	Pequeño
Gemma 3 1B	Qwen 3 1.7B	+0,0024	< 0,001	**Sí**	0,1033	Pequeño
Llama 3.2 1B	Qwen 3 1.7B	-0,0171	< 0,001	**Sí**	0,2174	Pequeño

D.2.4 Critical Difference Diagram (CD)

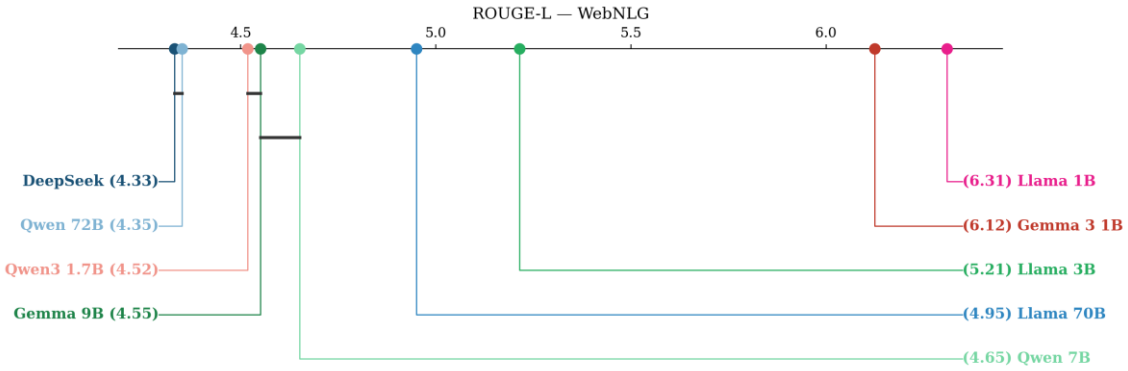


Figura Anexo 39: Diagrama de diferencia crítica para ROUGE-L en WebNLG. Fuente propia.

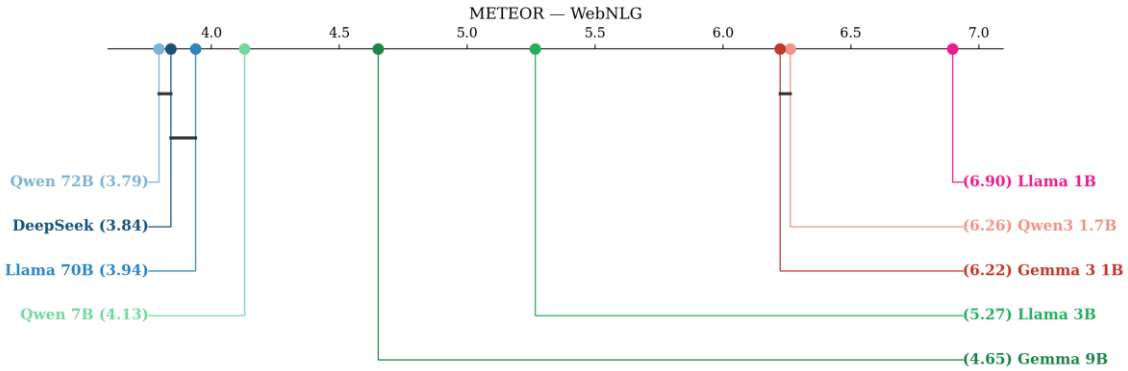


Figura Anexo 40: Diagrama de diferencia crítica para METEOR en WebNLG. Fuente propia.

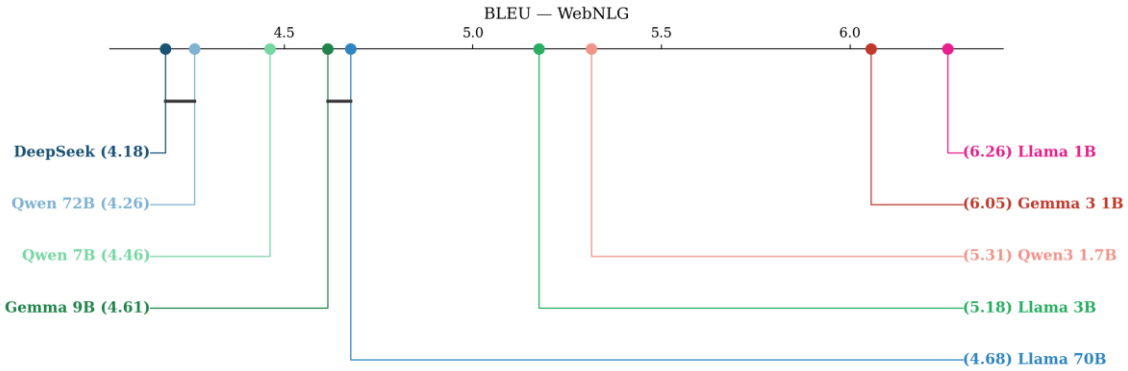


Figura Anexo 41: Diagrama de diferencia crítica para BLEU en WebNLG. Fuente propia.

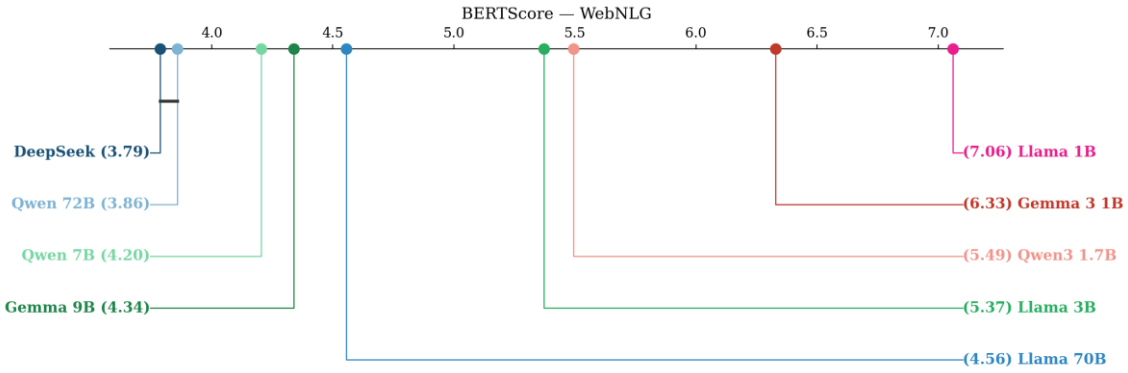


Figura Anexo 42: Diagrama de diferencia crítica para BERTScore en WebNLG. Fuente propia.

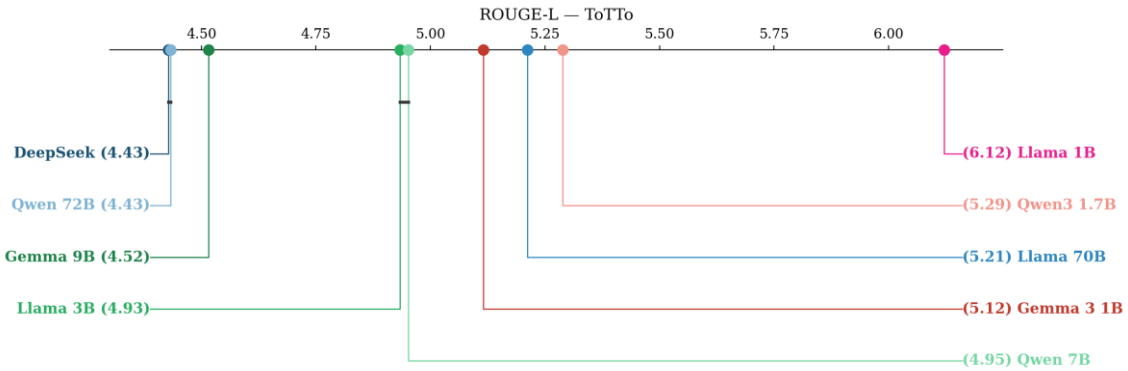


Figura Anexo 43: Diagrama de diferencia crítica para ROUGE-L en ToTTo. Fuente propia.

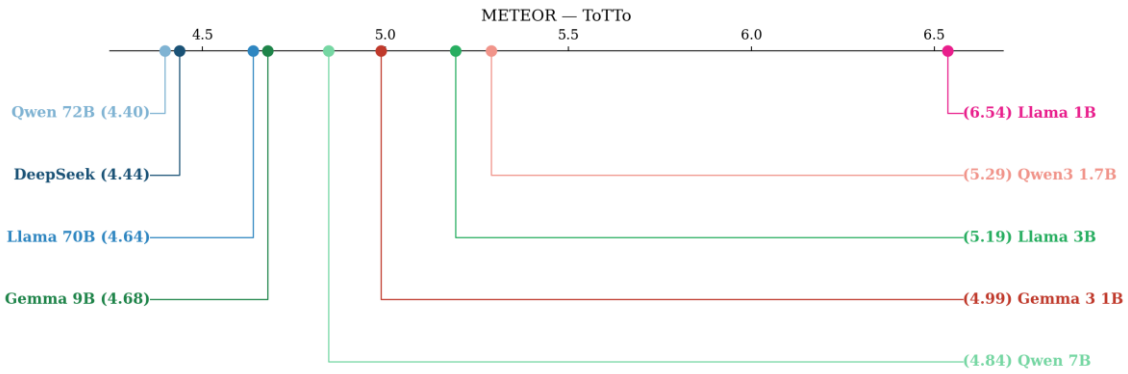


Figura Anexo 44: Diagrama de diferencia crítica para METEOR en ToTTo. Fuente propia.

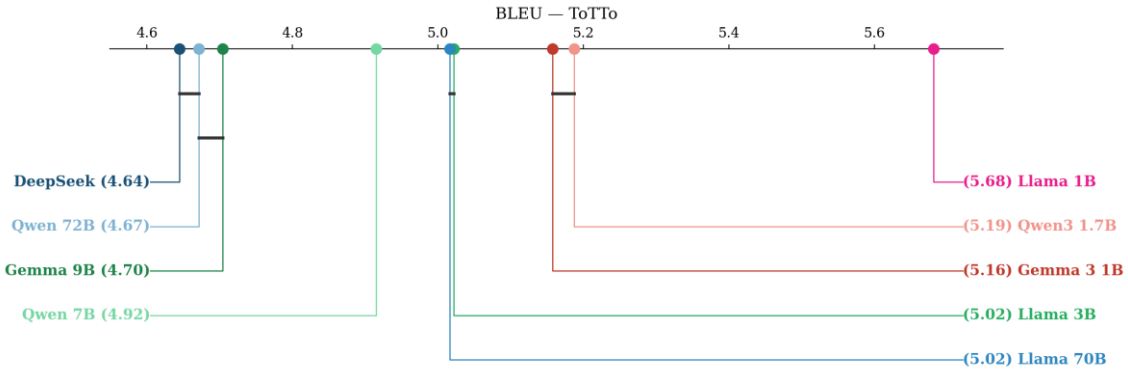


Figura Anexo 45: Diagrama de diferencia crítica para BLEU en ToTTo. Fuente propia.

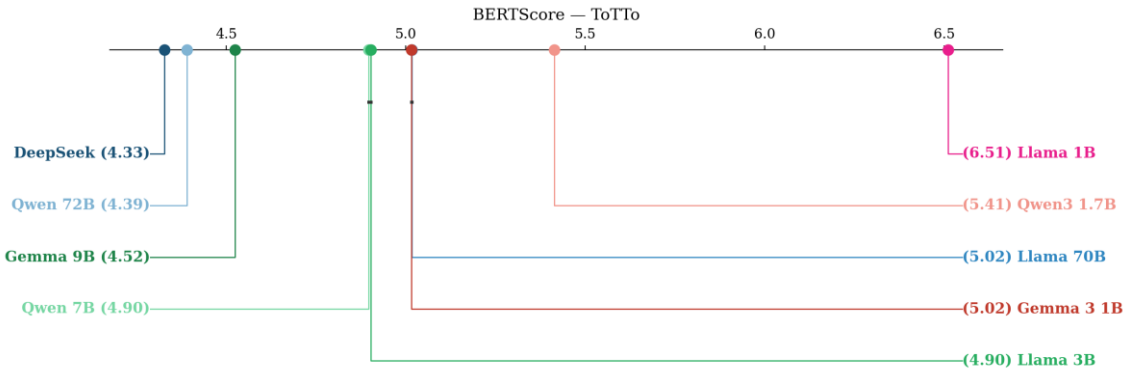


Figura Anexo 46: Diagrama de diferencia crítica para BERTScore en ToTTo. Fuente propia.

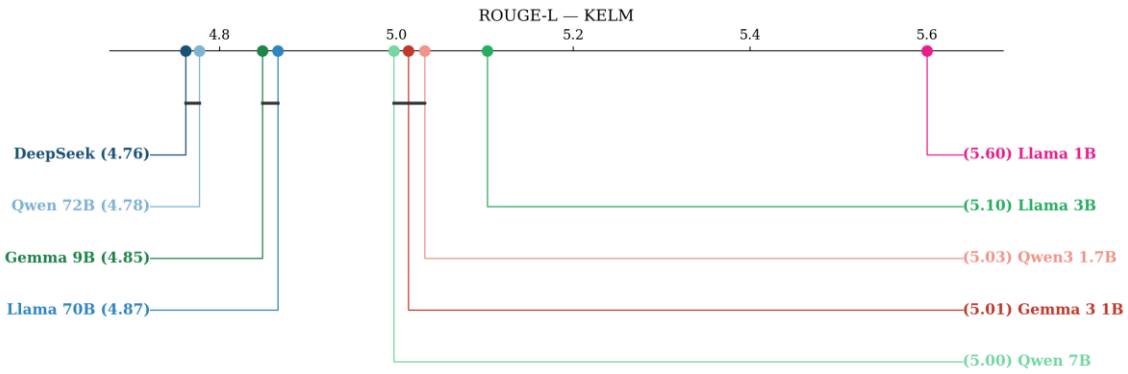


Figura Anexo 47: Diagrama de diferencia crítica para ROUGE-L en KELM. Fuente propia.

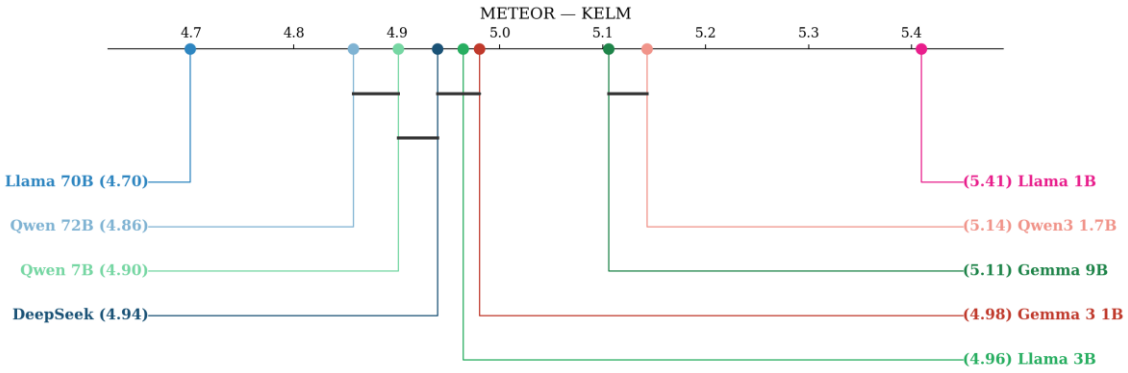


Figura Anexo 48: Diagrama de diferencia crítica para METEOR en KELM. Fuente propia.

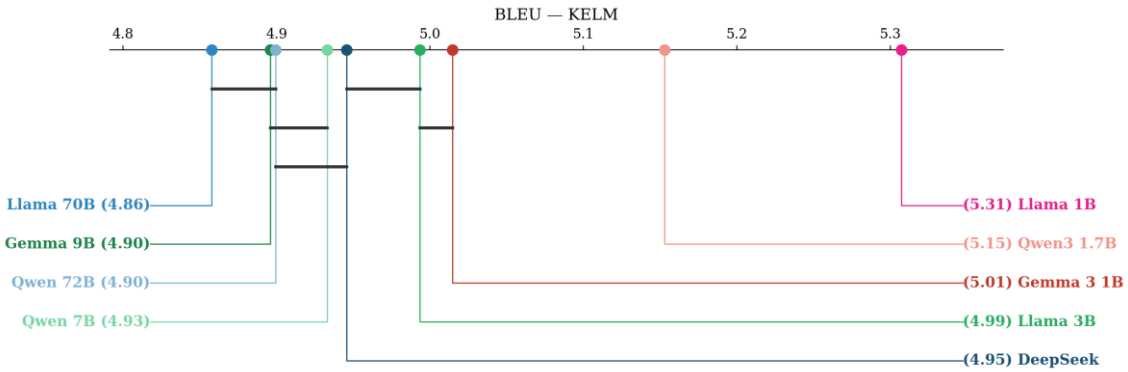


Figura Anexo 49: Diagrama de diferencia crítica para BLEU en KELM. Fuente propia.

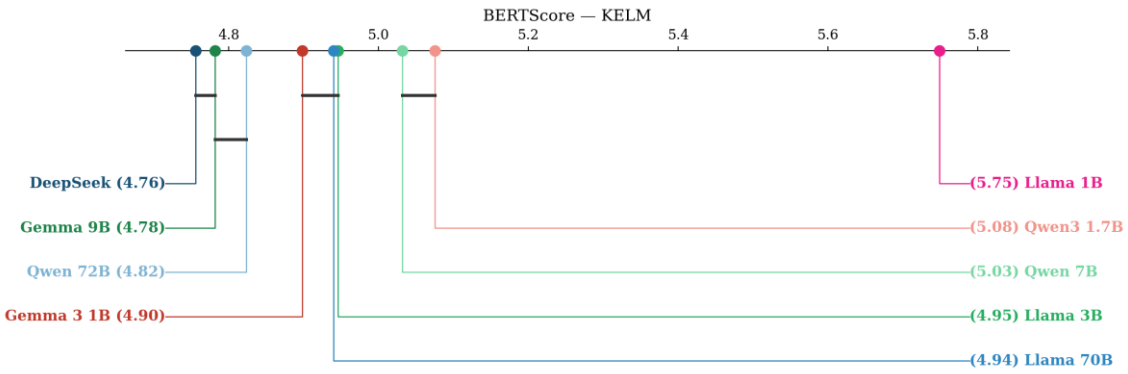


Figura Anexo 50: Diagrama de diferencia crítica para BERTScore en KELM. Fuente propia.

D.3 Comparativa 2: Mini-SLM vs Fine-tuning vs GAN

Esta sección presenta los resultados obtenidos en la comparativa entre los modelos Mini-SLM base, Mini-SLM *fine-tuned* y la GAN sobre el corpus propio de telecomunicaciones. Se incluyen las métricas de calidad textual (ROUGE-L, METEOR, BLEU y BERTScore), los tiempos de inferencia, los costes asociados, las emisiones de Co2 y las curvas de aprendizaje de la GAN y los modelos *fine-tuned*. Además, se aportan los resultados del test de Friedman, el análisis *post-hoc* de Nemenyi con tamaño del efecto de Wilcoxon y los diagramas de diferencia crítica correspondiente.

D.3.1 Métricas de calidad

Tabla Anexo 20: Costes y Co2 por modelo en el corpus propio.

Tipo	Modelo	Corpus Propio	
		Coste ($)	*Co2 (g)*
Mini-SLM	Gemma 3 1B	0,0843	40,041
Mini-SLM	Llama 3.2 1B	0,0440	35,381
Mini-SLM	Qwen 3 1B	0,2223	47,578
Fine-Tuning	Gemma 3 1B FT	0,0000	15,391
Fine-Tuning	Llama 3.2 1B FT	0,0000	33,795
Fine-Tuning	Qwen 3 1B FT	0,0000	20,419
GAN	GAN	0,0000	2,589

Tabla Anexo 21: Tiempo de inferencia por modelo en el corpus propio.

Tipo	Modelo	Corpus Propio
		Tiempo (s/fila)
Mini-SLM	Gemma 3 1B	2,9080
Mini-SLM	Llama 3.2 1B	2,6152
Mini-SLM	Qwen 3 1B	2,4696
Fine-Tuning	Gemma 3 1B FT	2,2837
Fine-Tuning	Llama 3.2 1B FT	4,8929
Fine-Tuning	Qwen 3 1B FT	2,4622
GAN	GAN	0,2064

Tabla Anexo 22: Métricas de calidad por modelo en el corpus propio.

Tipo	Modelo	ROUGE-L		METEOR		BLEU		BERTScore	
		Media + s	*IC 95%*	*Media + s*	*IC 95%*	*Media + s*	*IC 95%*	*Media + s*	*IC 95%*
Mini-SLM	Gemma 3 1B	0,1348 ± 0,0401	[0,1339; 0,1357]	0,2116 ± 0,0546	[0,2104; 0,2128]	0,0078 ± 0,0155	[0,0075; 0,0082]	0,8400 ± 0,0135	[0,8397; 0,8403]
Mini-SLM	Llama 3.2 1B	0,1321 ± 0,0475	[0,1311; 0,1332]	0,1982 ± 0,0583	[0,1969; 0,1995]	0,0092 ± 0,0213	[0,0087; 0,0097]	0,8451 ± 0,0178	[0,8447; 0,8455]
Mini-SLM	Qwen 3 1B	0,1585 ± 0,0511	[0,1574; 0,1596]	0,2321 ± 0,0639	[0,2306; 0,2335]	0,0110 ± 0,0215	[0,0105; 0,0115]	0,8459 ± 0,0158	[0,8455; 0,8462]
Fine-Tuning	Gemma 3 1B FT	0,2635 ± 0,0967	[0,2613; 0,2657]	0,2644 ± 0,1043	[0,2621; 0,2668]	0,0527 ± 0,0823	[0,0508; 0,0545]	0,8889 ± 0,0249	[0,8884; 0,8895]
Fine-Tuning	Llama 3.2 1B FT	0,1460 ± 0,0604	[0,1447; 0,1474]	0,2218 ± 0,0717	[0,2202; 0,2234]	0,0198 ± 0,0339	[0,0190; 0,0205]	0,8593 ± 0,0209	[0,8589; 0,8598]
Fine-Tuning	Qwen 3 1B FT	0,2860 ± 0,1031	[0,2837; 0,2883]	0,2891 ± 0,1116	[0,2867; 0,2917]	0,0693 ± 0,0957	[0,0672; 0,0714]	0,8947 ± 0,0236	[0,8942; 0,8952]
GAN	GAN	0,1466 ± 0,0586	[0,1453; 0,1479]	0,1155 ± 0,0508	[0,1144; 0,1167]	0,0009 ± 0,0086	[0,0007; 0,0011]	0,8558 ± 0,0221	[0,8553; 0,8563]

D.3.2 Test estadístico

Tabla Anexo 23: Resultados del test de Friedman por modelo en el corpus propio.

Métrica	Test	Estadístico	p-valor	Sig.	N
ROUGE-L	Friedman (χ^2)	1.392,24	$1{,}16 \cdot 10^{-297}$	**Sí**	7.695
METEOR	Friedman (χ^2)	1.021,41	$2{,}10 \cdot 10^{-217}$	**Sí**	7.695
BERTScore	Friedman (χ^2)	1.735,93	$< 10^{-300}$	**Sí**	7.695
BLEU	Friedman (χ^2)	403,25	$5{,}59 \cdot 10^{-84}$	**Sí**	7.695

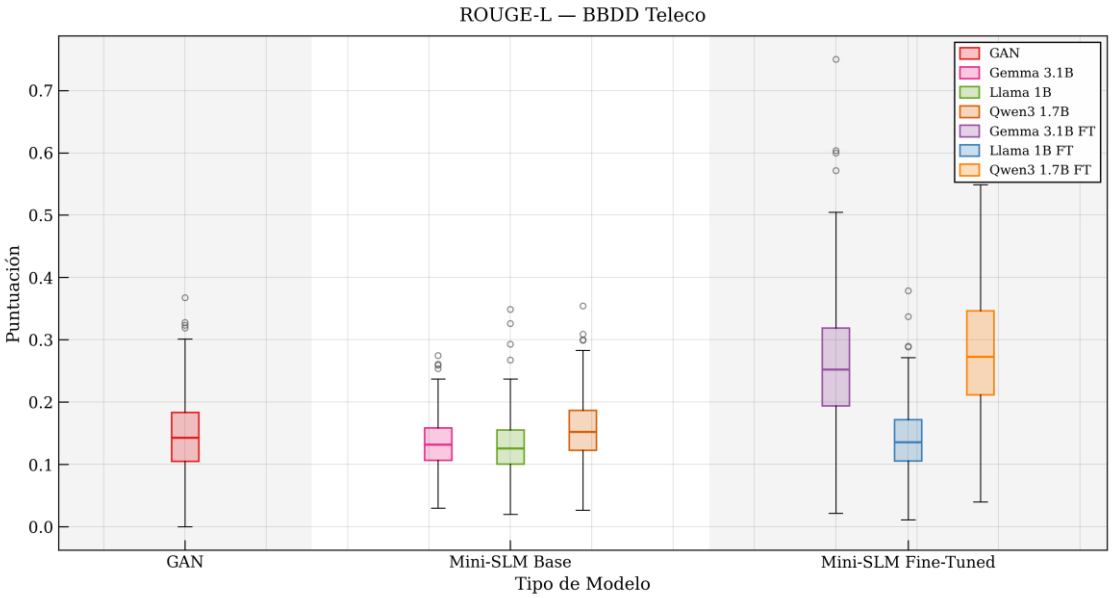


Figura Anexo 51: Boxplots de ROUGE-L por modelo en el corpus propio. Fuente propia.

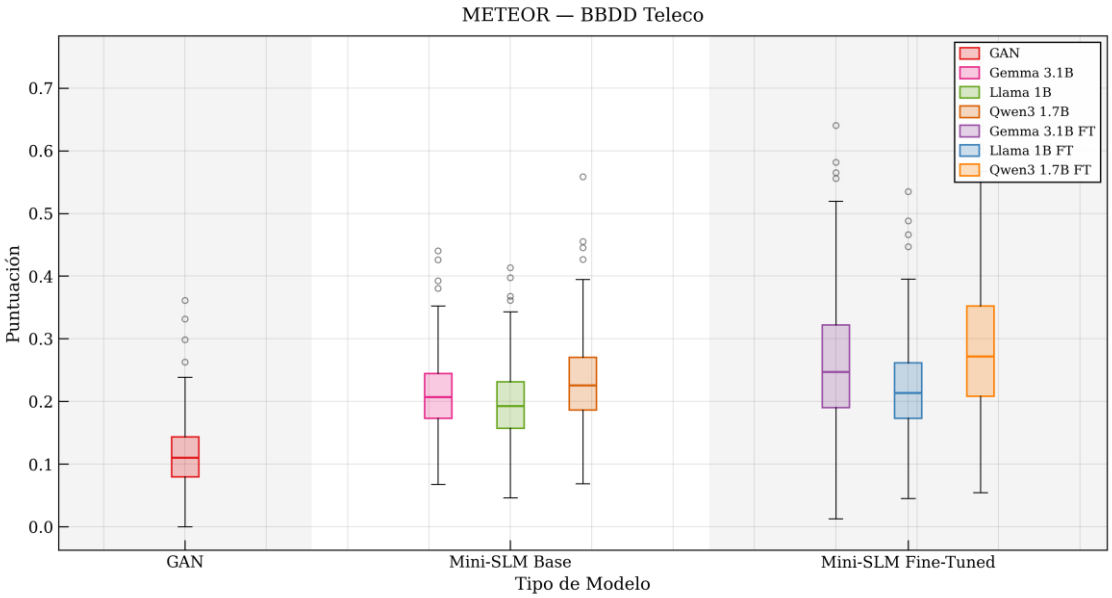


Figura Anexo 52: Boxplots de METEOR por modelo en el corpus propio. Fuente propia.

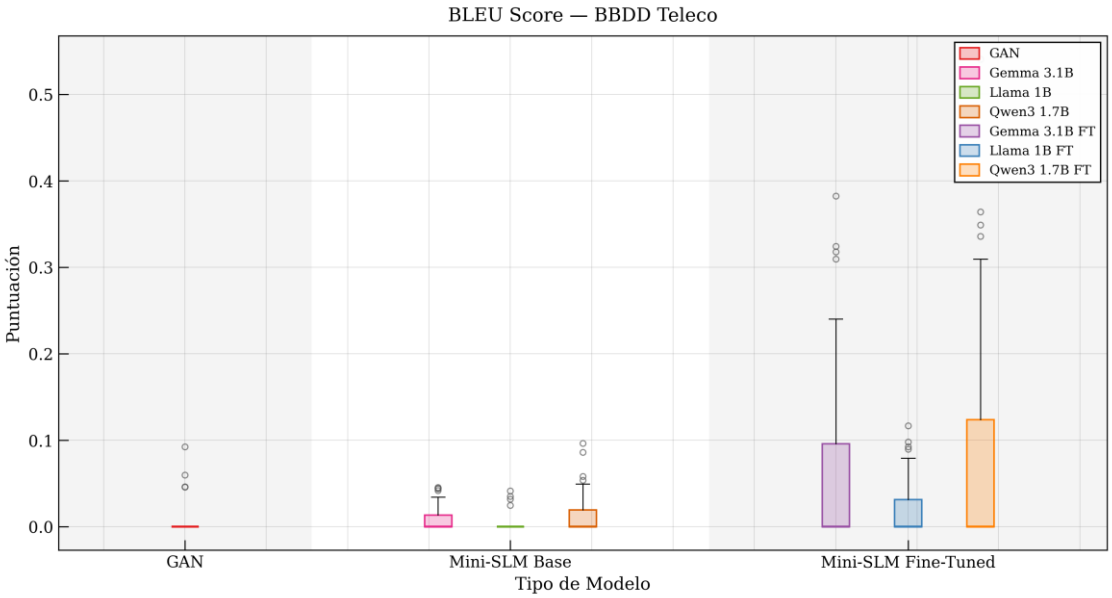


Figura Anexo 53: Boxplots de BLEU por modelo en el corpus propio. Fuente propia.

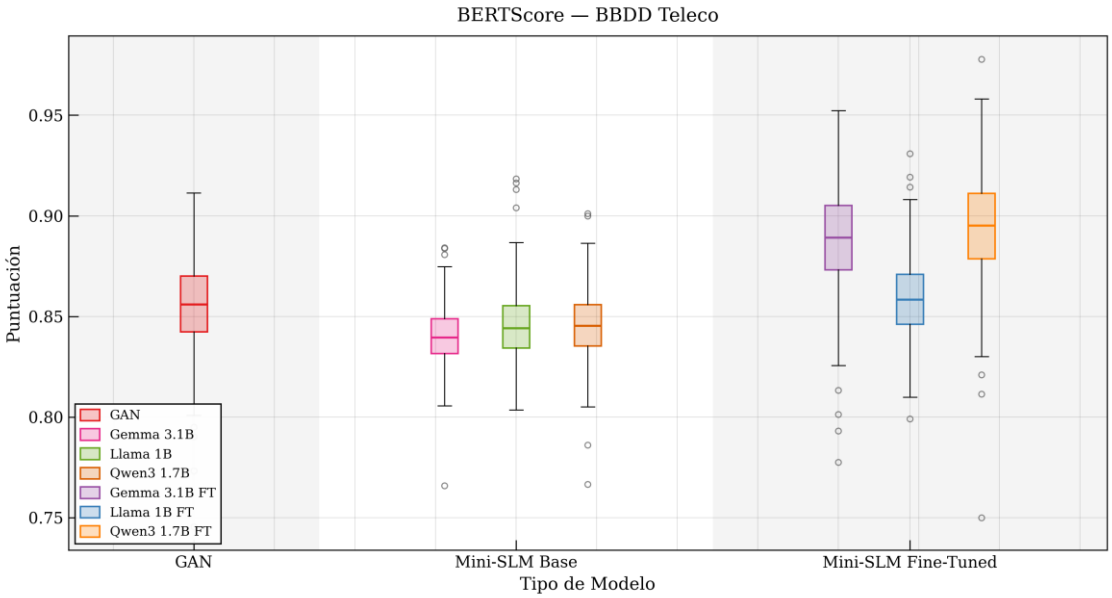


Figura Anexo 54: Boxplots de BERTScore por modelo en el corpus propio. Fuente propia.

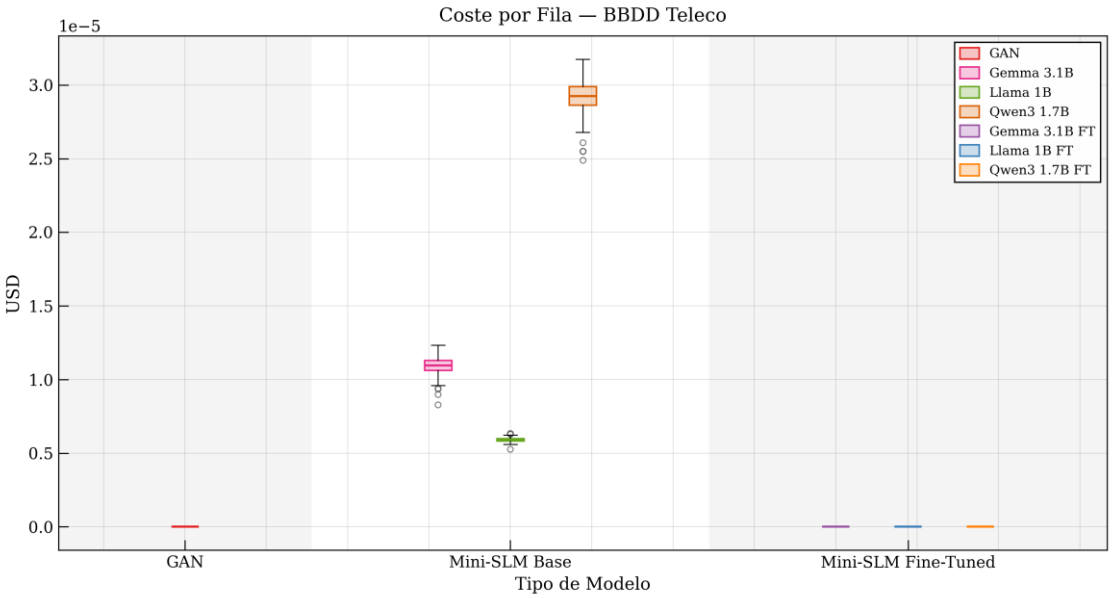


Figura Anexo 55: Boxplots de coste por modelo en el corpus propio. Fuente propia.

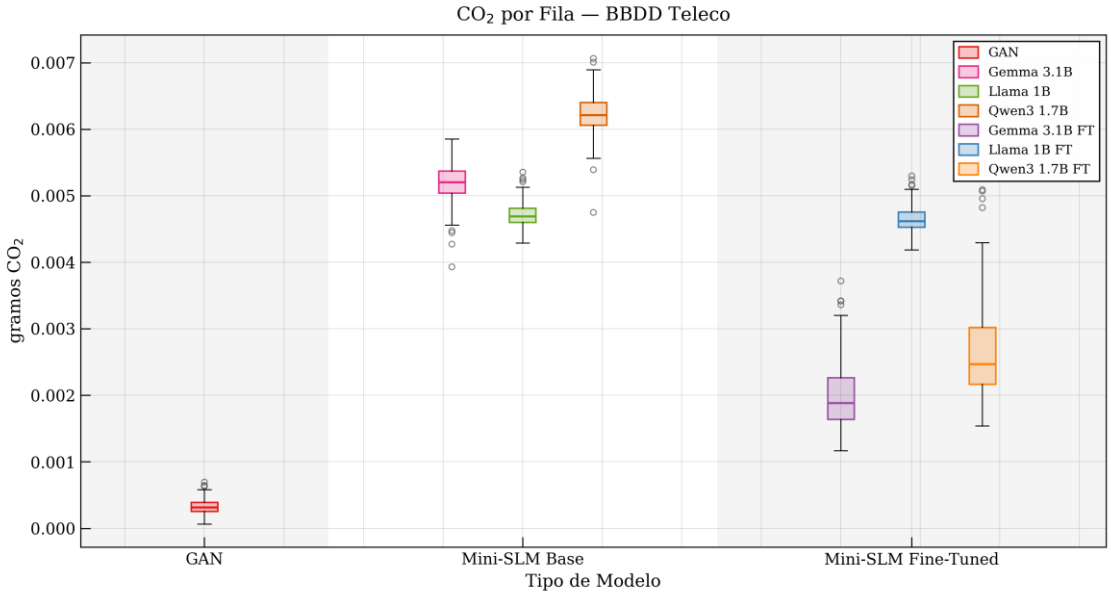


Figura Anexo 56: Boxplots de Co2 por modelo en el corpus propio. Fuente propia.

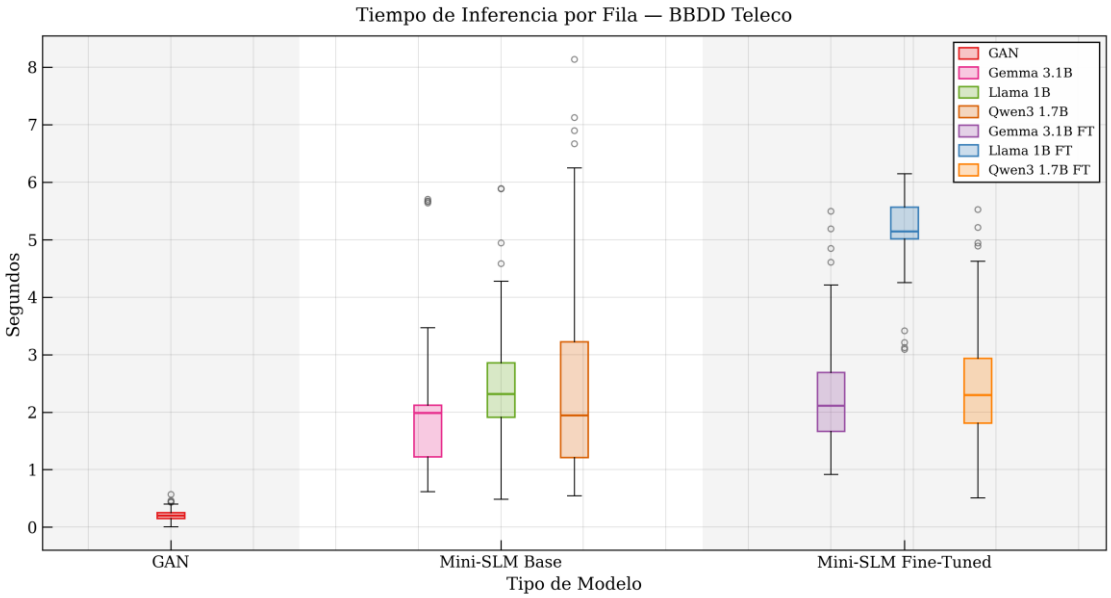


Figura Anexo 57: Boxplots de tiempo de inferencia por modelo en el corpus propio. Fuente propia.

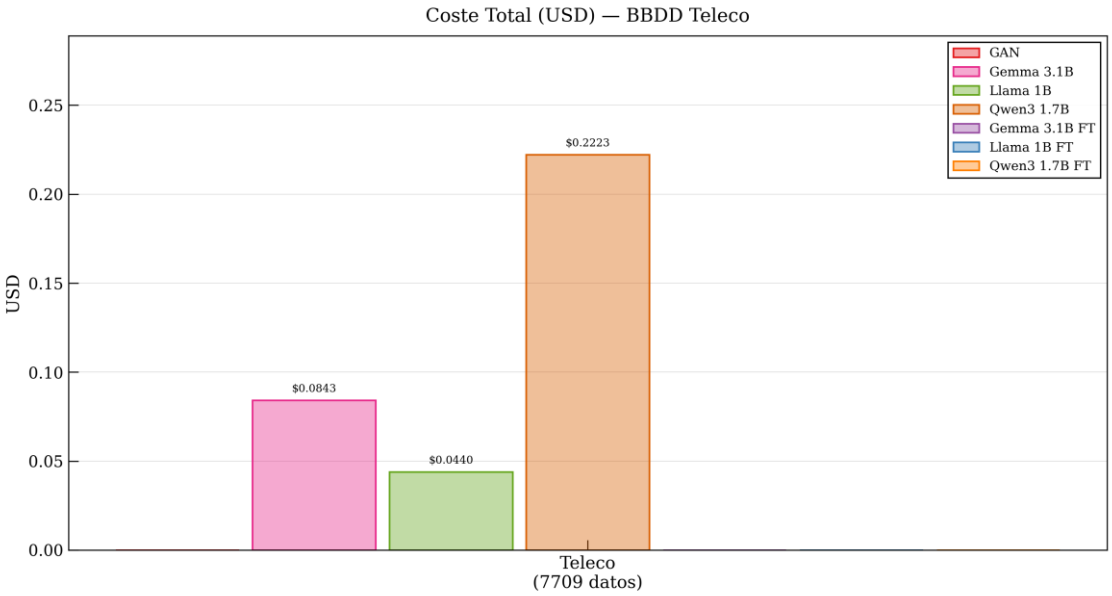


Figura Anexo 58: Barplots de coste total de todos los modelos en el corpus propio. Fuente propia.

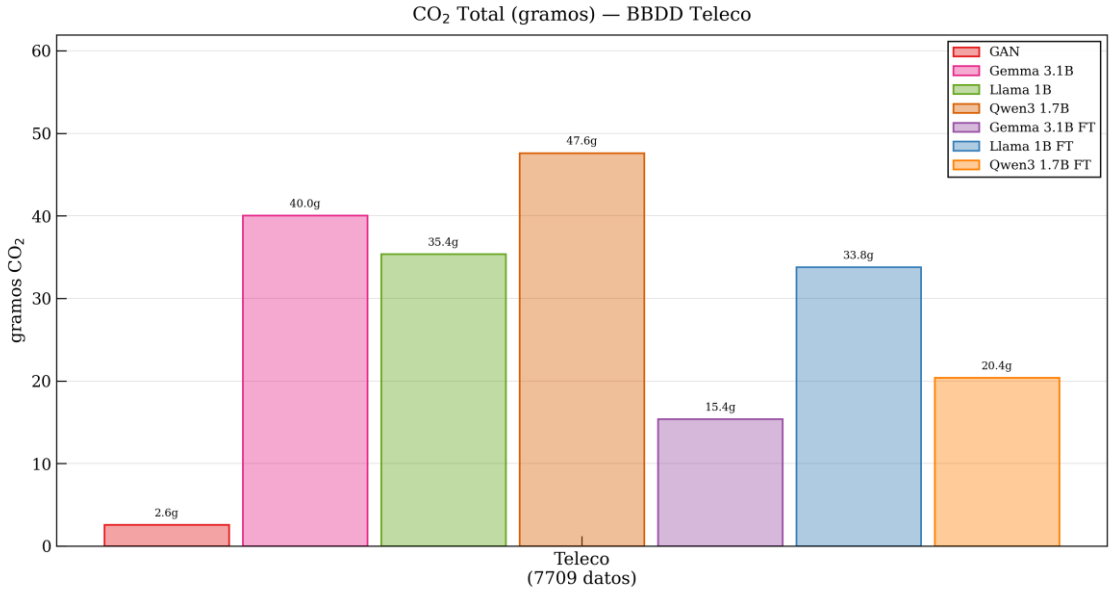


Figura Anexo 59: Barplots de Co2 total de todos los modelos en el corpus propio. Fuente propia.

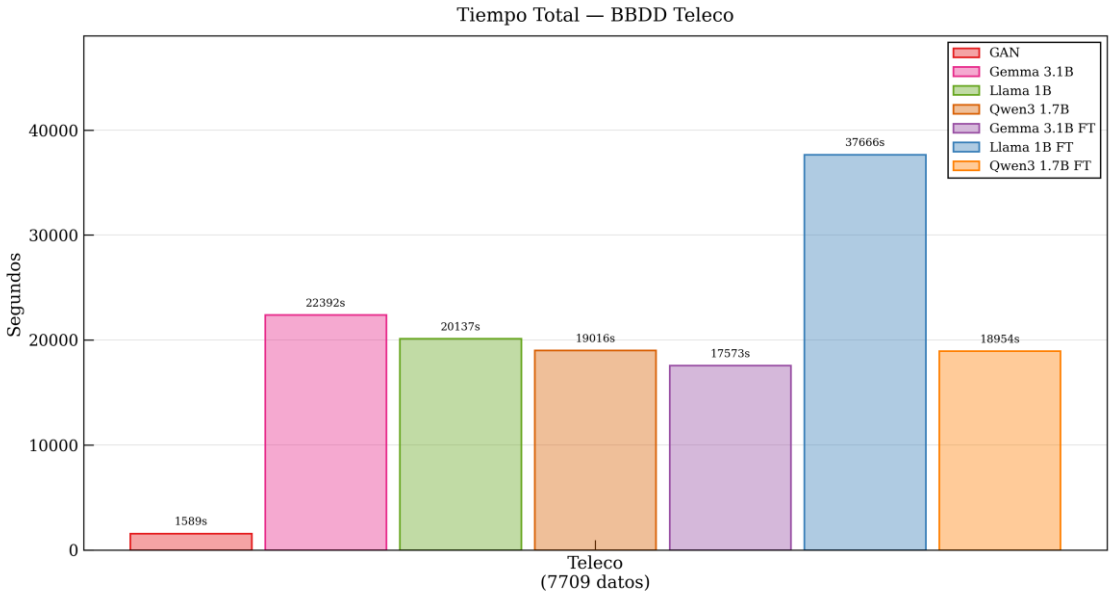


Figura Anexo 60: Barplots de tiempo total de todos los modelos en el corpus propio. Fuente propia.

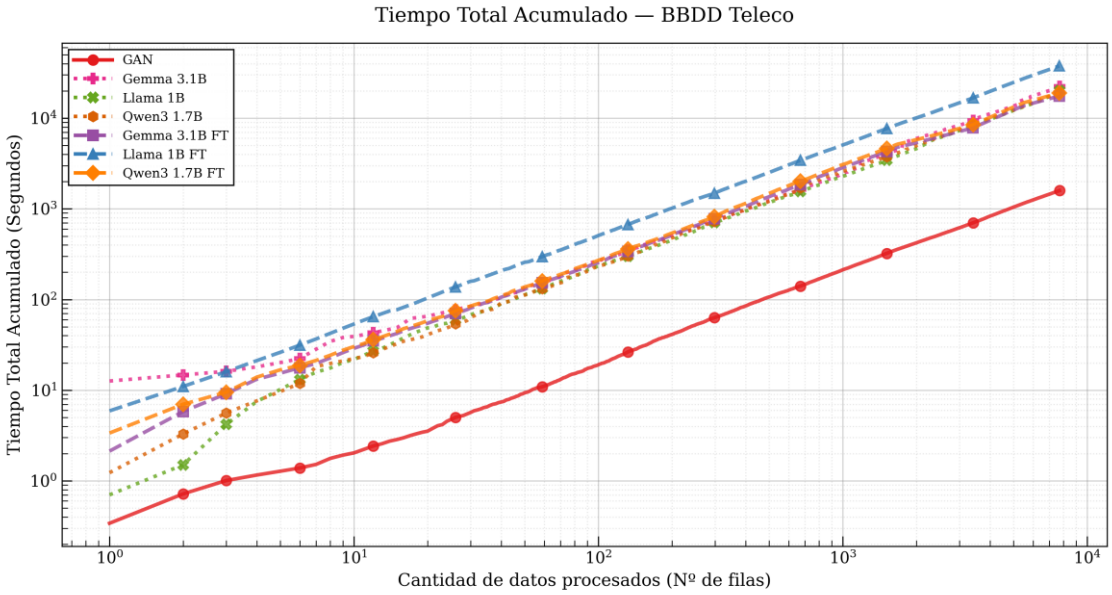


Figura Anexo 61: Tiempo total acumulado por modelo en el corpus propio. Fuente propia.

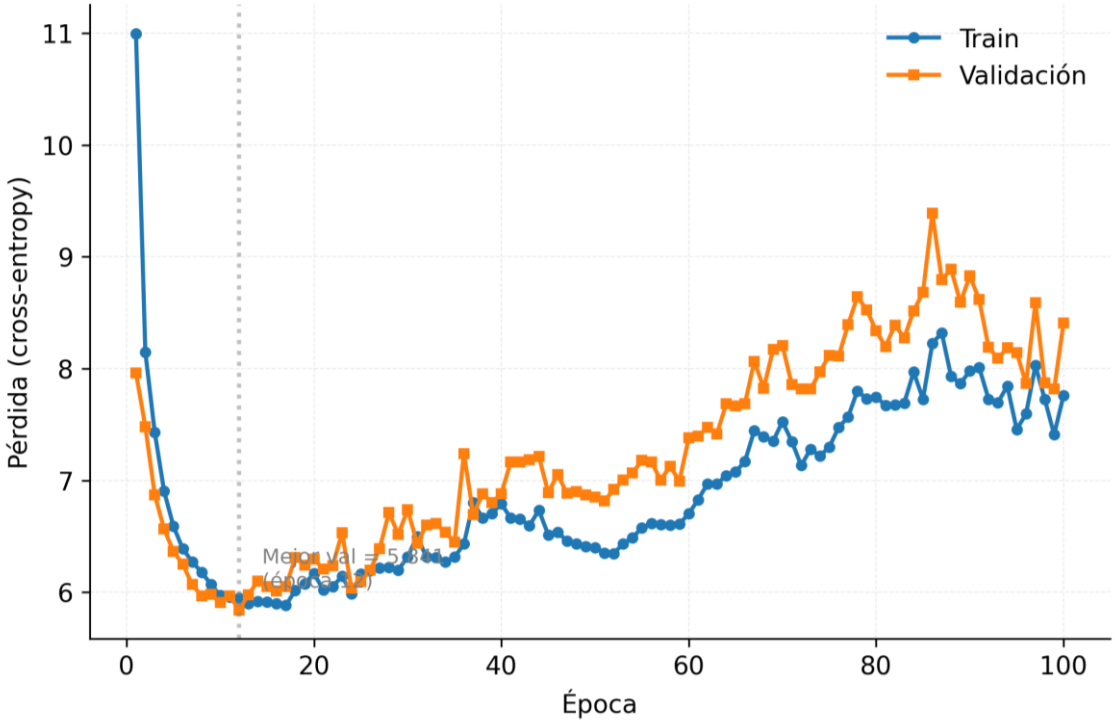


Figura Anexo 62: Curvas de aprendizaje de la GAN. Fuente propia.

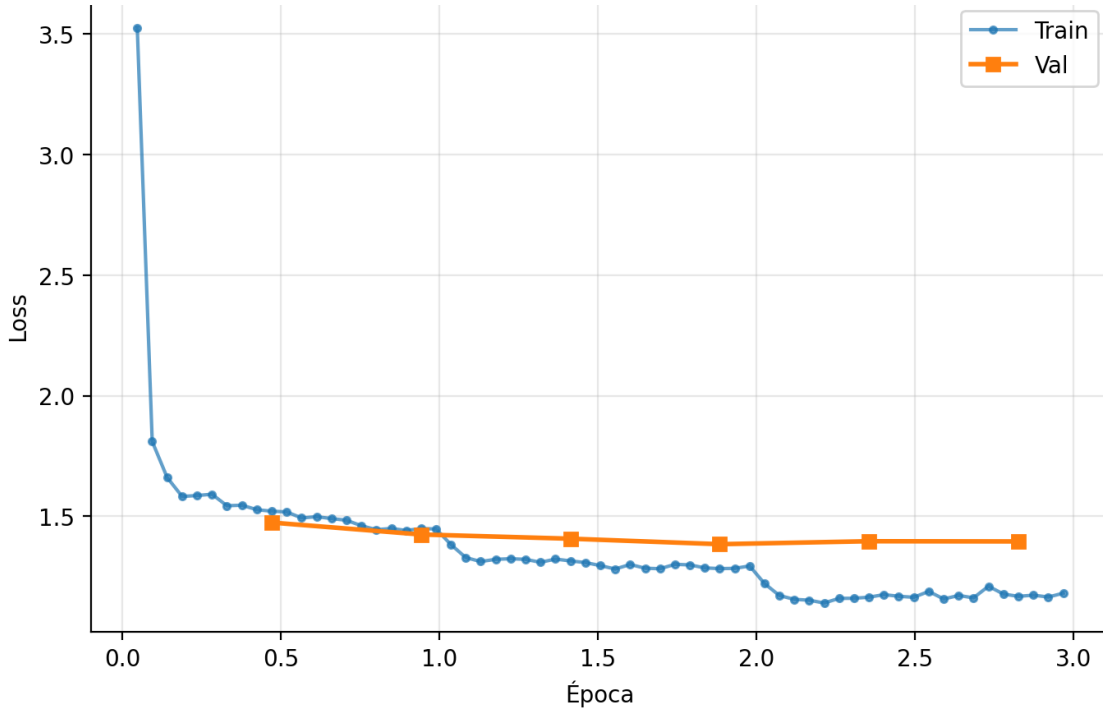


Figura Anexo 63: Curva de aprendizaje de Gemma 3 1B fine-tuned. Fuente propia.

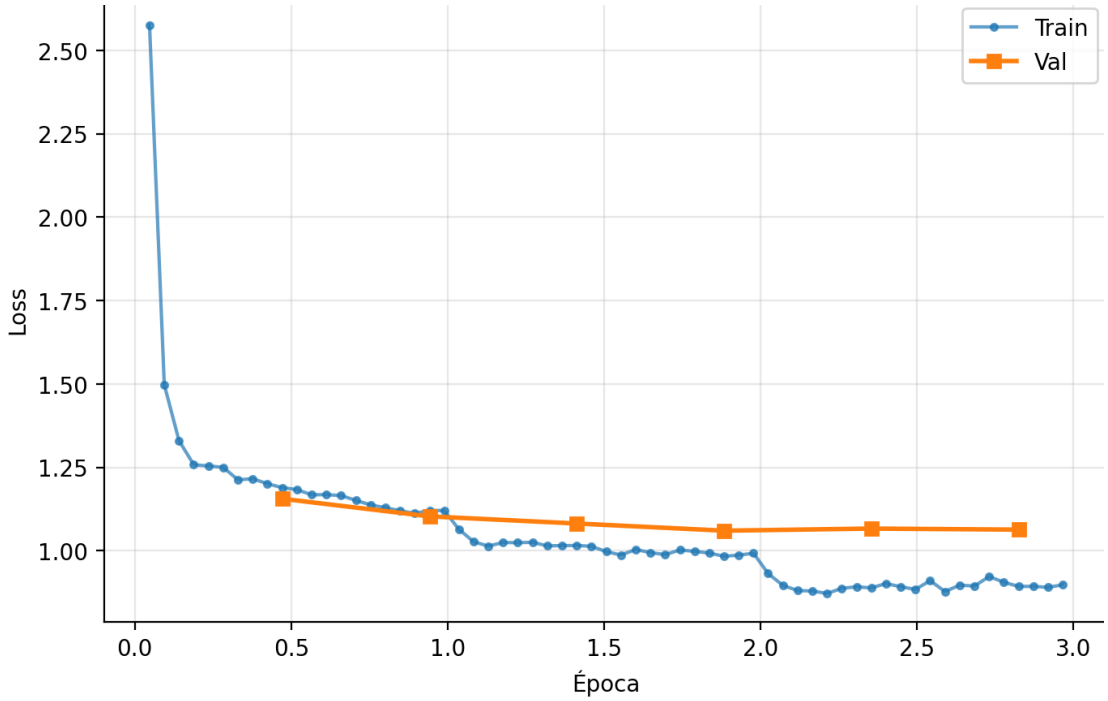


Figura Anexo 64: Curva de aprendizaje de Llama 3.2 1B fine-tuned. Fuente propia.

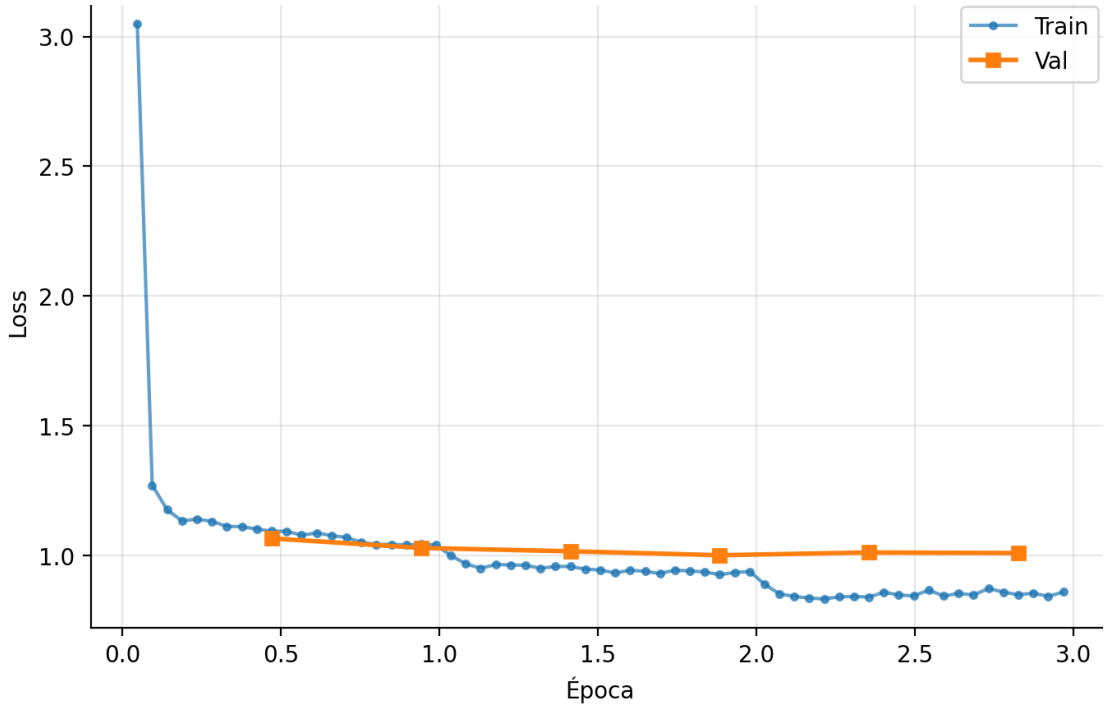


Figura Anexo 65: Curva de aprendizaje de Qwen 3 1.7B fine-tuned. Fuente propia.

D.3.3 Post hoc

Tabla Anexo 24: Análisis post-hoc (Nemenyi + Wilcoxon r) para ROUGE-L en el corpus propio.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
GAN	Gemma 3 1B	+0,0118	< 0,001	**Sí**	0,1476	Pequeño
GAN	Llama 3.2 1B	+0,0144	< 0,001	**Sí**	0,1951	Pequeño
GAN	Qwen 3 1.7B	-0,0119	< 0,001	**Sí**	0,1659	Pequeño
GAN	Gemma 3 1B FT	-0,1169	< 0,001	**Sí**	0,7927	Grande
GAN	Llama 3.2 1B FT	+0,0006	0,2775	**No**	0,0201	Nulo
GAN	Qwen 3 1.7B FT	-0,1394	< 0,001	**Sí**	0,8255	Grande
Gemma 3 1B	Llama 3.2 1B	+0,0026	< 0,001	**Sí**	0,0770	Nulo
Gemma 3 1B	Qwen 3 1.7B	-0,0237	< 0,001	**Sí**	0,4275	Mediano
Gemma 3 1B	Gemma 3 1B FT	-0,1287	< 0,001	**Sí**	0,8216	Grande
Gemma 3 1B	Llama 3.2 1B FT	-0,0113	< 0,001	**Sí**	0,1152	Pequeño
Gemma 3 1B	Qwen 3 1.7B FT	-0,1513	< 0,001	**Sí**	0,8442	Grande
Llama 3.2 1B	Qwen 3 1.7B	-0,0263	< 0,001	**Sí**	0,6510	Grande
Llama 3.2 1B	Gemma 3 1B FT	-0,1313	< 0,001	**Sí**	0,8165	Grande
Llama 3.2 1B	Llama 3.2 1B FT	-0,0139	< 0,001	**Sí**	0,1687	Pequeño
Llama 3.2 1B	Qwen 3 1.7B FT	-0,1539	< 0,001	**Sí**	0,8414	Grande
Qwen 3 1.7B	Gemma 3 1B FT	-0,1050	< 0,001	**Sí**	0,7448	Grande
Qwen 3 1.7B	Llama 3.2 1B FT	+0,0125	< 0,001	**Sí**	0,1988	Pequeño
Qwen 3 1.7B	Qwen3 1.7B FT	-0,1275	< 0,001	**Sí**	0,7986	Grande
Gemma 3 1B FT	Llama 3.2 1B FT	+0,1174	< 0,001	**Sí**	0,7598	Grande
Gemma 3 1B FT	Qwen 3 1.7B FT	-0,0226	< 0,001	**Sí**	0,1729	Pequeño
Llama 3.2 1B FT	Qwen 3 1.7B FT	-0,1400	< 0,001	**Sí**	0,8487	Grande

Tabla Anexo 25: Análisis post-hoc (Nemenyi + Wilcoxon r) para METEOR en el corpus propio.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
GAN	Gemma 3 1B	-0,0962	< 0,001	**Sí**	0,8038	Grande
GAN	Llama 3.2 1B	-0,0827	< 0,001	**Sí**	0,7527	Grande
GAN	Qwen 3 1.7B	-0,1166	< 0,001	**Sí**	0,8259	Grande
GAN	Gemma 3 1B FT	-0,1492	< 0,001	**Sí**	0,8400	Grande
GAN	Llama 3.2 1B FT	-0,1063	< 0,001	**Sí**	0,7977	Grande
GAN	Qwen 3 1.7B FT	-0,1737	< 0,001	**Sí**	0,8490	Grande
Gemma 3 1B	Llama 3.2 1B	+0,0135	< 0,001	**Sí**	0,2054	Pequeño
Gemma 3 1B	Qwen 3 1.7B	-0,0204	< 0,001	**Sí**	0,2741	Pequeño
Gemma 3 1B	Gemma 3 1B FT	-0,0530	< 0,001	**Sí**	0,3955	Mediano
Gemma 3 1B	Llama 3.2 1B FT	-0,0101	< 0,001	**Sí**	0,0931	Nulo
Gemma 3 1B	Qwen 3 1.7B FT	-0,0775	< 0,001	**Sí**	0,5438	Grande
Llama 3.2 1B	Qwen 3 1.7B	-0,0339	< 0,001	**Sí**	0,5493	Grande
Llama 3.2 1B	Gemma 3 1B FT	-0,0665	< 0,001	**Sí**	0,4963	Mediano
Llama 3.2 1B	Llama 3.2 1B FT	-0,0236	< 0,001	**Sí**	0,2493	Pequeño
Llama 3.2 1B	Qwen 3 1.7B FT	-0,0910	< 0,001	**Sí**	0,6307	Grande
Qwen 3 1.7B	Gemma 3 1B FT	-0,0326	< 0,001	**Sí**	0,2236	Pequeño
Qwen 3 1.7B	Llama 3.2 1B FT	+0,0103	< 0,001	**Sí**	0,1183	Pequeño
Qwen 3 1.7B	Qwen3 1.7B FT	-0,0571	< 0,001	**Sí**	0,4065	Mediano
Gemma 3 1B FT	Llama 3.2 1B FT	+0,0429	< 0,001	**Sí**	0,2984	Pequeño
Gemma 3 1B FT	Qwen 3 1.7B FT	-0,0245	< 0,001	**Sí**	0,1682	Pequeño
Llama 3.2 1B FT	Qwen 3 1.7B FT	-0,0674	< 0,001	**Sí**	0,5770	Grande

Tabla Anexo 26: Análisis post-hoc (Nemenyi + Wilcoxon r) para BLEU en el corpus propio.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
GAN	Gemma 3 1B	-0,0069	< 0,001	**Sí**	0,3706	Mediano
GAN	Llama 3.2 1B	-0,0083	< 0,001	**Sí**	0,3683	Mediano
GAN	Qwen 3 1.7B	-0,0100	< 0,001	**Sí**	0,4102	Mediano
GAN	Gemma 3 1B FT	-0,0518	< 0,001	**Sí**	0,5315	Grande
GAN	Llama 3.2 1B FT	-0,0188	< 0,001	**Sí**	0,5154	Grande
GAN	Qwen 3 1.7B FT	-0,0684	< 0,001	**Sí**	0,5767	Grande
Gemma 3 1B	Llama 3.2 1B	-0,0014	1,0000	**No**	0,0360	Nulo
Gemma 3 1B	Qwen 3 1.7B	-0,0032	< 0,001	**Sí**	0,1324	Pequeño
Gemma 3 1B	Gemma 3 1B FT	-0,0449	< 0,001	**Sí**	0,5070	Grande
Gemma 3 1B	Llama 3.2 1B FT	-0,0119	< 0,001	**Sí**	0,3188	Mediano
Gemma 3 1B	Qwen 3 1.7B FT	-0,0615	< 0,001	**Sí**	0,5761	Grande
Llama 3.2 1B	Qwen 3 1.7B	-0,0018	< 0,001	**Sí**	0,1078	Pequeño
Llama 3.2 1B	Gemma 3 1B FT	-0,0435	< 0,001	**Sí**	0,4921	Mediano
Llama 3.2 1B	Llama 3.2 1B FT	-0,0106	< 0,001	**Sí**	0,2772	Pequeño
Llama 3.2 1B	Qwen 3 1.7B FT	-0,0601	< 0,001	**Sí**	0,5649	Grande
Qwen 3 1.7B	Gemma 3 1B FT	-0,0417	< 0,001	**Sí**	0,4705	Mediano
Qwen 3 1.7B	Llama 3.2 1B FT	-0,0088	< 0,001	**Sí**	0,2117	Pequeño
Qwen 3 1.7B	Qwen3 1.7B FT	-0,0583	< 0,001	**Sí**	0,5507	Grande
Gemma 3 1B FT	Llama 3.2 1B FT	+0,0329	< 0,001	**Sí**	0,3503	Mediano
Gemma 3 1B FT	Qwen 3 1.7B FT	-0,0166	< 0,001	**Sí**	0,1340	Pequeño
Llama 3.2 1B FT	Qwen 3 1.7B FT	-0,0495	< 0,001	**Sí**	0,5437	Grande

Tabla Anexo 27: Análisis post-hoc (Nemenyi + Wilcoxon r) para BERTScore en el corpus propio.

Modelo A	Modelo B	Media	p ajust.	Sig.	R	Magnitud
GAN	Gemma 3 1B	+0,0158	< 0,001	**Sí**	0,5496	Grande
GAN	Llama 3.2 1B	+0,0107	< 0,001	**Sí**	0,3663	Mediano
GAN	Qwen 3 1.7B	+0,0099	< 0,001	**Sí**	0,3507	Mediano
GAN	Gemma 3 1B FT	-0,0331	< 0,001	**Sí**	0,8108	Grande
GAN	Llama 3.2 1B FT	-0,0035	< 0,001	**Sí**	0,1331	Pequeño
GAN	Qwen 3 1.7B FT	-0,0389	< 0,001	**Sí**	0,8397	Grande
Gemma 3 1B	Llama 3.2 1B	-0,0051	< 0,001	**Sí**	0,2720	Pequeño
Gemma 3 1B	Qwen 3 1.7B	-0,0059	< 0,001	**Sí**	0,3213	Mediano
Gemma 3 1B	Gemma 3 1B FT	-0,0489	< 0,001	**Sí**	0,8467	Grande
Gemma 3 1B	Llama 3.2 1B FT	-0,0193	< 0,001	**Sí**	0,6504	Grande
Gemma 3 1B	Qwen 3 1.7B FT	-0,0547	< 0,001	**Sí**	0,8603	Grande
Llama 3.2 1B	Qwen 3 1.7B	-0,0008	0,0409	**Sí**	0,0727	Nulo
Llama 3.2 1B	Gemma 3 1B FT	-0,0438	< 0,001	**Sí**	0,8251	Grande
Llama 3.2 1B	Llama 3.2 1B FT	-0,0142	< 0,001	**Sí**	0,4955	Mediano
Llama 3.2 1B	Qwen 3 1.7B FT	-0,0496	< 0,001	**Sí**	0,8512	Grande
Qwen 3 1.7B	Gemma 3 1B FT	-0,0430	< 0,001	**Sí**	0,8283	Grande
Qwen 3 1.7B	Llama 3.2 1B FT	-0,0134	< 0,001	**Sí**	0,4801	Mediano
Qwen 3 1.7B	Qwen3 1.7B FT	-0,0488	< 0,001	**Sí**	0,8529	Grande
Gemma 3 1B FT	Llama 3.2 1B FT	+0,0296	< 0,001	**Sí**	0,7376	Grande
Gemma 3 1B FT	Qwen 3 1.7B FT	-0,0058	< 0,001	**Sí**	0,2036	Pequeño
Llama 3.2 1B FT	Qwen 3 1.7B FT	-0,0354	< 0,001	**Sí**	0,8476	Grande

D.3.4 Critical Difference Diagram (CD)

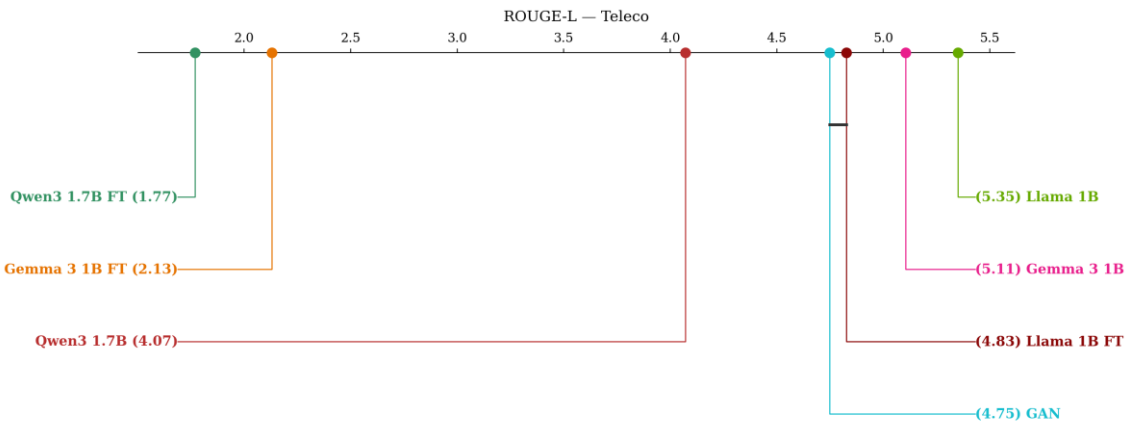


Figura Anexo 66: Diagrama de diferencia crítica para ROUGE-L en el corpus propio. Fuente propia.

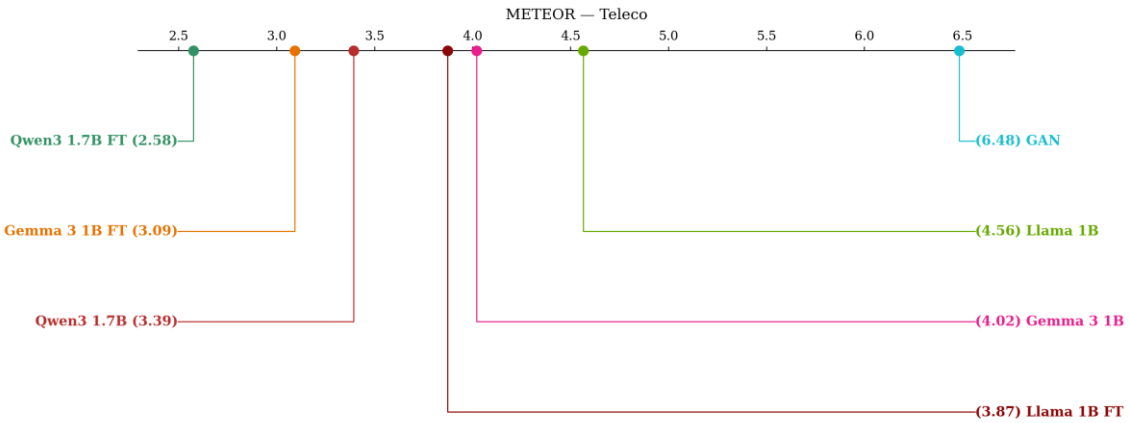


Figura Anexo 67: Diagrama de diferencia crítica para METEOR en el corpus propio. Fuente propia.

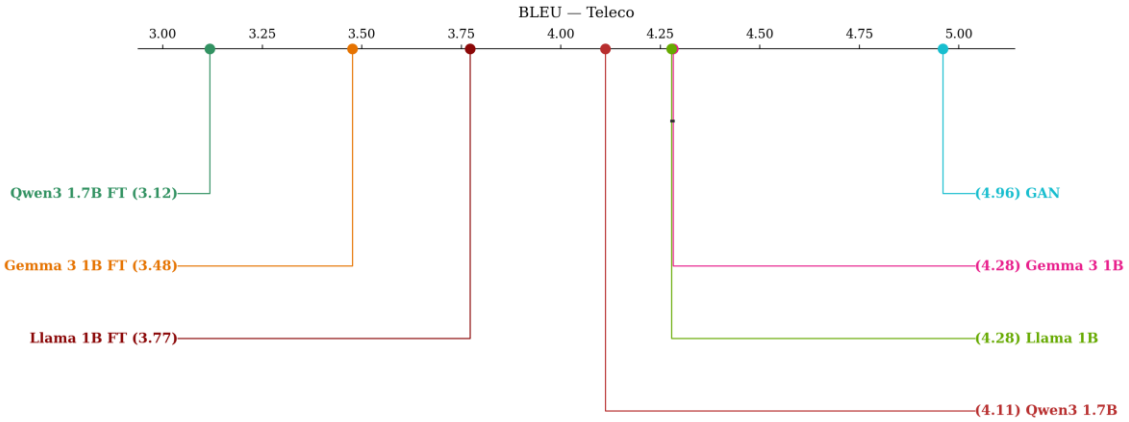


Figura Anexo 68: Diagrama de diferencia crítica para BLEU en el corpus propio. Fuente propia.

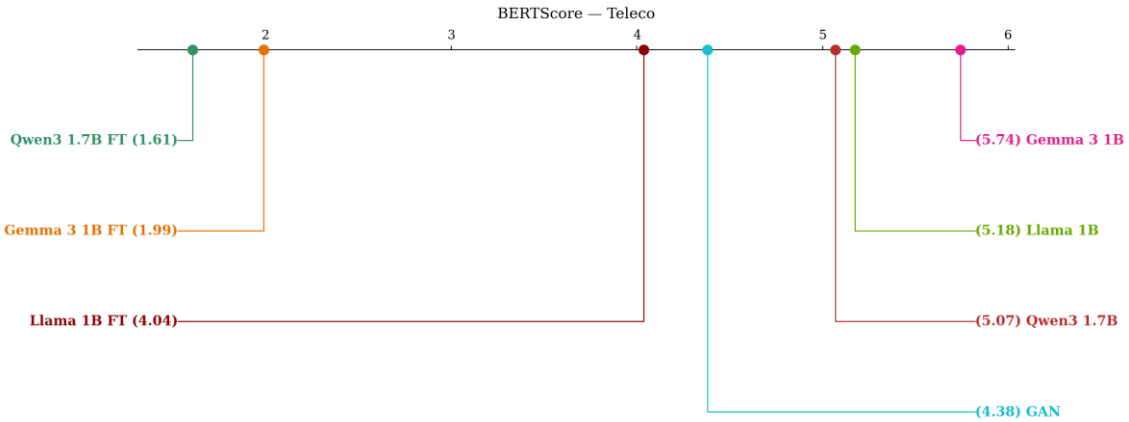


Figura Anexo 69: Diagrama de diferencia crítica para BERTScore en el corpus propio. Fuente propia.

Anexo E. Interfaz y funcionalidades de TelecoSLM

TelecoSLM es la aplicación desarrollada como parte de este trabajo para llevar el modelo Qwen 3 1.7B FT al dispositivo del usuario. La interfaz se organiza en tres pantallas, inicio, *chat* y configuración. Existe un manual de usuario bajo petición, que ofrece detalladamente el uso y la interfaz de la aplicación

E.1 Pantalla de inicio

En la parte superior se muestra el nombre de la aplicación junto con un *badge* de estado que cambia de color según la situación: gris cuando no hay ningún modelo cargado, naranja durante el proceso de carga y verde cuando la inferencia está disponible. Este indicador permanece visible en todo momento para que el usuario sepa si puede enviar preguntas.

El modelo utiliza un *system prompt* que condiciona el comportamiento del modelo sin necesidad de reentrenamiento. El *prompt* incluye cinco ejemplos concretos de pares pregunta-respuesta (*few-shot*) y reglas de brevedad que evitan que el modelo rellene las respuestas con información no solicitada.

En la parte inferior de la pantalla se muestran las conversaciones recientes, con un máximo de 20 almacenadas. Las más antiguas se eliminan automáticamente cuando se supera ese límite. Cada entrada del historial muestra el título de la conversación (derivado de la primera pregunta).

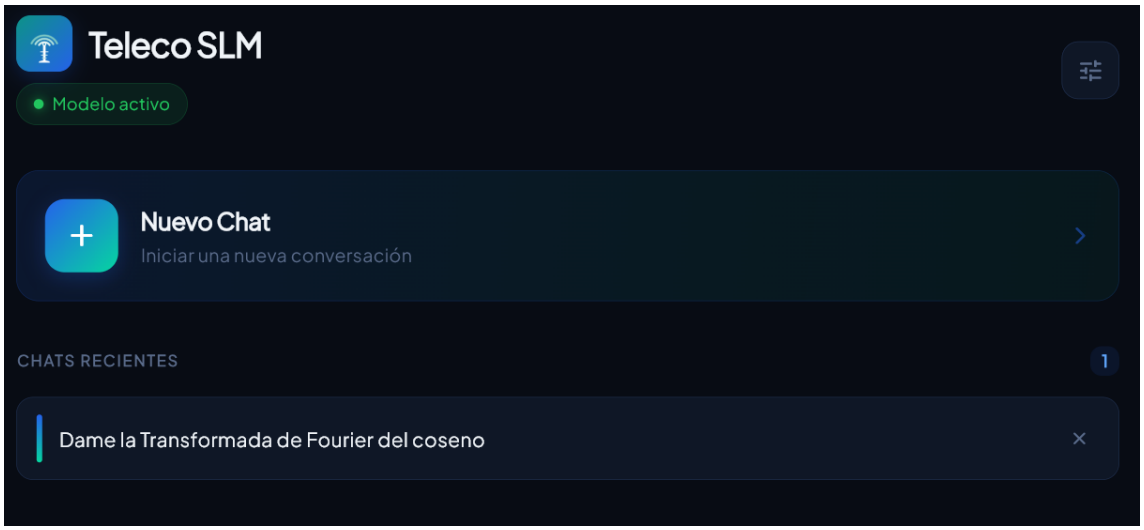


Figura Anexo 70: Pantalla de inicio de TelecoSLM. Fuente propia.

E.2 Pantalla de chat

Al seleccionar un modo o abrir una conversación del historial, la aplicación navega a la pantalla de chat. En la parte inferior de cada conversación aparece un texto fijo que advierte al usuario de que la aplicación utiliza inteligencia artificial generativa, que las respuestas pueden contener errores o información incompleta, y que no sustituye la consulta de fuentes académicas oficiales.

Los mensajes del usuario aparecen alineados a la derecha con un icono de persona, y las respuestas del modelo a la izquierda con un icono de robot. Las respuestas se generan *token* a *token* y aparecen progresivamente en la burbuja del asistente, de forma que el usuario puede empezar a leer antes de que la respuesta esté completa. Durante la generación se muestra un indicador visual de actividad. Mientras el modelo genera una respuesta, el botón de envío se transforma en un botón de parada. Al pulsarlo, la generación se interrumpe y el texto producido hasta ese momento se conserva en el historial, marcado como respuesta detenida.

Para no desbordar la ventana de contexto del modelo (2.048 *tokens* en *Android*, 4.096 en *Windows*), la aplicación envía únicamente el *system prompt* y el último mensaje del usuario en *Android*, o los últimos cinco mensajes en *Windows*. El historial completo de la conversación se mantiene almacenado en la base de datos y es visible al desplazarse hacia arriba, pero no se envía al modelo.

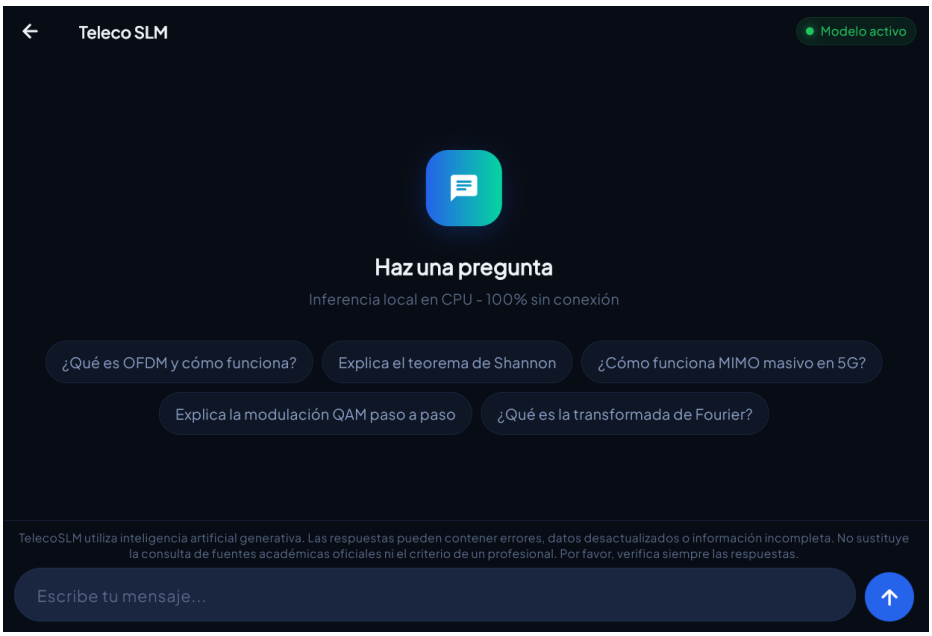


Figura Anexo 71: Pantalla de chat de TelecoSLM. Fuente propia.

E.3 Pantalla de configuración

Se accede a la pantalla de configuración desde el icono de configuración en la esquina superior derecha de la pantalla de inicio. La sección principal presenta las tres opciones disponibles para obtener el modelo, descarga desde *HuggingFace*, seleccionar archivo GGUF o servidor remoto. Solo una fuente puede estar activa a la vez.

Esta interfaz contiene un botón para borrar todo el historial de conversaciones. Al pulsarlo, la aplicación muestra un diálogo de confirmación antes de proceder, ya que la acción es irreversible.

Finalmente, una sección de "Acerca de" muestra información básica de la aplicación, nombre, versión, modelo utilizado y una breve descripción del proyecto.

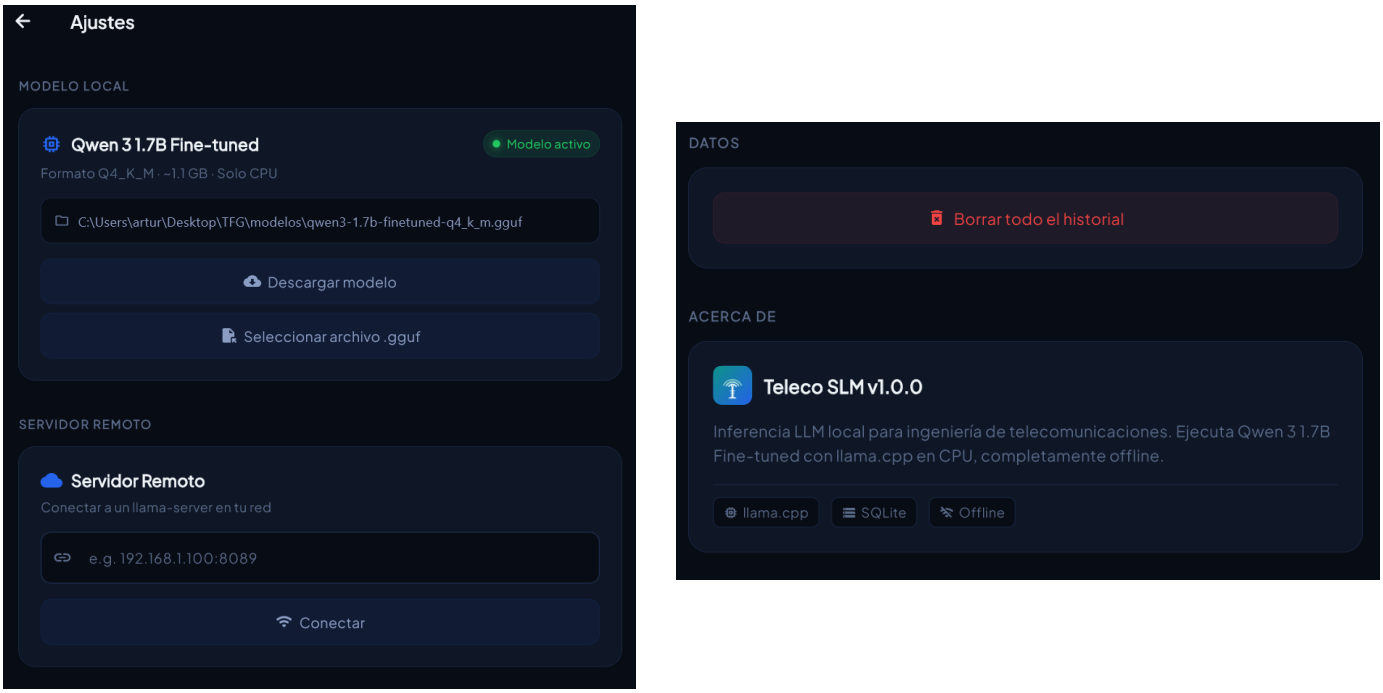


Figura Anexo 72: Pantalla de configuración de TelecoSLM. Fuente propia.